Autonomous Parafoil / CanSat Targeted-Landing System

Complete Technical Context, Iteration History, Code Archive, Tests, Logs, and Open Issues

Compiled from the conversation history, user-provided source files, test logs, and project memory

5 October 2026

Contents

1 Purpose of this document **3**

2 Executive project state **3**

3 Hardware and wiring history **3**
- 3.1 Teensy, GPS and buses . . . 3
- 3.2 ST-3009 servo bus . . . 4
- 3.3 Deployment servo . . . 4

4 Known verified sensor and calibration details **4**
- 4.1 GPS . . . 4
- 4.2 QMC5883 compass . . . 4
- 4.3 IMU . . . 4
- 4.4 BMP581 . . . 4

5 Servo control history and the "both motors move" issue **5**

6 Navigation algorithm history **5**
- 6.1 Original basic target-following algorithm . . . 5
- 6.2 Target capture realization . . . 5

7 Iteration timeline **5**

8 Python navigation logic testing **6**

9 Targeted landing simulation **6**

10 MATLAB live-monitor concept **7**

11 Most recent MATLAB screenshot diagnosis **7**

12 Why the standalone servo test works but the full code did not **8**

13 Current control law in equations **8**

14 How glide speed, descent rate, wind, and steering response are to be obtained 9
14.1 Glide speed . . . 9
14.2 Descent rate . . . 9
14.3 Wind . . . 9
14.4 Steering response . . . 9

15 Recommended next engineering sequence 9

16 Known open questions / do not silently assume 10

17 Compile/environment history 10

18 Command reference 10

A Source-code archive 11
A.1 Archived source 1: Autonomous_Parafoil_Teensy41_Final.ino . . . 11
A.2 Archived source 2: Autonomous_Parafoil_Teensy41_FINAL_WDT_T4.ino . . . 31
A.3 Archived source 3: Pasted code(10).cpp . . . 53
A.4 Archived source 4: Pasted code(20261005-071236).cpp . . . 106
A.5 Archived source 5: Pasted text(20261003-150406).txt . . . 158
A.6 Archived source 6: Teensy_Parafoil_Final_SingleServo_MATLAB.ino . . . 211
A.7 Archived source 7: Teensy_Parafoil_Final_MATLAB.ino . . . 266
A.8 Archived source 8: Teensy_Parafoil_FINAL_SERVO_FIXED.ino . . . 317
A.9 Archived source 9: ST3009_SERVO_BUS_TEST_ONLY.ino . . . 369
A.10 Archived source 10: parafoil_nav_test.py . . . 372
A.11 Archived source 11: parafoil_nav_test_corrected.py . . . 377
A.12 Archived source 12: targeted_landing_simulation.py . . . 382
A.13 Archived source 13: teensy_matlab_telemetry_addon.ino . . . 385
A.14 Archived source 14: live_parafoil_target_monitor.m . . . 387
A.15 Archived source 15: live_parafoil_target_monitor_FINAL.m . . . 393

B Artifact inventory and provenance notes 401

C Final hand-off summary for another AI 401

1 Purpose of this document

This document is a hand-off/context package for another AI model or engineer who will continue development of the autonomous parafoil targeted-landing system. It intentionally records the project history rather than silently replacing earlier designs. The archive distinguishes between verified observations, implemented code, proposed changes, and unresolved questions.

The immediate project goal is an autonomous parafoil/CanSat system in which a Teensy 4.1 receives GPS position, computes the direction to a fixed or user-specified target coordinate, determines the required left/right steering action, actuates two ST-3009 serial bus servos, and reports the live system state to MATLAB. A later stage is intended to move from simple bearing-following to a more robust precision-landing controller using measured glide speed, descent rate, wind, steering response, and eventually a PD or predicted-landing controller.

2 Executive project state

Item	Current state
MCU	Teensy 4.1
GPS	Foxeer M10Q 180; proven standalone GPS baud is 38400 on Serial3
GPS pins	GPS TX →Teensy 15 (RX3); GPS RX →Teensy 14 (TX3)
Navigation sensors	GPS course above speed threshold; QMC5883 compass as low-speed fallback in current code
Compass	QMC5883 at I2C 0x0D; calibration values documented below; navigation mirror enabled
IMU	LSM6DSO32 at 0x6A on I2C
Barometer	BMP581 0x46/0x47; diagnostic only in current controller
Steering servos	2 x ST-3009 serial bus servos, IDs 1 and 2, Serial1 half duplex, 1 Mbps
Servo neutral	2048
Current desired steering architecture	Single-servo bench/mechanical test: Servo 1 = left, Servo 2 = right; only requested brake servo moves from center
Deployment	SG90 on pin 8, currently manual in lab mode; prior automatic deployment implementation exists
Target capture	35 m reference exists, but capture action is disabled because passing within 35 m while still high above ground is not equivalent to final landing within 35 m
Control law today	Deadband + linear course-error steering, saturated at ±450 counts; not a PD controller
MATLAB	Live GPS/servo monitoring and logging over Teensy USB serial; machine-readable NAVMAT telemetry
Most recent observed failure	Servo-only test code moves the ST-3009s, but the full navigation code does not; screenshot shows HDOP 5.13 while the older/current strict navigation gate requires HDOP $\leq$ 3.5, causing navigation heading to become invalid and steering demand to remain zero

3 Hardware and wiring history

3.1 Teensy, GPS and buses

The verified GPS arrangement is:
GPS TX →Teensy pin 15 / RX3; GPS RX →Teensy pin 14 / TX3; common ground; GPS UART on Serial3 at 38400 baud. The PC/MATLAB USB serial is separate and normally runs at 115200 baud.

The I2C bus is shared on SDA pin 18 and SCL pin 19. The QMC5883 is at 0x0D, the LSM6DSO32 at 0x6A, and the BMP581 has been seen at 0x46 or 0x47.

3.2 ST-3009 servo bus

The ST-3009 devices are serial bus servos. Servo ID 1 and Servo ID 2 share the same half-duplex communication bus; the IDs distinguish the devices. They are not connected as "servo 1 to RX and servo 2 to TX". The physical bus must use the same known-good half-duplex interface/wiring that worked in the standalone servo test. The servo supply is external and must share ground with the Teensy.

The current software uses `SMS_STS st;` from SCServo library version 1.0.2. Earlier compilation failed when the wrong class name `SCServo` was used; the working class is `SMS_STS`.

3.3 Deployment servo

SG90 signal is on pin 8. The current lab version leaves automatic deployment disabled; manual `DEPLOY` is preferred during bench testing.

4 Known verified sensor and calibration details

4.1 GPS

A standalone TinyGPSPlus test on Serial3 at 38400 baud was explicitly verified as working. The final controller therefore uses 38400 rather than the earlier 115200 experiments.

4.2 QMC5883 compass

The standalone compass implementation was verified. Initialization used register 0x0B = 0x01 and control register 0x09 = 0x1D. Calibration values were:

$$\begin{aligned} \text{OFFSET}_X &= 180.0000, & \text{SCALE}_X &= 0.956484, \\ \text{OFFSET}_Y &= 336.0000, & \text{SCALE}_Y &= 0.904406, \\ \text{OFFSET}_Z &= 615.0000, & \text{SCALE}_Z &= 1.178127. \end{aligned}$$

The tested navigation orientation was mirrored, so the controller contains

$$\text{navHeading} = 360^\circ - \text{compassHeading}$$

when the mirror flag is enabled. The standalone compass test itself was not mirrored; the mirror was only added for navigation.

4.3 IMU

The working library/API is `Adafruit_LSM6DSOX`; the device address is 0x6A. Prior setup used a 16 g accelerometer range, 2000 dps gyro range, and 208 Hz accelerometer and gyro data rates.

4.4 BMP581

BMP581 is used as a diagnostic presence check in the current controller. No altitude-based steering or deployment decision has been integrated yet.

5 Servo control history and the "both motors move" issue

The original final-lab steering implementation used differential steering. A positive demand was mapped to

$$S_1 = 2048 - d, \qquad S_2 = 2048 + d$$

and a negative demand used the opposite signs. Both ST-3009s were therefore commanded on every steering change. This was intentional differential steering, not a software bug.

The project requirement was later clarified as a single-brake style bench/mechanical test: a LEFT command should move only the left servo from neutral while the right stays at 2048, and vice versa for RIGHT. The single-servo mapping is therefore:

$$\begin{aligned} \text{CENTER}: &\quad S_1 = 2048,\ S_2 = 2048, \\ \text{LEFT}: &\quad S_1 \neq 2048,\ S_2 = 2048, \\ \text{RIGHT}: &\quad S_1 = 2048,\ S_2 \neq 2048. \end{aligned}$$

A direct jump from LEFT to RIGHT can still make both physical servos move because one must return to center while the other leaves center.

The standalone ST-3009 test code uses a servo command speed of 2400 and acceleration 50; this became an important reference because the full controller initially used a slower speed of 500.

6 Navigation algorithm history

6.1 Original basic target-following algorithm

The original controller logic is:

1. Read current GPS latitude/longitude.
2. Compute distance to target.
3. Compute target bearing using the current position and target coordinate.
4. Select current direction: GPS course when speed is above 1.5 m/s, otherwise QMC5883 compass.
5. Compute course error as target bearing minus current direction.
6. Wrap course error to $-180°$ to $+180°$.
7. Apply a $\pm 5°$ deadband.
8. Linearly map error from the deadband to ± 450 servo counts at $\pm 90°$ error.
9. Apply slew limiting of 60 counts per navigation update.

This is a proportional-like controller with deadband, saturation, and slew limiting. It is **not** a PD controller.

6.2 Target capture realization

The early design used distance ≤ 35 m as "target captured" and centered the servos. Simulation later demonstrated why this is unsafe for a parafoil: entering a 35 m horizontal circle while hundreds of metres above the ground does not imply a final landing within 35 m. The current MATLAB/Teensy lab design therefore keeps the 35 m circle as a reference and disables the capture action by default.

7 Iteration timeline

Stage	Artifact / concept	Main result or change
Early hardware	GPS, compass, IMU, BMP, ST-3009, SG90 integration	Pin mapping and sensor addresses established.

Compass calibration	QMC5883 standalone test	Calibration offsets/scales established; navigation mirror discovered.
GPS standalone	TinyGPSPlus / Serial3	38400 baud verified.
Initial guided code	Autonomous parafoil controller	GPS-to-target bearing, GPS-course/compass heading, servo steering, state machine.
Servo-bus debugging	SCServo / ST-3009	Correct class identified as `SMS_STS`; IDs 1/2 and half-duplex bus verified.
Differential steering stage	Original `commandSteering()`	Both servos moved in opposite directions; later determined to be unsuitable for the desired single-brake bench test.
Python logic stage	`parafoil_nav_test.py` and corrected version	Offline distance/bearing/error/servo mapping checks; discovered that mislabeled test headings can make all commands saturate.
Targeted-flight simulation	`targeted_landing_simulation.py`	600 m, assumed 8 m/s glide speed, 4 m/s descent, no wind; a sample run produced 617.7 m final error despite entering the 35 m circle, exposing the capture-logic problem.
MATLAB live monitoring	MATLAB COM monitor + NAVMAT telemetry	Live GPS path, distance, target bearing, GPS course, heading, servo command/feedback and approach status.
Single-servo final lab code	`Teensy_Parafoil_Final_Single_Servo_MATLAB.ino`	Added single-servo mapping and MATLAB telemetry, but inherited strict GPS-quality gate.
Servo-fixed attempt	`Teensy_Parafoil_FINAL_SERVO_FIXED.ino`	Servo-fixed code and servo timing/speed improvements were proposed/generated.
Latest user test	User-pasted 20261005 controller	Standalone servo test works but final navigation code does not move servos when GPS is moved. Screenshot shows HDOP 5.13; strict quality threshold 3.5 blocks navigation.

8 Python navigation logic testing

The first Python test bench was designed to simulate current GPS position and heading. It calculated distance to target, target bearing, wrapped course error, the ±5° deadband, steering demand up to ±450, and servo positions around neutral 2048. A corrected normal Python file was produced after the user attempted to execute a Jupyter ".ipynb" as a normal Python script and got the error "NameError: name 'null' is not defined". The corrected distribution included both a normal `.py` script and a valid Jupyter notebook.

A key test mistake was identified in the initial run. With target latitude 32.7593400 and current latitude 32.6438366 at the same longitude, the target was north (bearing approximately 0°). A test labeled "ALIGNED" used heading 180°, which is actually exactly opposite the target. The saturated −450 result was therefore correct. This was an important validation of the angle-wrapping and saturation logic, not evidence of a controller bug.

9 Targeted landing simulation

The simplified 2D simulator used:

- Initial altitude = 600 m.
- Glide speed = 8 m/s (explicitly an assumed placeholder, not a measured value).
- Descent rate = 4 m/s (also an assumed placeholder).
- Constant or random wind options.
- Simplified maximum turn rate = 25 deg/s (model assumption).

- Servo demand to heading turn-rate model.
- 35 m target capture reference.

One sample output was:

```
Initial altitude   : 600.0 m
Glide speed        : 8.00 m/s
Descent rate       : 4.00 m/s
Wind               : N +0.00 m/s, E +0.00 m/s
Flight time        : 150.10 s
Landing offset     : N -529.68 m, E +317.81 m
Landing error      : 617.70 m
Target captured    : YES
35 m requirement   : FAIL
```

The result was not treated as a real-flight prediction because the parafoil parameters were guessed. It was instead used to expose a real algorithmic issue: current-distance capture is not the same as predicted landing-point capture.

10 MATLAB live-monitor concept

The intended data flow is:

$$\text{GPS} \rightarrow \text{Teensy Serial3} \rightarrow \text{navigation code}$$

and

$$\text{ST-3009 bus} \rightarrow \text{Teensy Serial1} \rightarrow \text{feedback},$$

with the Teensy forwarding a machine-readable telemetry line over USB to MATLAB.

The telemetry includes:

```
NAVMAT,millis,lat,lon,alt,speed,gpsCourse,heading,
distance,bearing,error,demand,cmdS1,cmdS2,
fbS1,fbS2,sats,hdop,state,headingSource,compass
```

MATLAB uses GPS movement direction and distance trend to tell whether the vehicle is approaching or moving away from the target. Servo feedback is read using `ReadPos()`, distinguishing actual feedback from commanded servo position.

11 Most recent MATLAB screenshot diagnosis

The latest screenshot contained approximately:

```
TARGET
LAT : 32.6323559
LON : 75.0143343

CURRENT GPS
LAT  : 32.6399383
LON  : 75.0137900
ALT  : 516.6 m
SAT  : 8
HDOP : 5.13

DISTANCE       : 878.37 m
TARGET BEARING : 176.45 deg
```

```
GPS COURSE     : 51.13 deg
CONTROLLER HDG : 148.57 deg
HEADING SOURCE : NONE
STEERING       : 0
```

The controller code currently defines a strict GPS maximum HDOP of 3.5 and aborts navigation calculations when `gpsQualityOK()` is false. Since $5.13 > 3.5$, the code marks heading as invalid and the steering function intentionally centers/does not command the servos. This provides a concrete explanation for the observed "motors do not move when GPS is moved" behavior in the latest full code.

The screenshot also reveals a display issue: the controller heading field can still show a stale numeric value while the source field says "NONE". The telemetry line prints the numeric `currentDirection` whether or not `headingValid` is true. A future MATLAB display should show "INVALID" whenever heading source is NONE rather than displaying a stale number.

12 Why the standalone servo test works but the full code did not

The pure ST-3009 test removes GPS quality, state-machine, heading-validity, and navigation conditions. It sends direct commands such as:

```
LEFT   -> ID1 = 1748, ID2 = 2048
RIGHT  -> ID1 = 2048, ID2 = 1748
CENTER -> ID1 = 2048, ID2 = 2048
```

Therefore, when this program works, it proves that the electrical bus, IDs, SCServo library, and basic WritePosEx() transaction can work.

The full controller contains additional gates:

1. Servo bus must be healthy.
2. State must be `STATE_AUTONAV` for autonomous steering.
3. GPS quality must pass the configured satellite, freshness, and HDOP conditions.
4. A valid heading must exist.
5. The course error must produce a nonzero demand outside the deadband.

The latest observed HDOP failure is therefore sufficient to explain a zero steering command even though the servo hardware itself works.

13 Current control law in equations

For valid heading,

$$e = \mathrm{wrap}_{[-180,180]}(B_{target} - H_{current}).$$

If $|e| \le 5^\circ$, then

$$u = 0.$$

Otherwise,

$$|u| = 450 \ \mathrm{sat}\left(\frac{|e| - 5}{90 - 5}, 0, 1\right),$$

with the sign of u equal to the sign of e. The single-servo mapping is then determined by the left/right configuration. With Servo 1 as LEFT,

$$u > 0: \quad S_1 = 2048 + d, \ S_2 = 2048,$$

where d is the positive steering magnitude adjusted by the configured direction sign, and for $u < 0$ the right servo moves from center.

The controller is therefore currently a proportional/lookup-like heading-error law. A future PD law would have the generic form

$$u = K_p e + K_d \dot{e},$$

or, preferably for a parafoil using an IMU, a damped form using yaw rate such as

$$u = K_p e - K_d r.$$

No final PD gains have been established or validated.

14 How glide speed, descent rate, wind, and steering response are to be obtained

These values must be measured rather than guessed.

14.1 Glide speed

GPS speed is ground speed, not necessarily airspeed. Measure representative straight descents and calculate horizontal ground travel over time. Combine with vertical descent to estimate glide ratio. If wind is separately estimated, a better air-relative speed can be inferred.

14.2 Descent rate

Measure altitude loss over time during a stable descent:

$$V_d = -\frac{\Delta h}{\Delta t}.$$

The current BMP581 is available for future altitude diagnostics, while GPS altitude may be logged for an initial experiment.

14.3 Wind

The fundamental relationship is

$$\vec{V}_{ground} = \vec{V}_{air} + \vec{V}_{wind}.$$

With enough straight-flight measurements in multiple directions, wind can be estimated as a vector. A simple out-and-back example can give a first scalar estimate, but a proper vector solution is preferred.

14.4 Steering response

For several controlled servo demands (for example 50, 100, 150, 200, 300, 450 counts), record GPS course and/or gyro yaw rate versus time. Derive a table or fitted model of servo demand versus turn rate and latency. Those measurements should drive the future simulation rather than the placeholder 25 deg/s used in the first simulator.

15 Recommended next engineering sequence

1. Prove the pure servo bus test.
2. In the full controller, prove `PING1/PING2`, positions and manual LEFT/RIGHT commands.
3. In lab mode, relax HDOP only for walking tests so the GPS course can be exercised; restore strict values for flight.
4. Verify that MATLAB shows heading source correctly and does not display stale heading values.
5. Move the system physically while logging GPS course, distance, altitude, servo commands, servo feedback, and gyro Z.

6. Characterize descent rate, glide speed, glide ratio, wind response and steering response.
7. Use the measured response to build a realistic MATLAB model.
8. Tune a PD/damped steering controller.
9. Add predicted landing point / energy-to-go logic instead of "current distance < 35 m" capture.
10. Validate the full flight-state machine and failsafe behavior.

16 Known open questions / do not silently assume

- Whether the Foxeer M10Q internal compass is actually accessible through the same interface has not been resolved in the final software; the implemented navigation code still uses the separate QMC5883 path.
- The exact physical half-duplex interface between Teensy Serial1 pins 0/1 and the ST-3009 DATA bus must remain the known-good hardware arrangement; do not invent a direct TX/RX wiring change without checking the existing working setup.
- The mechanical sign of LEFT/RIGHT steering must be verified with the actual parafoil linkage. Software sign flags alone cannot establish aerodynamic turn direction.
- Glide speed, descent rate, wind, and turn response are not yet measured; simulation values 8 m/s, 4 m/s, and 25 deg/s are placeholders only.
- The current algorithm is not yet a validated precision-landing controller.

17 Compile/environment history

The known compile environment included Teensy 4.1, Teensy core 1.62.0, TinyGPSPlus 1.0.3, SCServo 1.0.2, Adafruit LSM6DS 4.7.4, and the Teensy Servo library. One historical compilation issue was the use of a nonexistent `SCServo` class; the correct library class for the tested configuration is `SMS_STS`. A previous user log also showed a successful Teensy toolchain invocation with GCC/Teensy compile tools and the relevant library include paths.

18 Command reference

Command	Meaning
`STATUS`	Print complete system state, GPS quality, navigation, compass and servo information.
`PING1`, `PING2`	Directly test communication with ST-3009 IDs 1 and 2.
`POS1`, `POS2`	Read actual servo positions.
`CENTER`	Command both servos to 2048 and return to wait state.
`LEFT`	Manual left command; current single-servo design intends to move only the left servo.
`RIGHT`	Manual right command; current single-servo design intends to move only the right servo.
`MOTORTEST`	Automatic center-left-center-right-center sequence.
`AUTONAV`	Enable autonomous steering when GPS and servos pass readiness gates.
`TARGET lat lon`	Update the target coordinate without recompiling.
`DEPLOY`	Manually start the SG90 deployment sequence.
`PREP`	Move SG90 to preparation angle and detach.
`RESET`	Stop active tests/deployment state and return to waiting.
`HELP`	Print command list.

A Source-code archive

The following appendices contain the actual project source files available in the working environment. The archive includes near-duplicate files where those files existed as separate conversation artifacts so that a future AI can reconstruct the iteration history rather than relying only on a summarized diff.

A.1 Archived source 1: Autonomous_Parafoil_Teensy41_Final.ino

Source file: `Autonomous_Parafoil_Teensy41_Final.ino`

```
/*
======================================================================
       TEENSY 4.1 AUTONOMOUS PARAFOIL / TARGETED-LANDING SYSTEM
======================================================================

REFERENCE USED:
  R2Home_rls_V0_9

CURRENT HARDWARE:
  MCU       : Teensy 4.1
  GPS       : Foxeer M10Q 180 GPS + QMC5883 Compass
  Compass   : QMC5883 @ 0x0D
  IMU       : LSM6DSO32 @ 0x6A
  Barometer : BMP581 @ 0x46 / 0x47
  Servos    : 2 x ST-3009 serial bus TTL servos
              Servo 1 = ID 1
              Servo 2 = ID 2
  Deploy    : SG90
  SD        : Teensy 4.1 BUILTIN_SDCARD

UARTS:
  Serial1  : ST-3009 half-duplex servo bus
             TX1 = pin 1, RX1 = pin 0
  Serial3  : Foxeer GPS
             GPS TX -> pin 15 / RX3
             GPS RX -> pin 14 / TX3

I2C:
  SDA = pin 18
  SCL = pin 19

IMPORTANT:
  - System starts DISARMED.
  - Steering stays neutral until the wing is deployed.
  - TARGET,<lat>,<lon> sets the target.
  - ARM starts autonomous flight-state detection.
  - Deployment occurs at DEPLOY_ALTITUDE_M above the launch
    barometric level while descending.
  - The final left/right mechanical steering sign MUST be verified
    with the real parafoil/linkage. That mapping cannot be proven
    from bench servo rotation alone.

======================================================================
*/

#include <Wire.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

#include <TinyGPSPlus.h>
#include <Adafruit_LSM6DSOX.h>
#include <Adafruit_BMP5xx.h>
#include <SCServo.h>
#include <Servo.h>
#include <SD.h>
#include <Watchdog.h>

// ======================================================================
//                           USER CONFIGURATION
```

```
// ====================================================================

// ----------------------------- GPS ---------------------------------
#define GPS_SERIAL Serial3
#define GPS_BAUD 115200

// ----------------------------- I2C ---------------------------------
#define QMC5883_ADDR   0x0D
#define LSM6DSO32_ADDR 0x6A
#define BMP_ADDR_1     0x46
#define BMP_ADDR_2     0x47

// ----------------------------- GPS PINS ----------------------------
// Foxeer TX -> Teensy Pin 15 / RX3
// Foxeer RX -> Teensy Pin 14 / TX3

// ----------------------------- SERVO -------------------------------
#define SERVO1_ID 1
#define SERVO2_ID 2

// ST-3009 position range used by your tested library/code
const int SERVO_NEUTRAL = 2048;
const int SERVO_MIN_POS = 200;
const int SERVO_MAX_POS = 3895;

// Maximum steering displacement from neutral.
// 450 counts is about 39.6 degrees of encoder travel.
const int MAX_STEER_COUNTS = 450;

// Target-bearing error that produces full steering.
const float FULL_STEER_ERROR_DEG = 90.0f;

// Do not react aggressively to tiny GPS/heading errors.
const float HEADING_DEADBAND_DEG = 5.0f;

// GPS course is used when there is enough ground speed.
const float GPS_COURSE_MIN_SPEED_MPS = 1.5f;

// GPS navigation quality limits
const uint32_t GPS_DATA_TIMEOUT_MS = 1500;
const uint8_t  GPS_MIN_SATS = 5;
const float    GPS_MAX_HDOP = 3.5f;

// ------------------------ STEERING MAPPING --------------------------
//
// Positive steer means the target is to the RIGHT of the current
// direction. With the default mapping:
//     positive steer -> Servo 1 lower, Servo 2 higher
//     negative steer -> Servo 1 higher, Servo 2 lower
//
// Change TURN_SIGN to -1 after mechanical testing if the parafoil
// turns in the opposite direction.
//
const int TURN_SIGN = 1;

// Individual servo direction reversal options.
// The servos themselves were verified as:
// lower position -> CCW
// higher position -> CW
//
// These flags are for the actual linkage/mounting geometry.
const bool SERVO1_REVERSE = false;
const bool SERVO2_REVERSE = false;

// Servo command rate
const uint32_t SERVO_UPDATE_MS = 100;

// Servo movement settings from your working ST-3009 tests
const int SERVO_SPEED = 500;
const int SERVO_ACCEL = 50;

// -------------------------- DEPLOYMENT ------------------------------
// R2Home reference used a PWM command corresponding to about 140 deg
// before release and about 70 deg after release.
#define DEPLOY_SERVO_PIN 8
```

```
const int DEPLOY_PRELAUNCH_ANGLE = 140;
const int DEPLOY_RELEASE_ANGLE   = 70;

// Required deployment point above launch level.
const float DEPLOY_ALTITUDE_M = 600.0f;

// Wait for the parafoil to inflate/stabilize before steering.
const uint32_t DEPLOY_STABILIZE_MS = 5000;

// ------------------------- FLIGHT DETECTION -------------------------
const float LAUNCH_ASCENT_VS_MPS = 1.0f;
const uint32_t LAUNCH_CONFIRM_MS = 1000;

const float DESCENT_VS_MPS = -0.5f;
const uint32_t DESCENT_CONFIRM_MS = 1000;

// --------------------------- LANDING ------------------------------
const float LANDING_ALTITUDE_M = 12.0f;
const float LANDING_MAX_ABS_VS_MPS = 0.6f;
const uint32_t LANDING_CONFIRM_MS = 6000;

// Target capture / final approach
const float TARGET_CAPTURE_RADIUS_M = 35.0f;
const float TARGET_RELEASE_RADIUS_M = 60.0f;

// Reduce steering authority near the ground.
const float FINAL_APPROACH_ALT_M = 120.0f;
const float FINAL_STEER_SCALE = 0.70f;

// --------------------------- COMPASS ------------------------------
// Your existing compass code is used exactly for the numeric heading.
// Leave this at 0 unless a fixed installation offset is measured.
const float COMPASS_HEADING_OFFSET_DEG = 0.0f;

// Your existing calibration values
const float OFFSET_X = 180.0000f;
const float OFFSET_Y = 336.0000f;
const float OFFSET_Z = 615.0000f;

const float SCALE_X = 0.956484f;
const float SCALE_Y = 0.904406f;
const float SCALE_Z = 1.178127f;

// -------------------------- BAROMETER -----------------------------
// Relative altitude is computed from the pressure captured at ARM.
const float BARO_EXPONENT = 0.19029496f;

// ------------------------------ SD --------------------------------
const int SD_CS = BUILTIN_SDCARD;
const char *LOG_FILE = "AUTONAV.CSV";

// ========================================================================
//                           OBJECTS / GLOBALS
// ========================================================================

TinyGPSPlus gps;
Adafruit_LSM6DSOX lsm6dso;
Adafruit_BMP5xx bmp;
SMS_STS st;
Servo deployServo;
Watchdog watchdog;

bool compassOK = false;
bool imuOK = false;
bool bmpOK = false;
bool servo1OK = false;
bool servo2OK = false;
bool sdOK = false;

// ------------------------------ Target ------------------------------
double targetLat = 0.0;
double targetLon = 0.0;
bool targetSet = false;

// ------------------------------ GPS ---------------------------------
```

```
double gpsLat = 0.0;
double gpsLon = 0.0;
double gpsAlt = 0.0;
double gpsSpeedMps = 0.0;
double gpsCourseDeg = 0.0;
uint32_t gpsSats = 0;
double gpsHDOP = 99.9;
bool gpsFix = false;
uint32_t lastGPSLocationMs = 0;

// ----------------------------- Compass -----------------------------
int16_t rawX = 0;
int16_t rawY = 0;
int16_t rawZ = 0;
float magX = 0.0f;
float magY = 0.0f;
float magZ = 0.0f;
float compassHeading = 0.0f;

// ------------------------------ IMU ------------------------------
float ax = 0.0f;
float ay = 0.0f;
float az = 0.0f;
float gx = 0.0f;
float gy = 0.0f;
float gz = 0.0f;
float imuTemp = 0.0f;

// ---------------------------- BMP581 ----------------------------
float pressurePa = 0.0f;
float bmpTemp = 0.0f;
float baroAltRaw = 0.0f;
float baroAltFiltered = 0.0f;
float relativeAlt = 0.0f;
float verticalSpeed = 0.0f;
float launchPressurePa = 0.0f;
bool baroFilterInitialized = false;

// -------------------------- Navigation --------------------------
double targetDistanceM = 0.0;
double targetBearingDeg = 0.0;
float currentDirectionDeg = 0.0f;
float courseErrorDeg = 0.0f;
int steeringDemand = 0;
char directionSource[8] = "NONE";

// ---------------------------- Servos ----------------------------
int servo1Command = SERVO_NEUTRAL;
int servo2Command = SERVO_NEUTRAL;
int servo1Actual = -1;
int servo2Actual = -1;
uint32_t lastServoWriteMs = 0;
uint32_t lastServoHealthMs = 0;
uint32_t lastServoReadMs = 0;

// -------------------------- Deployment --------------------------
bool deployed = false;
uint32_t deploymentMs = 0;

// ------------------------ State machine ------------------------
enum FlightState
{
  STATE_DISARMED = 0,
  STATE_ARMED = 1,
  STATE_ASCENT = 2,
  STATE_DESCENT = 3,
  STATE_DEPLOY_STABILIZE = 4,
  STATE_AUTONAV = 5,
  STATE_CAPTURED = 6,
  STATE_LANDED = 7,
  STATE_FAILSAFE = 8
};

FlightState state = STATE_DISARMED;
FlightState previousState = STATE_DISARMED;
```

```
287
288 uint32_t ascentStartMs = 0;
289 uint32_t descentStartMs = 0;
290 uint32_t landingStartMs = 0;
291 uint32_t gpsGoodStartMs = 0;
292
293 // ------------------------------ Timers -----------------------------
294 uint32_t lastSensorMs = 0;
295 uint32_t lastPrintMs = 0;
296 uint32_t lastLogMs = 0;
297 uint32_t lastNavMs = 0;
298
299 // ========================================================================
300 //                            HELPER FUNCTIONS
301 // ========================================================================
302
303 float wrap360(float a)
304 {
305   while (a < 0.0f) a += 360.0f;
306   while (a >= 360.0f) a -= 360.0f;
307   return a;
308 }
309
310 float wrap180(float a)
311 {
312   while (a > 180.0f) a -= 360.0f;
313   while (a <= -180.0f) a += 360.0f;
314   return a;
315 }
316
317 int clampServoPosition(int p)
318 {
319   if (p < SERVO_MIN_POS) return SERVO_MIN_POS;
320   if (p > SERVO_MAX_POS) return SERVO_MAX_POS;
321   return p;
322 }
323
324 const char* stateName(FlightState s)
325 {
326   switch (s)
327   {
328     case STATE_DISARMED:         return "DISARMED";
329     case STATE_ARMED:            return "ARMED";
330     case STATE_ASCENT:           return "ASCENT";
331     case STATE_DESCENT:          return "DESCENT";
332     case STATE_DEPLOY_STABILIZE: return "DEPLOY_STAB";
333     case STATE_AUTONAV:          return "AUTONAV";
334     case STATE_CAPTURED:         return "CAPTURED";
335     case STATE_LANDED:           return "LANDED";
336     case STATE_FAILSAFE:         return "FAILSAFE";
337     default:                     return "UNKNOWN";
338   }
339 }
340
341 bool gpsQualityOK()
342 {
343   if (!gpsFix) return false;
344   if ((millis() - lastGPSLocationMs) > GPS_DATA_TIMEOUT_MS) return false;
345   if (gpsSats < GPS_MIN_SATS) return false;
346   if (gps.hdop.isValid() && gpsHDOP > GPS_MAX_HDOP) return false;
347   return true;
348 }
349
350 // ========================================================================
351 //                          QMC5883 COMPASS
352 //                 EXACT WORKING IMPLEMENTATION STYLE
353 // ========================================================================
354
355 void writeRegister(byte reg, byte value)
356 {
357   Wire.beginTransmission(QMC5883_ADDR);
358   Wire.write(reg);
359   Wire.write(value);
360   Wire.endTransmission();
361 }
```

```
362
363 bool readQMC5883(int16_t &x, int16_t &y, int16_t &z)
364 {
365   Wire.beginTransmission(QMC5883_ADDR);
366   Wire.write(0x00);
367
368   if (Wire.endTransmission(false) != 0)
369     return false;
370
371   Wire.requestFrom(QMC5883_ADDR, 6);
372
373   if (Wire.available() < 6)
374     return false;
375
376   uint8_t xl = Wire.read();
377   uint8_t xh = Wire.read();
378   uint8_t yl = Wire.read();
379   uint8_t yh = Wire.read();
380   uint8_t zl = Wire.read();
381   uint8_t zh = Wire.read();
382
383   x = (int16_t)((xh << 8) | xl);
384   y = (int16_t)((yh << 8) | yl);
385   z = (int16_t)((zh << 8) | zl);
386
387   return true;
388 }
389
390 bool initializeQMC5883()
391 {
392   Wire.beginTransmission(QMC5883_ADDR);
393
394   if (Wire.endTransmission() != 0)
395     return false;
396
397   // Same initialization as your verified compass code.
398   writeRegister(0x0B, 0x01);
399   delay(10);
400   writeRegister(0x09, 0x1D);
401   delay(100);
402
403   return true;
404 }
405
406 bool updateCompass()
407 {
408   if (!readQMC5883(rawX, rawY, rawZ))
409     return false;
410
411   magX = (rawX - OFFSET_X) * SCALE_X;
412   magY = (rawY - OFFSET_Y) * SCALE_Y;
413   magZ = (rawZ - OFFSET_Z) * SCALE_Z;
414
415   float angle = atan2(magY, magX) * 180.0f / PI;
416
417   if (angle < 0.0f)
418     angle += 360.0f;
419
420   compassHeading = wrap360(angle + COMPASS_HEADING_OFFSET_DEG);
421
422   return true;
423 }
424
425 // Same direction labels as your verified compass test. The numeric
426 // heading used by navigation is unaffected by these display labels.
427 const char* directionName(float h)
428 {
429   if (h >= 337.5f || h < 22.5f) return "NORTH";
430   if (h < 67.5f) return "NORTH-EAST";
431   if (h < 112.5f) return "WEST";
432   if (h < 157.5f) return "SOUTH-EAST";
433   if (h < 202.5f) return "SOUTH";
434   if (h < 247.5f) return "SOUTH-WEST";
435   if (h < 292.5f) return "EAST";
436   return "NORTH-WEST";
```

```
}

// ========================================================================
//                               IMU
// ========================================================================

bool initializeIMU()
{
  if (!lsm6dso.begin_I2C(LSM6DSO32_ADDR, &Wire))
    return false;

  // Same ranges/data rates as your known working sensor code.
  lsm6dso.setAccelRange(LSM6DS_ACCEL_RANGE_16_G);
  lsm6dso.setGyroRange(LSM6DS_GYRO_RANGE_2000_DPS);
  lsm6dso.setAccelDataRate(LSM6DS_RATE_208_HZ);
  lsm6dso.setGyroDataRate(LSM6DS_RATE_208_HZ);

  return true;
}

void updateIMU()
{
  if (!imuOK) return;

  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;

  lsm6dso.getEvent(&accel, &gyro, &temp);

  ax = accel.acceleration.x / 9.80665f;
  ay = accel.acceleration.y / 9.80665f;
  az = accel.acceleration.z / 9.80665f;

  gx = gyro.gyro.x * 57.2957795f;
  gy = gyro.gyro.y * 57.2957795f;
  gz = gyro.gyro.z * 57.2957795f;

  imuTemp = temp.temperature;
}

// ========================================================================
//                             BMP581
// ========================================================================

bool initializeBMP()
{
  if (bmp.begin(BMP_ADDR_1, &Wire))
    return true;

  if (bmp.begin(BMP_ADDR_2, &Wire))
    return true;

  return false;
}

float pressureToRelativeAltitude(float p)
{
  if (!isfinite(p) || p <= 0.0f || launchPressurePa <= 0.0f)
    return 0.0f;

  return 44330.0f *
         (1.0f - powf(p / launchPressurePa, BARO_EXPONENT));
}

void updateBMP()
{
  if (!bmpOK) return;

  float p = bmp.readPressure();
  float t = bmp.readTemperature();

  if (!isfinite(p) || !isfinite(t) || p <= 0.0f)
    return;

```

```
  pressurePa = p;
  bmpTemp = t;
  baroAltRaw = pressureToRelativeAltitude(p);

  // Slow filter to reduce pressure noise while retaining useful
  // vertical-speed information.
  if (!baroFilterInitialized)
  {
    baroAltFiltered = baroAltRaw;
    baroFilterInitialized = true;
  }
  else
  {
    const float ALT_ALPHA = 0.25f;
    baroAltFiltered += ALT_ALPHA * (baroAltRaw - baroAltFiltered);
  }

  relativeAlt = baroAltFiltered;
}

// ========================================================================
//                               GPS
// ========================================================================

void updateGPS()
{
  while (GPS_SERIAL.available())
    gps.encode(GPS_SERIAL.read());

  // Update the timestamp only when a NEW position sentence has been
  // decoded. This makes the GPS timeout a real communication timeout.
  if (gps.location.isUpdated())
  {
    gpsLat = gps.location.lat();
    gpsLon = gps.location.lng();
    lastGPSLocationMs = millis();
  }

  gpsFix = gps.location.isValid();

  if (gps.altitude.isValid())
    gpsAlt = gps.altitude.meters();

  if (gps.speed.isValid())
    gpsSpeedMps = gps.speed.mps();

  if (gps.course.isValid())
    gpsCourseDeg = gps.course.deg();

  if (gps.satellites.isValid())
    gpsSats = gps.satellites.value();

  if (gps.hdop.isValid())
    gpsHDOP = gps.hdop.hdop();
}

// ========================================================================
//                        NAVIGATION CALCULATION
// ========================================================================

void updateNavigation()
{
  if (!targetSet || !gpsFix)
  {
    targetDistanceM = 0.0;
    targetBearingDeg = 0.0;
    courseErrorDeg = 0.0f;
    steeringDemand = 0;
    strcpy(directionSource, "NONE");
    return;
  }

  targetDistanceM = TinyGPSPlus::distanceBetween(
    gpsLat,
    gpsLon,
```

```
    targetLat,
    targetLon
  );

  targetBearingDeg = TinyGPSPlus::courseTo(
    gpsLat,
    gpsLon,
    targetLat,
    targetLon
  );

  bool gpsCourseUsable =
    gpsQualityOK() &&
    gps.course.isValid() &&
    gpsSpeedMps >= GPS_COURSE_MIN_SPEED_MPS;

  if (gpsCourseUsable)
  {
    currentDirectionDeg = wrap360((float)gpsCourseDeg);
    strcpy(directionSource, "GPS");
  }
  else if (compassOK)
  {
    currentDirectionDeg = compassHeading;
    strcpy(directionSource, "COMPASS");
  }
  else
  {
    steeringDemand = 0;
    courseErrorDeg = 0.0f;
    strcpy(directionSource, "NONE");
    return;
  }

  courseErrorDeg = wrap180(
    (float)targetBearingDeg - currentDirectionDeg
  );

  float absError = fabsf(courseErrorDeg);

  if (absError <= HEADING_DEADBAND_DEG)
  {
    steeringDemand = 0;
    return;
  }

  float normalized = absError / FULL_STEER_ERROR_DEG;
  if (normalized > 1.0f) normalized = 1.0f;

  float demand = normalized * MAX_STEER_COUNTS;

  // Positive error = target to the right.
  if (courseErrorDeg > 0.0f)
    demand = fabsf(demand);
  else
    demand = -fabsf(demand);

  demand *= TURN_SIGN;

  // Reduce authority during final low-altitude approach.
  if (relativeAlt < FINAL_APPROACH_ALT_M)
    demand *= FINAL_STEER_SCALE;

  steeringDemand = (int)roundf(demand);

  // Capture zone: stop aggressive steering inside the target radius.
  if (targetDistanceM <= TARGET_CAPTURE_RADIUS_M)
  {
    steeringDemand = 0;
  }
}

// ========================================================================
//                          SERVO CONTROL
// ========================================================================
```

```
662
663 int applyReverse(int position, bool reverse)
664 {
665   if (!reverse)
666     return position;
667
668   return SERVO_NEUTRAL - (position - SERVO_NEUTRAL);
669 }
670
671 void setSteeringNeutral(bool immediate = false)
672 {
673   servo1Command = SERVO_NEUTRAL;
674   servo2Command = SERVO_NEUTRAL;
675
676   if (immediate)
677   {
678     st.WritePosEx(SERVO1_ID, servo1Command, SERVO_SPEED, SERVO_ACCEL);
679     st.WritePosEx(SERVO2_ID, servo2Command, SERVO_SPEED, SERVO_ACCEL);
680   }
681 }
682
683 void setSteeringFromDemand(int demand)
684 {
685   demand = constrain(demand, -MAX_STEER_COUNTS, MAX_STEER_COUNTS);
686
687   // Default differential-brake mapping:
688   // demand > 0: Servo 1 lower, Servo 2 higher
689   int s1 = SERVO_NEUTRAL - demand;
690   int s2 = SERVO_NEUTRAL + demand;
691
692   s1 = applyReverse(s1, SERVO1_REVERSE);
693   s2 = applyReverse(s2, SERVO2_REVERSE);
694
695   servo1Command = clampServoPosition(s1);
696   servo2Command = clampServoPosition(s2);
697 }
698
699 void updateServos()
700 {
701   if (millis() - lastServoWriteMs < SERVO_UPDATE_MS)
702     return;
703
704   lastServoWriteMs = millis();
705
706   if (!deployed || state != STATE_AUTONAV)
707   {
708     servo1Command = SERVO_NEUTRAL;
709     servo2Command = SERVO_NEUTRAL;
710   }
711
712   st.WritePosEx(
713     SERVO1_ID,
714     servo1Command,
715     SERVO_SPEED,
716     SERVO_ACCEL
717   );
718
719   st.WritePosEx(
720     SERVO2_ID,
721     servo2Command,
722     SERVO_SPEED,
723     SERVO_ACCEL
724   );
725 }
726
727 void checkServoHealth()
728 {
729   if (millis() - lastServoHealthMs >= 2000)
730   {
731     lastServoHealthMs = millis();
732
733     servo1OK = (st.Ping(SERVO1_ID) != -1);
734     servo2OK = (st.Ping(SERVO2_ID) != -1);
735   }
736
```

```
  if (millis() - lastServoReadMs >= 500)
  {
    lastServoReadMs = millis();

    servo1Actual = st.ReadPos(SERVO1_ID);
    servo2Actual = st.ReadPos(SERVO2_ID);
  }
}

bool servoBusOK()
{
  return servo1OK && servo2OK;
}

// ========================================================================
//                            DEPLOYMENT
// ========================================================================

void deployWing()
{
  if (deployed)
    return;

  deployed = true;
  deploymentMs = millis();

  // R2Home reference behavior:
  // pre-deployment approximately 140 deg, then return to approximately
  // 70 deg after release.
  deployServo.write(DEPLOY_RELEASE_ANGLE);

  setSteeringNeutral(true);

  Serial.println("*** WING DEPLOYED ***");
}

// ========================================================================
//                        LAUNCH / LANDING LOGIC
// ========================================================================

void updateLaunchAndDescentDetection()
{
  if (state == STATE_ARMED)
  {
    if (verticalSpeed >= LAUNCH_ASCENT_VS_MPS)
    {
      if (ascentStartMs == 0)
        ascentStartMs = millis();

      if (millis() - ascentStartMs >= LAUNCH_CONFIRM_MS)
      {
        state = STATE_ASCENT;
        ascentStartMs = 0;
      }
    }
    else
    {
      ascentStartMs = 0;
    }
  }

  if (state == STATE_ASCENT)
  {
    if (verticalSpeed <= DESCENT_VS_MPS)
    {
      if (descentStartMs == 0)
        descentStartMs = millis();

      if (millis() - descentStartMs >= DESCENT_CONFIRM_MS)
      {
        state = STATE_DESCENT;
        descentStartMs = 0;
      }
    }
    else
```

```
    {
      descentStartMs = 0;
    }
  }

  if (state == STATE_DESCENT)
  {
    if (relativeAlt <= DEPLOY_ALTITUDE_M && verticalSpeed < 0.0f)
    {
      deployWing();
      state = STATE_DEPLOY_STABILIZE;
    }
  }
}

void updateLandingDetection()
{
  if (!deployed)
    return;

  if (relativeAlt <= LANDING_ALTITUDE_M &&
      fabsf(verticalSpeed) <= LANDING_MAX_ABS_VS_MPS)
  {
    if (landingStartMs == 0)
      landingStartMs = millis();

    if (millis() - landingStartMs >= LANDING_CONFIRM_MS)
    {
      state = STATE_LANDED;
      setSteeringNeutral(true);
    }
  }
  else
  {
    landingStartMs = 0;
  }
}

// ============================================================================
//                          AUTONOMOUS STATE MACHINE
// ============================================================================

void handleStateMachine()
{
  switch (state)
  {
    case STATE_DISARMED:
      setSteeringNeutral(false);
      break;

    case STATE_ARMED:
      setSteeringNeutral(false);
      updateLaunchAndDescentDetection();
      break;

    case STATE_ASCENT:
      setSteeringNeutral(false);
      updateLaunchAndDescentDetection();
      break;

    case STATE_DESCENT:
      setSteeringNeutral(false);
      updateLaunchAndDescentDetection();
      break;

    case STATE_DEPLOY_STABILIZE:
      setSteeringNeutral(false);

      if (millis() - deploymentMs >= DEPLOY_STABILIZE_MS)
      {
        if (gpsQualityOK() && targetSet && servoBusOK())
          state = STATE_AUTONAV;
        else
          state = STATE_FAILSAFE;
      }
```

```
      break;

    case STATE_AUTONAV:
      if (!targetSet || !gpsQualityOK() || !servoBusOK())
      {
        state = STATE_FAILSAFE;
        setSteeringNeutral(false);
        break;
      }

      if (targetDistanceM <= TARGET_CAPTURE_RADIUS_M)
      {
        state = STATE_CAPTURED;
        setSteeringNeutral(false);
        break;
      }

      if (targetDistanceM > TARGET_RELEASE_RADIUS_M)
      {
        state = STATE_AUTONAV;
      }
      break;

    case STATE_CAPTURED:
      setSteeringNeutral(false);

      // Allow re-entry if the vehicle drifts back out of the capture
      // radius before landing.
      if (targetDistanceM > TARGET_RELEASE_RADIUS_M)
      {
        state = STATE_AUTONAV;
      }
      break;

    case STATE_LANDED:
      setSteeringNeutral(false);
      break;

    case STATE_FAILSAFE:
      setSteeringNeutral(false);

      // Recover automatically after a stable GPS/servo recovery.
      if (deployed && targetSet && gpsQualityOK() && servoBusOK())
      {
        if (gpsGoodStartMs == 0)
          gpsGoodStartMs = millis();

        if (millis() - gpsGoodStartMs >= 3000)
        {
          state = STATE_AUTONAV;
          gpsGoodStartMs = 0;
        }
      }
      else
      {
        gpsGoodStartMs = 0;
      }
      break;
  }

  // Navigation commands are generated only in AUTONAV.
  if (state == STATE_AUTONAV)
  {
    if (millis() - lastNavMs >= 100)
    {
      lastNavMs = millis();
      updateNavigation();
      setSteeringFromDemand(steeringDemand);
    }
  }
  else
  {
    steeringDemand = 0;
  }

```

```
  updateLandingDetection();

  // Immediate neutral outside autonomous navigation.
  if (state != STATE_AUTONAV)
  {
    servo1Command = SERVO_NEUTRAL;
    servo2Command = SERVO_NEUTRAL;
  }
}

// ========================================================================
//                          ARM / DISARM
// ========================================================================

bool armSystem()
{
  if (!targetSet)
  {
    Serial.println("ARM rejected: TARGET not set.");
    return false;
  }

  if (!gpsQualityOK())
  {
    Serial.println("ARM rejected: GPS quality is not good.");
    return false;
  }

  if (!bmpOK)
  {
    Serial.println("ARM rejected: BMP581 unavailable.");
    return false;
  }

  if (!servoBusOK())
  {
    Serial.println("ARM rejected: ST-3009 servos not detected.");
    return false;
  }

  float p = bmp.readPressure();
  if (!isfinite(p) || p <= 0.0f)
  {
    Serial.println("ARM rejected: invalid launch pressure.");
    return false;
  }

  launchPressurePa = p;
  baroAltFiltered = 0.0f;
  baroFilterInitialized = false;
  relativeAlt = 0.0f;
  verticalSpeed = 0.0f;

  deployed = false;
  deploymentMs = 0;
  ascentStartMs = 0;
  descentStartMs = 0;
  landingStartMs = 0;
  gpsGoodStartMs = 0;

  deployServo.write(DEPLOY_PRELAUNCH_ANGLE);
  setSteeringNeutral(true);

  state = STATE_ARMED;

  Serial.println("SYSTEM ARMED.");
  Serial.println("Waiting for ascent...");

  return true;
}

void disarmSystem()
{
  state = STATE_DISARMED;
  deployed = false;
```

```
  steeringDemand = 0;
  setSteeringNeutral(true);
  deployServo.write(DEPLOY_PRELAUNCH_ANGLE);

  Serial.println("SYSTEM DISARMED.");
}

// ========================================================================
//                          SERIAL COMMANDS
// ========================================================================

void printHelp()
{
  Serial.println();
  Serial.println("COMMANDS:");
  Serial.println("  TARGET,<latitude>,<longitude>");
  Serial.println("  ARM");
  Serial.println("  DISARM");
  Serial.println("  STATUS");
  Serial.println("  ZERO     (reset launch barometer while disarmed)");
  Serial.println("  NEUTRAL  (center steering servos)");
  Serial.println("  HELP");
  Serial.println();
}

void printStatus()
{
  Serial.println();
  Serial.println("================ STATUS ================");
  Serial.print("State             : "); Serial.println(stateName(state));
  Serial.print("Target set        : "); Serial.println(targetSet ? "YES" : "NO");

  if (targetSet)
  {
    Serial.print("Target latitude   : "); Serial.println(targetLat, 7);
    Serial.print("Target longitude  : "); Serial.println(targetLon, 7);
  }

  Serial.print("GPS fix           : "); Serial.println(gpsFix ? "YES" : "NO");
  Serial.print("GPS sats          : "); Serial.println(gpsSats);
  Serial.print("GPS HDOP          : "); Serial.println(gpsHDOP, 2);
  Serial.print("GPS speed         : "); Serial.println(gpsSpeedMps, 2);
  Serial.print("GPS course        : "); Serial.println(gpsCourseDeg, 1);

  Serial.print("Compass heading   : "); Serial.print(compassHeading, 1);
  Serial.print(" deg ("); Serial.print(directionName(compassHeading)); Serial.println(")");
  Serial.print("Baro rel altitude : "); Serial.println(relativeAlt, 2);
  Serial.print("Vertical speed    : "); Serial.println(verticalSpeed, 2);

  Serial.print("Distance to target: "); Serial.println(targetDistanceM, 1);
  Serial.print("Target bearing    : "); Serial.println(targetBearingDeg, 1);
  Serial.print("Current direction : "); Serial.println(currentDirectionDeg, 1);
  Serial.print("Direction source  : "); Serial.println(directionSource);
  Serial.print("Course error      : "); Serial.println(courseErrorDeg, 1);
  Serial.print("Steering demand   : "); Serial.println(steeringDemand);

  Serial.print("Servo 1 command   : "); Serial.println(servo1Command);
  Serial.print("Servo 2 command   : "); Serial.println(servo2Command);
  Serial.print("Servo 1 actual    : "); Serial.println(servo1Actual);
  Serial.print("Servo 2 actual    : "); Serial.println(servo2Actual);

  Serial.print("Deployed          : "); Serial.println(deployed ? "YES" : "NO");
  Serial.print("Servo 1 bus       : "); Serial.println(servo1OK ? "OK" : "FAIL");
  Serial.print("Servo 2 bus       : "); Serial.println(servo2OK ? "OK" : "FAIL");
  Serial.println("========================================");
}

void processCommand(char *cmd)
{
  // Convert command to uppercase only for the command word.
  // TARGET numeric values are left untouched because they are parsed
  // from the remainder of the string.

  if (strncmp(cmd, "TARGET,", 7) == 0)
  {
```

```
    char *p = cmd + 7;
    char *comma = strchr(p, ',');

    if (comma == NULL)
    {
      Serial.println("Invalid TARGET format.");
      Serial.println("Use: TARGET,latitude,longitude");
      return;
    }

    *comma = '\0';

    double newLat = atof(p);
    double newLon = atof(comma + 1);

    if (newLat < -90.0 || newLat > 90.0 ||
        newLon < -180.0 || newLon > 180.0)
    {
      Serial.println("Invalid latitude/longitude range.");
      return;
    }

    targetLat = newLat;
    targetLon = newLon;
    targetSet = true;

    Serial.print("TARGET SET: ");
    Serial.print(targetLat, 7);
    Serial.print(", ");
    Serial.println(targetLon, 7);
    return;
  }

  if (strcmp(cmd, "ARM") == 0)
  {
    armSystem();
    return;
  }

  if (strcmp(cmd, "DISARM") == 0)
  {
    disarmSystem();
    return;
  }

  if (strcmp(cmd, "STATUS") == 0)
  {
    printStatus();
    return;
  }

  if (strcmp(cmd, "ZERO") == 0)
  {
    if (state != STATE_DISARMED)
    {
      Serial.println("ZERO allowed only when DISARMED.");
      return;
    }

    if (bmpOK)
    {
      float p = bmp.readPressure();
      if (isfinite(p) && p > 0.0f)
      {
        launchPressurePa = p;
        relativeAlt = 0.0f;
        baroAltFiltered = 0.0f;
        Serial.println("Barometer zeroed.");
      }
    }
    return;
  }

  if (strcmp(cmd, "NEUTRAL") == 0)
  {
```

```
    setSteeringNeutral(true);
    Serial.println("Steering servos commanded to 2048.");
    return;
  }

  if (strcmp(cmd, "HELP") == 0)
  {
    printHelp();
    return;
  }

  Serial.print("Unknown command: ");
  Serial.println(cmd);
}

void processSerialCommands()
{
  static char line[100];
  static uint8_t index = 0;

  while (Serial.available())
  {
    char c = Serial.read();

    if (c == '\r')
      continue;

    if (c == '\n')
    {
      line[index] = '\0';

      if (index > 0)
      {
        processCommand(line);
      }

      index = 0;
    }
    else
    {
      if (index < sizeof(line) - 1)
      {
        line[index++] = c;
      }
    }
  }
}

// ============================================================================
//                              SD LOGGING
// ============================================================================

void initSD()
{
  if (!SD.begin(SD_CS))
  {
    sdOK = false;
    return;
  }

  sdOK = true;

  if (!SD.exists(LOG_FILE))
  {
    File f = SD.open(LOG_FILE, FILE_WRITE);

    if (f)
    {
      f.println(
        "ms,state,targetLat,targetLon,lat,lon,gpsAlt,gpsSpeed,gpsCourse,sats,hdop,"
        "compass,baroAlt,verticalSpeed,targetDistance,targetBearing,currentDirection,"
        "directionSource,courseError,steerDemand,servo1Cmd,servo2Cmd,servo1Actual,"
        "servo2Actual,deployed,ax,ay,az,gx,gy,gz,bmpTemp,pressure"
      );
      f.close();
```

```
    }
  }
}

void logData()
{
  if (!sdOK)
    return;

  if (millis() - lastLogMs < 500)
    return;

  lastLogMs = millis();

  File f = SD.open(LOG_FILE, FILE_WRITE);

  if (!f)
    return;

  f.print(millis()); f.print(',');
  f.print(stateName(state)); f.print(',');
  f.print(targetLat, 7); f.print(',');
  f.print(targetLon, 7); f.print(',');
  f.print(gpsLat, 7); f.print(',');
  f.print(gpsLon, 7); f.print(',');
  f.print(gpsAlt, 2); f.print(',');
  f.print(gpsSpeedMps, 2); f.print(',');
  f.print(gpsCourseDeg, 2); f.print(',');
  f.print(gpsSats); f.print(',');
  f.print(gpsHDOP, 2); f.print(',');
  f.print(compassHeading, 2); f.print(',');
  f.print(relativeAlt, 2); f.print(',');
  f.print(verticalSpeed, 2); f.print(',');
  f.print(targetDistanceM, 2); f.print(',');
  f.print(targetBearingDeg, 2); f.print(',');
  f.print(currentDirectionDeg, 2); f.print(',');
  f.print(directionSource); f.print(',');
  f.print(courseErrorDeg, 2); f.print(',');
  f.print(steeringDemand); f.print(',');
  f.print(servo1Command); f.print(',');
  f.print(servo2Command); f.print(',');
  f.print(servo1Actual); f.print(',');
  f.print(servo2Actual); f.print(',');
  f.print(deployed ? 1 : 0); f.print(',');
  f.print(ax, 3); f.print(',');
  f.print(ay, 3); f.print(',');
  f.print(az, 3); f.print(',');
  f.print(gx, 2); f.print(',');
  f.print(gy, 2); f.print(',');
  f.print(gz, 2); f.print(',');
  f.print(bmpTemp, 2); f.print(',');
  f.println(pressurePa, 1);

  f.close();
}

// ============================================================================
//                          SERIAL MONITOR
// ============================================================================

void printLiveData()
{
  if (millis() - lastPrintMs < 500)
    return;

  lastPrintMs = millis();

  Serial.println();
  Serial.println("------------------------------------------------------------");
  Serial.print("STATE = "); Serial.println(stateName(state));

  Serial.print("GPS: ");
  if (gpsFix)
  {
    Serial.print(gpsLat, 7);
```

```
    Serial.print(", ");
    Serial.print(gpsLon, 7);
    Serial.print("  Sats=");
    Serial.print(gpsSats);
    Serial.print(" HDOP=");
    Serial.print(gpsHDOP, 1);
    Serial.print(" Speed=");
    Serial.print(gpsSpeedMps, 2);
    Serial.print("m/s Course=");
    Serial.print(gpsCourseDeg, 1);
    Serial.println("deg");
  }
  else
  {
    Serial.println("NO FIX");
  }

  Serial.print("Compass: ");
  if (compassOK)
  {
    Serial.print(compassHeading, 1);
    Serial.println(" deg");
  }
  else
  {
    Serial.println("FAIL");
  }

  Serial.print("Baro: Alt=");
  Serial.print(relativeAlt, 2);
  Serial.print("m VS=");
  Serial.print(verticalSpeed, 2);
  Serial.println("m/s");

  if (targetSet && gpsFix)
  {
    Serial.print("NAV: Bearing=");
    Serial.print(targetBearingDeg, 1);
    Serial.print("deg Distance=");
    Serial.print(targetDistanceM, 1);
    Serial.print("m Current=");
    Serial.print(currentDirectionDeg, 1);
    Serial.print("deg Error=");
    Serial.print(courseErrorDeg, 1);
    Serial.print("deg Source=");
    Serial.println(directionSource);
  }
  else
  {
    Serial.println("NAV: target/GPS not ready");
  }

  Serial.print("Steering: ");
  Serial.print(servo1Command);
  Serial.print(" / ");
  Serial.print(servo2Command);
  Serial.print("  actual=");
  Serial.print(servo1Actual);
  Serial.print(" / ");
  Serial.println(servo2Actual);

  Serial.print("Deploy=");
  Serial.print(deployed ? "YES" : "NO");
  Serial.print("  ServoBus=");
  Serial.println(servoBusOK() ? "OK" : "FAIL");

  Serial.println("------------------------------------------------------------");
}

// ======================================================================
//                               SETUP
// ======================================================================

void setup()
{
```

```
  Serial.begin(115200);
  Serial.setTimeout(20);

  delay(1500);

  Serial.println();
  Serial.println("============================================================");
  Serial.println("      TEENSY 4.1 AUTONOMOUS PARAFOIL CONTROLLER");
  Serial.println("============================================================");

  // ------------------------------ I2C ------------------------------
  Wire.begin();
  Wire.setClock(400000);

  Serial.println("I2C: SDA 18 / SCL 19");

  // ---------------------------- Compass ----------------------------
  compassOK = initializeQMC5883();
  Serial.print("QMC5883: ");
  Serial.println(compassOK ? "OK @ 0x0D" : "FAIL");

  // ------------------------------ IMU ------------------------------
  imuOK = initializeIMU();
  Serial.print("LSM6DSO32: ");
  Serial.println(imuOK ? "OK @ 0x6A" : "FAIL");

  // ---------------------------- BMP581 ----------------------------
  bmpOK = initializeBMP();
  Serial.print("BMP581: ");
  Serial.println(bmpOK ? "OK" : "FAIL");

  // ------------------------------ GPS ------------------------------
  GPS_SERIAL.begin(GPS_BAUD);
  Serial.println("GPS: Serial3 @ 115200");
  Serial.println("     GPS TX -> pin 15 / RX3");
  Serial.println("     GPS RX -> pin 14 / TX3");

  // ------------------------------ Servo ------------------------------
  Serial1.begin(1000000, SERIAL_8N1_HALF_DUPLEX);
  st.pSerial = &Serial1;

  delay(1000);

  servo1OK = (st.Ping(SERVO1_ID) != -1);
  servo2OK = (st.Ping(SERVO2_ID) != -1);

  Serial.print("ST-3009 ID1: ");
  Serial.println(servo1OK ? "FOUND" : "NOT FOUND");

  Serial.print("ST-3009 ID2: ");
  Serial.println(servo2OK ? "FOUND" : "NOT FOUND");

  setSteeringNeutral(true);

  // ------------------------- Deploy servo -------------------------
  deployServo.attach(DEPLOY_SERVO_PIN);
  deployServo.write(DEPLOY_PRELAUNCH_ANGLE);
  Serial.print("SG90 deploy pin: ");
  Serial.println(DEPLOY_SERVO_PIN);

  // ------------------------------ SD ------------------------------
  initSD();
  Serial.print("SD logging: ");
  Serial.println(sdOK ? "OK" : "FAIL");

  // --------------------------- Watchdog ---------------------------
  watchdog.enable(Watchdog::TIMEOUT_2S);

  // ------------------------------ Info ------------------------------
  Serial.println();
  Serial.println("SYSTEM STARTED DISARMED");
  Serial.println();
  printHelp();
}

```

```
// ====================================================================
//                                LOOP
// ====================================================================

void loop()
{
  // Keep serial parsing responsive.
  processSerialCommands();

  // GPS must be serviced continuously.
  updateGPS();

  // Sensors at approximately 100 Hz.
  if (millis() - lastSensorMs >= 10)
  {
    lastSensorMs = millis();

    compassOK = updateCompass();
    updateIMU();
    updateBMP();
  }

  // Navigation/flight-state logic.
  updateLaunchAndDescentDetection();
  handleStateMachine();

  // Servo bus health / encoder positions.
  checkServoHealth();

  // In an active flight, losing both servos immediately forces neutral.
  if (deployed && state == STATE_AUTONAV && !servoBusOK())
  {
    state = STATE_FAILSAFE;
    setSteeringNeutral(true);
    Serial.println("FAILSAFE: SERVO BUS FAULT");
  }

  updateServos();

  logData();
  printLiveData();

  watchdog.reset();
}
```

A.2 Archived source 2: Autonomous_Parafoil_Teensy41_FINAL_WDT_T4.ino

Source file: `Autonomous_Parafoil_Teensy41_FINAL_WDT_T4.ino`

```
/*
======================================================================
       TEENSY 4.1 AUTONOMOUS PARAFOIL / TARGETED-LANDING SYSTEM
======================================================================

REFERENCE USED:
  R2Home_rls_V0_9

CURRENT HARDWARE:
  MCU       : Teensy 4.1
  GPS       : Foxeer M10Q 180 GPS + QMC5883 Compass
  Compass   : QMC5883 @ 0x0D
  IMU       : LSM6DSO32 @ 0x6A
  Barometer : BMP581 @ 0x46 / 0x47
  Servos    : 2 x ST-3009 serial bus TTL servos
              Servo 1 = ID 1
              Servo 2 = ID 2
  Deploy    : SG90
  SD        : Teensy 4.1 BUILTIN_SDCARD

UARTS:
  Serial1  : ST-3009 half-duplex servo bus
             TX1 = pin 1, RX1 = pin 0
  Serial3  : Foxeer GPS
```

```
             GPS TX -> pin 15 / RX3
             GPS RX -> pin 14 / TX3

I2C:
  SDA = pin 18
  SCL = pin 19

IMPORTANT:
  - System starts DISARMED.
  - Steering stays neutral until the wing is deployed.
  - TARGET,<lat>,<lon> sets the target.
  - ARM starts autonomous flight-state detection.
  - Deployment occurs at DEPLOY_ALTITUDE_M above the launch
    barometric level while descending.
  - The final left/right mechanical steering sign MUST be verified
    with the real parafoil/linkage. That mapping cannot be proven
    from bench servo rotation alone.

======================================================================
*/

#include <Wire.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

#include <TinyGPSPlus.h>
#include <Adafruit_LSM6DSOX.h>
#include <Adafruit_BMP5xx.h>
#include <SCServo.h>
#include <Servo.h>
#include <SD.h>
// Optional Teensy 4.x hardware watchdog.
// Install "Watchdog_t4" from Arduino Library Manager.
#if __has_include(<Watchdog_t4.h>)
  #include <Watchdog_t4.h>
  #define HAVE_WATCHDOG_T4 1
  WDT_T4<WDT1> watchdog;
#else
  #define HAVE_WATCHDOG_T4 0
#endif

// ======================================================================
//                            USER CONFIGURATION
// ======================================================================

// ------------------------------ GPS ---------------------------------
#define GPS_SERIAL Serial3
#define GPS_BAUD 115200

// ------------------------------ I2C ---------------------------------
#define QMC5883_ADDR   0x0D
#define LSM6DSO32_ADDR 0x6A
#define BMP_ADDR_1     0x46
#define BMP_ADDR_2     0x47

// ------------------------------ GPS PINS ----------------------------
// Foxeer TX -> Teensy Pin 15 / RX3
// Foxeer RX -> Teensy Pin 14 / TX3

// ------------------------------ SERVO -------------------------------
#define SERVO1_ID 1
#define SERVO2_ID 2

// ST-3009 position range used by your tested library/code
const int SERVO_NEUTRAL = 2048;
const int SERVO_MIN_POS = 200;
const int SERVO_MAX_POS = 3895;

// Maximum steering displacement from neutral.
// 450 counts is about 39.6 degrees of encoder travel.
const int MAX_STEER_COUNTS = 450;

// Target-bearing error that produces full steering.
```

```
const float FULL_STEER_ERROR_DEG = 90.0f;

// Do not react aggressively to tiny GPS/heading errors.
const float HEADING_DEADBAND_DEG = 5.0f;

// GPS course is used when there is enough ground speed.
const float GPS_COURSE_MIN_SPEED_MPS = 1.5f;

// GPS navigation quality limits
const uint32_t GPS_DATA_TIMEOUT_MS = 1500;
const uint8_t  GPS_MIN_SATS = 5;
const float    GPS_MAX_HDOP = 3.5f;

// ------------------------ STEERING MAPPING --------------------------
//
// Positive steer means the target is to the RIGHT of the current
// direction. With the default mapping:
//     positive steer -> Servo 1 lower, Servo 2 higher
//     negative steer -> Servo 1 higher, Servo 2 lower
//
// Change TURN_SIGN to -1 after mechanical testing if the parafoil
// turns in the opposite direction.
//
const int TURN_SIGN = 1;

// Individual servo direction reversal options.
// The servos themselves were verified as:
// lower position -> CCW
// higher position -> CW
//
// These flags are for the actual linkage/mounting geometry.
const bool SERVO1_REVERSE = false;
const bool SERVO2_REVERSE = false;

// Servo command rate
const uint32_t SERVO_UPDATE_MS = 100;

// Servo movement settings from your working ST-3009 tests
const int SERVO_SPEED = 500;
const int SERVO_ACCEL = 50;

// -------------------------- DEPLOYMENT ------------------------------
// R2Home reference used a PWM command corresponding to about 140 deg
// before release and about 70 deg after release.
#define DEPLOY_SERVO_PIN 8
const int DEPLOY_PRELAUNCH_ANGLE = 140;
const int DEPLOY_RELEASE_ANGLE   = 70;

// Required deployment point above launch level.
const float DEPLOY_ALTITUDE_M = 600.0f;

// Wait for the parafoil to inflate/stabilize before steering.
const uint32_t DEPLOY_STABILIZE_MS = 5000;

// ------------------------ FLIGHT DETECTION ------------------------
const float LAUNCH_ASCENT_VS_MPS = 1.0f;
const uint32_t LAUNCH_CONFIRM_MS = 1000;

const float DESCENT_VS_MPS = -0.5f;
const uint32_t DESCENT_CONFIRM_MS = 1000;

// -------------------------- LANDING -------------------------------
const float LANDING_ALTITUDE_M = 12.0f;
const float LANDING_MAX_ABS_VS_MPS = 0.6f;
const uint32_t LANDING_CONFIRM_MS = 6000;

// Target capture / final approach
const float TARGET_CAPTURE_RADIUS_M = 35.0f;
const float TARGET_RELEASE_RADIUS_M = 60.0f;

// Reduce steering authority near the ground.
const float FINAL_APPROACH_ALT_M = 120.0f;
const float FINAL_STEER_SCALE = 0.70f;

// ------------------------ COMPASS -------------------------------
```

```
// Your existing compass code is used exactly for the numeric heading.
// Leave this at 0 unless a fixed installation offset is measured.
const float COMPASS_HEADING_OFFSET_DEG = 0.0f;
// Your earlier 360-degree test showed the compass angle changed in the
// opposite rotational sense to a normal clockwise compass heading.
// Keep the tested raw code, but mirror the numeric heading for navigation
// so GNSS true-course and compass fallback use the same clockwise frame.
const bool COMPASS_MIRROR_FOR_NAV = true;

// Your existing calibration values
const float OFFSET_X = 180.0000f;
const float OFFSET_Y = 336.0000f;
const float OFFSET_Z = 615.0000f;

const float SCALE_X = 0.956484f;
const float SCALE_Y = 0.904406f;
const float SCALE_Z = 1.178127f;

// ------------------------- BAROMETER ------------------------------
// Relative altitude is computed from the pressure captured at ARM.
const float BARO_EXPONENT = 0.19029496f;

// ------------------------------ SD ---------------------------------
const int SD_CS = BUILTIN_SDCARD;
const char *LOG_FILE = "AUTONAV.CSV";

// ========================================================================
//                          OBJECTS / GLOBALS
// ========================================================================

TinyGPSPlus gps;
Adafruit_LSM6DSOX lsm6dso;
Adafruit_BMP5xx bmp;
SMS_STS st;
Servo deployServo;

bool compassOK = false;
bool imuOK = false;
bool bmpOK = false;
bool servo1OK = false;
bool servo2OK = false;
bool sdOK = false;

// ------------------------------ Target -----------------------------
double targetLat = 0.0;
double targetLon = 0.0;
bool targetSet = false;

// ------------------------------ GPS --------------------------------
double gpsLat = 0.0;
double gpsLon = 0.0;
double gpsAlt = 0.0;
double gpsSpeedMps = 0.0;
double gpsCourseDeg = 0.0;
uint32_t gpsSats = 0;
double gpsHDOP = 99.9;
bool gpsFix = false;
uint32_t lastGPSLocationMs = 0;

// ---------------------------- Compass ----------------------------
int16_t rawX = 0;
int16_t rawY = 0;
int16_t rawZ = 0;
float magX = 0.0f;
float magY = 0.0f;
float magZ = 0.0f;
float compassHeading = 0.0f;
uint8_t compassReadFailCount = 0;
uint32_t lastCompassRetryMs = 0;

// ------------------------------ IMU ------------------------------
float ax = 0.0f;
float ay = 0.0f;
float az = 0.0f;
float gx = 0.0f;
```

```
float gy = 0.0f;
float gz = 0.0f;
float imuTemp = 0.0f;

// ---------------------------- BMP581 ----------------------------
float pressurePa = 0.0f;
float bmpTemp = 0.0f;
float baroAltRaw = 0.0f;
float baroAltFiltered = 0.0f;
float relativeAlt = 0.0f;
float verticalSpeed = 0.0f;
float launchPressurePa = 0.0f;
bool baroFilterInitialized = false;
uint32_t lastBaroAltitudeMs = 0;
float lastBaroAltitude = 0.0f;

// ------------------------- Navigation ----------------------------
double targetDistanceM = 0.0;
double targetBearingDeg = 0.0;
float currentDirectionDeg = 0.0f;
float courseErrorDeg = 0.0f;
int steeringDemand = 0;
char directionSource[8] = "NONE";

// ---------------------------- Servos ----------------------------
int servo1Command = SERVO_NEUTRAL;
int servo2Command = SERVO_NEUTRAL;
int servo1Actual = -1;
int servo2Actual = -1;
uint32_t lastServoWriteMs = 0;
uint32_t lastServoHealthMs = 0;
uint32_t lastServoReadMs = 0;

// --------------------------- Deployment ----------------------------
bool deployed = false;
uint32_t deploymentMs = 0;

// ------------------------- State machine ----------------------------
enum FlightState
{
  STATE_DISARMED = 0,
  STATE_ARMED = 1,
  STATE_ASCENT = 2,
  STATE_DESCENT = 3,
  STATE_DEPLOY_STABILIZE = 4,
  STATE_AUTONAV = 5,
  STATE_CAPTURED = 6,
  STATE_LANDED = 7,
  STATE_FAILSAFE = 8
};

FlightState state = STATE_DISARMED;
FlightState previousState = STATE_DISARMED;

uint32_t ascentStartMs = 0;
uint32_t descentStartMs = 0;
uint32_t landingStartMs = 0;
uint32_t gpsGoodStartMs = 0;

// ---------------------------- Timers ----------------------------
uint32_t lastSensorMs = 0;
uint32_t lastPrintMs = 0;
uint32_t lastLogMs = 0;
uint32_t lastNavMs = 0;

// ========================================================================
//                            HELPER FUNCTIONS
// ========================================================================

float wrap360(float a)
{
  while (a < 0.0f) a += 360.0f;
  while (a >= 360.0f) a -= 360.0f;
  return a;
}
```

```
325
326 float wrap180(float a)
327 {
328   while (a > 180.0f) a -= 360.0f;
329   while (a <= -180.0f) a += 360.0f;
330   return a;
331 }
332
333 int clampServoPosition(int p)
334 {
335   if (p < SERVO_MIN_POS) return SERVO_MIN_POS;
336   if (p > SERVO_MAX_POS) return SERVO_MAX_POS;
337   return p;
338 }
339
340 const char* stateName(FlightState s)
341 {
342   switch (s)
343   {
344     case STATE_DISARMED:         return "DISARMED";
345     case STATE_ARMED:            return "ARMED";
346     case STATE_ASCENT:           return "ASCENT";
347     case STATE_DESCENT:          return "DESCENT";
348     case STATE_DEPLOY_STABILIZE: return "DEPLOY_STAB";
349     case STATE_AUTONAV:          return "AUTONAV";
350     case STATE_CAPTURED:         return "CAPTURED";
351     case STATE_LANDED:           return "LANDED";
352     case STATE_FAILSAFE:         return "FAILSAFE";
353     default:                     return "UNKNOWN";
354   }
355 }
356
357 bool gpsQualityOK()
358 {
359   if (!gpsFix) return false;
360   if ((millis() - lastGPSLocationMs) > GPS_DATA_TIMEOUT_MS) return false;
361   if (gpsSats < GPS_MIN_SATS) return false;
362   if (gps.hdop.isValid() && gpsHDOP > GPS_MAX_HDOP) return false;
363   return true;
364 }
365
366 // ========================================================================
367 //                          QMC5883 COMPASS
368 //                 EXACT WORKING IMPLEMENTATION STYLE
369 // ========================================================================
370
371 void writeRegister(byte reg, byte value)
372 {
373   Wire.beginTransmission(QMC5883_ADDR);
374   Wire.write(reg);
375   Wire.write(value);
376   Wire.endTransmission();
377 }
378
379 bool readQMC5883(int16_t &x, int16_t &y, int16_t &z)
380 {
381   Wire.beginTransmission(QMC5883_ADDR);
382   Wire.write(0x00);
383
384   if (Wire.endTransmission(false) != 0)
385     return false;
386
387   Wire.requestFrom(QMC5883_ADDR, 6);
388
389   if (Wire.available() < 6)
390     return false;
391
392   uint8_t xl = Wire.read();
393   uint8_t xh = Wire.read();
394   uint8_t yl = Wire.read();
395   uint8_t yh = Wire.read();
396   uint8_t zl = Wire.read();
397   uint8_t zh = Wire.read();
398
399   x = (int16_t)((xh << 8) | xl);
```

```
  y = (int16_t)((yh << 8) | yl);
  z = (int16_t)((zh << 8) | zl);

  return true;
}

bool initializeQMC5883()
{
  Wire.beginTransmission(QMC5883_ADDR);

  if (Wire.endTransmission() != 0)
    return false;

  // Same initialization as your verified compass code.
  writeRegister(0x0B, 0x01);
  delay(10);
  writeRegister(0x09, 0x1D);
  delay(100);

  return true;
}

bool updateCompass()
{
  if (!readQMC5883(rawX, rawY, rawZ))
    return false;

  magX = (rawX - OFFSET_X) * SCALE_X;
  magY = (rawY - OFFSET_Y) * SCALE_Y;
  magZ = (rawZ - OFFSET_Z) * SCALE_Z;

  // This is the exact numeric calculation from your proven compass test.
  float angle = atan2(magY, magX) * 180.0f / PI;

  if (angle < 0.0f)
    angle += 360.0f;

  compassHeading = wrap360(angle + COMPASS_HEADING_OFFSET_DEG);

  return true;
}

float navigationCompassHeading()
{
  if (COMPASS_MIRROR_FOR_NAV)
    return wrap360(360.0f - compassHeading);

  return compassHeading;
}

// Same direction labels as your verified compass test. The numeric
// heading used by navigation is unaffected by these display labels.
const char* directionName(float h)
{
  if (h >= 337.5f || h < 22.5f) return "NORTH";
  if (h < 67.5f) return "NORTH-EAST";
  if (h < 112.5f) return "WEST";
  if (h < 157.5f) return "SOUTH-EAST";
  if (h < 202.5f) return "SOUTH";
  if (h < 247.5f) return "SOUTH-WEST";
  if (h < 292.5f) return "EAST";
  return "NORTH-WEST";
}

// ========================================================================
//                               IMU
// ========================================================================

bool initializeIMU()
{
  if (!lsm6dso.begin_I2C(LSM6DSO32_ADDR, &Wire))
    return false;

  // Same ranges/data rates as your known working sensor code.
  lsm6dso.setAccelRange(LSM6DS_ACCEL_RANGE_16_G);
```

```
  lsm6dso.setGyroRange(LSM6DS_GYRO_RANGE_2000_DPS);
  lsm6dso.setAccelDataRate(LSM6DS_RATE_208_HZ);
  lsm6dso.setGyroDataRate(LSM6DS_RATE_208_HZ);

  return true;
}

void updateIMU()
{
  if (!imuOK) return;

  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;

  lsm6dso.getEvent(&accel, &gyro, &temp);

  ax = accel.acceleration.x / 9.80665f;
  ay = accel.acceleration.y / 9.80665f;
  az = accel.acceleration.z / 9.80665f;

  gx = gyro.gyro.x * 57.2957795f;
  gy = gyro.gyro.y * 57.2957795f;
  gz = gyro.gyro.z * 57.2957795f;

  imuTemp = temp.temperature;
}

// ========================================================================
//                             BMP581
// ========================================================================

bool initializeBMP()
{
  if (bmp.begin(BMP_ADDR_1, &Wire))
    return true;

  if (bmp.begin(BMP_ADDR_2, &Wire))
    return true;

  return false;
}

float pressureToRelativeAltitude(float p)
{
  if (!isfinite(p) || p <= 0.0f || launchPressurePa <= 0.0f)
    return 0.0f;

  return 44330.0f *
         (1.0f - powf(p / launchPressurePa, BARO_EXPONENT));
}

void updateBMP()
{
  if (!bmpOK) return;

  float p = bmp.readPressure();
  float t = bmp.readTemperature();

  if (!isfinite(p) || !isfinite(t) || p <= 0.0f)
    return;

  pressurePa = p;
  bmpTemp = t;
  baroAltRaw = pressureToRelativeAltitude(p);

  // Slow filter to reduce pressure noise.
  if (!baroFilterInitialized)
  {
    baroAltFiltered = baroAltRaw;
    baroFilterInitialized = true;
    relativeAlt = baroAltFiltered;
    lastBaroAltitude = relativeAlt;
    lastBaroAltitudeMs = millis();
    verticalSpeed = 0.0f;
```

```
    return;
  }
  else
  {
    const float ALT_ALPHA = 0.25f;
    baroAltFiltered += ALT_ALPHA * (baroAltRaw - baroAltFiltered);
  }

  relativeAlt = baroAltFiltered;

  // Calculate vertical speed from filtered relative altitude.
  uint32_t now = millis();
  uint32_t dtMs = now - lastBaroAltitudeMs;

  if (dtMs >= 20 && dtMs <= 1000)
  {
    float dt = dtMs * 0.001f;
    float instantVS = (relativeAlt - lastBaroAltitude) / dt;

    // Low-pass the derivative to suppress barometric noise.
    const float VS_ALPHA = 0.20f;
    verticalSpeed += VS_ALPHA * (instantVS - verticalSpeed);

    lastBaroAltitude = relativeAlt;
    lastBaroAltitudeMs = now;
  }
}

// ========================================================================
//                                GPS
// ========================================================================

void updateGPS()
{
  while (GPS_SERIAL.available())
    gps.encode(GPS_SERIAL.read());

  // Update the timestamp only when a NEW position sentence has been
  // decoded. This makes the GPS timeout a real communication timeout.
  if (gps.location.isUpdated())
  {
    gpsLat = gps.location.lat();
    gpsLon = gps.location.lng();
    lastGPSLocationMs = millis();
  }

  gpsFix = gps.location.isValid();

  if (gps.altitude.isValid())
    gpsAlt = gps.altitude.meters();

  if (gps.speed.isValid())
    gpsSpeedMps = gps.speed.mps();

  if (gps.course.isValid())
    gpsCourseDeg = gps.course.deg();

  if (gps.satellites.isValid())
    gpsSats = gps.satellites.value();

  if (gps.hdop.isValid())
    gpsHDOP = gps.hdop.hdop();
}

// ========================================================================
//                         NAVIGATION CALCULATION
// ========================================================================

void updateNavigation()
{
  if (!targetSet || !gpsFix)
  {
    targetDistanceM = 0.0;
    targetBearingDeg = 0.0;
    courseErrorDeg = 0.0f;
```

```
    steeringDemand = 0;
    strcpy(directionSource, "NONE");
    return;
  }

  targetDistanceM = TinyGPSPlus::distanceBetween(
    gpsLat,
    gpsLon,
    targetLat,
    targetLon
  );

  targetBearingDeg = TinyGPSPlus::courseTo(
    gpsLat,
    gpsLon,
    targetLat,
    targetLon
  );

  bool gpsCourseUsable =
    gpsQualityOK() &&
    gps.course.isValid() &&
    gpsSpeedMps >= GPS_COURSE_MIN_SPEED_MPS;

  if (gpsCourseUsable)
  {
    currentDirectionDeg = wrap360((float)gpsCourseDeg);
    strcpy(directionSource, "GPS");
  }
  else if (compassOK)
  {
    currentDirectionDeg = navigationCompassHeading();
    strcpy(directionSource, "COMPASS");
  }
  else
  {
    steeringDemand = 0;
    courseErrorDeg = 0.0f;
    strcpy(directionSource, "NONE");
    return;
  }

  courseErrorDeg = wrap180(
    (float)targetBearingDeg - currentDirectionDeg
  );

  float absError = fabsf(courseErrorDeg);

  if (absError <= HEADING_DEADBAND_DEG)
  {
    steeringDemand = 0;
    return;
  }

  float normalized = absError / FULL_STEER_ERROR_DEG;
  if (normalized > 1.0f) normalized = 1.0f;

  float demand = normalized * MAX_STEER_COUNTS;

  // Positive error = target to the right.
  if (courseErrorDeg > 0.0f)
    demand = fabsf(demand);
  else
    demand = -fabsf(demand);

  demand *= TURN_SIGN;

  // Reduce authority during final low-altitude approach.
  if (relativeAlt < FINAL_APPROACH_ALT_M)
    demand *= FINAL_STEER_SCALE;

  steeringDemand = (int)roundf(demand);

  // Capture zone: stop aggressive steering inside the target radius.
  if (targetDistanceM <= TARGET_CAPTURE_RADIUS_M)
```

```
  {
    steeringDemand = 0;
  }
}

// ========================================================================
//                          SERVO CONTROL
// ========================================================================

int applyReverse(int position, bool reverse)
{
  if (!reverse)
    return position;

  return SERVO_NEUTRAL - (position - SERVO_NEUTRAL);
}

void setSteeringNeutral(bool immediate = false)
{
  servo1Command = SERVO_NEUTRAL;
  servo2Command = SERVO_NEUTRAL;

  if (immediate)
  {
    st.WritePosEx(SERVO1_ID, servo1Command, SERVO_SPEED, SERVO_ACCEL);
    st.WritePosEx(SERVO2_ID, servo2Command, SERVO_SPEED, SERVO_ACCEL);
  }
}

void setSteeringFromDemand(int demand)
{
  demand = constrain(demand, -MAX_STEER_COUNTS, MAX_STEER_COUNTS);

  // Default differential-brake mapping:
  // demand > 0: Servo 1 lower, Servo 2 higher
  int s1 = SERVO_NEUTRAL - demand;
  int s2 = SERVO_NEUTRAL + demand;

  s1 = applyReverse(s1, SERVO1_REVERSE);
  s2 = applyReverse(s2, SERVO2_REVERSE);

  servo1Command = clampServoPosition(s1);
  servo2Command = clampServoPosition(s2);
}

void updateServos()
{
  if (millis() - lastServoWriteMs < SERVO_UPDATE_MS)
    return;

  lastServoWriteMs = millis();

  if (!deployed || state != STATE_AUTONAV)
  {
    servo1Command = SERVO_NEUTRAL;
    servo2Command = SERVO_NEUTRAL;
  }

  st.WritePosEx(
    SERVO1_ID,
    servo1Command,
    SERVO_SPEED,
    SERVO_ACCEL
  );

  st.WritePosEx(
    SERVO2_ID,
    servo2Command,
    SERVO_SPEED,
    SERVO_ACCEL
  );
}

void checkServoHealth()
{
```

```
  if (millis() - lastServoHealthMs >= 2000)
  {
    lastServoHealthMs = millis();

    servo1OK = (st.Ping(SERVO1_ID) != -1);
    servo2OK = (st.Ping(SERVO2_ID) != -1);
  }

  if (millis() - lastServoReadMs >= 500)
  {
    lastServoReadMs = millis();

    servo1Actual = st.ReadPos(SERVO1_ID);
    servo2Actual = st.ReadPos(SERVO2_ID);
  }
}

bool servoBusOK()
{
  return servo1OK && servo2OK;
}

// ========================================================================
//                            DEPLOYMENT
// ========================================================================

void deployWing()
{
  if (deployed)
    return;

  deployed = true;
  deploymentMs = millis();

  // R2Home reference behavior:
  // pre-deployment approximately 140 deg, then return to approximately
  // 70 deg after release.
  deployServo.write(DEPLOY_RELEASE_ANGLE);

  setSteeringNeutral(true);

  Serial.println("*** WING DEPLOYED ***");
}

// ========================================================================
//                        LAUNCH / LANDING LOGIC
// ========================================================================

void updateLaunchAndDescentDetection()
{
  if (state == STATE_ARMED)
  {
    if (verticalSpeed >= LAUNCH_ASCENT_VS_MPS)
    {
      if (ascentStartMs == 0)
        ascentStartMs = millis();

      if (millis() - ascentStartMs >= LAUNCH_CONFIRM_MS)
      {
        state = STATE_ASCENT;
        ascentStartMs = 0;
      }
    }
    else
    {
      ascentStartMs = 0;
    }
  }

  if (state == STATE_ASCENT)
  {
    if (verticalSpeed <= DESCENT_VS_MPS)
    {
      if (descentStartMs == 0)
        descentStartMs = millis();
```

```
      if (millis() - descentStartMs >= DESCENT_CONFIRM_MS)
      {
        state = STATE_DESCENT;
        descentStartMs = 0;
      }
    }
    else
    {
      descentStartMs = 0;
    }
  }

  if (state == STATE_DESCENT)
  {
    if (relativeAlt <= DEPLOY_ALTITUDE_M && verticalSpeed < 0.0f)
    {
      deployWing();
      state = STATE_DEPLOY_STABILIZE;
    }
  }
}

void updateLandingDetection()
{
  if (!deployed)
    return;

  if (relativeAlt <= LANDING_ALTITUDE_M &&
      fabsf(verticalSpeed) <= LANDING_MAX_ABS_VS_MPS)
  {
    if (landingStartMs == 0)
      landingStartMs = millis();

    if (millis() - landingStartMs >= LANDING_CONFIRM_MS)
    {
      state = STATE_LANDED;
      setSteeringNeutral(true);
    }
  }
  else
  {
    landingStartMs = 0;
  }
}

// ============================================================================
//                          AUTONOMOUS STATE MACHINE
// ============================================================================

void handleStateMachine()
{
  switch (state)
  {
    case STATE_DISARMED:
      setSteeringNeutral(false);
      break;

    case STATE_ARMED:
      setSteeringNeutral(false);
      updateLaunchAndDescentDetection();
      break;

    case STATE_ASCENT:
      setSteeringNeutral(false);
      updateLaunchAndDescentDetection();
      break;

    case STATE_DESCENT:
      setSteeringNeutral(false);
      updateLaunchAndDescentDetection();
      break;

    case STATE_DEPLOY_STABILIZE:
      setSteeringNeutral(false);
```

```
925
926       if (millis() - deploymentMs >= DEPLOY_STABILIZE_MS)
927       {
928         if (gpsQualityOK() && targetSet && servoBusOK())
929           state = STATE_AUTONAV;
930         else
931           state = STATE_FAILSAFE;
932       }
933       break;
934
935     case STATE_AUTONAV:
936       if (!targetSet || !gpsQualityOK() || !servoBusOK())
937       {
938         state = STATE_FAILSAFE;
939         setSteeringNeutral(false);
940         break;
941       }
942
943       if (targetDistanceM <= TARGET_CAPTURE_RADIUS_M)
944       {
945         state = STATE_CAPTURED;
946         setSteeringNeutral(false);
947         break;
948       }
949
950       if (targetDistanceM > TARGET_RELEASE_RADIUS_M)
951       {
952         state = STATE_AUTONAV;
953       }
954       break;
955
956     case STATE_CAPTURED:
957       setSteeringNeutral(false);
958
959       // Allow re-entry if the vehicle drifts back out of the capture
960       // radius before landing.
961       if (targetDistanceM > TARGET_RELEASE_RADIUS_M)
962       {
963         state = STATE_AUTONAV;
964       }
965       break;
966
967     case STATE_LANDED:
968       setSteeringNeutral(false);
969       break;
970
971     case STATE_FAILSAFE:
972       setSteeringNeutral(false);
973
974       // Recover automatically after a stable GPS/servo recovery.
975       if (deployed && targetSet && gpsQualityOK() && servoBusOK())
976       {
977         if (gpsGoodStartMs == 0)
978           gpsGoodStartMs = millis();
979
980         if (millis() - gpsGoodStartMs >= 3000)
981         {
982           state = STATE_AUTONAV;
983           gpsGoodStartMs = 0;
984         }
985       }
986       else
987       {
988         gpsGoodStartMs = 0;
989       }
990       break;
991   }
992
993   // Navigation commands are generated only in AUTONAV.
994   if (state == STATE_AUTONAV)
995   {
996     if (millis() - lastNavMs >= 100)
997     {
998       lastNavMs = millis();
999       updateNavigation();
```

```
      setSteeringFromDemand(steeringDemand);
    }
  }
  else
  {
    steeringDemand = 0;
  }

  updateLandingDetection();

  // Immediate neutral outside autonomous navigation.
  if (state != STATE_AUTONAV)
  {
    servo1Command = SERVO_NEUTRAL;
    servo2Command = SERVO_NEUTRAL;
  }
}

// ============================================================================
//                          ARM / DISARM
// ============================================================================

bool armSystem()
{
  if (!targetSet)
  {
    Serial.println("ARM rejected: TARGET not set.");
    return false;
  }

  if (!gpsQualityOK())
  {
    Serial.println("ARM rejected: GPS quality is not good.");
    return false;
  }

  if (!bmpOK)
  {
    Serial.println("ARM rejected: BMP581 unavailable.");
    return false;
  }

  if (!servoBusOK())
  {
    Serial.println("ARM rejected: ST-3009 servos not detected.");
    return false;
  }

  float p = bmp.readPressure();
  if (!isfinite(p) || p <= 0.0f)
  {
    Serial.println("ARM rejected: invalid launch pressure.");
    return false;
  }

  launchPressurePa = p;
  baroAltFiltered = 0.0f;
  baroFilterInitialized = false;
  relativeAlt = 0.0f;
  verticalSpeed = 0.0f;
  lastBaroAltitude = 0.0f;
  lastBaroAltitudeMs = millis();

  deployed = false;
  deploymentMs = 0;
  ascentStartMs = 0;
  descentStartMs = 0;
  landingStartMs = 0;
  gpsGoodStartMs = 0;

  deployServo.write(DEPLOY_PRELAUNCH_ANGLE);
  setSteeringNeutral(true);

  state = STATE_ARMED;

```

```
  Serial.println("SYSTEM ARMED.");
  Serial.println("Waiting for ascent...");

  return true;
}

void disarmSystem()
{
  state = STATE_DISARMED;
  deployed = false;
  steeringDemand = 0;
  setSteeringNeutral(true);
  deployServo.write(DEPLOY_PRELAUNCH_ANGLE);

  Serial.println("SYSTEM DISARMED.");
}

// ========================================================================
//                          SERIAL COMMANDS
// ========================================================================

void printHelp()
{
  Serial.println();
  Serial.println("COMMANDS:");
  Serial.println("  TARGET,<latitude>,<longitude>");
  Serial.println("  ARM");
  Serial.println("  DISARM");
  Serial.println("  STATUS");
  Serial.println("  ZERO     (reset launch barometer while disarmed)");
  Serial.println("  NEUTRAL  (center steering servos)");
  Serial.println("  HELP");
  Serial.println();
}

void printStatus()
{
  Serial.println();
  Serial.println("================ STATUS ================");
  Serial.print("State             : "); Serial.println(stateName(state));
  Serial.print("Target set        : "); Serial.println(targetSet ? "YES" : "NO");

  if (targetSet)
  {
    Serial.print("Target latitude   : "); Serial.println(targetLat, 7);
    Serial.print("Target longitude  : "); Serial.println(targetLon, 7);
  }

  Serial.print("GPS fix           : "); Serial.println(gpsFix ? "YES" : "NO");
  Serial.print("GPS sats          : "); Serial.println(gpsSats);
  Serial.print("GPS HDOP          : "); Serial.println(gpsHDOP, 2);
  Serial.print("GPS speed         : "); Serial.println(gpsSpeedMps, 2);
  Serial.print("GPS course        : "); Serial.println(gpsCourseDeg, 1);

  Serial.print("Compass heading   : "); Serial.print(compassHeading, 1);
  Serial.print(" deg ("); Serial.print(directionName(compassHeading)); Serial.println(")");
  Serial.print("Compass nav head. : "); Serial.print(navigationCompassHeading(), 1);
  Serial.println(" deg");
  Serial.print("Baro rel altitude : "); Serial.println(relativeAlt, 2);
  Serial.print("Vertical speed    : "); Serial.println(verticalSpeed, 2);

  Serial.print("Distance to target: "); Serial.println(targetDistanceM, 1);
  Serial.print("Target bearing    : "); Serial.println(targetBearingDeg, 1);
  Serial.print("Current direction : "); Serial.println(currentDirectionDeg, 1);
  Serial.print("Direction source  : "); Serial.println(directionSource);
  Serial.print("Course error      : "); Serial.println(courseErrorDeg, 1);
  Serial.print("Steering demand   : "); Serial.println(steeringDemand);

  Serial.print("Servo 1 command   : "); Serial.println(servo1Command);
  Serial.print("Servo 2 command   : "); Serial.println(servo2Command);
  Serial.print("Servo 1 actual    : "); Serial.println(servo1Actual);
  Serial.print("Servo 2 actual    : "); Serial.println(servo2Actual);

  Serial.print("Deployed          : "); Serial.println(deployed ? "YES" : "NO");
  Serial.print("Servo 1 bus       : "); Serial.println(servo1OK ? "OK" : "FAIL");
```

```
  Serial.print("Servo 2 bus       : "); Serial.println(servo2OK ? "OK" : "FAIL");
  Serial.println("========================================");
}

void processCommand(char *cmd)
{
  // Convert command to uppercase only for the command word.
  // TARGET numeric values are left untouched because they are parsed
  // from the remainder of the string.

  if (strncmp(cmd, "TARGET,", 7) == 0)
  {
    char *p = cmd + 7;
    char *comma = strchr(p, ',');

    if (comma == NULL)
    {
      Serial.println("Invalid TARGET format.");
      Serial.println("Use: TARGET,latitude,longitude");
      return;
    }

    *comma = '\0';

    double newLat = atof(p);
    double newLon = atof(comma + 1);

    if (newLat < -90.0 || newLat > 90.0 ||
        newLon < -180.0 || newLon > 180.0)
    {
      Serial.println("Invalid latitude/longitude range.");
      return;
    }

    targetLat = newLat;
    targetLon = newLon;
    targetSet = true;

    Serial.print("TARGET SET: ");
    Serial.print(targetLat, 7);
    Serial.print(", ");
    Serial.println(targetLon, 7);
    return;
  }

  if (strcmp(cmd, "ARM") == 0)
  {
    armSystem();
    return;
  }

  if (strcmp(cmd, "DISARM") == 0)
  {
    disarmSystem();
    return;
  }

  if (strcmp(cmd, "STATUS") == 0)
  {
    printStatus();
    return;
  }

  if (strcmp(cmd, "ZERO") == 0)
  {
    if (state != STATE_DISARMED)
    {
      Serial.println("ZERO allowed only when DISARMED.");
      return;
    }

    if (bmpOK)
    {
      float p = bmp.readPressure();
      if (isfinite(p) && p > 0.0f)
```

```
      {
        launchPressurePa = p;
        relativeAlt = 0.0f;
        baroAltFiltered = 0.0f;
        verticalSpeed = 0.0f;
        lastBaroAltitude = 0.0f;
        lastBaroAltitudeMs = millis();
        baroFilterInitialized = false;
        Serial.println("Barometer zeroed.");
      }
    }
    return;
  }

  if (strcmp(cmd, "NEUTRAL") == 0)
  {
    setSteeringNeutral(true);
    Serial.println("Steering servos commanded to 2048.");
    return;
  }

  if (strcmp(cmd, "HELP") == 0)
  {
    printHelp();
    return;
  }

  Serial.print("Unknown command: ");
  Serial.println(cmd);
}

void processSerialCommands()
{
  static char line[100];
  static uint8_t index = 0;

  while (Serial.available())
  {
    char c = Serial.read();

    if (c == '\r')
      continue;

    if (c == '\n')
    {
      line[index] = '\0';

      if (index > 0)
      {
        processCommand(line);
      }

      index = 0;
    }
    else
    {
      if (index < sizeof(line) - 1)
      {
        line[index++] = c;
      }
    }
  }
}

// ============================================================================
//                              SD LOGGING
// ============================================================================

void initSD()
{
  if (!SD.begin(SD_CS))
  {
    sdOK = false;
    return;
  }
```

```
  sdOK = true;

  if (!SD.exists(LOG_FILE))
  {
    File f = SD.open(LOG_FILE, FILE_WRITE);

    if (f)
    {
      f.println(
        "ms,state,targetLat,targetLon,lat,lon,gpsAlt,gpsSpeed,gpsCourse,sats,hdop,"
        "compass,baroAlt,verticalSpeed,targetDistance,targetBearing,currentDirection,"
        "directionSource,courseError,steerDemand,servo1Cmd,servo2Cmd,servo1Actual,"
        "servo2Actual,deployed,ax,ay,az,gx,gy,gz,bmpTemp,pressure"
      );
      f.close();
    }
  }
}

void logData()
{
  if (!sdOK)
    return;

  if (millis() - lastLogMs < 500)
    return;

  lastLogMs = millis();

  File f = SD.open(LOG_FILE, FILE_WRITE);

  if (!f)
    return;

  f.print(millis()); f.print(',');
  f.print(stateName(state)); f.print(',');
  f.print(targetLat, 7); f.print(',');
  f.print(targetLon, 7); f.print(',');
  f.print(gpsLat, 7); f.print(',');
  f.print(gpsLon, 7); f.print(',');
  f.print(gpsAlt, 2); f.print(',');
  f.print(gpsSpeedMps, 2); f.print(',');
  f.print(gpsCourseDeg, 2); f.print(',');
  f.print(gpsSats); f.print(',');
  f.print(gpsHDOP, 2); f.print(',');
  f.print(compassHeading, 2); f.print(',');
  f.print(relativeAlt, 2); f.print(',');
  f.print(verticalSpeed, 2); f.print(',');
  f.print(targetDistanceM, 2); f.print(',');
  f.print(targetBearingDeg, 2); f.print(',');
  f.print(currentDirectionDeg, 2); f.print(',');
  f.print(directionSource); f.print(',');
  f.print(courseErrorDeg, 2); f.print(',');
  f.print(steeringDemand); f.print(',');
  f.print(servo1Command); f.print(',');
  f.print(servo2Command); f.print(',');
  f.print(servo1Actual); f.print(',');
  f.print(servo2Actual); f.print(',');
  f.print(deployed ? 1 : 0); f.print(',');
  f.print(ax, 3); f.print(',');
  f.print(ay, 3); f.print(',');
  f.print(az, 3); f.print(',');
  f.print(gx, 2); f.print(',');
  f.print(gy, 2); f.print(',');
  f.print(gz, 2); f.print(',');
  f.print(bmpTemp, 2); f.print(',');
  f.println(pressurePa, 1);

  f.close();
}

// ============================================================================
//                            SERIAL MONITOR
// ============================================================================
```

```
1375
1376 void printLiveData()
1377 {
1378   if (millis() - lastPrintMs < 500)
1379     return;
1380
1381   lastPrintMs = millis();
1382
1383   Serial.println();
1384   Serial.println("------------------------------------------------------------");
1385   Serial.print("STATE = "); Serial.println(stateName(state));
1386
1387   Serial.print("GPS: ");
1388   if (gpsFix)
1389   {
1390     Serial.print(gpsLat, 7);
1391     Serial.print(", ");
1392     Serial.print(gpsLon, 7);
1393     Serial.print("  Sats=");
1394     Serial.print(gpsSats);
1395     Serial.print(" HDOP=");
1396     Serial.print(gpsHDOP, 1);
1397     Serial.print(" Speed=");
1398     Serial.print(gpsSpeedMps, 2);
1399     Serial.print("m/s Course=");
1400     Serial.print(gpsCourseDeg, 1);
1401     Serial.println("deg");
1402   }
1403   else
1404   {
1405     Serial.println("NO FIX");
1406   }
1407
1408   Serial.print("Compass: ");
1409   if (compassOK)
1410   {
1411     Serial.print(compassHeading, 1);
1412     Serial.println(" deg");
1413   }
1414   else
1415   {
1416     Serial.println("FAIL");
1417   }
1418
1419   Serial.print("Baro: Alt=");
1420   Serial.print(relativeAlt, 2);
1421   Serial.print("m VS=");
1422   Serial.print(verticalSpeed, 2);
1423   Serial.println("m/s");
1424
1425   if (targetSet && gpsFix)
1426   {
1427     Serial.print("NAV: Bearing=");
1428     Serial.print(targetBearingDeg, 1);
1429     Serial.print("deg Distance=");
1430     Serial.print(targetDistanceM, 1);
1431     Serial.print("m Current=");
1432     Serial.print(currentDirectionDeg, 1);
1433     Serial.print("deg Error=");
1434     Serial.print(courseErrorDeg, 1);
1435     Serial.print("deg Source=");
1436     Serial.println(directionSource);
1437   }
1438   else
1439   {
1440     Serial.println("NAV: target/GPS not ready");
1441   }
1442
1443   Serial.print("Steering: ");
1444   Serial.print(servo1Command);
1445   Serial.print(" / ");
1446   Serial.print(servo2Command);
1447   Serial.print("  actual=");
1448   Serial.print(servo1Actual);
1449   Serial.print(" / ");
```

```
  Serial.println(servo2Actual);

  Serial.print("Deploy=");
  Serial.print(deployed ? "YES" : "NO");
  Serial.print("  ServoBus=");
  Serial.println(servoBusOK() ? "OK" : "FAIL");

  Serial.println("------------------------------------------------------------");
}

// ====================================================================
//                              SETUP
// ====================================================================

void setup()
{
  Serial.begin(115200);
  Serial.setTimeout(20);

  delay(1500);

  Serial.println();
  Serial.println("============================================================");
  Serial.println("      TEENSY 4.1 AUTONOMOUS PARAFOIL CONTROLLER");
  Serial.println("============================================================");

  // ------------------------------ I2C ------------------------------
  Wire.begin();
  Wire.setClock(400000);

  Serial.println("I2C: SDA 18 / SCL 19");

  // ---------------------------- Compass ----------------------------
  compassOK = initializeQMC5883();
  compassReadFailCount = 0;
  lastCompassRetryMs = millis();
  Serial.print("QMC5883: ");
  Serial.println(compassOK ? "OK @ 0x0D" : "FAIL");

  // ------------------------------ IMU ------------------------------
  imuOK = initializeIMU();
  Serial.print("LSM6DSO32: ");
  Serial.println(imuOK ? "OK @ 0x6A" : "FAIL");

  // ---------------------------- BMP581 ----------------------------
  bmpOK = initializeBMP();
  Serial.print("BMP581: ");
  Serial.println(bmpOK ? "OK" : "FAIL");

  // ------------------------------ GPS ------------------------------
  GPS_SERIAL.begin(GPS_BAUD);
  Serial.println("GPS: Serial3 @ 115200");
  Serial.println("     GPS TX -> pin 15 / RX3");
  Serial.println("     GPS RX -> pin 14 / TX3");

  // ------------------------------ Servo ----------------------------
  Serial1.begin(1000000, SERIAL_8N1_HALF_DUPLEX);
  st.pSerial = &Serial1;

  delay(1000);

  servo1OK = (st.Ping(SERVO1_ID) != -1);
  servo2OK = (st.Ping(SERVO2_ID) != -1);

  Serial.print("ST-3009 ID1: ");
  Serial.println(servo1OK ? "FOUND" : "NOT FOUND");

  Serial.print("ST-3009 ID2: ");
  Serial.println(servo2OK ? "FOUND" : "NOT FOUND");

  setSteeringNeutral(true);

  // ------------------------- Deploy servo -------------------------
  deployServo.attach(DEPLOY_SERVO_PIN);
  deployServo.write(DEPLOY_PRELAUNCH_ANGLE);
```

```
  Serial.print("SG90 deploy pin: ");
  Serial.println(DEPLOY_SERVO_PIN);

  // ------------------------------ SD -----------------------------
  initSD();
  Serial.print("SD logging: ");
  Serial.println(sdOK ? "OK" : "FAIL");

  // --------------------------- Watchdog ------------------------
#if HAVE_WATCHDOG_T4
  WDT_timings_t wdtConfig;
  wdtConfig.timeout = 2000;
  watchdog.begin(wdtConfig);
  Serial.println("Watchdog: Watchdog_t4 / 2 s");
#else
  Serial.println("Watchdog: NOT INSTALLED (install Watchdog_t4 for flight)");
#endif

  // ------------------------------ Info ---------------------------
  Serial.println();
  Serial.println("SYSTEM STARTED DISARMED");
  Serial.println();
  printHelp();
}

// ========================================================================
//                                LOOP
// ========================================================================

void loop()
{
  // Keep serial parsing responsive.
  processSerialCommands();

  // GPS must be serviced continuously.
  updateGPS();

  // Sensors at approximately 100 Hz.
  if (millis() - lastSensorMs >= 10)
  {
    lastSensorMs = millis();

    if (compassOK)
    {
      if (updateCompass())
      {
        compassReadFailCount = 0;
      }
      else
      {
        if (compassReadFailCount < 255)
          compassReadFailCount++;

        // Ignore isolated I2C glitches. Only mark the compass bad after
        // several consecutive failed reads.
        if (compassReadFailCount >= 10)
        {
          compassOK = false;
          lastCompassRetryMs = millis();
          Serial.println("QMC5883: repeated read failure");
        }
      }
    }
    else if (millis() - lastCompassRetryMs >= 1000)
    {
      lastCompassRetryMs = millis();
      if (initializeQMC5883())
      {
        compassOK = true;
        compassReadFailCount = 0;
        Serial.println("QMC5883: reinitialized");
      }
    }

    updateIMU();
```

```
    updateBMP();
  }

  // Navigation/flight-state logic.
  handleStateMachine();

  // Servo bus health / encoder positions.
  checkServoHealth();

  // In an active flight, losing both servos immediately forces neutral.
  if (deployed && state == STATE_AUTONAV && !servoBusOK())
  {
    state = STATE_FAILSAFE;
    setSteeringNeutral(true);
    Serial.println("FAILSAFE: SERVO BUS FAULT");
  }

  updateServos();

  logData();
  printLiveData();

#if HAVE_WATCHDOG_T4
  watchdog.feed();
#endif
}
```

A.3 Archived source 3: Pasted code(10).cpp

Source file: `Pasted code(10).cpp`

```
/*
  ================================================================
  TEENSY 4.1 AUTONOMOUS PARAFOIL
  GUIDED LANDING - FINAL LAB TEST
  ================================================================

  TARGET IS FIXED IN CODE.

  CHANGE ONLY:
      TARGET_LAT
      TARGET_LON


  ================================================================
  HARDWARE
  ================================================================

  TEENSY 4.1

  ----------------------------------------------------------------
  FOXEER M10Q 180 GPS
  ----------------------------------------------------------------

      GPS TX  -> Teensy Pin 15 (RX3)
      GPS RX  -> Teensy Pin 14 (TX3)
      GPS GND -> Teensy GND

      GPS UART = Serial3
      GPS BAUD = 38400

  ----------------------------------------------------------------
  I2C BUS
  ----------------------------------------------------------------

      SDA = Teensy Pin 18
      SCL = Teensy Pin 19

  ----------------------------------------------------------------
  QMC5883 COMPASS
  ----------------------------------------------------------------

      Address = 0x0D
```

```
----------------------------------------------------------------
LSM6DSO32 IMU
----------------------------------------------------------------

    Address = 0x6A

----------------------------------------------------------------
BMP581
----------------------------------------------------------------

    Address = 0x46 or 0x47
    Diagnostic only

----------------------------------------------------------------
ST-3009
----------------------------------------------------------------

    Servo 1 ID = 1
    Servo 2 ID = 2

    Serial1
    Pin 0 = RX1
    Pin 1 = TX1

    1 Mbps
    SERIAL_8N1_HALF_DUPLEX

    Servo neutral = 2048

----------------------------------------------------------------
SG90 DEPLOYMENT SERVO
----------------------------------------------------------------

    Signal = Pin 8

================================================================
NAVIGATION
================================================================

Target bearing:
    GPS position -> target

Current heading:

    If speed >= 1.5 m/s:
        GPS course

    If speed < 1.5 m/s:
        QMC5883 compass

Course error:

    target bearing - current heading

    wrapped to -180 ... +180 degrees

Steering:

    error <= +/-5 degrees:
        neutral

    error reaches +/-90 degrees:
        maximum steering

Maximum steering:
    +/-450 counts

Differential:

    Positive demand:
        Servo 1 = 2048 - demand
        Servo 2 = 2048 + demand

    Negative demand:
```

```
          Servo 1 = 2048 - demand
          Servo 2 = 2048 + demand

  Target capture:
      35 m

  ================================================================
  STATE MACHINE
  ================================================================

      WAIT_READY
           |
           | GPS GOOD + Servo 1 + Servo 2
           v
      DEPLOYING
           |
           | 1.5 sec
           v
      STABILIZING
           |
           | 5 sec + GPS + servos
           v
       AUTONAV
           |
           | <=35 m
           v
       CAPTURED

  GPS or servo failure during AUTONAV:
       AUTONAV -> FAILSAFE

  3 sec good GPS + servos:
       FAILSAFE -> AUTONAV

  ================================================================
  SERIAL COMMANDS
  ================================================================

      STATUS
      MOTORTEST
      LEFT
      RIGHT
      CENTER
      AUTONAV
      DEPLOY
      PREP
      RESET
      HELP

  There is NO TARGET serial command.

  ================================================================
*/


// ================================================================
// LIBRARIES
// ================================================================

#include <Arduino.h>
#include <Wire.h>
#include <Servo.h>
#include <TinyGPS++.h>
#include <SCServo.h>
#include <Adafruit_LSM6DSOX.h>

#include <math.h>
#include <string.h>


// ================================================================
// TARGET
// ================================================================

const double TARGET_LAT = 32.6321769;
```

```
const double TARGET_LON = 75.0143889;


// ================================================================
// AUTO DEPLOY
// ================================================================
//
// true  = deploy automatically after GPS + both ST-3009 ready
// false = wait for DEPLOY command
//
// For the autonomous lab sequence use TRUE.
// ================================================================

const bool AUTO_DEPLOY_AFTER_READY = true;


// ================================================================
// GPS
// ================================================================

#define GPS_SERIAL Serial3

const uint32_t GPS_BAUD = 38400;

const uint8_t GPS_RX_PIN = 15;
const uint8_t GPS_TX_PIN = 14;

// GPS data considered stale after this time
const uint32_t GPS_DATA_TIMEOUT_MS = 2500;

// Minimum satellites
const uint8_t GPS_MIN_SATS = 5;

// Maximum acceptable HDOP
const double GPS_MAX_HDOP = 3.5;

// GPS course is used only above this speed
const double GPS_COURSE_MIN_SPEED_MPS = 1.5;


// ================================================================
// I2C
// ================================================================

const uint8_t I2C_SDA_PIN = 18;
const uint8_t I2C_SCL_PIN = 19;


// ================================================================
// QMC5883
// ================================================================

const uint8_t QMC5883_ADDR = 0x0D;


// ----------------------------------------------------------------
// Compass navigation orientation
// ----------------------------------------------------------------
//
// Your tested compass direction was mirrored for navigation.
//

const bool COMPASS_MIRROR_FOR_NAV = true;

const float COMPASS_HEADING_OFFSET_DEG = 0.0f;


// ----------------------------------------------------------------
// Calibration values from your working compass test
// ----------------------------------------------------------------

const float OFFSET_X = 180.0000f;
const float OFFSET_Y = 336.0000f;
const float OFFSET_Z = 615.0000f;

```

```
const float SCALE_X = 0.956484f;
const float SCALE_Y = 0.904406f;
const float SCALE_Z = 1.178127f;


// ================================================================
// LSM6DSO32
// ================================================================

const uint8_t LSM6DSO32_ADDR = 0x6A;

Adafruit_LSM6DSOX lsm6dso;

bool imuOK = false;


// ================================================================
// BMP581
// ================================================================

const uint8_t BMP581_ADDR_1 = 0x46;
const uint8_t BMP581_ADDR_2 = 0x47;

bool bmpOK = false;

uint8_t bmpAddress = 0;


// ================================================================
// GPS VARIABLES
// ================================================================

TinyGPSPlus gps;

bool gpsFix = false;

uint8_t gpsSats = 0;

double gpsHDOP = 99.9;

double currentSpeed = 0.0;

unsigned long lastGPSLocationMs = 0;


// ================================================================
// COMPASS VARIABLES
// ================================================================

bool compassOK = false;

int16_t rawMagX = 0;
int16_t rawMagY = 0;
int16_t rawMagZ = 0;

float magX = 0.0f;
float magY = 0.0f;
float magZ = 0.0f;

float compassHeading = 0.0f;
float navCompassHeading = 0.0f;

uint8_t compassFailCount = 0;


// ================================================================
// IMU VARIABLES
// ================================================================

float accelX = 0.0f;
float accelY = 0.0f;
float accelZ = 0.0f;

float gyroX = 0.0f;
float gyroY = 0.0f;
```

```
float gyroZ = 0.0f;

unsigned long lastSensorUpdateMs = 0;


// ================================================================
// ST-3009
// ================================================================
//
// IMPORTANT:
// SCServo 1.0.2 uses:
//
//     SMS_STS
//

SMS_STS st;


// ----------------------------------------------------------------
// Servo IDs
// ----------------------------------------------------------------

const uint8_t SERVO1_ID = 1;
const uint8_t SERVO2_ID = 2;


// ----------------------------------------------------------------
// Servo positions
// ----------------------------------------------------------------

const int SERVO_NEUTRAL = 2048;

const int SERVO_MIN_POS = 200;
const int SERVO_MAX_POS = 3895;


// ----------------------------------------------------------------
// Steering
// ----------------------------------------------------------------

const int MAX_STEERING_COUNTS = 450;

const int STEERING_SERVO_SPEED = 500;
const int STEERING_SERVO_ACCEL = 50;

const int MAX_SLEW_COUNTS = 60;


// ----------------------------------------------------------------
// Servo health
// ----------------------------------------------------------------

const unsigned long SERVO_HEALTH_INTERVAL_MS = 2000;

bool servo1OK = false;
bool servo2OK = false;
bool servoBusReady = false;

unsigned long lastServoHealthMs = 0;


// ----------------------------------------------------------------
// Current positions
// ----------------------------------------------------------------

int servo1Position = SERVO_NEUTRAL;
int servo2Position = SERVO_NEUTRAL;


// ----------------------------------------------------------------
// Current navigation demand
// ----------------------------------------------------------------

int steeringDemand = 0;

```

```
418
419 // ================================================================
420 // SG90 DEPLOYMENT
421 // ================================================================
422
423 Servo deployServo;
424
425 const uint8_t DEPLOY_SERVO_PIN = 8;
426
427 const int DEPLOY_ANGLE = 180;
428
429 const int PREP_ANGLE = 140;
430
431 const unsigned long DEPLOY_MOVE_TIME_MS = 1500;
432
433 const unsigned long DEPLOY_STABILIZE_TIME_MS = 5000;
434
435 bool deploymentServoActive = false;
436
437 bool deploymentDone = false;
438
439 unsigned long deploymentStartMs = 0;
440
441 unsigned long stabilizationStartMs = 0;
442
443
444 // ================================================================
445 // NAVIGATION
446 // ================================================================
447
448 const double TARGET_CAPTURE_RADIUS_M = 35.0;
449
450 const double STEERING_DEADBAND_DEG = 5.0;
451
452 const double FULL_STEERING_ERROR_DEG = 90.0;
453
454
455 // ================================================================
456 // STATE MACHINE
457 // ================================================================
458
459 enum SystemState
460 {
461   STATE_WAIT_READY,
462   STATE_DEPLOYING,
463   STATE_STABILIZING,
464   STATE_AUTONAV,
465   STATE_CAPTURED,
466   STATE_FAILSAFE,
467   STATE_MANUAL_TEST
468 };
469
470 SystemState state = STATE_WAIT_READY;
471
472 unsigned long stateEnteredMs = 0;
473
474
475 // ================================================================
476 // NAVIGATION VARIABLES
477 // ================================================================
478
479 double currentLatitude = 0.0;
480
481 double currentLongitude = 0.0;
482
483 double distanceToTarget = 0.0;
484
485 double targetBearing = 0.0;
486
487 double currentDirection = 0.0;
488
489 double courseError = 0.0;
490
491 bool headingValid = false;
492
```

```
493 const char *headingSource = "NONE";
494
495
496 // ================================================================
497 // FAILSAFE
498 // ================================================================
499
500 unsigned long failsafeGoodSinceMs = 0;
501
502 const unsigned long FAILSAFE_RECOVERY_MS = 3000;
503
504
505 // ================================================================
506 // SERIAL COMMAND BUFFER
507 // ================================================================
508
509 #define COMMAND_BUFFER_SIZE 120
510
511 char commandBuffer[COMMAND_BUFFER_SIZE];
512
513 uint16_t commandIndex = 0;
514
515
516 // ================================================================
517 // TIMERS
518 // ================================================================
519
520 unsigned long lastNavigationUpdateMs = 0;
521
522 const unsigned long NAVIGATION_UPDATE_INTERVAL_MS = 100;
523
524 unsigned long lastLivePrintMs = 0;
525
526 const unsigned long LIVE_PRINT_INTERVAL_MS = 1000;
527
528 unsigned long lastBMPCheckMs = 0;
529
530 const unsigned long BMP_CHECK_INTERVAL_MS = 3000;
531
532
533 // ================================================================
534 // MOTOR TEST
535 // ================================================================
536
537 bool motorTestActive = false;
538
539 uint8_t motorTestStep = 0;
540
541 unsigned long motorTestTimer = 0;
542
543
544 // ================================================================
545 // FORWARD DECLARATIONS
546 // ================================================================
547
548 void processSerialCommands();
549 void processCommand(char *cmd);
550
551 void updateGPS();
552 void updateCompass();
553 void updateIMU();
554 void updateServoHealth();
555
556 void updateNavigation();
557 void updateSteering();
558 void updateStateMachine();
559
560 void writeSteeringPositions(int pos1, int pos2);
561 void centerServos();
562 void commandSteering(int demand);
563
564 void startDeployment();
565 void prepareDeploymentServo();
566 void resetSystem();
567
```

```
void startMotorTest();
void updateMotorTest();

void printStatus();
void printLiveData();

bool gpsQualityOK();
bool readQMC5883();
bool initQMC5883();

bool checkI2CDevice(uint8_t address);
bool initIMU();

bool servosReady();

float wrap360(float angle);
double wrap180(double angle);
float navigationCompassHeading();

const char *stateName(SystemState s);

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep
);


// ================================================================
// SETUP
// ================================================================

void setup()
{
  // ------------------------------------------------------------
  // USB SERIAL
  // ------------------------------------------------------------

  Serial.begin(115200);

  delay(1500);


  Serial.println();
  Serial.println(
    "================================================"
  );

  Serial.println(
    " TEENSY 4.1 AUTONOMOUS PARAFOIL"
  );

  Serial.println(
    " GUIDED LANDING - FINAL LAB TEST"
  );

  Serial.println(
    "================================================"
  );


  // ------------------------------------------------------------
  // TARGET
  // ------------------------------------------------------------

  Serial.println();

  Serial.print(
    "TARGET LATITUDE  : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );
```

```
643
644
645   Serial.print(
646     "TARGET LONGITUDE : "
647   );
648
649   Serial.println(
650     TARGET_LON,
651     7
652   );
653
654
655   // ---------------------------------------------------------------
656   // I2C
657   // ---------------------------------------------------------------
658
659   Wire.begin();
660
661   Wire.setClock(400000);
662
663
664   Serial.println();
665
666   Serial.println(
667     "I2C initialized"
668   );
669
670   Serial.println(
671     "SDA = Pin 18"
672   );
673
674   Serial.println(
675     "SCL = Pin 19"
676   );
677
678
679   // ---------------------------------------------------------------
680   // QMC5883
681   // ---------------------------------------------------------------
682
683   if (initQMC5883())
684   {
685     compassOK = true;
686
687     Serial.println(
688       "QMC5883 : OK"
689     );
690   }
691   else
692   {
693     compassOK = false;
694
695     Serial.println(
696       "QMC5883 : NOT FOUND"
697     );
698   }
699
700
701   // ---------------------------------------------------------------
702   // LSM6DSO32
703   // ---------------------------------------------------------------
704
705   if (initIMU())
706   {
707     imuOK = true;
708
709     Serial.println(
710       "LSM6DSO32 : OK"
711     );
712   }
713   else
714   {
715     imuOK = false;
716
717     Serial.println(
```

```
      "LSM6DSO32 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // BMP581
  // ---------------------------------------------------------------

  if (checkI2CDevice(
        BMP581_ADDR_1))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_1;
  }
  else if (checkI2CDevice(
             BMP581_ADDR_2))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_2;
  }
  else
  {
    bmpOK = false;

    bmpAddress = 0;
  }


  if (bmpOK)
  {
    Serial.print(
      "BMP581 : FOUND at 0x"
    );

    if (bmpAddress < 16)
      Serial.print("0");

    Serial.println(
      bmpAddress,
      HEX
    );
  }
  else
  {
    Serial.println(
      "BMP581 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // GPS
  // ---------------------------------------------------------------
  //
  // EXACTLY MATCHES YOUR WORKING GPS TEST
  //

  GPS_SERIAL.begin(
    GPS_BAUD
  );


  Serial.println();

  Serial.println(
    "GPS UART  : Serial3"
  );

  Serial.print(
    "GPS RX    : Pin "
```

```
  );

  Serial.println(
    GPS_RX_PIN
  );

  Serial.print(
    "GPS TX    : Pin "
  );

  Serial.println(
    GPS_TX_PIN
  );

  Serial.print(
    "GPS BAUD  : "
  );

  Serial.println(
    GPS_BAUD
  );


  // ---------------------------------------------------------------
  // ST-3009
  // ---------------------------------------------------------------

  Serial.println();

  Serial.println(
    "Initializing ST-3009 bus..."
  );


  Serial1.begin(
    1000000,
    SERIAL_8N1_HALF_DUPLEX
  );


  st.pSerial =
    &Serial1;


  delay(500);


  // Force immediate first health test
  lastServoHealthMs =
    millis() -
    SERVO_HEALTH_INTERVAL_MS;


  updateServoHealth();


  if (servosReady())
  {
    Serial.println(
      "ST-3009 ID 1 : OK"
    );

    Serial.println(
      "ST-3009 ID 2 : OK"
    );


    centerServos();
  }
  else
  {
    Serial.println(
      "ST-3009 : WAITING"
    );
  }
```

```
868
869
870   // ---------------------------------------------------------------
871   // SG90
872   // ---------------------------------------------------------------
873
874   deployServo.detach();
875
876   deploymentServoActive =
877     false;
878
879
880   // ---------------------------------------------------------------
881   // State
882   // ---------------------------------------------------------------
883
884   state =
885     STATE_WAIT_READY;
886
887   stateEnteredMs =
888     millis();
889
890
891   Serial.println();
892
893   Serial.println(
894     "================================================"
895   );
896
897   Serial.println(
898     "SYSTEM READY"
899   );
900
901   Serial.println(
902     "Target is fixed in code."
903   );
904
905   Serial.println(
906     "================================================"
907   );
908
909
910   Serial.println();
911
912   Serial.println(
913     "COMMANDS:"
914   );
915
916   Serial.println(
917     "STATUS"
918   );
919
920   Serial.println(
921     "MOTORTEST"
922   );
923
924   Serial.println(
925     "LEFT"
926   );
927
928   Serial.println(
929     "RIGHT"
930   );
931
932   Serial.println(
933     "CENTER"
934   );
935
936   Serial.println(
937     "AUTONAV"
938   );
939
940   Serial.println(
941     "DEPLOY"
942   );
```

```
943
944   Serial.println(
945     "PREP"
946   );
947
948   Serial.println(
949     "RESET"
950   );
951
952   Serial.println(
953     "HELP"
954   );
955
956
957   Serial.println();
958
959   Serial.println(
960     "Waiting for GPS + ST-3009..."
961   );
962 }
963
964
965 // ================================================================
966 // LOOP
967 // ================================================================
968
969 void loop()
970 {
971   // GPS continuously
972   updateGPS();
973
974
975   // Serial commands
976   processSerialCommands();
977
978
979   // Sensors
980   if (millis() -
981       lastSensorUpdateMs >=
982       100)
983   {
984     lastSensorUpdateMs =
985       millis();
986
987     updateCompass();
988     updateIMU();
989   }
990
991
992   // Servo health
993   updateServoHealth();
994
995
996   // Motor test
997   if (motorTestActive)
998   {
999     updateMotorTest();
1000  }
1001
1002
1003  // State
1004  updateStateMachine();
1005
1006
1007  // Navigation
1008  if (millis() -
1009      lastNavigationUpdateMs >=
1010      NAVIGATION_UPDATE_INTERVAL_MS)
1011  {
1012    lastNavigationUpdateMs =
1013      millis();
1014
1015    updateNavigation();
1016    updateSteering();
1017  }
```

```

  // BMP check
  if (millis() -
      lastBMPCheckMs >=
      BMP_CHECK_INTERVAL_MS)
  {
    lastBMPCheckMs =
      millis();


    if (!bmpOK)
    {
      if (checkI2CDevice(
            BMP581_ADDR_1))
      {
        bmpOK = true;

        bmpAddress =
          BMP581_ADDR_1;
      }
      else if (checkI2CDevice(
                 BMP581_ADDR_2))
      {
        bmpOK = true;

        bmpAddress =
          BMP581_ADDR_2;
      }
    }
  }


  // Live data
  if (millis() -
      lastLivePrintMs >=
      LIVE_PRINT_INTERVAL_MS)
  {
    lastLivePrintMs =
      millis();

    printLiveData();
  }
}


// ================================================================
// GPS UPDATE
// ================================================================

void updateGPS()
{
  while (GPS_SERIAL.available() > 0)
  {
    char c =
      (char)GPS_SERIAL.read();

    gps.encode(c);
  }


  // Fresh GPS location
  if (gps.location.isUpdated())
  {
    lastGPSLocationMs =
      millis();
  }


  // Position
  if (gps.location.isValid())
  {
    gpsFix = true;

    currentLatitude =
```

```
      gps.location.lat();

    currentLongitude =
      gps.location.lng();
  }
  else
  {
    gpsFix = false;
  }


  // Satellites
  if (gps.satellites.isValid())
  {
    gpsSats =
      gps.satellites.value();
  }
  else
  {
    gpsSats = 0;
  }


  // HDOP
  if (gps.hdop.isValid())
  {
    gpsHDOP =
      gps.hdop.hdop();
  }
  else
  {
    gpsHDOP = 99.9;
  }


  // Speed
  if (gps.speed.isValid())
  {
    currentSpeed =
      gps.speed.mps();
  }
  else
  {
    currentSpeed = 0.0;
  }
}


// ================================================================
// GPS QUALITY
// ================================================================

bool gpsQualityOK()
{
  if (!gpsFix)
    return false;


  if (lastGPSLocationMs == 0)
    return false;


  if (millis() -
      lastGPSLocationMs >
      GPS_DATA_TIMEOUT_MS)
  {
    return false;
  }


  if (gpsSats <
      GPS_MIN_SATS)
  {
    return false;
  }
```

```
1168
1169
1170    if (gps.hdop.isValid() &&
1171        gpsHDOP >
1172        GPS_MAX_HDOP)
1173    {
1174      return false;
1175    }
1176
1177
1178    return true;
1179  }
1180
1181
1182  // ================================================================
1183  // QMC5883 INIT
1184  // ================================================================
1185
1186  bool initQMC5883()
1187  {
1188    // Check device
1189    Wire.beginTransmission(
1190      QMC5883_ADDR
1191    );
1192
1193
1194    if (Wire.endTransmission() != 0)
1195    {
1196      return false;
1197    }
1198
1199
1200    // SET/RESET period
1201    Wire.beginTransmission(
1202      QMC5883_ADDR
1203    );
1204
1205    Wire.write(0x0B);
1206    Wire.write(0x01);
1207
1208    if (Wire.endTransmission() != 0)
1209    {
1210      return false;
1211    }
1212
1213
1214    delay(10);
1215
1216
1217    // Control register
1218    // Same setting used by your
1219    // working compass test.
1220    Wire.beginTransmission(
1221      QMC5883_ADDR
1222    );
1223
1224    Wire.write(0x09);
1225    Wire.write(0x1D);
1226
1227    if (Wire.endTransmission() != 0)
1228    {
1229      return false;
1230    }
1231
1232
1233    delay(100);
1234
1235
1236    return true;
1237  }
1238
1239
1240  // ================================================================
1241  // QMC5883 READ
1242  // ================================================================
```

```

bool readQMC5883()
{
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x00);


  if (Wire.endTransmission(false) != 0)
  {
    return false;
  }


  uint8_t count =
    Wire.requestFrom(
      QMC5883_ADDR,
      (uint8_t)6
    );


  if (count != 6)
  {
    while (Wire.available())
    {
      Wire.read();
    }

    return false;
  }


  uint8_t xl =
    Wire.read();

  uint8_t xh =
    Wire.read();


  uint8_t yl =
    Wire.read();

  uint8_t yh =
    Wire.read();


  uint8_t zl =
    Wire.read();

  uint8_t zh =
    Wire.read();


  rawMagX =
    (int16_t)(
      (xh << 8) |
      xl
    );


  rawMagY =
    (int16_t)(
      (yh << 8) |
      yl
    );


  rawMagZ =
    (int16_t)(
      (zh << 8) |
      zl
    );

```

```
1318
1319    return true;
1320  }
1321
1322
1323  // ================================================================
1324  // COMPASS UPDATE
1325  // ================================================================
1326
1327  void updateCompass()
1328  {
1329    // ------------------------------------------------------------
1330    // Read sensor
1331    // ------------------------------------------------------------
1332
1333    if (!readQMC5883())
1334    {
1335      compassFailCount++;
1336
1337
1338      // Do not immediately declare compass dead.
1339      // Reinitialize after 5 consecutive failures.
1340
1341      if (compassFailCount >= 5)
1342      {
1343        if (initQMC5883())
1344        {
1345          compassOK = true;
1346
1347          compassFailCount = 0;
1348
1349          Serial.println(
1350            "[COMPASS] Reinitialized OK"
1351          );
1352        }
1353        else
1354        {
1355          compassOK = false;
1356        }
1357      }
1358
1359
1360      return;
1361    }
1362
1363
1364    // Successful reading
1365    compassFailCount = 0;
1366    compassOK = true;
1367
1368
1369    // ------------------------------------------------------------
1370    // Apply your proven calibration
1371    // ------------------------------------------------------------
1372
1373    magX =
1374      ((float)rawMagX -
1375       OFFSET_X) *
1376      SCALE_X;
1377
1378
1379    magY =
1380      ((float)rawMagY -
1381       OFFSET_Y) *
1382      SCALE_Y;
1383
1384
1385    magZ =
1386      ((float)rawMagZ -
1387       OFFSET_Z) *
1388      SCALE_Z;
1389
1390
1391    // ------------------------------------------------------------
1392    // Calculate heading
```

```
  // ------------------------------------------------------------

  float angle =
    atan2(
      magY,
      magX
    ) *
    180.0f /
    PI;


  if (angle < 0.0f)
    angle += 360.0f;


  compassHeading =
    wrap360(
      angle +
      COMPASS_HEADING_OFFSET_DEG
    );


  // Navigation heading
  navCompassHeading =
    navigationCompassHeading();
}


// ================================================================
// NAVIGATION COMPASS
// ================================================================

float navigationCompassHeading()
{
  if (!COMPASS_MIRROR_FOR_NAV)
  {
    return compassHeading;
  }


  return wrap360(
    360.0f -
    compassHeading
  );
}


// ================================================================
// WRAP 0...360
// ================================================================

float wrap360(float angle)
{
  while (angle >= 360.0f)
    angle -= 360.0f;


  while (angle < 0.0f)
    angle += 360.0f;


  return angle;
}


// ================================================================
// WRAP -180...180
// ================================================================

double wrap180(double angle)
{
  while (angle > 180.0)
    angle -= 360.0;


```

```
  while (angle < -180.0)
    angle += 360.0;


  return angle;
}


// ================================================================
// IMU INIT
// ================================================================

bool initIMU()
{
  if (!lsm6dso.begin_I2C(
        LSM6DSO32_ADDR,
        &Wire))
  {
    return false;
  }


  lsm6dso.setAccelRange(
    LSM6DS_ACCEL_RANGE_16_G
  );


  lsm6dso.setGyroRange(
    LSM6DS_GYRO_RANGE_2000_DPS
  );


  lsm6dso.setAccelDataRate(
    LSM6DS_RATE_208_HZ
  );


  lsm6dso.setGyroDataRate(
    LSM6DS_RATE_208_HZ
  );


  return true;
}


// ================================================================
// IMU UPDATE
// ================================================================

void updateIMU()
{
  if (!imuOK)
    return;


  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;


  lsm6dso.getEvent(
    &accel,
    &gyro,
    &temp
  );


  accelX =
    accel.acceleration.x;


  accelY =
    accel.acceleration.y;

```

```
  accelZ =
    accel.acceleration.z;


  gyroX =
    gyro.gyro.x;


  gyroY =
    gyro.gyro.y;


  gyroZ =
    gyro.gyro.z;
}


// ================================================================
// I2C CHECK
// ================================================================

bool checkI2CDevice(
  uint8_t address)
{
  Wire.beginTransmission(
    address
  );


  return (
    Wire.endTransmission() == 0
  );
}


// ================================================================
// SERVO HEALTH
// ================================================================

void updateServoHealth()
{
  if (millis() -
      lastServoHealthMs <
      SERVO_HEALTH_INTERVAL_MS)
  {
    return;
  }


  lastServoHealthMs =
    millis();


  // Ping ID 1
  int result1 =
    st.Ping(
      SERVO1_ID
    );


  delay(10);


  // Ping ID 2
  int result2 =
    st.Ping(
      SERVO2_ID
    );


  servo1OK =
    (result1 >= 0);


```

```
  servo2OK =
    (result2 >= 0);


  servoBusReady =
    servo1OK &&
    servo2OK;
}


// ================================================================
// SERVO READY
// ================================================================

bool servosReady()
{
  return
    servoBusReady &&
    servo1OK &&
    servo2OK;
}


// ================================================================
// CENTER BOTH SERVOS
// ================================================================

void centerServos()
{
  if (!servosReady())
    return;


  st.WritePosEx(
    SERVO1_ID,
    SERVO_NEUTRAL,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  st.WritePosEx(
    SERVO2_ID,
    SERVO_NEUTRAL,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  servo1Position =
    SERVO_NEUTRAL;


  servo2Position =
    SERVO_NEUTRAL;


  steeringDemand = 0;
}


// ================================================================
// DIRECT STEERING COMMAND
// ================================================================
//
// demand = +300:
//
//   Servo 1 = 2048 - 300 = 1748
//   Servo 2 = 2048 + 300 = 2348
//
// demand = -300:
//
//   Servo 1 = 2048 + 300 = 2348
//   Servo 2 = 2048 - 300 = 1748
//
```

```

void commandSteering(
  int demand)
{
  if (!servosReady())
  {
    Serial.println(
      "ERROR: ST-3009 BUS NOT READY"
    );

    return;
  }


  demand =
    constrain(
      demand,
      -MAX_STEERING_COUNTS,
      MAX_STEERING_COUNTS
    );


  int pos1 =
    SERVO_NEUTRAL -
    demand;


  int pos2 =
    SERVO_NEUTRAL +
    demand;


  pos1 =
    constrain(
      pos1,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


  pos2 =
    constrain(
      pos2,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


  st.WritePosEx(
    SERVO1_ID,
    pos1,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  st.WritePosEx(
    SERVO2_ID,
    pos2,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  servo1Position =
    pos1;


  servo2Position =
    pos2;


  steeringDemand =
    demand;

```

```
  Serial.print(
    "DEMAND="
  );

  Serial.print(
    demand
  );

  Serial.print(
    "  S1="
  );

  Serial.print(
    pos1
  );

  Serial.print(
    "  S2="
  );

  Serial.println(
    pos2
  );
}


// ================================================================
// MOTOR TEST START
// ================================================================

void startMotorTest()
{
  if (!servosReady())
  {
    Serial.println(
      "MOTORTEST BLOCKED: "
      "ST-3009 NOT READY"
    );

    return;
  }


  motorTestActive =
    true;


  state =
    STATE_MANUAL_TEST;


  motorTestStep = 0;

  motorTestTimer =
    millis();


  centerServos();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "       ST-3009 MOTOR TEST"
  );

  Serial.println(
    "========================================"
  );

```

```
  Serial.println(
    "CENTER -> LEFT -> CENTER -> RIGHT -> CENTER"
  );

  Serial.println();
}


// ================================================================
// MOTOR TEST UPDATE
// ================================================================

void updateMotorTest()
{
  if (!servosReady())
  {
    motorTestActive =
      false;

    state =
      STATE_WAIT_READY;

    return;
  }


  if (millis() -
      motorTestTimer <
      1800)
  {
    return;
  }


  motorTestTimer =
    millis();


  switch (motorTestStep)
  {
    case 0:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    case 1:

      Serial.println(
        "TEST: LEFT"
      );

      commandSteering(
        +300
      );

      motorTestStep++;

      break;


    case 2:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);
```

```
      motorTestStep++;

      break;


    case 3:

      Serial.println(
        "TEST: RIGHT"
      );

      commandSteering(
        -300
      );

      motorTestStep++;

      break;


    case 4:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    default:

      motorTestActive =
        false;


      state =
        STATE_WAIT_READY;


      centerServos();


      Serial.println();

      Serial.println(
        "MOTOR TEST COMPLETE"
      );

      Serial.println();

      break;
  }
}


// ================================================================
// NAVIGATION CALCULATION
// ================================================================

void updateNavigation()
{
  if (!gpsQualityOK())
  {
    headingValid =
      false;

    headingSource =
      "NONE";

    return;
```

```
}


// ---------------------------------------------------------------
// GPS position
// ---------------------------------------------------------------

currentLatitude =
  gps.location.lat();


currentLongitude =
  gps.location.lng();


// ---------------------------------------------------------------
// Distance
// ---------------------------------------------------------------

distanceToTarget =
  TinyGPSPlus::distanceBetween(
    currentLatitude,
    currentLongitude,
    TARGET_LAT,
    TARGET_LON
  );


// ---------------------------------------------------------------
// Bearing
// ---------------------------------------------------------------

targetBearing =
  TinyGPSPlus::courseTo(
    currentLatitude,
    currentLongitude,
    TARGET_LAT,
    TARGET_LON
  );


targetBearing =
  wrap360(
    targetBearing
  );


// ---------------------------------------------------------------
// Determine heading
// ---------------------------------------------------------------

headingValid =
  false;


headingSource =
  "NONE";


// GPS course when moving
if (gps.speed.isValid() &&
    gps.course.isValid() &&
    currentSpeed >=
    GPS_COURSE_MIN_SPEED_MPS)
{
  currentDirection =
    wrap360(
      gps.course.deg()
    );


  headingValid =
    true;

```

```
    headingSource =
      "GPS COURSE";
  }


  // Compass when slow/stationary
  else if (compassOK)
  {
    currentDirection =
      navigationCompassHeading();


    headingValid =
      true;


    headingSource =
      "COMPASS";
  }


  // ------------------------------------------------------------
  // Course error
  // ------------------------------------------------------------

  if (headingValid)
  {
    courseError =
      wrap180(
        targetBearing -
        currentDirection
      );
  }
}


// ====================================================================
// AUTONOMOUS STEERING
// ====================================================================

void updateSteering()
{
  // Manual motor test owns the servos
  if (motorTestActive)
    return;


  // Servo unavailable
  if (!servosReady())
  {
    steeringDemand = 0;

    return;
  }


  // Only steer during AUTONAV
  if (state !=
      STATE_AUTONAV)
  {
    return;
  }


  // GPS required
  if (!gpsQualityOK())
  {
    centerServos();

    return;
  }


  // Target captured
  if (distanceToTarget <=
```

```
    TARGET_CAPTURE_RADIUS_M)
{
  centerServos();

  return;
}


// No heading
if (!headingValid)
{
  centerServos();

  return;
}


// ------------------------------------------------------------
// Absolute error
// ------------------------------------------------------------

double absoluteError =
  fabs(courseError);


// ------------------------------------------------------------
// Deadband
// ------------------------------------------------------------

if (absoluteError <=
    STEERING_DEADBAND_DEG)
{
  steeringDemand = 0;
}
else
{
  double usableError =
    absoluteError -
    STEERING_DEADBAND_DEG;


  double usableRange =
    FULL_STEERING_ERROR_DEG -
    STEERING_DEADBAND_DEG;


  double normalized =
    usableError /
    usableRange;


  normalized =
    constrain(
      normalized,
      0.0,
      1.0
    );


  int demand =
    (int)round(
      normalized *
      MAX_STEERING_COUNTS
    );


  if (courseError > 0.0)
  {
    steeringDemand =
      demand;
  }
  else
  {
    steeringDemand =
      -demand;
```

```
  }
}


// ---------------------------------------------------------
// Target positions
// ---------------------------------------------------------

int target1 =
  SERVO_NEUTRAL -
  steeringDemand;


int target2 =
  SERVO_NEUTRAL +
  steeringDemand;


target1 =
  constrain(
    target1,
    SERVO_MIN_POS,
    SERVO_MAX_POS
  );


target2 =
  constrain(
    target2,
    SERVO_MIN_POS,
    SERVO_MAX_POS
  );


// ---------------------------------------------------------
// Slew limit
// ---------------------------------------------------------

int new1 =
  slewTo(
    servo1Position,
    target1,
    MAX_SLEW_COUNTS
  );


int new2 =
  slewTo(
    servo2Position,
    target2,
    MAX_SLEW_COUNTS
  );


// ---------------------------------------------------------
// Send only if changed
// ---------------------------------------------------------

if (new1 != servo1Position ||
    new2 != servo2Position)
{
  st.WritePosEx(
    SERVO1_ID,
    new1,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  st.WritePosEx(
    SERVO2_ID,
    new2,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );
```

```
2293
2294
2295      servo1Position =
2296        new1;
2297
2298
2299      servo2Position =
2300        new2;
2301    }
2302  }
2303
2304
2305  // ================================================================
2306  // SLEW LIMIT
2307  // ================================================================
2308
2309  int slewTo(
2310    int currentValue,
2311    int targetValue,
2312    int maxStep)
2313  {
2314    if (targetValue >
2315        currentValue)
2316    {
2317      int difference =
2318        targetValue -
2319        currentValue;
2320
2321
2322      if (difference >
2323          maxStep)
2324      {
2325        return currentValue +
2326               maxStep;
2327      }
2328
2329
2330      return targetValue;
2331    }
2332
2333
2334    if (targetValue <
2335        currentValue)
2336    {
2337      int difference =
2338        currentValue -
2339        targetValue;
2340
2341
2342      if (difference >
2343          maxStep)
2344      {
2345        return currentValue -
2346               maxStep;
2347      }
2348
2349
2350      return targetValue;
2351    }
2352
2353
2354    return currentValue;
2355  }
2356
2357
2358  // ================================================================
2359  // DEPLOYMENT
2360  // ================================================================
2361
2362  void startDeployment()
2363  {
2364    if (deploymentDone)
2365    {
2366      Serial.println(
2367        "DEPLOYMENT ALREADY COMPLETED"
```

```
    );

    return;
  }


  if (!gpsQualityOK())
  {
    Serial.println(
      "DEPLOY BLOCKED: GPS NOT READY"
    );

    return;
  }


  if (!servosReady())
  {
    Serial.println(
      "DEPLOY BLOCKED: ST-3009 NOT READY"
    );

    return;
  }


  // Center steering first
  centerServos();


  delay(100);


  // Attach SG90
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );


  delay(100);


  // Deployment position
  deployServo.write(
    DEPLOY_ANGLE
  );


  deploymentStartMs =
    millis();


  deploymentServoActive =
    true;


  deploymentDone =
    true;


  state =
    STATE_DEPLOYING;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
```

```
    "STARTING DEPLOYMENT"
  );

  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    DEPLOY_ANGLE
  );

  Serial.println(
    " degrees"
  );
}


// ================================================================
// SG90 PREP
// ================================================================

void prepareDeploymentServo()
{
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );


  delay(100);


  deployServo.write(
    PREP_ANGLE
  );


  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    PREP_ANGLE
  );

  Serial.println(
    " degrees"
  );


  delay(1000);


  deployServo.detach();


  Serial.println(
    "SG90 detached"
  );
}


// ================================================================
// STATE MACHINE
// ================================================================

void updateStateMachine()
{
  // ------------------------------------------------------------
  // Manual test
  // ------------------------------------------------------------

  if (state ==
      STATE_MANUAL_TEST)
  {
    return;
```

```
  }


  // ------------------------------------------------------------
  // WAIT READY
  // ------------------------------------------------------------

  if (state ==
      STATE_WAIT_READY)
  {
    if (AUTO_DEPLOY_AFTER_READY &&
        !deploymentDone &&
        gpsQualityOK() &&
        servosReady())
    {
      Serial.println();

      Serial.println(
        "GPS + ST-3009 READY"
      );

      Serial.println(
        "AUTO DEPLOY ENABLED"
      );


      startDeployment();
    }


    return;
  }


  // ------------------------------------------------------------
  // DEPLOYING
  // ------------------------------------------------------------

  if (state ==
      STATE_DEPLOYING)
  {
    steeringDemand =
      0;


    if (millis() -
        deploymentStartMs >=
        DEPLOY_MOVE_TIME_MS)
    {
      // Detach SG90
      deployServo.detach();


      deploymentServoActive =
        false;


      // Center steering
      if (servosReady())
      {
        centerServos();
      }


      // Stabilization starts
      stabilizationStartMs =
        millis();


      state =
        STATE_STABILIZING;


      Serial.println();

```

```
      Serial.println(
        "SG90 detached"
      );

      Serial.println(
        "5 second stabilization started"
      );
    }


    return;
  }


  // ------------------------------------------------------------
  // STABILIZING
  // ------------------------------------------------------------

  if (state ==
      STATE_STABILIZING)
  {
    steeringDemand =
      0;


    if (millis() -
        stabilizationStartMs >=
        DEPLOY_STABILIZE_TIME_MS)
    {
      if (gpsQualityOK() &&
          servosReady())
      {
        state =
          STATE_AUTONAV;


        Serial.println();

        Serial.println(
          "========================================"
        );

        Serial.println(
          "AUTONOMOUS NAVIGATION STARTED"
        );

        Serial.println(
          "========================================"
        );
      }
    }


    return;
  }


  // ------------------------------------------------------------
  // AUTONAV
  // ------------------------------------------------------------

  if (state ==
      STATE_AUTONAV)
  {
    // GPS / servo failure
    if (!gpsQualityOK() ||
        !servosReady())
    {
      steeringDemand =
        0;


      if (servosReady())
      {
        centerServos();
```

```
      }


      state =
        STATE_FAILSAFE;


      failsafeGoodSinceMs =
        0;


      Serial.println();

      Serial.println(
        "!!! NAVIGATION FAILSAFE !!!"
      );

      Serial.println(
        "GPS or ST-3009 unavailable"
      );


      return;
    }


    // Target capture
    if (distanceToTarget <=
        TARGET_CAPTURE_RADIUS_M)
    {
      centerServos();


      state =
        STATE_CAPTURED;


      Serial.println();

      Serial.println(
        "========================================"
      );

      Serial.println(
        "TARGET CAPTURED"
      );

      Serial.print(
        "Distance = "
      );

      Serial.print(
        distanceToTarget,
        2
      );

      Serial.println(
        " m"
      );

      Serial.println(
        "ST-3009 -> CENTER"
      );

      Serial.println(
        "========================================"
      );
    }


    return;
  }


  // ------------------------------------------------------------
```

```
  // CAPTURED
  // ---------------------------------------------------------------

  if (state ==
      STATE_CAPTURED)
  {
    steeringDemand =
      0;

    return;
  }


  // ---------------------------------------------------------------
  // FAILSAFE
  // ---------------------------------------------------------------

  if (state ==
      STATE_FAILSAFE)
  {
    steeringDemand =
      0;


    if (servosReady())
    {
      centerServos();
    }


    if (gpsQualityOK() &&
        servosReady())
    {
      if (failsafeGoodSinceMs ==
          0)
      {
        failsafeGoodSinceMs =
          millis();


        Serial.println(
          "GPS + ST-3009 recovered"
        );


        Serial.println(
          "Waiting 3 seconds..."
        );
      }
      else if (millis() -
               failsafeGoodSinceMs >=
               FAILSAFE_RECOVERY_MS)
      {
        state =
          STATE_AUTONAV;


        failsafeGoodSinceMs =
          0;


        Serial.println(
          "FAILSAFE RECOVERY COMPLETE"
        );


        Serial.println(
          "AUTONAV RESUMED"
        );
      }
    }
    else
    {
      failsafeGoodSinceMs =
        0;
```

```
    }


    return;
  }
}


// ================================================================
// SERIAL INPUT
// ================================================================

void processSerialCommands()
{
  while (Serial.available() > 0)
  {
    char c =
      (char)Serial.read();


    // ------------------------------------------------------------
    // Newline
    // ------------------------------------------------------------

    if (c == '\r' ||
        c == '\n')
    {
      if (commandIndex > 0)
      {
        commandBuffer[
          commandIndex] =
          '\0';


        processCommand(
          commandBuffer
        );


        commandIndex =
          0;
      }


      continue;
    }


    // ------------------------------------------------------------
    // Store
    // ------------------------------------------------------------

    if (commandIndex <
        COMMAND_BUFFER_SIZE - 1)
    {
      commandBuffer[
        commandIndex++] =
        c;
    }
    else
    {
      commandIndex = 0;

      Serial.println(
        "ERROR: COMMAND TOO LONG"
      );
    }
  }
}


// ================================================================
// COMMAND PROCESSOR
// ================================================================

```

```
void processCommand(
  char *cmd)
{
  // Remove leading spaces
  while (*cmd == ' ' ||
         *cmd == '\t')
  {
    cmd++;
  }


  // Remove trailing spaces
  int len =
    strlen(cmd);


  while (len > 0 &&
         (cmd[len - 1] == ' ' ||
          cmd[len - 1] == '\t'))
  {
    cmd[len - 1] =
      '\0';

    len--;
  }


  if (len == 0)
    return;


  Serial.print(
    "CMD RECEIVED: ["
  );

  Serial.print(
    cmd
  );

  Serial.println(
    "]"
  );


  // ============================================================
  // STATUS
  // ============================================================

  if (strcmp(
        cmd,
        "STATUS") == 0)
  {
    printStatus();
    return;
  }


  // ============================================================
  // MOTORTEST
  // ============================================================

  if (strcmp(
        cmd,
        "MOTORTEST") == 0)
  {
    startMotorTest();
    return;
  }


  // ============================================================
  // LEFT
  // ============================================================

  if (strcmp(
```

```
      cmd,
      "LEFT") == 0)
{
  motorTestActive =
    false;


  state =
    STATE_MANUAL_TEST;


  commandSteering(
    +300
  );


  return;
}


// ============================================================
// RIGHT
// ============================================================

if (strcmp(
      cmd,
      "RIGHT") == 0)
{
  motorTestActive =
    false;


  state =
    STATE_MANUAL_TEST;


  commandSteering(
    -300
  );


  return;
}


// ============================================================
// CENTER
// ============================================================

if (strcmp(
      cmd,
      "CENTER") == 0)
{
  motorTestActive =
    false;


  if (servosReady())
  {
    centerServos();


    Serial.println(
      "ST-3009 -> CENTER"
    );
  }
  else
  {
    Serial.println(
      "ST-3009 BUS NOT READY"
    );
  }


  state =
```

```
      STATE_WAIT_READY;


    return;
  }


  // ============================================================
  // AUTONAV
  // ============================================================

  if (strcmp(
        cmd,
        "AUTONAV") == 0)
  {
    motorTestActive =
      false;


    if (gpsQualityOK() &&
        servosReady())
    {
      state =
        STATE_AUTONAV;


      Serial.println(
        "AUTONAV ENABLED"
      );
    }
    else
    {
      Serial.println(
        "AUTONAV BLOCKED"
      );


      Serial.print(
        "GPS = "
      );

      Serial.println(
        gpsQualityOK()
          ? "GOOD"
          : "BAD"
      );


      Serial.print(
        "SERVO = "
      );

      Serial.println(
        servosReady()
          ? "GOOD"
          : "BAD"
      );
    }


    return;
  }


  // ============================================================
  // DEPLOY
  // ============================================================

  if (strcmp(
        cmd,
        "DEPLOY") == 0)
  {
    motorTestActive =
      false;

```

```
    startDeployment();

    return;
  }


  // ============================================================
  // PREP
  // ============================================================

  if (strcmp(
        cmd,
        "PREP") == 0)
  {
    motorTestActive =
      false;


    prepareDeploymentServo();

    return;
  }


  // ============================================================
  // RESET
  // ============================================================

  if (strcmp(
        cmd,
        "RESET") == 0)
  {
    motorTestActive =
      false;


    resetSystem();

    return;
  }


  // ============================================================
  // HELP
  // ============================================================

  if (strcmp(
        cmd,
        "HELP") == 0)
  {
    Serial.println();

    Serial.println(
      "STATUS"
    );

    Serial.println(
      "MOTORTEST"
    );

    Serial.println(
      "LEFT"
    );

    Serial.println(
      "RIGHT"
    );

    Serial.println(
      "CENTER"
    );

    Serial.println(
      "AUTONAV"
```

```
    );

    Serial.println(
      "DEPLOY"
    );

    Serial.println(
      "PREP"
    );

    Serial.println(
      "RESET"
    );

    Serial.println(
      "HELP"
    );


    Serial.println();

    Serial.println(
      "TARGET IS FIXED IN CODE"
    );


    return;
  }


  // Unknown
  Serial.print(
    "UNKNOWN COMMAND: "
  );

  Serial.println(
    cmd
  );
}


// ================================================================
// RESET
// ================================================================

void resetSystem()
{
  // Stop deployment servo
  if (deploymentServoActive)
  {
    deployServo.detach();

    deploymentServoActive =
      false;
  }


  // Stop motor test
  motorTestActive =
    false;


  // Center if available
  if (servosReady())
  {
    centerServos();
  }


  // Reset flags
  deploymentDone =
    false;


  steeringDemand =
```

```
    0;


  failsafeGoodSinceMs =
    0;


  // Return to wait state
  state =
    STATE_WAIT_READY;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "SYSTEM RESET"
  );

  Serial.print(
    "TARGET = "
  );

  Serial.print(
    TARGET_LAT,
    7
  );

  Serial.print(
    ", "
  );

  Serial.println(
    TARGET_LON,
    7
  );

  Serial.println(
    "========================================"
  );
}


// ================================================================
// STATUS
// ================================================================

void printStatus()
{
  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "                    STATUS"
  );

  Serial.println(
    "================================================"
  );


  // System
  Serial.print(
    "State             : "
  );
```

```
3343
3344  Serial.println(
3345    stateName(state)
3346  );
3347
3348
3349  // Target
3350  Serial.print(
3351    "Target latitude   : "
3352  );
3353
3354  Serial.println(
3355    TARGET_LAT,
3356    7
3357  );
3358
3359
3360  Serial.print(
3361    "Target longitude  : "
3362  );
3363
3364  Serial.println(
3365    TARGET_LON,
3366    7
3367  );
3368
3369
3370  // GPS
3371  Serial.print(
3372    "GPS fix           : "
3373  );
3374
3375  Serial.println(
3376    gpsFix
3377      ? "YES"
3378      : "NO"
3379  );
3380
3381
3382  Serial.print(
3383    "GPS quality       : "
3384  );
3385
3386  Serial.println(
3387    gpsQualityOK()
3388      ? "GOOD"
3389      : "BAD"
3390  );
3391
3392
3393  Serial.print(
3394    "Satellites        : "
3395  );
3396
3397  Serial.println(
3398    gpsSats
3399  );
3400
3401
3402  Serial.print(
3403    "HDOP              : "
3404  );
3405
3406  Serial.println(
3407    gpsHDOP,
3408    2
3409  );
3410
3411
3412  Serial.print(
3413    "Latitude          : "
3414  );
3415
3416  Serial.println(
3417    currentLatitude,
```

```
    7
  );


  Serial.print(
    "Longitude         : "
  );

  Serial.println(
    currentLongitude,
    7
  );


  Serial.print(
    "Speed             : "
  );

  Serial.print(
    currentSpeed,
    2
  );

  Serial.println(
    " m/s"
  );


  // Navigation
  Serial.print(
    "Distance to target: "
  );

  Serial.print(
    distanceToTarget,
    2
  );

  Serial.println(
    " m"
  );


  Serial.print(
    "Target bearing    : "
  );

  Serial.print(
    targetBearing,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Current heading   : "
  );

  Serial.print(
    currentDirection,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Heading source    : "
  );

```

```
  Serial.println(
    headingSource
  );


  Serial.print(
    "Course error      : "
  );

  Serial.print(
    courseError,
    2
  );

  Serial.println(
    " deg"
  );


  // Compass
  Serial.print(
    "Compass status    : "
  );

  Serial.println(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Raw magnetometer  : "
  );

  Serial.print(
    rawMagX
  );

  Serial.print(
    ", "
  );

  Serial.print(
    rawMagY
  );

  Serial.print(
    ", "
  );

  Serial.println(
    rawMagZ
  );


  Serial.print(
    "Compass heading   : "
  );

  Serial.print(
    compassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Navigation heading: "
  );

  Serial.print(
```

```
    navCompassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  // Servos
  Serial.print(
    "Servo 1           : "
  );

  Serial.println(
    servo1OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 2           : "
  );

  Serial.println(
    servo2OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 1 position  : "
  );

  Serial.println(
    servo1Position
  );


  Serial.print(
    "Servo 2 position  : "
  );

  Serial.println(
    servo2Position
  );


  Serial.print(
    "Steering demand   : "
  );

  Serial.println(
    steeringDemand
  );


  // IMU
  Serial.print(
    "IMU               : "
  );

  Serial.println(
    imuOK
      ? "OK"
      : "FAIL"
  );


  // BMP
  Serial.print(
    "BMP581            : "
  );
```

```
3643
3644
3645    if (bmpOK)
3646    {
3647      Serial.print(
3648        "OK at 0x"
3649      );
3650
3651
3652      if (bmpAddress < 16)
3653        Serial.print("0");
3654
3655
3656      Serial.println(
3657        bmpAddress,
3658        HEX
3659      );
3660    }
3661    else
3662    {
3663      Serial.println(
3664        "NOT FOUND"
3665      );
3666    }
3667
3668
3669    // Deployment
3670    Serial.print(
3671      "Deployment done   : "
3672    );
3673
3674    Serial.println(
3675      deploymentDone
3676        ? "YES"
3677        : "NO"
3678    );
3679
3680
3681    Serial.println(
3682      "================================================"
3683    );
3684  }
3685
3686
3687  // ================================================================
3688  // LIVE DATA
3689  // ================================================================
3690
3691  void printLiveData()
3692  {
3693    Serial.print(
3694      "[LIVE] State="
3695    );
3696
3697    Serial.print(
3698      stateName(state)
3699    );
3700
3701
3702    // GPS
3703    Serial.print(
3704      " GPS="
3705    );
3706
3707    Serial.print(
3708      gpsQualityOK()
3709        ? "GOOD"
3710        : "BAD"
3711    );
3712
3713
3714    Serial.print(
3715      " SAT="
3716    );
3717
```

```
  Serial.print(
    gpsSats
  );


  Serial.print(
    " HDOP="
  );

  Serial.print(
    gpsHDOP,
    1
  );


  if (gpsFix)
  {
    Serial.print(
      " LAT="
    );

    Serial.print(
      currentLatitude,
      6
    );


    Serial.print(
      " LON="
    );

    Serial.print(
      currentLongitude,
      6
    );
  }


  // Navigation
  if (gpsQualityOK())
  {
    Serial.print(
      " DIST="
    );

    Serial.print(
      distanceToTarget,
      1
    );


    Serial.print(
      "m BRG="
    );

    Serial.print(
      targetBearing,
      1
    );
  }


  // Heading
  if (headingValid)
  {
    Serial.print(
      " HDG="
    );

    Serial.print(
      currentDirection,
      1
    );

```

```
    Serial.print(
      " SRC="
    );

    Serial.print(
      headingSource
    );


    Serial.print(
      " ERR="
    );

    Serial.print(
      courseError,
      1
    );
  }
  else
  {
    Serial.print(
      " HDG=NONE"
    );
  }


  // Steering
  Serial.print(
    " DEM="
  );

  Serial.print(
    steeringDemand
  );


  Serial.print(
    " S1="
  );

  Serial.print(
    servo1Position
  );


  Serial.print(
    " S2="
  );

  Serial.print(
    servo2Position
  );


  // Compass
  Serial.print(
    " CMP="
  );

  Serial.print(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    " CHP="
  );

  Serial.print(
    compassHeading,
    1
  );

```

```
  Serial.print(
    " NAV="
  );

  Serial.print(
    navCompassHeading,
    1
  );


  // IMU
  Serial.print(
    " IMU="
  );

  Serial.print(
    imuOK
      ? "OK"
      : "FAIL"
  );


  // Servos
  Serial.print(
    " SERVO="
  );

  Serial.println(
    servosReady()
      ? "OK"
      : "FAIL"
  );
}


// ================================================================
// STATE NAME
// ================================================================

const char *stateName(
  SystemState s)
{
  switch (s)
  {
    case STATE_WAIT_READY:
      return "WAIT_READY";


    case STATE_DEPLOYING:
      return "DEPLOYING";


    case STATE_STABILIZING:
      return "STABILIZING";


    case STATE_AUTONAV:
      return "AUTONAV";


    case STATE_CAPTURED:
      return "CAPTURED";


    case STATE_FAILSAFE:
      return "FAILSAFE";


    case STATE_MANUAL_TEST:
      return "MANUAL_TEST";


    default:
      return "UNKNOWN";
```

```
  }
}
```

A.4 Archived source 4: Pasted code(20261005-071236).cpp

Source file: `Pasted code(20261005-071236).cpp`

```
/*
  ================================================================
  TEENSY 4.1 AUTONOMOUS PARAFOIL
  GUIDED LANDING - FINAL LAB TEST
  ================================================================

  TARGET IS FIXED IN CODE.

  CHANGE ONLY:
      TARGET_LAT
      TARGET_LON


  ================================================================
  HARDWARE
  ================================================================

  TEENSY 4.1

  ----------------------------------------------------------------
  FOXEER M10Q 180 GPS
  ----------------------------------------------------------------

      GPS TX  -> Teensy Pin 15 (RX3)
      GPS RX  -> Teensy Pin 14 (TX3)
      GPS GND -> Teensy GND

      GPS UART = Serial3
      GPS BAUD = 38400

  ----------------------------------------------------------------
  I2C BUS
  ----------------------------------------------------------------

      SDA = Teensy Pin 18
      SCL = Teensy Pin 19

  ----------------------------------------------------------------
  QMC5883 COMPASS
  ----------------------------------------------------------------

      Address = 0x0D

  ----------------------------------------------------------------
  LSM6DSO32 IMU
  ----------------------------------------------------------------

      Address = 0x6A

  ----------------------------------------------------------------
  BMP581
  ----------------------------------------------------------------

      Address = 0x46 or 0x47
      Diagnostic only

  ----------------------------------------------------------------
  ST-3009
  ----------------------------------------------------------------

      Servo 1 ID = 1
      Servo 2 ID = 2

      Serial1
      Pin 0 = RX1
      Pin 1 = TX1
```

```
    1 Mbps
    SERIAL_8N1_HALF_DUPLEX

    Servo neutral = 2048

------------------------------------------------------------------
SG90 DEPLOYMENT SERVO
------------------------------------------------------------------

    Signal = Pin 8

==================================================================
NAVIGATION
==================================================================

Target bearing:
    GPS position -> target

Current heading:

    If speed >= 1.5 m/s:
        GPS course

    If speed < 1.5 m/s:
        QMC5883 compass

Course error:

    target bearing - current heading

    wrapped to -180 ... +180 degrees

Steering:

    error <= +/-5 degrees:
        neutral

    error reaches +/-90 degrees:
        maximum steering

Maximum steering:
    +/-450 counts

Differential:

    Positive demand:
        Servo 1 = 2048 - demand
        Servo 2 = 2048 + demand

    Negative demand:
        Servo 1 = 2048 - demand
        Servo 2 = 2048 + demand

Target capture:
    35 m

==================================================================
STATE MACHINE
==================================================================

    WAIT_READY
         |
         | GPS GOOD + Servo 1 + Servo 2
         v
    DEPLOYING
         |
         | 1.5 sec
         v
    STABILIZING
         |
         | 5 sec + GPS + servos
         v
     AUTONAV
         |
```

```
            | <=35 m
            v
        CAPTURED

  GPS or servo failure during AUTONAV:
       AUTONAV -> FAILSAFE

  3 sec good GPS + servos:
       FAILSAFE -> AUTONAV

  ====================================================================
  SERIAL COMMANDS
  ====================================================================

      STATUS
      MOTORTEST
      LEFT
      RIGHT
      CENTER
      AUTONAV
      DEPLOY
      PREP
      RESET
      HELP

  There is NO TARGET serial command.

  ====================================================================
*/


// ====================================================================
// LIBRARIES
// ====================================================================

#include <Arduino.h>
#include <Wire.h>
#include <Servo.h>
#include <TinyGPS++.h>
#include <SCServo.h>
#include <Adafruit_LSM6DSOX.h>

#include <math.h>
#include <string.h>


// ====================================================================
// TARGET
// ====================================================================

const double TARGET_LAT = 32.6321769;
const double TARGET_LON = 75.0143889;


// ====================================================================
// AUTO DEPLOY
// ====================================================================
//
// true  = deploy automatically after GPS + both ST-3009 ready
// false = wait for DEPLOY command
//
// For the autonomous lab sequence use TRUE.
// ====================================================================

const bool AUTO_DEPLOY_AFTER_READY = true;


// ====================================================================
// GPS
// ====================================================================

#define GPS_SERIAL Serial3

const uint32_t GPS_BAUD = 38400;

```

```
const uint8_t GPS_RX_PIN = 15;
const uint8_t GPS_TX_PIN = 14;

// GPS data considered stale after this time
const uint32_t GPS_DATA_TIMEOUT_MS = 2500;

// Minimum satellites
const uint8_t GPS_MIN_SATS = 5;

// Maximum acceptable HDOP
const double GPS_MAX_HDOP = 3.5;

// GPS course is used only above this speed
const double GPS_COURSE_MIN_SPEED_MPS = 1.5;


// ================================================================
// I2C
// ================================================================

const uint8_t I2C_SDA_PIN = 18;
const uint8_t I2C_SCL_PIN = 19;


// ================================================================
// QMC5883
// ================================================================

const uint8_t QMC5883_ADDR = 0x0D;


// ----------------------------------------------------------------
// Compass navigation orientation
// ----------------------------------------------------------------
//
// Your tested compass direction was mirrored for navigation.
//

const bool COMPASS_MIRROR_FOR_NAV = true;

const float COMPASS_HEADING_OFFSET_DEG = 0.0f;


// ----------------------------------------------------------------
// Calibration values from your working compass test
// ----------------------------------------------------------------

const float OFFSET_X = 180.0000f;
const float OFFSET_Y = 336.0000f;
const float OFFSET_Z = 615.0000f;

const float SCALE_X = 0.956484f;
const float SCALE_Y = 0.904406f;
const float SCALE_Z = 1.178127f;


// ================================================================
// LSM6DSO32
// ================================================================

const uint8_t LSM6DSO32_ADDR = 0x6A;

Adafruit_LSM6DSOX lsm6dso;

bool imuOK = false;


// ================================================================
// BMP581
// ================================================================

const uint8_t BMP581_ADDR_1 = 0x46;
const uint8_t BMP581_ADDR_2 = 0x47;

bool bmpOK = false;
```

```
292
293 uint8_t bmpAddress = 0;
294
295
296 // ================================================================
297 // GPS VARIABLES
298 // ================================================================
299
300 TinyGPSPlus gps;
301
302 bool gpsFix = false;
303
304 uint8_t gpsSats = 0;
305
306 double gpsHDOP = 99.9;
307
308 double currentSpeed = 0.0;
309
310 unsigned long lastGPSLocationMs = 0;
311
312
313 // ================================================================
314 // COMPASS VARIABLES
315 // ================================================================
316
317 bool compassOK = false;
318
319 int16_t rawMagX = 0;
320 int16_t rawMagY = 0;
321 int16_t rawMagZ = 0;
322
323 float magX = 0.0f;
324 float magY = 0.0f;
325 float magZ = 0.0f;
326
327 float compassHeading = 0.0f;
328 float navCompassHeading = 0.0f;
329
330 uint8_t compassFailCount = 0;
331
332
333 // ================================================================
334 // IMU VARIABLES
335 // ================================================================
336
337 float accelX = 0.0f;
338 float accelY = 0.0f;
339 float accelZ = 0.0f;
340
341 float gyroX = 0.0f;
342 float gyroY = 0.0f;
343 float gyroZ = 0.0f;
344
345 unsigned long lastSensorUpdateMs = 0;
346
347
348 // ================================================================
349 // ST-3009
350 // ================================================================
351 //
352 // IMPORTANT:
353 // SCServo 1.0.2 uses:
354 //
355 //     SMS_STS
356 //
357
358 SMS_STS st;
359
360
361 // ----------------------------------------------------------------
362 // Servo IDs
363 // ----------------------------------------------------------------
364
365 const uint8_t SERVO1_ID = 1;
366 const uint8_t SERVO2_ID = 2;
```

```
367
368
369 // ----------------------------------------------------------------
370 // Servo positions
371 // ----------------------------------------------------------------
372
373 const int SERVO_NEUTRAL = 2048;
374
375 const int SERVO_MIN_POS = 200;
376 const int SERVO_MAX_POS = 3895;
377
378
379 // ----------------------------------------------------------------
380 // Steering
381 // ----------------------------------------------------------------
382
383 const int MAX_STEERING_COUNTS = 450;
384
385 const int STEERING_SERVO_SPEED = 500;
386 const int STEERING_SERVO_ACCEL = 50;
387
388 const int MAX_SLEW_COUNTS = 60;
389
390
391 // ----------------------------------------------------------------
392 // Servo health
393 // ----------------------------------------------------------------
394
395 const unsigned long SERVO_HEALTH_INTERVAL_MS = 2000;
396
397 bool servo1OK = false;
398 bool servo2OK = false;
399 bool servoBusReady = false;
400
401 unsigned long lastServoHealthMs = 0;
402
403
404 // ----------------------------------------------------------------
405 // Current positions
406 // ----------------------------------------------------------------
407
408 int servo1Position = SERVO_NEUTRAL;
409 int servo2Position = SERVO_NEUTRAL;
410
411
412 // ----------------------------------------------------------------
413 // Current navigation demand
414 // ----------------------------------------------------------------
415
416 int steeringDemand = 0;
417
418
419 // ================================================================
420 // SG90 DEPLOYMENT
421 // ================================================================
422
423 Servo deployServo;
424
425 const uint8_t DEPLOY_SERVO_PIN = 8;
426
427 const int DEPLOY_ANGLE = 180;
428
429 const int PREP_ANGLE = 140;
430
431 const unsigned long DEPLOY_MOVE_TIME_MS = 1500;
432
433 const unsigned long DEPLOY_STABILIZE_TIME_MS = 5000;
434
435 bool deploymentServoActive = false;
436
437 bool deploymentDone = false;
438
439 unsigned long deploymentStartMs = 0;
440
441 unsigned long stabilizationStartMs = 0;
```

```
442
443
444 // ================================================================
445 // NAVIGATION
446 // ================================================================
447
448 const double TARGET_CAPTURE_RADIUS_M = 35.0;
449
450 const double STEERING_DEADBAND_DEG = 5.0;
451
452 const double FULL_STEERING_ERROR_DEG = 90.0;
453
454
455 // ================================================================
456 // STATE MACHINE
457 // ================================================================
458
459 enum SystemState
460 {
461   STATE_WAIT_READY,
462   STATE_DEPLOYING,
463   STATE_STABILIZING,
464   STATE_AUTONAV,
465   STATE_CAPTURED,
466   STATE_FAILSAFE,
467   STATE_MANUAL_TEST
468 };
469
470 SystemState state = STATE_WAIT_READY;
471
472 unsigned long stateEnteredMs = 0;
473
474
475 // ================================================================
476 // NAVIGATION VARIABLES
477 // ================================================================
478
479 double currentLatitude = 0.0;
480
481 double currentLongitude = 0.0;
482
483 double distanceToTarget = 0.0;
484
485 double targetBearing = 0.0;
486
487 double currentDirection = 0.0;
488
489 double courseError = 0.0;
490
491 bool headingValid = false;
492
493 const char *headingSource = "NONE";
494
495
496 // ================================================================
497 // FAILSAFE
498 // ================================================================
499
500 unsigned long failsafeGoodSinceMs = 0;
501
502 const unsigned long FAILSAFE_RECOVERY_MS = 3000;
503
504
505 // ================================================================
506 // SERIAL COMMAND BUFFER
507 // ================================================================
508
509 #define COMMAND_BUFFER_SIZE 120
510
511 char commandBuffer[COMMAND_BUFFER_SIZE];
512
513 uint16_t commandIndex = 0;
514
515
516 // ================================================================
```

```
// TIMERS
// ================================================================

unsigned long lastNavigationUpdateMs = 0;

const unsigned long NAVIGATION_UPDATE_INTERVAL_MS = 100;

unsigned long lastLivePrintMs = 0;

const unsigned long LIVE_PRINT_INTERVAL_MS = 1000;

unsigned long lastBMPCheckMs = 0;

const unsigned long BMP_CHECK_INTERVAL_MS = 3000;


// ================================================================
// MOTOR TEST
// ================================================================

bool motorTestActive = false;

uint8_t motorTestStep = 0;

unsigned long motorTestTimer = 0;


// ================================================================
// FORWARD DECLARATIONS
// ================================================================

void processSerialCommands();
void processCommand(char *cmd);

void updateGPS();
void updateCompass();
void updateIMU();
void updateServoHealth();

void updateNavigation();
void updateSteering();
void updateStateMachine();

void writeSteeringPositions(int pos1, int pos2);
void centerServos();
void commandSteering(int demand);

void startDeployment();
void prepareDeploymentServo();
void resetSystem();

void startMotorTest();
void updateMotorTest();

void printStatus();
void printLiveData();

bool gpsQualityOK();
bool readQMC5883();
bool initQMC5883();

bool checkI2CDevice(uint8_t address);
bool initIMU();

bool servosReady();

float wrap360(float angle);
double wrap180(double angle);
float navigationCompassHeading();

const char *stateName(SystemState s);

int slewTo(
  int currentValue,
  int targetValue,
```

```
  int maxStep
);


// ================================================================
// SETUP
// ================================================================

void setup()
{
  // ----------------------------------------------------------------
  // USB SERIAL
  // ----------------------------------------------------------------

  Serial.begin(115200);

  delay(1500);


  Serial.println();
  Serial.println(
    "================================================"
  );

  Serial.println(
    " TEENSY 4.1 AUTONOMOUS PARAFOIL"
  );

  Serial.println(
    " GUIDED LANDING - FINAL LAB TEST"
  );

  Serial.println(
    "================================================"
  );


  // ----------------------------------------------------------------
  // TARGET
  // ----------------------------------------------------------------

  Serial.println();

  Serial.print(
    "TARGET LATITUDE  : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "TARGET LONGITUDE : "
  );

  Serial.println(
    TARGET_LON,
    7
  );


  // ----------------------------------------------------------------
  // I2C
  // ----------------------------------------------------------------

  Wire.begin();

  Wire.setClock(400000);


  Serial.println();

  Serial.println(
```

```
    "I2C initialized"
  );

  Serial.println(
    "SDA = Pin 18"
  );

  Serial.println(
    "SCL = Pin 19"
  );


  // ---------------------------------------------------------------
  // QMC5883
  // ---------------------------------------------------------------

  if (initQMC5883())
  {
    compassOK = true;

    Serial.println(
      "QMC5883 : OK"
    );
  }
  else
  {
    compassOK = false;

    Serial.println(
      "QMC5883 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // LSM6DSO32
  // ---------------------------------------------------------------

  if (initIMU())
  {
    imuOK = true;

    Serial.println(
      "LSM6DSO32 : OK"
    );
  }
  else
  {
    imuOK = false;

    Serial.println(
      "LSM6DSO32 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // BMP581
  // ---------------------------------------------------------------

  if (checkI2CDevice(
        BMP581_ADDR_1))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_1;
  }
  else if (checkI2CDevice(
             BMP581_ADDR_2))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_2;
```

```
  }
  else
  {
    bmpOK = false;

    bmpAddress = 0;
  }


  if (bmpOK)
  {
    Serial.print(
      "BMP581 : FOUND at 0x"
    );

    if (bmpAddress < 16)
      Serial.print("0");

    Serial.println(
      bmpAddress,
      HEX
    );
  }
  else
  {
    Serial.println(
      "BMP581 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // GPS
  // ---------------------------------------------------------------
  //
  // EXACTLY MATCHES YOUR WORKING GPS TEST
  //

  GPS_SERIAL.begin(
    GPS_BAUD
  );


  Serial.println();

  Serial.println(
    "GPS UART  : Serial3"
  );

  Serial.print(
    "GPS RX    : Pin "
  );

  Serial.println(
    GPS_RX_PIN
  );

  Serial.print(
    "GPS TX    : Pin "
  );

  Serial.println(
    GPS_TX_PIN
  );

  Serial.print(
    "GPS BAUD  : "
  );

  Serial.println(
    GPS_BAUD
  );


  // ---------------------------------------------------------------
```

```
  // ST-3009
  // ----------------------------------------------------------------

  Serial.println();

  Serial.println(
    "Initializing ST-3009 bus..."
  );


  Serial1.begin(
    1000000,
    SERIAL_8N1_HALF_DUPLEX
  );


  st.pSerial =
    &Serial1;


  delay(500);


  // Force immediate first health test
  lastServoHealthMs =
    millis() -
    SERVO_HEALTH_INTERVAL_MS;


  updateServoHealth();


  if (servosReady())
  {
    Serial.println(
      "ST-3009 ID 1 : OK"
    );

    Serial.println(
      "ST-3009 ID 2 : OK"
    );


    centerServos();
  }
  else
  {
    Serial.println(
      "ST-3009 : WAITING"
    );
  }


  // ----------------------------------------------------------------
  // SG90
  // ----------------------------------------------------------------

  deployServo.detach();

  deploymentServoActive =
    false;


  // ----------------------------------------------------------------
  // State
  // ----------------------------------------------------------------

  state =
    STATE_WAIT_READY;

  stateEnteredMs =
    millis();


  Serial.println();
```

```
892
893   Serial.println(
894     "================================================"
895   );
896
897   Serial.println(
898     "SYSTEM READY"
899   );
900
901   Serial.println(
902     "Target is fixed in code."
903   );
904
905   Serial.println(
906     "================================================"
907   );
908
909
910   Serial.println();
911
912   Serial.println(
913     "COMMANDS:"
914   );
915
916   Serial.println(
917     "STATUS"
918   );
919
920   Serial.println(
921     "MOTORTEST"
922   );
923
924   Serial.println(
925     "LEFT"
926   );
927
928   Serial.println(
929     "RIGHT"
930   );
931
932   Serial.println(
933     "CENTER"
934   );
935
936   Serial.println(
937     "AUTONAV"
938   );
939
940   Serial.println(
941     "DEPLOY"
942   );
943
944   Serial.println(
945     "PREP"
946   );
947
948   Serial.println(
949     "RESET"
950   );
951
952   Serial.println(
953     "HELP"
954   );
955
956
957   Serial.println();
958
959   Serial.println(
960     "Waiting for GPS + ST-3009..."
961   );
962 }
963
964
965 // ================================================================
966 // LOOP
```

```
// ================================================================

void loop()
{
  // GPS continuously
  updateGPS();


  // Serial commands
  processSerialCommands();


  // Sensors
  if (millis() -
      lastSensorUpdateMs >=
      100)
  {
    lastSensorUpdateMs =
      millis();

    updateCompass();
    updateIMU();
  }


  // Servo health
  updateServoHealth();


  // Motor test
  if (motorTestActive)
  {
    updateMotorTest();
  }


  // State
  updateStateMachine();


  // Navigation
  if (millis() -
      lastNavigationUpdateMs >=
      NAVIGATION_UPDATE_INTERVAL_MS)
  {
    lastNavigationUpdateMs =
      millis();

    updateNavigation();
    updateSteering();
  }


  // BMP check
  if (millis() -
      lastBMPCheckMs >=
      BMP_CHECK_INTERVAL_MS)
  {
    lastBMPCheckMs =
      millis();


    if (!bmpOK)
    {
      if (checkI2CDevice(
            BMP581_ADDR_1))
      {
        bmpOK = true;

        bmpAddress =
          BMP581_ADDR_1;
      }
      else if (checkI2CDevice(
                 BMP581_ADDR_2))
      {
```

```
        bmpOK = true;

        bmpAddress =
          BMP581_ADDR_2;
      }
    }
  }


  // Live data
  if (millis() -
      lastLivePrintMs >=
      LIVE_PRINT_INTERVAL_MS)
  {
    lastLivePrintMs =
      millis();

    printLiveData();
  }
}


// ================================================================
// GPS UPDATE
// ================================================================

void updateGPS()
{
  while (GPS_SERIAL.available() > 0)
  {
    char c =
      (char)GPS_SERIAL.read();

    gps.encode(c);
  }


  // Fresh GPS location
  if (gps.location.isUpdated())
  {
    lastGPSLocationMs =
      millis();
  }


  // Position
  if (gps.location.isValid())
  {
    gpsFix = true;

    currentLatitude =
      gps.location.lat();

    currentLongitude =
      gps.location.lng();
  }
  else
  {
    gpsFix = false;
  }


  // Satellites
  if (gps.satellites.isValid())
  {
    gpsSats =
      gps.satellites.value();
  }
  else
  {
    gpsSats = 0;
  }


  // HDOP
```

```
  if (gps.hdop.isValid())
  {
    gpsHDOP =
      gps.hdop.hdop();
  }
  else
  {
    gpsHDOP = 99.9;
  }


  // Speed
  if (gps.speed.isValid())
  {
    currentSpeed =
      gps.speed.mps();
  }
  else
  {
    currentSpeed = 0.0;
  }
}


// ================================================================
// GPS QUALITY
// ================================================================

bool gpsQualityOK()
{
  if (!gpsFix)
    return false;


  if (lastGPSLocationMs == 0)
    return false;


  if (millis() -
      lastGPSLocationMs >
      GPS_DATA_TIMEOUT_MS)
  {
    return false;
  }


  if (gpsSats <
      GPS_MIN_SATS)
  {
    return false;
  }


  if (gps.hdop.isValid() &&
      gpsHDOP >
      GPS_MAX_HDOP)
  {
    return false;
  }


  return true;
}


// ================================================================
// QMC5883 INIT
// ================================================================

bool initQMC5883()
{
  // Check device
  Wire.beginTransmission(
    QMC5883_ADDR
  );
```

```

  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  // SET/RESET period
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x0B);
  Wire.write(0x01);

  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  delay(10);


  // Control register
  // Same setting used by your
  // working compass test.
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x09);
  Wire.write(0x1D);

  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  delay(100);


  return true;
}


// ================================================================
// QMC5883 READ
// ================================================================

bool readQMC5883()
{
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x00);


  if (Wire.endTransmission(false) != 0)
  {
    return false;
  }


  uint8_t count =
    Wire.requestFrom(
      QMC5883_ADDR,
      (uint8_t)6
    );


  if (count != 6)
```

```
  {
    while (Wire.available())
    {
      Wire.read();
    }

    return false;
  }


  uint8_t xl =
    Wire.read();

  uint8_t xh =
    Wire.read();


  uint8_t yl =
    Wire.read();

  uint8_t yh =
    Wire.read();


  uint8_t zl =
    Wire.read();

  uint8_t zh =
    Wire.read();


  rawMagX =
    (int16_t)(
      (xh << 8) |
      xl
    );


  rawMagY =
    (int16_t)(
      (yh << 8) |
      yl
    );


  rawMagZ =
    (int16_t)(
      (zh << 8) |
      zl
    );


  return true;
}


// ================================================================
// COMPASS UPDATE
// ================================================================

void updateCompass()
{
  // ------------------------------------------------------------
  // Read sensor
  // ------------------------------------------------------------

  if (!readQMC5883())
  {
    compassFailCount++;


    // Do not immediately declare compass dead.
    // Reinitialize after 5 consecutive failures.

    if (compassFailCount >= 5)
```

```
    {
      if (initQMC5883())
      {
        compassOK = true;

        compassFailCount = 0;

        Serial.println(
          "[COMPASS] Reinitialized OK"
        );
      }
      else
      {
        compassOK = false;
      }
    }


    return;
  }


  // Successful reading
  compassFailCount = 0;
  compassOK = true;


  // ---------------------------------------------------------------
  // Apply your proven calibration
  // ---------------------------------------------------------------

  magX =
    ((float)rawMagX -
     OFFSET_X) *
    SCALE_X;


  magY =
    ((float)rawMagY -
     OFFSET_Y) *
    SCALE_Y;


  magZ =
    ((float)rawMagZ -
     OFFSET_Z) *
    SCALE_Z;


  // ---------------------------------------------------------------
  // Calculate heading
  // ---------------------------------------------------------------

  float angle =
    atan2(
      magY,
      magX
    ) *
    180.0f /
    PI;


  if (angle < 0.0f)
    angle += 360.0f;


  compassHeading =
    wrap360(
      angle +
      COMPASS_HEADING_OFFSET_DEG
    );


  // Navigation heading
  navCompassHeading =
```

```
    navigationCompassHeading();
}


// ================================================================
// NAVIGATION COMPASS
// ================================================================

float navigationCompassHeading()
{
  if (!COMPASS_MIRROR_FOR_NAV)
  {
    return compassHeading;
  }


  return wrap360(
    360.0f -
    compassHeading
  );
}


// ================================================================
// WRAP 0...360
// ================================================================

float wrap360(float angle)
{
  while (angle >= 360.0f)
    angle -= 360.0f;


  while (angle < 0.0f)
    angle += 360.0f;


  return angle;
}


// ================================================================
// WRAP -180...180
// ================================================================

double wrap180(double angle)
{
  while (angle > 180.0)
    angle -= 360.0;


  while (angle < -180.0)
    angle += 360.0;


  return angle;
}


// ================================================================
// IMU INIT
// ================================================================

bool initIMU()
{
  if (!lsm6dso.begin_I2C(
        LSM6DSO32_ADDR,
        &Wire))
  {
    return false;
  }


  lsm6dso.setAccelRange(
    LSM6DS_ACCEL_RANGE_16_G
```

```
  );


  lsm6dso.setGyroRange(
    LSM6DS_GYRO_RANGE_2000_DPS
  );


  lsm6dso.setAccelDataRate(
    LSM6DS_RATE_208_HZ
  );


  lsm6dso.setGyroDataRate(
    LSM6DS_RATE_208_HZ
  );


  return true;
}


// ================================================================
// IMU UPDATE
// ================================================================

void updateIMU()
{
  if (!imuOK)
    return;


  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;


  lsm6dso.getEvent(
    &accel,
    &gyro,
    &temp
  );


  accelX =
    accel.acceleration.x;


  accelY =
    accel.acceleration.y;


  accelZ =
    accel.acceleration.z;


  gyroX =
    gyro.gyro.x;


  gyroY =
    gyro.gyro.y;


  gyroZ =
    gyro.gyro.z;
}


// ================================================================
// I2C CHECK
// ================================================================

bool checkI2CDevice(
  uint8_t address)
```

```
{
  Wire.beginTransmission(
    address
  );


  return (
    Wire.endTransmission() == 0
  );
}


// ================================================================
// SERVO HEALTH
// ================================================================

void updateServoHealth()
{
  if (millis() -
      lastServoHealthMs <
      SERVO_HEALTH_INTERVAL_MS)
  {
    return;
  }


  lastServoHealthMs =
    millis();


  // Ping ID 1
  int result1 =
    st.Ping(
      SERVO1_ID
    );


  delay(10);


  // Ping ID 2
  int result2 =
    st.Ping(
      SERVO2_ID
    );


  servo1OK =
    (result1 >= 0);


  servo2OK =
    (result2 >= 0);


  servoBusReady =
    servo1OK &&
    servo2OK;
}


// ================================================================
// SERVO READY
// ================================================================

bool servosReady()
{
  return
    servoBusReady &&
    servo1OK &&
    servo2OK;
}


// ================================================================
```

```
// CENTER BOTH SERVOS
// ================================================================

void centerServos()
{
  if (!servosReady())
    return;


  st.WritePosEx(
    SERVO1_ID,
    SERVO_NEUTRAL,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  st.WritePosEx(
    SERVO2_ID,
    SERVO_NEUTRAL,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  servo1Position =
    SERVO_NEUTRAL;


  servo2Position =
    SERVO_NEUTRAL;


  steeringDemand = 0;
}


// ================================================================
// DIRECT STEERING COMMAND
// ================================================================
//
// demand = +300:
//
//   Servo 1 = 2048 - 300 = 1748
//   Servo 2 = 2048 + 300 = 2348
//
// demand = -300:
//
//   Servo 1 = 2048 + 300 = 2348
//   Servo 2 = 2048 - 300 = 1748
//

void commandSteering(
  int demand)
{
  if (!servosReady())
  {
    Serial.println(
      "ERROR: ST-3009 BUS NOT READY"
    );

    return;
  }


  demand =
    constrain(
      demand,
      -MAX_STEERING_COUNTS,
      MAX_STEERING_COUNTS
    );


  int pos1 =
    SERVO_NEUTRAL -
```

```
    demand;


  int pos2 =
    SERVO_NEUTRAL +
    demand;


  pos1 =
    constrain(
      pos1,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


  pos2 =
    constrain(
      pos2,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


  st.WritePosEx(
    SERVO1_ID,
    pos1,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  st.WritePosEx(
    SERVO2_ID,
    pos2,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  servo1Position =
    pos1;


  servo2Position =
    pos2;


  steeringDemand =
    demand;


  Serial.print(
    "DEMAND="
  );

  Serial.print(
    demand
  );

  Serial.print(
    "  S1="
  );

  Serial.print(
    pos1
  );

  Serial.print(
    "  S2="
  );

  Serial.println(
    pos2
  );
```

```
}


// ================================================================
// MOTOR TEST START
// ================================================================

void startMotorTest()
{
  if (!servosReady())
  {
    Serial.println(
      "MOTORTEST BLOCKED: "
      "ST-3009 NOT READY"
    );

    return;
  }


  motorTestActive =
    true;


  state =
    STATE_MANUAL_TEST;


  motorTestStep = 0;

  motorTestTimer =
    millis();


  centerServos();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "       ST-3009 MOTOR TEST"
  );

  Serial.println(
    "========================================"
  );

  Serial.println(
    "CENTER -> LEFT -> CENTER -> RIGHT -> CENTER"
  );

  Serial.println();
}


// ================================================================
// MOTOR TEST UPDATE
// ================================================================

void updateMotorTest()
{
  if (!servosReady())
  {
    motorTestActive =
      false;

    state =
      STATE_WAIT_READY;

    return;
  }
```

```

if (millis() -
    motorTestTimer <
    1800)
{
  return;
}


motorTestTimer =
  millis();


switch (motorTestStep)
{
  case 0:

    Serial.println(
      "TEST: CENTER"
    );

    commandSteering(0);

    motorTestStep++;

    break;


  case 1:

    Serial.println(
      "TEST: LEFT"
    );

    commandSteering(
      +300
    );

    motorTestStep++;

    break;


  case 2:

    Serial.println(
      "TEST: CENTER"
    );

    commandSteering(0);

    motorTestStep++;

    break;


  case 3:

    Serial.println(
      "TEST: RIGHT"
    );

    commandSteering(
      -300
    );

    motorTestStep++;

    break;


  case 4:

    Serial.println(
```

```
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    default:

      motorTestActive =
        false;


      state =
        STATE_WAIT_READY;


      centerServos();


      Serial.println();

      Serial.println(
        "MOTOR TEST COMPLETE"
      );

      Serial.println();

      break;
  }
}


// ======================================================================
// NAVIGATION CALCULATION
// ======================================================================

void updateNavigation()
{
  if (!gpsQualityOK())
  {
    headingValid =
      false;

    headingSource =
      "NONE";

    return;
  }


  // ------------------------------------------------------------
  // GPS position
  // ------------------------------------------------------------

  currentLatitude =
    gps.location.lat();


  currentLongitude =
    gps.location.lng();


  // ------------------------------------------------------------
  // Distance
  // ------------------------------------------------------------

  distanceToTarget =
    TinyGPSPlus::distanceBetween(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
```

```
    TARGET_LON
  );


// ------------------------------------------------------------
// Bearing
// ------------------------------------------------------------

targetBearing =
  TinyGPSPlus::courseTo(
    currentLatitude,
    currentLongitude,
    TARGET_LAT,
    TARGET_LON
  );


targetBearing =
  wrap360(
    targetBearing
  );


// ------------------------------------------------------------
// Determine heading
// ------------------------------------------------------------

headingValid =
  false;


headingSource =
  "NONE";


// GPS course when moving
if (gps.speed.isValid() &&
    gps.course.isValid() &&
    currentSpeed >=
    GPS_COURSE_MIN_SPEED_MPS)
{
  currentDirection =
    wrap360(
      gps.course.deg()
    );


  headingValid =
    true;


  headingSource =
    "GPS COURSE";
}


// Compass when slow/stationary
else if (compassOK)
{
  currentDirection =
    navigationCompassHeading();


  headingValid =
    true;


  headingSource =
    "COMPASS";
}


// ------------------------------------------------------------
// Course error
// ------------------------------------------------------------
```

```
2092
2093    if (headingValid)
2094    {
2095      courseError =
2096        wrap180(
2097          targetBearing -
2098          currentDirection
2099        );
2100    }
2101  }
2102
2103
2104  // ================================================================
2105  // AUTONOMOUS STEERING
2106  // ================================================================
2107
2108  void updateSteering()
2109  {
2110    // Manual motor test owns the servos
2111    if (motorTestActive)
2112      return;
2113
2114
2115    // Servo unavailable
2116    if (!servosReady())
2117    {
2118      steeringDemand = 0;
2119
2120      return;
2121    }
2122
2123
2124    // Only steer during AUTONAV
2125    if (state !=
2126        STATE_AUTONAV)
2127    {
2128      return;
2129    }
2130
2131
2132    // GPS required
2133    if (!gpsQualityOK())
2134    {
2135      centerServos();
2136
2137      return;
2138    }
2139
2140
2141    // Target captured
2142    if (distanceToTarget <=
2143        TARGET_CAPTURE_RADIUS_M)
2144    {
2145      centerServos();
2146
2147      return;
2148    }
2149
2150
2151    // No heading
2152    if (!headingValid)
2153    {
2154      centerServos();
2155
2156      return;
2157    }
2158
2159
2160    // ---------------------------------------------------------
2161    // Absolute error
2162    // ---------------------------------------------------------
2163
2164    double absoluteError =
2165      fabs(courseError);
2166
```

```
2167
2168  // ------------------------------------------------------------
2169  // Deadband
2170  // ------------------------------------------------------------
2171
2172  if (absoluteError <=
2173      STEERING_DEADBAND_DEG)
2174  {
2175    steeringDemand = 0;
2176  }
2177  else
2178  {
2179    double usableError =
2180      absoluteError -
2181      STEERING_DEADBAND_DEG;
2182
2183
2184    double usableRange =
2185      FULL_STEERING_ERROR_DEG -
2186      STEERING_DEADBAND_DEG;
2187
2188
2189    double normalized =
2190      usableError /
2191      usableRange;
2192
2193
2194    normalized =
2195      constrain(
2196        normalized,
2197        0.0,
2198        1.0
2199      );
2200
2201
2202    int demand =
2203      (int)round(
2204        normalized *
2205        MAX_STEERING_COUNTS
2206      );
2207
2208
2209    if (courseError > 0.0)
2210    {
2211      steeringDemand =
2212        demand;
2213    }
2214    else
2215    {
2216      steeringDemand =
2217        -demand;
2218    }
2219  }
2220
2221
2222  // ------------------------------------------------------------
2223  // Target positions
2224  // ------------------------------------------------------------
2225
2226  int target1 =
2227    SERVO_NEUTRAL -
2228    steeringDemand;
2229
2230
2231  int target2 =
2232    SERVO_NEUTRAL +
2233    steeringDemand;
2234
2235
2236  target1 =
2237    constrain(
2238      target1,
2239      SERVO_MIN_POS,
2240      SERVO_MAX_POS
2241    );
```

```

  target2 =
    constrain(
      target2,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


  // ------------------------------------------------------------
  // Slew limit
  // ------------------------------------------------------------

  int new1 =
    slewTo(
      servo1Position,
      target1,
      MAX_SLEW_COUNTS
    );


  int new2 =
    slewTo(
      servo2Position,
      target2,
      MAX_SLEW_COUNTS
    );


  // ------------------------------------------------------------
  // Send only if changed
  // ------------------------------------------------------------

  if (new1 != servo1Position ||
      new2 != servo2Position)
  {
    st.WritePosEx(
      SERVO1_ID,
      new1,
      STEERING_SERVO_SPEED,
      STEERING_SERVO_ACCEL
    );


    st.WritePosEx(
      SERVO2_ID,
      new2,
      STEERING_SERVO_SPEED,
      STEERING_SERVO_ACCEL
    );


    servo1Position =
      new1;


    servo2Position =
      new2;
  }
}


// ================================================================
// SLEW LIMIT
// ================================================================

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep)
{
  if (targetValue >
      currentValue)
  {
```

```
    int difference =
      targetValue -
      currentValue;


    if (difference >
        maxStep)
    {
      return currentValue +
             maxStep;
    }


    return targetValue;
  }


  if (targetValue <
      currentValue)
  {
    int difference =
      currentValue -
      targetValue;


    if (difference >
        maxStep)
    {
      return currentValue -
             maxStep;
    }


    return targetValue;
  }


  return currentValue;
}


// ================================================================
// DEPLOYMENT
// ================================================================

void startDeployment()
{
  if (deploymentDone)
  {
    Serial.println(
      "DEPLOYMENT ALREADY COMPLETED"
    );

    return;
  }


  if (!gpsQualityOK())
  {
    Serial.println(
      "DEPLOY BLOCKED: GPS NOT READY"
    );

    return;
  }


  if (!servosReady())
  {
    Serial.println(
      "DEPLOY BLOCKED: ST-3009 NOT READY"
    );

    return;
  }
```

```

  // Center steering first
  centerServos();


  delay(100);


  // Attach SG90
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );


  delay(100);


  // Deployment position
  deployServo.write(
    DEPLOY_ANGLE
  );


  deploymentStartMs =
    millis();


  deploymentServoActive =
    true;


  deploymentDone =
    true;


  state =
    STATE_DEPLOYING;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "STARTING DEPLOYMENT"
  );

  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    DEPLOY_ANGLE
  );

  Serial.println(
    " degrees"
  );
}


// ================================================================
// SG90 PREP
// ================================================================

void prepareDeploymentServo()
{
  deployServo.attach(
```

```
    DEPLOY_SERVO_PIN
  );


  delay(100);


  deployServo.write(
    PREP_ANGLE
  );


  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    PREP_ANGLE
  );

  Serial.println(
    " degrees"
  );


  delay(1000);


  deployServo.detach();


  Serial.println(
    "SG90 detached"
  );
}


// ================================================================
// STATE MACHINE
// ================================================================

void updateStateMachine()
{
  // ------------------------------------------------------------
  // Manual test
  // ------------------------------------------------------------

  if (state ==
      STATE_MANUAL_TEST)
  {
    return;
  }


  // ------------------------------------------------------------
  // WAIT READY
  // ------------------------------------------------------------

  if (state ==
      STATE_WAIT_READY)
  {
    if (AUTO_DEPLOY_AFTER_READY &&
        !deploymentDone &&
        gpsQualityOK() &&
        servosReady())
    {
      Serial.println();

      Serial.println(
        "GPS + ST-3009 READY"
      );

      Serial.println(
        "AUTO DEPLOY ENABLED"
      );
```

```
2542
2543
2544        startDeployment();
2545      }
2546
2547
2548      return;
2549    }
2550
2551
2552    // ---------------------------------------------------------------
2553    // DEPLOYING
2554    // ---------------------------------------------------------------
2555
2556    if (state ==
2557        STATE_DEPLOYING)
2558    {
2559      steeringDemand =
2560        0;
2561
2562
2563      if (millis() -
2564          deploymentStartMs >=
2565          DEPLOY_MOVE_TIME_MS)
2566      {
2567        // Detach SG90
2568        deployServo.detach();
2569
2570
2571        deploymentServoActive =
2572          false;
2573
2574
2575        // Center steering
2576        if (servosReady())
2577        {
2578          centerServos();
2579        }
2580
2581
2582        // Stabilization starts
2583        stabilizationStartMs =
2584          millis();
2585
2586
2587        state =
2588          STATE_STABILIZING;
2589
2590
2591        Serial.println();
2592
2593        Serial.println(
2594          "SG90 detached"
2595        );
2596
2597        Serial.println(
2598          "5 second stabilization started"
2599        );
2600      }
2601
2602
2603      return;
2604    }
2605
2606
2607    // ---------------------------------------------------------------
2608    // STABILIZING
2609    // ---------------------------------------------------------------
2610
2611    if (state ==
2612        STATE_STABILIZING)
2613    {
2614      steeringDemand =
2615        0;
2616
```

```
    if (millis() -
        stabilizationStartMs >=
        DEPLOY_STABILIZE_TIME_MS)
    {
      if (gpsQualityOK() &&
          servosReady())
      {
        state =
          STATE_AUTONAV;


        Serial.println();

        Serial.println(
          "======================================="
        );

        Serial.println(
          "AUTONOMOUS NAVIGATION STARTED"
        );

        Serial.println(
          "======================================="
        );
      }
    }


    return;
  }


  // ------------------------------------------------------------
  // AUTONAV
  // ------------------------------------------------------------

  if (state ==
      STATE_AUTONAV)
  {
    // GPS / servo failure
    if (!gpsQualityOK() ||
        !servosReady())
    {
      steeringDemand =
        0;


      if (servosReady())
      {
        centerServos();
      }


      state =
        STATE_FAILSAFE;


      failsafeGoodSinceMs =
        0;


      Serial.println();

      Serial.println(
        "!!! NAVIGATION FAILSAFE !!!"
      );

      Serial.println(
        "GPS or ST-3009 unavailable"
      );


      return;
    }
```

```

  // Target capture
  if (distanceToTarget <=
      TARGET_CAPTURE_RADIUS_M)
  {
    centerServos();


    state =
      STATE_CAPTURED;


    Serial.println();

    Serial.println(
      "========================================"
    );

    Serial.println(
      "TARGET CAPTURED"
    );

    Serial.print(
      "Distance = "
    );

    Serial.print(
      distanceToTarget,
      2
    );

    Serial.println(
      " m"
    );

    Serial.println(
      "ST-3009 -> CENTER"
    );

    Serial.println(
      "========================================"
    );
  }


  return;
}


// ------------------------------------------------------------
// CAPTURED
// ------------------------------------------------------------

if (state ==
    STATE_CAPTURED)
{
  steeringDemand =
    0;

  return;
}


// ------------------------------------------------------------
// FAILSAFE
// ------------------------------------------------------------

if (state ==
    STATE_FAILSAFE)
{
  steeringDemand =
    0;


```

```
    if (servosReady())
    {
      centerServos();
    }


    if (gpsQualityOK() &&
        servosReady())
    {
      if (failsafeGoodSinceMs ==
          0)
      {
        failsafeGoodSinceMs =
          millis();


        Serial.println(
          "GPS + ST-3009 recovered"
        );


        Serial.println(
          "Waiting 3 seconds..."
        );
      }
      else if (millis() -
               failsafeGoodSinceMs >=
               FAILSAFE_RECOVERY_MS)
      {
        state =
          STATE_AUTONAV;


        failsafeGoodSinceMs =
          0;


        Serial.println(
          "FAILSAFE RECOVERY COMPLETE"
        );


        Serial.println(
          "AUTONAV RESUMED"
        );
      }
    }
    else
    {
      failsafeGoodSinceMs =
        0;
    }


    return;
  }
}


// ================================================================
// SERIAL INPUT
// ================================================================

void processSerialCommands()
{
  while (Serial.available() > 0)
  {
    char c =
      (char)Serial.read();


    // ------------------------------------------------------------
    // Newline
    // ------------------------------------------------------------

```

```
    if (c == '\r' ||
        c == '\n')
    {
      if (commandIndex > 0)
      {
        commandBuffer[
          commandIndex] =
          '\0';


        processCommand(
          commandBuffer
        );


        commandIndex =
          0;
      }


      continue;
    }


    // ---------------------------------------------------------
    // Store
    // ---------------------------------------------------------

    if (commandIndex <
        COMMAND_BUFFER_SIZE - 1)
    {
      commandBuffer[
        commandIndex++] =
        c;
    }
    else
    {
      commandIndex = 0;

      Serial.println(
        "ERROR: COMMAND TOO LONG"
      );
    }
  }
}


// ================================================================
// COMMAND PROCESSOR
// ================================================================

void processCommand(
  char *cmd)
{
  // Remove leading spaces
  while (*cmd == ' ' ||
         *cmd == '\t')
  {
    cmd++;
  }


  // Remove trailing spaces
  int len =
    strlen(cmd);


  while (len > 0 &&
         (cmd[len - 1] == ' ' ||
          cmd[len - 1] == '\t'))
  {
    cmd[len - 1] =
      '\0';

    len--;
```

```
  }


  if (len == 0)
    return;


  Serial.print(
    "CMD RECEIVED: ["
  );

  Serial.print(
    cmd
  );

  Serial.println(
    "]"
  );


  // ================================================================
  // STATUS
  // ================================================================

  if (strcmp(
        cmd,
        "STATUS") == 0)
  {
    printStatus();
    return;
  }


  // ================================================================
  // MOTORTEST
  // ================================================================

  if (strcmp(
        cmd,
        "MOTORTEST") == 0)
  {
    startMotorTest();
    return;
  }


  // ================================================================
  // LEFT
  // ================================================================

  if (strcmp(
        cmd,
        "LEFT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      +300
    );


    return;
  }


  // ================================================================
  // RIGHT
  // ================================================================

```

```
  if (strcmp(
        cmd,
        "RIGHT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      -300
    );


    return;
  }


  // ============================================================
  // CENTER
  // ============================================================

  if (strcmp(
        cmd,
        "CENTER") == 0)
  {
    motorTestActive =
      false;


    if (servosReady())
    {
      centerServos();


      Serial.println(
        "ST-3009 -> CENTER"
      );
    }
    else
    {
      Serial.println(
        "ST-3009 BUS NOT READY"
      );
    }


    state =
      STATE_WAIT_READY;


    return;
  }


  // ============================================================
  // AUTONAV
  // ============================================================

  if (strcmp(
        cmd,
        "AUTONAV") == 0)
  {
    motorTestActive =
      false;


    if (gpsQualityOK() &&
        servosReady())
    {
      state =
        STATE_AUTONAV;
```

```

      Serial.println(
        "AUTONAV ENABLED"
      );
    }
    else
    {
      Serial.println(
        "AUTONAV BLOCKED"
      );


      Serial.print(
        "GPS = "
      );

      Serial.println(
        gpsQualityOK()
          ? "GOOD"
          : "BAD"
      );


      Serial.print(
        "SERVO = "
      );

      Serial.println(
        servosReady()
          ? "GOOD"
          : "BAD"
      );
    }


    return;
  }


  // ============================================================
  // DEPLOY
  // ============================================================

  if (strcmp(
        cmd,
        "DEPLOY") == 0)
  {
    motorTestActive =
      false;


    startDeployment();

    return;
  }


  // ============================================================
  // PREP
  // ============================================================

  if (strcmp(
        cmd,
        "PREP") == 0)
  {
    motorTestActive =
      false;


    prepareDeploymentServo();

    return;
  }

```

```
3142
3143  // ================================================================
3144  // RESET
3145  // ================================================================
3146
3147  if (strcmp(
3148        cmd,
3149        "RESET") == 0)
3150  {
3151    motorTestActive =
3152      false;
3153
3154
3155    resetSystem();
3156
3157    return;
3158  }
3159
3160
3161  // ================================================================
3162  // HELP
3163  // ================================================================
3164
3165  if (strcmp(
3166        cmd,
3167        "HELP") == 0)
3168  {
3169    Serial.println();
3170
3171    Serial.println(
3172      "STATUS"
3173    );
3174
3175    Serial.println(
3176      "MOTORTEST"
3177    );
3178
3179    Serial.println(
3180      "LEFT"
3181    );
3182
3183    Serial.println(
3184      "RIGHT"
3185    );
3186
3187    Serial.println(
3188      "CENTER"
3189    );
3190
3191    Serial.println(
3192      "AUTONAV"
3193    );
3194
3195    Serial.println(
3196      "DEPLOY"
3197    );
3198
3199    Serial.println(
3200      "PREP"
3201    );
3202
3203    Serial.println(
3204      "RESET"
3205    );
3206
3207    Serial.println(
3208      "HELP"
3209    );
3210
3211
3212    Serial.println();
3213
3214    Serial.println(
3215      "TARGET IS FIXED IN CODE"
3216    );
```

```
3217
3218
3219      return;
3220    }
3221
3222
3223    // Unknown
3224    Serial.print(
3225      "UNKNOWN COMMAND: "
3226    );
3227
3228    Serial.println(
3229      cmd
3230    );
3231  }
3232
3233
3234  // ================================================================
3235  // RESET
3236  // ================================================================
3237
3238  void resetSystem()
3239  {
3240    // Stop deployment servo
3241    if (deploymentServoActive)
3242    {
3243      deployServo.detach();
3244
3245      deploymentServoActive =
3246        false;
3247    }
3248
3249
3250    // Stop motor test
3251    motorTestActive =
3252      false;
3253
3254
3255    // Center if available
3256    if (servosReady())
3257    {
3258      centerServos();
3259    }
3260
3261
3262    // Reset flags
3263    deploymentDone =
3264      false;
3265
3266
3267    steeringDemand =
3268      0;
3269
3270
3271    failsafeGoodSinceMs =
3272      0;
3273
3274
3275    // Return to wait state
3276    state =
3277      STATE_WAIT_READY;
3278
3279
3280    stateEnteredMs =
3281      millis();
3282
3283
3284    Serial.println();
3285
3286    Serial.println(
3287      "========================================"
3288    );
3289
3290    Serial.println(
3291      "SYSTEM RESET"
```

```
  );

  Serial.print(
    "TARGET = "
  );

  Serial.print(
    TARGET_LAT,
    7
  );

  Serial.print(
    ", "
  );

  Serial.println(
    TARGET_LON,
    7
  );

  Serial.println(
    "========================================"
  );
}


// ================================================================
// STATUS
// ================================================================

void printStatus()
{
  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "                    STATUS"
  );

  Serial.println(
    "================================================"
  );


  // System
  Serial.print(
    "State             : "
  );

  Serial.println(
    stateName(state)
  );


  // Target
  Serial.print(
    "Target latitude   : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "Target longitude  : "
  );

  Serial.println(
    TARGET_LON,
    7
```

```
  );


  // GPS
  Serial.print(
    "GPS fix           : "
  );

  Serial.println(
    gpsFix
      ? "YES"
      : "NO"
  );


  Serial.print(
    "GPS quality       : "
  );

  Serial.println(
    gpsQualityOK()
      ? "GOOD"
      : "BAD"
  );


  Serial.print(
    "Satellites        : "
  );

  Serial.println(
    gpsSats
  );


  Serial.print(
    "HDOP              : "
  );

  Serial.println(
    gpsHDOP,
    2
  );


  Serial.print(
    "Latitude          : "
  );

  Serial.println(
    currentLatitude,
    7
  );


  Serial.print(
    "Longitude         : "
  );

  Serial.println(
    currentLongitude,
    7
  );


  Serial.print(
    "Speed             : "
  );

  Serial.print(
    currentSpeed,
    2
  );

  Serial.println(
```

```
    " m/s"
  );


  // Navigation
  Serial.print(
    "Distance to target: "
  );

  Serial.print(
    distanceToTarget,
    2
  );

  Serial.println(
    " m"
  );


  Serial.print(
    "Target bearing    : "
  );

  Serial.print(
    targetBearing,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Current heading   : "
  );

  Serial.print(
    currentDirection,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Heading source    : "
  );

  Serial.println(
    headingSource
  );


  Serial.print(
    "Course error      : "
  );

  Serial.print(
    courseError,
    2
  );

  Serial.println(
    " deg"
  );


  // Compass
  Serial.print(
    "Compass status    : "
  );

```

```
  Serial.println(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Raw magnetometer  : "
  );

  Serial.print(
    rawMagX
  );

  Serial.print(
    ", "
  );

  Serial.print(
    rawMagY
  );

  Serial.print(
    ", "
  );

  Serial.println(
    rawMagZ
  );


  Serial.print(
    "Compass heading   : "
  );

  Serial.print(
    compassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Navigation heading: "
  );

  Serial.print(
    navCompassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  // Servos
  Serial.print(
    "Servo 1           : "
  );

  Serial.println(
    servo1OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 2           : "
  );
```

```
3592
3593  Serial.println(
3594    servo2OK
3595      ? "OK"
3596      : "FAIL"
3597  );
3598
3599
3600  Serial.print(
3601    "Servo 1 position  : "
3602  );
3603
3604  Serial.println(
3605    servo1Position
3606  );
3607
3608
3609  Serial.print(
3610    "Servo 2 position  : "
3611  );
3612
3613  Serial.println(
3614    servo2Position
3615  );
3616
3617
3618  Serial.print(
3619    "Steering demand   : "
3620  );
3621
3622  Serial.println(
3623    steeringDemand
3624  );
3625
3626
3627  // IMU
3628  Serial.print(
3629    "IMU               : "
3630  );
3631
3632  Serial.println(
3633    imuOK
3634      ? "OK"
3635      : "FAIL"
3636  );
3637
3638
3639  // BMP
3640  Serial.print(
3641    "BMP581            : "
3642  );
3643
3644
3645  if (bmpOK)
3646  {
3647    Serial.print(
3648      "OK at 0x"
3649    );
3650
3651
3652    if (bmpAddress < 16)
3653      Serial.print("0");
3654
3655
3656    Serial.println(
3657      bmpAddress,
3658      HEX
3659    );
3660  }
3661  else
3662  {
3663    Serial.println(
3664      "NOT FOUND"
3665    );
3666  }
```

```
3667
3668
3669    // Deployment
3670    Serial.print(
3671      "Deployment done   : "
3672    );
3673
3674    Serial.println(
3675      deploymentDone
3676        ? "YES"
3677        : "NO"
3678    );
3679
3680
3681    Serial.println(
3682      "================================================"
3683    );
3684  }
3685
3686
3687  // ================================================================
3688  // LIVE DATA
3689  // ================================================================
3690
3691  void printLiveData()
3692  {
3693    Serial.print(
3694      "[LIVE] State="
3695    );
3696
3697    Serial.print(
3698      stateName(state)
3699    );
3700
3701
3702    // GPS
3703    Serial.print(
3704      " GPS="
3705    );
3706
3707    Serial.print(
3708      gpsQualityOK()
3709        ? "GOOD"
3710        : "BAD"
3711    );
3712
3713
3714    Serial.print(
3715      " SAT="
3716    );
3717
3718    Serial.print(
3719      gpsSats
3720    );
3721
3722
3723    Serial.print(
3724      " HDOP="
3725    );
3726
3727    Serial.print(
3728      gpsHDOP,
3729      1
3730    );
3731
3732
3733    if (gpsFix)
3734    {
3735      Serial.print(
3736        " LAT="
3737      );
3738
3739      Serial.print(
3740        currentLatitude,
3741        6
```

```
    );


    Serial.print(
      " LON="
    );

    Serial.print(
      currentLongitude,
      6
    );
  }


  // Navigation
  if (gpsQualityOK())
  {
    Serial.print(
      " DIST="
    );

    Serial.print(
      distanceToTarget,
      1
    );


    Serial.print(
      "m BRG="
    );

    Serial.print(
      targetBearing,
      1
    );
  }


  // Heading
  if (headingValid)
  {
    Serial.print(
      " HDG="
    );

    Serial.print(
      currentDirection,
      1
    );


    Serial.print(
      " SRC="
    );

    Serial.print(
      headingSource
    );


    Serial.print(
      " ERR="
    );

    Serial.print(
      courseError,
      1
    );
  }
  else
  {
    Serial.print(
      " HDG=NONE"
    );
  }
```

```
3817
3818
3819    // Steering
3820    Serial.print(
3821      " DEM="
3822    );
3823
3824    Serial.print(
3825      steeringDemand
3826    );
3827
3828
3829    Serial.print(
3830      " S1="
3831    );
3832
3833    Serial.print(
3834      servo1Position
3835    );
3836
3837
3838    Serial.print(
3839      " S2="
3840    );
3841
3842    Serial.print(
3843      servo2Position
3844    );
3845
3846
3847    // Compass
3848    Serial.print(
3849      " CMP="
3850    );
3851
3852    Serial.print(
3853      compassOK
3854        ? "OK"
3855        : "FAIL"
3856    );
3857
3858
3859    Serial.print(
3860      " CHP="
3861    );
3862
3863    Serial.print(
3864      compassHeading,
3865      1
3866    );
3867
3868
3869    Serial.print(
3870      " NAV="
3871    );
3872
3873    Serial.print(
3874      navCompassHeading,
3875      1
3876    );
3877
3878
3879    // IMU
3880    Serial.print(
3881      " IMU="
3882    );
3883
3884    Serial.print(
3885      imuOK
3886        ? "OK"
3887        : "FAIL"
3888    );
3889
3890
3891    // Servos
```

```
  Serial.print(
    " SERVO="
  );

  Serial.println(
    servosReady()
      ? "OK"
      : "FAIL"
  );
}


// ================================================================
// STATE NAME
// ================================================================

const char *stateName(
  SystemState s)
{
  switch (s)
  {
    case STATE_WAIT_READY:
      return "WAIT_READY";


    case STATE_DEPLOYING:
      return "DEPLOYING";


    case STATE_STABILIZING:
      return "STABILIZING";


    case STATE_AUTONAV:
      return "AUTONAV";


    case STATE_CAPTURED:
      return "CAPTURED";


    case STATE_FAILSAFE:
      return "FAILSAFE";


    case STATE_MANUAL_TEST:
      return "MANUAL_TEST";


    default:
      return "UNKNOWN";
  }
}
```

A.5 Archived source 5: Pasted text(20261003-150406).txt

Source file: `Pasted text(20261003-150406).txt`

```
/*
  ================================================================
  TEENSY 4.1 AUTONOMOUS PARAFOIL
  GUIDED LANDING - FINAL LAB TEST
  ================================================================

  TARGET IS FIXED IN CODE.

  CHANGE ONLY:
      TARGET_LAT
      TARGET_LON


  ================================================================
  HARDWARE
```

```
================================================================

TEENSY 4.1

----------------------------------------------------------------
FOXEER M10Q 180 GPS
----------------------------------------------------------------

    GPS TX  -> Teensy Pin 15 (RX3)
    GPS RX  -> Teensy Pin 14 (TX3)
    GPS GND -> Teensy GND

    GPS UART = Serial3
    GPS BAUD = 38400

----------------------------------------------------------------
I2C BUS
----------------------------------------------------------------

    SDA = Teensy Pin 18
    SCL = Teensy Pin 19

----------------------------------------------------------------
QMC5883 COMPASS
----------------------------------------------------------------

    Address = 0x0D

----------------------------------------------------------------
LSM6DSO32 IMU
----------------------------------------------------------------

    Address = 0x6A

----------------------------------------------------------------
BMP581
----------------------------------------------------------------

    Address = 0x46 or 0x47
    Diagnostic only

----------------------------------------------------------------
ST-3009
----------------------------------------------------------------

    Servo 1 ID = 1
    Servo 2 ID = 2

    Serial1
    Pin 0 = RX1
    Pin 1 = TX1

    1 Mbps
    SERIAL_8N1_HALF_DUPLEX

    Servo neutral = 2048

----------------------------------------------------------------
SG90 DEPLOYMENT SERVO
----------------------------------------------------------------

    Signal = Pin 8

================================================================
NAVIGATION
================================================================

Target bearing:
    GPS position -> target

Current heading:

    If speed >= 1.5 m/s:
        GPS course

```

```
    If speed < 1.5 m/s:
        QMC5883 compass

Course error:

    target bearing - current heading

    wrapped to -180 ... +180 degrees

Steering:

    error <= +/-5 degrees:
        neutral

    error reaches +/-90 degrees:
        maximum steering

Maximum steering:
    +/-450 counts

Differential:

    Positive demand:
        Servo 1 = 2048 - demand
        Servo 2 = 2048 + demand

    Negative demand:
        Servo 1 = 2048 - demand
        Servo 2 = 2048 + demand

Target capture:
    35 m

================================================================
STATE MACHINE
================================================================

    WAIT_READY
         |
         | GPS GOOD + Servo 1 + Servo 2
         v
    DEPLOYING
         |
         | 1.5 sec
         v
    STABILIZING
         |
         | 5 sec + GPS + servos
         v
     AUTONAV
         |
         | <=35 m
         v
     CAPTURED

GPS or servo failure during AUTONAV:
     AUTONAV -> FAILSAFE

3 sec good GPS + servos:
     FAILSAFE -> AUTONAV

================================================================
SERIAL COMMANDS
================================================================

    STATUS
    MOTORTEST
    LEFT
    RIGHT
    CENTER
    AUTONAV
    DEPLOY
    PREP
    RESET
    HELP
```

```
166
167   There is NO TARGET serial command.
168
169   ====================================================================
170 */
171
172
173 // ====================================================================
174 // LIBRARIES
175 // ====================================================================
176
177 #include <Arduino.h>
178 #include <Wire.h>
179 #include <Servo.h>
180 #include <TinyGPS++.h>
181 #include <SCServo.h>
182 #include <Adafruit_LSM6DSOX.h>
183
184 #include <math.h>
185 #include <string.h>
186
187
188 // ====================================================================
189 // TARGET
190 // ====================================================================
191
192 const double TARGET_LAT = 32.639340;
193 const double TARGET_LON = 75.014533;
194
195
196 // ====================================================================
197 // AUTO DEPLOY
198 // ====================================================================
199 //
200 // true  = deploy automatically after GPS + both ST-3009 ready
201 // false = wait for DEPLOY command
202 //
203 // For the autonomous lab sequence use TRUE.
204 // ====================================================================
205
206 const bool AUTO_DEPLOY_AFTER_READY = true;
207
208
209 // ====================================================================
210 // GPS
211 // ====================================================================
212
213 #define GPS_SERIAL Serial3
214
215 const uint32_t GPS_BAUD = 38400;
216
217 const uint8_t GPS_RX_PIN = 15;
218 const uint8_t GPS_TX_PIN = 14;
219
220 // GPS data considered stale after this time
221 const uint32_t GPS_DATA_TIMEOUT_MS = 2500;
222
223 // Minimum satellites
224 const uint8_t GPS_MIN_SATS = 5;
225
226 // Maximum acceptable HDOP
227 const double GPS_MAX_HDOP = 3.5;
228
229 // GPS course is used only above this speed
230 const double GPS_COURSE_MIN_SPEED_MPS = 1.5;
231
232
233 // ====================================================================
234 // I2C
235 // ====================================================================
236
237 const uint8_t I2C_SDA_PIN = 18;
238 const uint8_t I2C_SCL_PIN = 19;
239
240
```

```
// ================================================================
// QMC5883
// ================================================================

const uint8_t QMC5883_ADDR = 0x0D;


// ----------------------------------------------------------------
// Compass navigation orientation
// ----------------------------------------------------------------
//
// Your tested compass direction was mirrored for navigation.
//

const bool COMPASS_MIRROR_FOR_NAV = true;

const float COMPASS_HEADING_OFFSET_DEG = 0.0f;


// ----------------------------------------------------------------
// Calibration values from your working compass test
// ----------------------------------------------------------------

const float OFFSET_X = 180.0000f;
const float OFFSET_Y = 336.0000f;
const float OFFSET_Z = 615.0000f;

const float SCALE_X = 0.956484f;
const float SCALE_Y = 0.904406f;
const float SCALE_Z = 1.178127f;


// ================================================================
// LSM6DSO32
// ================================================================

const uint8_t LSM6DSO32_ADDR = 0x6A;

Adafruit_LSM6DSOX lsm6dso;

bool imuOK = false;


// ================================================================
// BMP581
// ================================================================

const uint8_t BMP581_ADDR_1 = 0x46;
const uint8_t BMP581_ADDR_2 = 0x47;

bool bmpOK = false;

uint8_t bmpAddress = 0;


// ================================================================
// GPS VARIABLES
// ================================================================

TinyGPSPlus gps;

bool gpsFix = false;

uint8_t gpsSats = 0;

double gpsHDOP = 99.9;

double currentSpeed = 0.0;

unsigned long lastGPSLocationMs = 0;


// ================================================================
// COMPASS VARIABLES
// ================================================================
```

```

bool compassOK = false;

int16_t rawMagX = 0;
int16_t rawMagY = 0;
int16_t rawMagZ = 0;

float magX = 0.0f;
float magY = 0.0f;
float magZ = 0.0f;

float compassHeading = 0.0f;
float navCompassHeading = 0.0f;

uint8_t compassFailCount = 0;


// ================================================================
// IMU VARIABLES
// ================================================================

float accelX = 0.0f;
float accelY = 0.0f;
float accelZ = 0.0f;

float gyroX = 0.0f;
float gyroY = 0.0f;
float gyroZ = 0.0f;

unsigned long lastSensorUpdateMs = 0;


// ================================================================
// ST-3009
// ================================================================
//
// IMPORTANT:
// SCServo 1.0.2 uses:
//
//     SMS_STS
//

SMS_STS st;


// ----------------------------------------------------------------
// Servo IDs
// ----------------------------------------------------------------

const uint8_t SERVO1_ID = 1;
const uint8_t SERVO2_ID = 2;


// ----------------------------------------------------------------
// Servo positions
// ----------------------------------------------------------------

const int SERVO_NEUTRAL = 2048;

const int SERVO_MIN_POS = 200;
const int SERVO_MAX_POS = 3895;


// ----------------------------------------------------------------
// Steering
// ----------------------------------------------------------------

const int MAX_STEERING_COUNTS = 450;

const int STEERING_SERVO_SPEED = 500;
const int STEERING_SERVO_ACCEL = 50;

const int MAX_SLEW_COUNTS = 60;


```

```
// ----------------------------------------------------------------
// Servo health
// ----------------------------------------------------------------

const unsigned long SERVO_HEALTH_INTERVAL_MS = 2000;

bool servo1OK = false;
bool servo2OK = false;
bool servoBusReady = false;

unsigned long lastServoHealthMs = 0;


// ----------------------------------------------------------------
// Current positions
// ----------------------------------------------------------------

int servo1Position = SERVO_NEUTRAL;
int servo2Position = SERVO_NEUTRAL;


// ----------------------------------------------------------------
// Current navigation demand
// ----------------------------------------------------------------

int steeringDemand = 0;


// ================================================================
// SG90 DEPLOYMENT
// ================================================================

Servo deployServo;

const uint8_t DEPLOY_SERVO_PIN = 8;

const int DEPLOY_ANGLE = 180;

const int PREP_ANGLE = 140;

const unsigned long DEPLOY_MOVE_TIME_MS = 1500;

const unsigned long DEPLOY_STABILIZE_TIME_MS = 5000;

bool deploymentServoActive = false;

bool deploymentDone = false;

unsigned long deploymentStartMs = 0;

unsigned long stabilizationStartMs = 0;


// ================================================================
// NAVIGATION
// ================================================================

const double TARGET_CAPTURE_RADIUS_M = 35.0;

const double STEERING_DEADBAND_DEG = 5.0;

const double FULL_STEERING_ERROR_DEG = 90.0;


// ================================================================
// STATE MACHINE
// ================================================================

enum SystemState
{
  STATE_WAIT_READY,
  STATE_DEPLOYING,
  STATE_STABILIZING,
  STATE_AUTONAV,
  STATE_CAPTURED,
```

```
  STATE_FAILSAFE,
  STATE_MANUAL_TEST
};

SystemState state = STATE_WAIT_READY;

unsigned long stateEnteredMs = 0;


// ================================================================
// NAVIGATION VARIABLES
// ================================================================

double currentLatitude = 0.0;

double currentLongitude = 0.0;

double distanceToTarget = 0.0;

double targetBearing = 0.0;

double currentDirection = 0.0;

double courseError = 0.0;

bool headingValid = false;

const char *headingSource = "NONE";


// ================================================================
// FAILSAFE
// ================================================================

unsigned long failsafeGoodSinceMs = 0;

const unsigned long FAILSAFE_RECOVERY_MS = 3000;


// ================================================================
// SERIAL COMMAND BUFFER
// ================================================================

#define COMMAND_BUFFER_SIZE 120

char commandBuffer[COMMAND_BUFFER_SIZE];

uint16_t commandIndex = 0;


// ================================================================
// TIMERS
// ================================================================

unsigned long lastNavigationUpdateMs = 0;

const unsigned long NAVIGATION_UPDATE_INTERVAL_MS = 100;

unsigned long lastLivePrintMs = 0;

const unsigned long LIVE_PRINT_INTERVAL_MS = 1000;

unsigned long lastBMPCheckMs = 0;

const unsigned long BMP_CHECK_INTERVAL_MS = 3000;


// ================================================================
// MOTOR TEST
// ================================================================

bool motorTestActive = false;

uint8_t motorTestStep = 0;

```

```
unsigned long motorTestTimer = 0;


// ================================================================
// FORWARD DECLARATIONS
// ================================================================

void processSerialCommands();
void processCommand(char *cmd);

void updateGPS();
void updateCompass();
void updateIMU();
void updateServoHealth();

void updateNavigation();
void updateSteering();
void updateStateMachine();

void writeSteeringPositions(int pos1, int pos2);
void centerServos();
void commandSteering(int demand);

void startDeployment();
void prepareDeploymentServo();
void resetSystem();

void startMotorTest();
void updateMotorTest();

void printStatus();
void printLiveData();

bool gpsQualityOK();
bool readQMC5883();
bool initQMC5883();

bool checkI2CDevice(uint8_t address);
bool initIMU();

bool servosReady();

float wrap360(float angle);
double wrap180(double angle);
float navigationCompassHeading();

const char *stateName(SystemState s);

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep
);


// ================================================================
// SETUP
// ================================================================

void setup()
{
  // ------------------------------------------------------------
  // USB SERIAL
  // ------------------------------------------------------------

  Serial.begin(115200);

  delay(1500);


  Serial.println();
  Serial.println(
    "================================================"
  );

```

```
  Serial.println(
    " TEENSY 4.1 AUTONOMOUS PARAFOIL"
  );

  Serial.println(
    " GUIDED LANDING - FINAL LAB TEST"
  );

  Serial.println(
    "================================================"
  );


  // ---------------------------------------------------------------
  // TARGET
  // ---------------------------------------------------------------

  Serial.println();

  Serial.print(
    "TARGET LATITUDE  : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "TARGET LONGITUDE : "
  );

  Serial.println(
    TARGET_LON,
    7
  );


  // ---------------------------------------------------------------
  // I2C
  // ---------------------------------------------------------------

  Wire.begin();

  Wire.setClock(400000);


  Serial.println();

  Serial.println(
    "I2C initialized"
  );

  Serial.println(
    "SDA = Pin 18"
  );

  Serial.println(
    "SCL = Pin 19"
  );


  // ---------------------------------------------------------------
  // QMC5883
  // ---------------------------------------------------------------

  if (initQMC5883())
  {
    compassOK = true;

    Serial.println(
      "QMC5883 : OK"
    );
  }
```

```
  else
  {
    compassOK = false;

    Serial.println(
      "QMC5883 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // LSM6DSO32
  // ---------------------------------------------------------------

  if (initIMU())
  {
    imuOK = true;

    Serial.println(
      "LSM6DSO32 : OK"
    );
  }
  else
  {
    imuOK = false;

    Serial.println(
      "LSM6DSO32 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // BMP581
  // ---------------------------------------------------------------

  if (checkI2CDevice(
        BMP581_ADDR_1))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_1;
  }
  else if (checkI2CDevice(
             BMP581_ADDR_2))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_2;
  }
  else
  {
    bmpOK = false;

    bmpAddress = 0;
  }


  if (bmpOK)
  {
    Serial.print(
      "BMP581 : FOUND at 0x"
    );

    if (bmpAddress < 16)
      Serial.print("0");

    Serial.println(
      bmpAddress,
      HEX
    );
  }
  else
```

```
  {
    Serial.println(
      "BMP581 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // GPS
  // ---------------------------------------------------------------
  //
  // EXACTLY MATCHES YOUR WORKING GPS TEST
  //

  GPS_SERIAL.begin(
    GPS_BAUD
  );


  Serial.println();

  Serial.println(
    "GPS UART  : Serial3"
  );

  Serial.print(
    "GPS RX    : Pin "
  );

  Serial.println(
    GPS_RX_PIN
  );

  Serial.print(
    "GPS TX    : Pin "
  );

  Serial.println(
    GPS_TX_PIN
  );

  Serial.print(
    "GPS BAUD  : "
  );

  Serial.println(
    GPS_BAUD
  );


  // ---------------------------------------------------------------
  // ST-3009
  // ---------------------------------------------------------------

  Serial.println();

  Serial.println(
    "Initializing ST-3009 bus..."
  );


  Serial1.begin(
    1000000,
    SERIAL_8N1_HALF_DUPLEX
  );


  st.pSerial =
    &Serial1;


  delay(500);


  // Force immediate first health test
```

```
  lastServoHealthMs =
    millis() -
    SERVO_HEALTH_INTERVAL_MS;


  updateServoHealth();


  if (servosReady())
  {
    Serial.println(
      "ST-3009 ID 1 : OK"
    );

    Serial.println(
      "ST-3009 ID 2 : OK"
    );


    centerServos();
  }
  else
  {
    Serial.println(
      "ST-3009 : WAITING"
    );
  }


  // ---------------------------------------------------------------
  // SG90
  // ---------------------------------------------------------------

  deployServo.detach();

  deploymentServoActive =
    false;


  // ---------------------------------------------------------------
  // State
  // ---------------------------------------------------------------

  state =
    STATE_WAIT_READY;

  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "SYSTEM READY"
  );

  Serial.println(
    "Target is fixed in code."
  );

  Serial.println(
    "================================================"
  );


  Serial.println();

  Serial.println(
    "COMMANDS:"
  );

```

```
  Serial.println(
    "STATUS"
  );

  Serial.println(
    "MOTORTEST"
  );

  Serial.println(
    "LEFT"
  );

  Serial.println(
    "RIGHT"
  );

  Serial.println(
    "CENTER"
  );

  Serial.println(
    "AUTONAV"
  );

  Serial.println(
    "DEPLOY"
  );

  Serial.println(
    "PREP"
  );

  Serial.println(
    "RESET"
  );

  Serial.println(
    "HELP"
  );


  Serial.println();

  Serial.println(
    "Waiting for GPS + ST-3009..."
  );
}


// ================================================================
// LOOP
// ================================================================

void loop()
{
  // GPS continuously
  updateGPS();


  // Serial commands
  processSerialCommands();


  // Sensors
  if (millis() -
      lastSensorUpdateMs >=
      100)
  {
    lastSensorUpdateMs =
      millis();

    updateCompass();
    updateIMU();
  }

```

```
991
992   // Servo health
993   updateServoHealth();
994
995
996   // Motor test
997   if (motorTestActive)
998   {
999     updateMotorTest();
1000  }
1001
1002
1003  // State
1004  updateStateMachine();
1005
1006
1007  // Navigation
1008  if (millis() -
1009      lastNavigationUpdateMs >=
1010      NAVIGATION_UPDATE_INTERVAL_MS)
1011  {
1012    lastNavigationUpdateMs =
1013      millis();
1014
1015    updateNavigation();
1016    updateSteering();
1017  }
1018
1019
1020  // BMP check
1021  if (millis() -
1022      lastBMPCheckMs >=
1023      BMP_CHECK_INTERVAL_MS)
1024  {
1025    lastBMPCheckMs =
1026      millis();
1027
1028
1029    if (!bmpOK)
1030    {
1031      if (checkI2CDevice(
1032            BMP581_ADDR_1))
1033      {
1034        bmpOK = true;
1035
1036        bmpAddress =
1037          BMP581_ADDR_1;
1038      }
1039      else if (checkI2CDevice(
1040                 BMP581_ADDR_2))
1041      {
1042        bmpOK = true;
1043
1044        bmpAddress =
1045          BMP581_ADDR_2;
1046      }
1047    }
1048  }
1049
1050
1051  // Live data
1052  if (millis() -
1053      lastLivePrintMs >=
1054      LIVE_PRINT_INTERVAL_MS)
1055  {
1056    lastLivePrintMs =
1057      millis();
1058
1059    printLiveData();
1060  }
1061}
1062
1063
1064// ================================================================
1065// GPS UPDATE
```

```
// ================================================================

void updateGPS()
{
  while (GPS_SERIAL.available() > 0)
  {
    char c =
      (char)GPS_SERIAL.read();

    gps.encode(c);
  }


  // Fresh GPS location
  if (gps.location.isUpdated())
  {
    lastGPSLocationMs =
      millis();
  }


  // Position
  if (gps.location.isValid())
  {
    gpsFix = true;

    currentLatitude =
      gps.location.lat();

    currentLongitude =
      gps.location.lng();
  }
  else
  {
    gpsFix = false;
  }


  // Satellites
  if (gps.satellites.isValid())
  {
    gpsSats =
      gps.satellites.value();
  }
  else
  {
    gpsSats = 0;
  }


  // HDOP
  if (gps.hdop.isValid())
  {
    gpsHDOP =
      gps.hdop.hdop();
  }
  else
  {
    gpsHDOP = 99.9;
  }


  // Speed
  if (gps.speed.isValid())
  {
    currentSpeed =
      gps.speed.mps();
  }
  else
  {
    currentSpeed = 0.0;
  }
}


```

```
// ================================================================
// GPS QUALITY
// ================================================================

bool gpsQualityOK()
{
  if (!gpsFix)
    return false;


  if (lastGPSLocationMs == 0)
    return false;


  if (millis() -
      lastGPSLocationMs >
      GPS_DATA_TIMEOUT_MS)
  {
    return false;
  }


  if (gpsSats <
      GPS_MIN_SATS)
  {
    return false;
  }


  if (gps.hdop.isValid() &&
      gpsHDOP >
      GPS_MAX_HDOP)
  {
    return false;
  }


  return true;
}


// ================================================================
// QMC5883 INIT
// ================================================================

bool initQMC5883()
{
  // Check device
  Wire.beginTransmission(
    QMC5883_ADDR
  );


  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  // SET/RESET period
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x0B);
  Wire.write(0x01);

  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  delay(10);

```

```
  // Control register
  // Same setting used by your
  // working compass test.
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x09);
  Wire.write(0x1D);

  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  delay(100);


  return true;
}


// ================================================================
// QMC5883 READ
// ================================================================

bool readQMC5883()
{
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x00);


  if (Wire.endTransmission(false) != 0)
  {
    return false;
  }


  uint8_t count =
    Wire.requestFrom(
      QMC5883_ADDR,
      (uint8_t)6
    );


  if (count != 6)
  {
    while (Wire.available())
    {
      Wire.read();
    }

    return false;
  }


  uint8_t xl =
    Wire.read();

  uint8_t xh =
    Wire.read();


  uint8_t yl =
    Wire.read();

  uint8_t yh =
    Wire.read();


```

```
  uint8_t zl =
    Wire.read();

  uint8_t zh =
    Wire.read();


  rawMagX =
    (int16_t)(
      (xh << 8) |
      xl
    );


  rawMagY =
    (int16_t)(
      (yh << 8) |
      yl
    );


  rawMagZ =
    (int16_t)(
      (zh << 8) |
      zl
    );


  return true;
}


// ================================================================
// COMPASS UPDATE
// ================================================================

void updateCompass()
{
  // ------------------------------------------------------------
  // Read sensor
  // ------------------------------------------------------------

  if (!readQMC5883())
  {
    compassFailCount++;


    // Do not immediately declare compass dead.
    // Reinitialize after 5 consecutive failures.

    if (compassFailCount >= 5)
    {
      if (initQMC5883())
      {
        compassOK = true;

        compassFailCount = 0;

        Serial.println(
          "[COMPASS] Reinitialized OK"
        );
      }
      else
      {
        compassOK = false;
      }
    }


    return;
  }


  // Successful reading
  compassFailCount = 0;
```

```
  compassOK = true;


  // ------------------------------------------------------------
  // Apply your proven calibration
  // ------------------------------------------------------------

  magX =
    ((float)rawMagX -
     OFFSET_X) *
    SCALE_X;


  magY =
    ((float)rawMagY -
     OFFSET_Y) *
    SCALE_Y;


  magZ =
    ((float)rawMagZ -
     OFFSET_Z) *
    SCALE_Z;


  // ------------------------------------------------------------
  // Calculate heading
  // ------------------------------------------------------------

  float angle =
    atan2(
      magY,
      magX
    ) *
    180.0f /
    PI;


  if (angle < 0.0f)
    angle += 360.0f;


  compassHeading =
    wrap360(
      angle +
      COMPASS_HEADING_OFFSET_DEG
    );


  // Navigation heading
  navCompassHeading =
    navigationCompassHeading();
}


// ================================================================
// NAVIGATION COMPASS
// ================================================================

float navigationCompassHeading()
{
  if (!COMPASS_MIRROR_FOR_NAV)
  {
    return compassHeading;
  }


  return wrap360(
    360.0f -
    compassHeading
  );
}


// ================================================================
```

```
// WRAP 0...360
// ================================================================

float wrap360(float angle)
{
  while (angle >= 360.0f)
    angle -= 360.0f;


  while (angle < 0.0f)
    angle += 360.0f;


  return angle;
}


// ================================================================
// WRAP -180...180
// ================================================================

double wrap180(double angle)
{
  while (angle > 180.0)
    angle -= 360.0;


  while (angle < -180.0)
    angle += 360.0;


  return angle;
}


// ================================================================
// IMU INIT
// ================================================================

bool initIMU()
{
  if (!lsm6dso.begin_I2C(
        LSM6DSO32_ADDR,
        &Wire))
  {
    return false;
  }


  lsm6dso.setAccelRange(
    LSM6DS_ACCEL_RANGE_16_G
  );


  lsm6dso.setGyroRange(
    LSM6DS_GYRO_RANGE_2000_DPS
  );


  lsm6dso.setAccelDataRate(
    LSM6DS_RATE_208_HZ
  );


  lsm6dso.setGyroDataRate(
    LSM6DS_RATE_208_HZ
  );


  return true;
}


// ================================================================
// IMU UPDATE
```

```
// ================================================================

void updateIMU()
{
  if (!imuOK)
    return;


  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;


  lsm6dso.getEvent(
    &accel,
    &gyro,
    &temp
  );


  accelX =
    accel.acceleration.x;


  accelY =
    accel.acceleration.y;


  accelZ =
    accel.acceleration.z;


  gyroX =
    gyro.gyro.x;


  gyroY =
    gyro.gyro.y;


  gyroZ =
    gyro.gyro.z;
}


// ================================================================
// I2C CHECK
// ================================================================

bool checkI2CDevice(
  uint8_t address)
{
  Wire.beginTransmission(
    address
  );


  return (
    Wire.endTransmission() == 0
  );
}


// ================================================================
// SERVO HEALTH
// ================================================================

void updateServoHealth()
{
  if (millis() -
      lastServoHealthMs <
      SERVO_HEALTH_INTERVAL_MS)
  {
    return;
  }
```

```
1591
1592
1593    lastServoHealthMs =
1594      millis();
1595
1596
1597    // Ping ID 1
1598    int result1 =
1599      st.Ping(
1600        SERVO1_ID
1601      );
1602
1603
1604    delay(10);
1605
1606
1607    // Ping ID 2
1608    int result2 =
1609      st.Ping(
1610        SERVO2_ID
1611      );
1612
1613
1614    servo1OK =
1615      (result1 >= 0);
1616
1617
1618    servo2OK =
1619      (result2 >= 0);
1620
1621
1622    servoBusReady =
1623      servo1OK &&
1624      servo2OK;
1625  }
1626
1627
1628  // ================================================================
1629  // SERVO READY
1630  // ================================================================
1631
1632  bool servosReady()
1633  {
1634    return
1635      servoBusReady &&
1636      servo1OK &&
1637      servo2OK;
1638  }
1639
1640
1641  // ================================================================
1642  // CENTER BOTH SERVOS
1643  // ================================================================
1644
1645  void centerServos()
1646  {
1647    if (!servosReady())
1648      return;
1649
1650
1651    st.WritePosEx(
1652      SERVO1_ID,
1653      SERVO_NEUTRAL,
1654      STEERING_SERVO_SPEED,
1655      STEERING_SERVO_ACCEL
1656    );
1657
1658
1659    st.WritePosEx(
1660      SERVO2_ID,
1661      SERVO_NEUTRAL,
1662      STEERING_SERVO_SPEED,
1663      STEERING_SERVO_ACCEL
1664    );
1665
```

```

  servo1Position =
    SERVO_NEUTRAL;


  servo2Position =
    SERVO_NEUTRAL;


  steeringDemand = 0;
}


// ================================================================
// DIRECT STEERING COMMAND
// ================================================================
//
// demand = +300:
//
//   Servo 1 = 2048 - 300 = 1748
//   Servo 2 = 2048 + 300 = 2348
//
// demand = -300:
//
//   Servo 1 = 2048 + 300 = 2348
//   Servo 2 = 2048 - 300 = 1748
//

void commandSteering(
  int demand)
{
  if (!servosReady())
  {
    Serial.println(
      "ERROR: ST-3009 BUS NOT READY"
    );

    return;
  }


  demand =
    constrain(
      demand,
      -MAX_STEERING_COUNTS,
      MAX_STEERING_COUNTS
    );


  int pos1 =
    SERVO_NEUTRAL -
    demand;


  int pos2 =
    SERVO_NEUTRAL +
    demand;


  pos1 =
    constrain(
      pos1,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


  pos2 =
    constrain(
      pos2,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


```

```
  st.WritePosEx(
    SERVO1_ID,
    pos1,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  st.WritePosEx(
    SERVO2_ID,
    pos2,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  servo1Position =
    pos1;


  servo2Position =
    pos2;


  steeringDemand =
    demand;


  Serial.print(
    "DEMAND="
  );

  Serial.print(
    demand
  );

  Serial.print(
    "  S1="
  );

  Serial.print(
    pos1
  );

  Serial.print(
    "  S2="
  );

  Serial.println(
    pos2
  );
}


// ================================================================
// MOTOR TEST START
// ================================================================

void startMotorTest()
{
  if (!servosReady())
  {
    Serial.println(
      "MOTORTEST BLOCKED: "
      "ST-3009 NOT READY"
    );

    return;
  }


  motorTestActive =
    true;


```

```
  state =
    STATE_MANUAL_TEST;


  motorTestStep = 0;

  motorTestTimer =
    millis();


  centerServos();


  Serial.println();

  Serial.println(
    "======================================="
  );

  Serial.println(
    "       ST-3009 MOTOR TEST"
  );

  Serial.println(
    "======================================="
  );

  Serial.println(
    "CENTER -> LEFT -> CENTER -> RIGHT -> CENTER"
  );

  Serial.println();
}


// ================================================================
// MOTOR TEST UPDATE
// ================================================================

void updateMotorTest()
{
  if (!servosReady())
  {
    motorTestActive =
      false;

    state =
      STATE_WAIT_READY;

    return;
  }


  if (millis() -
      motorTestTimer <
      1800)
  {
    return;
  }


  motorTestTimer =
    millis();


  switch (motorTestStep)
  {
    case 0:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

```

```
      motorTestStep++;

      break;


    case 1:

      Serial.println(
        "TEST: LEFT"
      );

      commandSteering(
        +300
      );

      motorTestStep++;

      break;


    case 2:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    case 3:

      Serial.println(
        "TEST: RIGHT"
      );

      commandSteering(
        -300
      );

      motorTestStep++;

      break;


    case 4:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    default:

      motorTestActive =
        false;


      state =
        STATE_WAIT_READY;


      centerServos();


      Serial.println();
```

```
      Serial.println(
        "MOTOR TEST COMPLETE"
      );

      Serial.println();

      break;
  }
}


// ================================================================
// NAVIGATION CALCULATION
// ================================================================

void updateNavigation()
{
  if (!gpsQualityOK())
  {
    headingValid =
      false;

    headingSource =
      "NONE";

    return;
  }


  // ---------------------------------------------------------------
  // GPS position
  // ---------------------------------------------------------------

  currentLatitude =
    gps.location.lat();


  currentLongitude =
    gps.location.lng();


  // ---------------------------------------------------------------
  // Distance
  // ---------------------------------------------------------------

  distanceToTarget =
    TinyGPSPlus::distanceBetween(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
      TARGET_LON
    );


  // ---------------------------------------------------------------
  // Bearing
  // ---------------------------------------------------------------

  targetBearing =
    TinyGPSPlus::courseTo(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
      TARGET_LON
    );


  targetBearing =
    wrap360(
      targetBearing
    );


  // ---------------------------------------------------------------
```

```
  // Determine heading
  // ------------------------------------------------------------

  headingValid =
    false;


  headingSource =
    "NONE";


  // GPS course when moving
  if (gps.speed.isValid() &&
      gps.course.isValid() &&
      currentSpeed >=
      GPS_COURSE_MIN_SPEED_MPS)
  {
    currentDirection =
      wrap360(
        gps.course.deg()
      );


    headingValid =
      true;


    headingSource =
      "GPS COURSE";
  }


  // Compass when slow/stationary
  else if (compassOK)
  {
    currentDirection =
      navigationCompassHeading();


    headingValid =
      true;


    headingSource =
      "COMPASS";
  }


  // ------------------------------------------------------------
  // Course error
  // ------------------------------------------------------------

  if (headingValid)
  {
    courseError =
      wrap180(
        targetBearing -
        currentDirection
      );
  }
}


// ================================================================
// AUTONOMOUS STEERING
// ================================================================

void updateSteering()
{
  // Manual motor test owns the servos
  if (motorTestActive)
    return;


  // Servo unavailable
```

```
  if (!servosReady())
  {
    steeringDemand = 0;

    return;
  }


  // Only steer during AUTONAV
  if (state !=
      STATE_AUTONAV)
  {
    return;
  }


  // GPS required
  if (!gpsQualityOK())
  {
    centerServos();

    return;
  }


  // Target captured
  if (distanceToTarget <=
      TARGET_CAPTURE_RADIUS_M)
  {
    centerServos();

    return;
  }


  // No heading
  if (!headingValid)
  {
    centerServos();

    return;
  }


  // ------------------------------------------------------------
  // Absolute error
  // ------------------------------------------------------------

  double absoluteError =
    fabs(courseError);


  // ------------------------------------------------------------
  // Deadband
  // ------------------------------------------------------------

  if (absoluteError <=
      STEERING_DEADBAND_DEG)
  {
    steeringDemand = 0;
  }
  else
  {
    double usableError =
      absoluteError -
      STEERING_DEADBAND_DEG;


    double usableRange =
      FULL_STEERING_ERROR_DEG -
      STEERING_DEADBAND_DEG;


    double normalized =
      usableError /
```

```
      usableRange;


    normalized =
      constrain(
        normalized,
        0.0,
        1.0
      );


    int demand =
      (int)round(
        normalized *
        MAX_STEERING_COUNTS
      );


    if (courseError > 0.0)
    {
      steeringDemand =
        demand;
    }
    else
    {
      steeringDemand =
        -demand;
    }
  }


  // ---------------------------------------------------------------
  // Target positions
  // ---------------------------------------------------------------

  int target1 =
    SERVO_NEUTRAL -
    steeringDemand;


  int target2 =
    SERVO_NEUTRAL +
    steeringDemand;


  target1 =
    constrain(
      target1,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


  target2 =
    constrain(
      target2,
      SERVO_MIN_POS,
      SERVO_MAX_POS
    );


  // ---------------------------------------------------------------
  // Slew limit
  // ---------------------------------------------------------------

  int new1 =
    slewTo(
      servo1Position,
      target1,
      MAX_SLEW_COUNTS
    );


  int new2 =
    slewTo(
```

```
      servo2Position,
      target2,
      MAX_SLEW_COUNTS
    );


  // ------------------------------------------------------------
  // Send only if changed
  // ------------------------------------------------------------

  if (new1 != servo1Position ||
      new2 != servo2Position)
  {
    st.WritePosEx(
      SERVO1_ID,
      new1,
      STEERING_SERVO_SPEED,
      STEERING_SERVO_ACCEL
    );


    st.WritePosEx(
      SERVO2_ID,
      new2,
      STEERING_SERVO_SPEED,
      STEERING_SERVO_ACCEL
    );


    servo1Position =
      new1;


    servo2Position =
      new2;
  }
}


// ================================================================
// SLEW LIMIT
// ================================================================

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep)
{
  if (targetValue >
      currentValue)
  {
    int difference =
      targetValue -
      currentValue;


    if (difference >
        maxStep)
    {
      return currentValue +
             maxStep;
    }


    return targetValue;
  }


  if (targetValue <
      currentValue)
  {
    int difference =
      currentValue -
      targetValue;

```

```
2341
2342      if (difference >
2343          maxStep)
2344      {
2345        return currentValue -
2346               maxStep;
2347      }
2348
2349
2350      return targetValue;
2351    }
2352
2353
2354    return currentValue;
2355  }
2356
2357
2358  // ================================================================
2359  // DEPLOYMENT
2360  // ================================================================
2361
2362  void startDeployment()
2363  {
2364    if (deploymentDone)
2365    {
2366      Serial.println(
2367        "DEPLOYMENT ALREADY COMPLETED"
2368      );
2369
2370      return;
2371    }
2372
2373
2374    if (!gpsQualityOK())
2375    {
2376      Serial.println(
2377        "DEPLOY BLOCKED: GPS NOT READY"
2378      );
2379
2380      return;
2381    }
2382
2383
2384    if (!servosReady())
2385    {
2386      Serial.println(
2387        "DEPLOY BLOCKED: ST-3009 NOT READY"
2388      );
2389
2390      return;
2391    }
2392
2393
2394    // Center steering first
2395    centerServos();
2396
2397
2398    delay(100);
2399
2400
2401    // Attach SG90
2402    deployServo.attach(
2403      DEPLOY_SERVO_PIN
2404    );
2405
2406
2407    delay(100);
2408
2409
2410    // Deployment position
2411    deployServo.write(
2412      DEPLOY_ANGLE
2413    );
2414
2415
```

```
  deploymentStartMs =
    millis();


  deploymentServoActive =
    true;


  deploymentDone =
    true;


  state =
    STATE_DEPLOYING;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "======================================="
  );

  Serial.println(
    "STARTING DEPLOYMENT"
  );

  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    DEPLOY_ANGLE
  );

  Serial.println(
    " degrees"
  );
}


// ================================================================
// SG90 PREP
// ================================================================

void prepareDeploymentServo()
{
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );


  delay(100);


  deployServo.write(
    PREP_ANGLE
  );


  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    PREP_ANGLE
  );

  Serial.println(
    " degrees"
  );

```

```
  delay(1000);


  deployServo.detach();


  Serial.println(
    "SG90 detached"
  );
}


// ================================================================
// STATE MACHINE
// ================================================================

void updateStateMachine()
{
  // ----------------------------------------------------------------
  // Manual test
  // ----------------------------------------------------------------

  if (state ==
      STATE_MANUAL_TEST)
  {
    return;
  }


  // ----------------------------------------------------------------
  // WAIT READY
  // ----------------------------------------------------------------

  if (state ==
      STATE_WAIT_READY)
  {
    if (AUTO_DEPLOY_AFTER_READY &&
        !deploymentDone &&
        gpsQualityOK() &&
        servosReady())
    {
      Serial.println();

      Serial.println(
        "GPS + ST-3009 READY"
      );

      Serial.println(
        "AUTO DEPLOY ENABLED"
      );


      startDeployment();
    }


    return;
  }


  // ----------------------------------------------------------------
  // DEPLOYING
  // ----------------------------------------------------------------

  if (state ==
      STATE_DEPLOYING)
  {
    steeringDemand =
      0;


    if (millis() -
        deploymentStartMs >=
        DEPLOY_MOVE_TIME_MS)
```

```
    {
      // Detach SG90
      deployServo.detach();


      deploymentServoActive =
        false;


      // Center steering
      if (servosReady())
      {
        centerServos();
      }


      // Stabilization starts
      stabilizationStartMs =
        millis();


      state =
        STATE_STABILIZING;


      Serial.println();

      Serial.println(
        "SG90 detached"
      );

      Serial.println(
        "5 second stabilization started"
      );
    }


    return;
  }


  // ------------------------------------------------------------
  // STABILIZING
  // ------------------------------------------------------------

  if (state ==
      STATE_STABILIZING)
  {
    steeringDemand =
      0;


    if (millis() -
        stabilizationStartMs >=
        DEPLOY_STABILIZE_TIME_MS)
    {
      if (gpsQualityOK() &&
          servosReady())
      {
        state =
          STATE_AUTONAV;


        Serial.println();

        Serial.println(
          "========================================"
        );

        Serial.println(
          "AUTONOMOUS NAVIGATION STARTED"
        );

        Serial.println(
          "========================================"
```

```
        );
      }
    }


    return;
  }


  // ------------------------------------------------------------
  // AUTONAV
  // ------------------------------------------------------------

  if (state ==
      STATE_AUTONAV)
  {
    // GPS / servo failure
    if (!gpsQualityOK() ||
        !servosReady())
    {
      steeringDemand =
        0;


      if (servosReady())
      {
        centerServos();
      }


      state =
        STATE_FAILSAFE;


      failsafeGoodSinceMs =
        0;


      Serial.println();

      Serial.println(
        "!!! NAVIGATION FAILSAFE !!!"
      );

      Serial.println(
        "GPS or ST-3009 unavailable"
      );


      return;
    }


    // Target capture
    if (distanceToTarget <=
        TARGET_CAPTURE_RADIUS_M)
    {
      centerServos();


      state =
        STATE_CAPTURED;


      Serial.println();

      Serial.println(
        "========================================"
      );

      Serial.println(
        "TARGET CAPTURED"
      );

      Serial.print(
```

```
        "Distance = "
      );

      Serial.print(
        distanceToTarget,
        2
      );

      Serial.println(
        " m"
      );

      Serial.println(
        "ST-3009 -> CENTER"
      );

      Serial.println(
        "========================================"
      );
    }


    return;
  }


  // ------------------------------------------------------------
  // CAPTURED
  // ------------------------------------------------------------

  if (state ==
      STATE_CAPTURED)
  {
    steeringDemand =
      0;

    return;
  }


  // ------------------------------------------------------------
  // FAILSAFE
  // ------------------------------------------------------------

  if (state ==
      STATE_FAILSAFE)
  {
    steeringDemand =
      0;


    if (servosReady())
    {
      centerServos();
    }


    if (gpsQualityOK() &&
        servosReady())
    {
      if (failsafeGoodSinceMs ==
          0)
      {
        failsafeGoodSinceMs =
          millis();


        Serial.println(
          "GPS + ST-3009 recovered"
        );


        Serial.println(
          "Waiting 3 seconds..."
        );
```

```
      }
      else if (millis() -
               failsafeGoodSinceMs >=
               FAILSAFE_RECOVERY_MS)
      {
        state =
          STATE_AUTONAV;


        failsafeGoodSinceMs =
          0;


        Serial.println(
          "FAILSAFE RECOVERY COMPLETE"
        );


        Serial.println(
          "AUTONAV RESUMED"
        );
      }
    }
    else
    {
      failsafeGoodSinceMs =
        0;
    }


    return;
  }
}


// ================================================================
// SERIAL INPUT
// ================================================================

void processSerialCommands()
{
  while (Serial.available() > 0)
  {
    char c =
      (char)Serial.read();


    // ------------------------------------------------------------
    // Newline
    // ------------------------------------------------------------

    if (c == '\r' ||
        c == '\n')
    {
      if (commandIndex > 0)
      {
        commandBuffer[
          commandIndex] =
          '\0';


        processCommand(
          commandBuffer
        );


        commandIndex =
          0;
      }


      continue;
    }


```

```
    // ---------------------------------------------------------------
    // Store
    // ---------------------------------------------------------------

    if (commandIndex <
        COMMAND_BUFFER_SIZE - 1)
    {
      commandBuffer[
        commandIndex++] =
        c;
    }
    else
    {
      commandIndex = 0;

      Serial.println(
        "ERROR: COMMAND TOO LONG"
      );
    }
  }
}


// ================================================================
// COMMAND PROCESSOR
// ================================================================

void processCommand(
  char *cmd)
{
  // Remove leading spaces
  while (*cmd == ' ' ||
         *cmd == '\t')
  {
    cmd++;
  }


  // Remove trailing spaces
  int len =
    strlen(cmd);


  while (len > 0 &&
         (cmd[len - 1] == ' ' ||
          cmd[len - 1] == '\t'))
  {
    cmd[len - 1] =
      '\0';

    len--;
  }


  if (len == 0)
    return;


  Serial.print(
    "CMD RECEIVED: ["
  );

  Serial.print(
    cmd
  );

  Serial.println(
    "]"
  );


  // ================================================================
  // STATUS
  // ================================================================

```

```
  if (strcmp(
        cmd,
        "STATUS") == 0)
  {
    printStatus();
    return;
  }


  // ============================================================
  // MOTORTEST
  // ============================================================

  if (strcmp(
        cmd,
        "MOTORTEST") == 0)
  {
    startMotorTest();
    return;
  }


  // ============================================================
  // LEFT
  // ============================================================

  if (strcmp(
        cmd,
        "LEFT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      +300
    );


    return;
  }


  // ============================================================
  // RIGHT
  // ============================================================

  if (strcmp(
        cmd,
        "RIGHT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      -300
    );


    return;
  }


  // ============================================================
  // CENTER
  // ============================================================
```

```

  if (strcmp(
        cmd,
        "CENTER") == 0)
  {
    motorTestActive =
      false;


    if (servosReady())
    {
      centerServos();


      Serial.println(
        "ST-3009 -> CENTER"
      );
    }
    else
    {
      Serial.println(
        "ST-3009 BUS NOT READY"
      );
    }


    state =
      STATE_WAIT_READY;


    return;
  }


  // ============================================================
  // AUTONAV
  // ============================================================

  if (strcmp(
        cmd,
        "AUTONAV") == 0)
  {
    motorTestActive =
      false;


    if (gpsQualityOK() &&
        servosReady())
    {
      state =
        STATE_AUTONAV;


      Serial.println(
        "AUTONAV ENABLED"
      );
    }
    else
    {
      Serial.println(
        "AUTONAV BLOCKED"
      );


      Serial.print(
        "GPS = "
      );

      Serial.println(
        gpsQualityOK()
          ? "GOOD"
          : "BAD"
      );


```

```
      Serial.print(
        "SERVO = "
      );

      Serial.println(
        servosReady()
          ? "GOOD"
          : "BAD"
      );
    }


    return;
  }


  // ============================================================
  // DEPLOY
  // ============================================================

  if (strcmp(
        cmd,
        "DEPLOY") == 0)
  {
    motorTestActive =
      false;


    startDeployment();

    return;
  }


  // ============================================================
  // PREP
  // ============================================================

  if (strcmp(
        cmd,
        "PREP") == 0)
  {
    motorTestActive =
      false;


    prepareDeploymentServo();

    return;
  }


  // ============================================================
  // RESET
  // ============================================================

  if (strcmp(
        cmd,
        "RESET") == 0)
  {
    motorTestActive =
      false;


    resetSystem();

    return;
  }


  // ============================================================
  // HELP
  // ============================================================

  if (strcmp(
```

```
        cmd,
        "HELP") == 0)
  {
    Serial.println();

    Serial.println(
      "STATUS"
    );

    Serial.println(
      "MOTORTEST"
    );

    Serial.println(
      "LEFT"
    );

    Serial.println(
      "RIGHT"
    );

    Serial.println(
      "CENTER"
    );

    Serial.println(
      "AUTONAV"
    );

    Serial.println(
      "DEPLOY"
    );

    Serial.println(
      "PREP"
    );

    Serial.println(
      "RESET"
    );

    Serial.println(
      "HELP"
    );


    Serial.println();

    Serial.println(
      "TARGET IS FIXED IN CODE"
    );


    return;
  }


  // Unknown
  Serial.print(
    "UNKNOWN COMMAND: "
  );

  Serial.println(
    cmd
  );
}


// ================================================================
// RESET
// ================================================================

void resetSystem()
{
  // Stop deployment servo
```

```
  if (deploymentServoActive)
  {
    deployServo.detach();

    deploymentServoActive =
      false;
  }


  // Stop motor test
  motorTestActive =
    false;


  // Center if available
  if (servosReady())
  {
    centerServos();
  }


  // Reset flags
  deploymentDone =
    false;


  steeringDemand =
    0;


  failsafeGoodSinceMs =
    0;


  // Return to wait state
  state =
    STATE_WAIT_READY;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "SYSTEM RESET"
  );

  Serial.print(
    "TARGET = "
  );

  Serial.print(
    TARGET_LAT,
    7
  );

  Serial.print(
    ", "
  );

  Serial.println(
    TARGET_LON,
    7
  );

  Serial.println(
    "========================================"
  );
}
```

```

// ================================================================
// STATUS
// ================================================================

void printStatus()
{
  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "                    STATUS"
  );

  Serial.println(
    "================================================"
  );


  // System
  Serial.print(
    "State             : "
  );

  Serial.println(
    stateName(state)
  );


  // Target
  Serial.print(
    "Target latitude   : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "Target longitude  : "
  );

  Serial.println(
    TARGET_LON,
    7
  );


  // GPS
  Serial.print(
    "GPS fix           : "
  );

  Serial.println(
    gpsFix
      ? "YES"
      : "NO"
  );


  Serial.print(
    "GPS quality       : "
  );

  Serial.println(
    gpsQualityOK()
      ? "GOOD"
      : "BAD"
  );
```

```

  Serial.print(
    "Satellites        : "
  );

  Serial.println(
    gpsSats
  );


  Serial.print(
    "HDOP              : "
  );

  Serial.println(
    gpsHDOP,
    2
  );


  Serial.print(
    "Latitude          : "
  );

  Serial.println(
    currentLatitude,
    7
  );


  Serial.print(
    "Longitude         : "
  );

  Serial.println(
    currentLongitude,
    7
  );


  Serial.print(
    "Speed             : "
  );

  Serial.print(
    currentSpeed,
    2
  );

  Serial.println(
    " m/s"
  );


  // Navigation
  Serial.print(
    "Distance to target: "
  );

  Serial.print(
    distanceToTarget,
    2
  );

  Serial.println(
    " m"
  );


  Serial.print(
    "Target bearing    : "
  );

  Serial.print(
```

```
    targetBearing,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Current heading   : "
  );

  Serial.print(
    currentDirection,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Heading source    : "
  );

  Serial.println(
    headingSource
  );


  Serial.print(
    "Course error      : "
  );

  Serial.print(
    courseError,
    2
  );

  Serial.println(
    " deg"
  );


  // Compass
  Serial.print(
    "Compass status    : "
  );

  Serial.println(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Raw magnetometer  : "
  );

  Serial.print(
    rawMagX
  );

  Serial.print(
    ", "
  );

  Serial.print(
    rawMagY
  );

  Serial.print(
```

```
    ", "
  );

  Serial.println(
    rawMagZ
  );


  Serial.print(
    "Compass heading   : "
  );

  Serial.print(
    compassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Navigation heading: "
  );

  Serial.print(
    navCompassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  // Servos
  Serial.print(
    "Servo 1           : "
  );

  Serial.println(
    servo1OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 2           : "
  );

  Serial.println(
    servo2OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 1 position  : "
  );

  Serial.println(
    servo1Position
  );


  Serial.print(
    "Servo 2 position  : "
  );

  Serial.println(
    servo2Position
  );
```

```
3616
3617
3618    Serial.print(
3619      "Steering demand   : "
3620    );
3621
3622    Serial.println(
3623      steeringDemand
3624    );
3625
3626
3627    // IMU
3628    Serial.print(
3629      "IMU               : "
3630    );
3631
3632    Serial.println(
3633      imuOK
3634        ? "OK"
3635        : "FAIL"
3636    );
3637
3638
3639    // BMP
3640    Serial.print(
3641      "BMP581            : "
3642    );
3643
3644
3645    if (bmpOK)
3646    {
3647      Serial.print(
3648        "OK at 0x"
3649      );
3650
3651
3652      if (bmpAddress < 16)
3653        Serial.print("0");
3654
3655
3656      Serial.println(
3657        bmpAddress,
3658        HEX
3659      );
3660    }
3661    else
3662    {
3663      Serial.println(
3664        "NOT FOUND"
3665      );
3666    }
3667
3668
3669    // Deployment
3670    Serial.print(
3671      "Deployment done   : "
3672    );
3673
3674    Serial.println(
3675      deploymentDone
3676        ? "YES"
3677        : "NO"
3678    );
3679
3680
3681    Serial.println(
3682      "================================================"
3683    );
3684  }
3685
3686
3687  // ================================================================
3688  // LIVE DATA
3689  // ================================================================
3690
```

```
void printLiveData()
{
  Serial.print(
    "[LIVE] State="
  );

  Serial.print(
    stateName(state)
  );


  // GPS
  Serial.print(
    " GPS="
  );

  Serial.print(
    gpsQualityOK()
      ? "GOOD"
      : "BAD"
  );


  Serial.print(
    " SAT="
  );

  Serial.print(
    gpsSats
  );


  Serial.print(
    " HDOP="
  );

  Serial.print(
    gpsHDOP,
    1
  );


  if (gpsFix)
  {
    Serial.print(
      " LAT="
    );

    Serial.print(
      currentLatitude,
      6
    );


    Serial.print(
      " LON="
    );

    Serial.print(
      currentLongitude,
      6
    );
  }


  // Navigation
  if (gpsQualityOK())
  {
    Serial.print(
      " DIST="
    );

    Serial.print(
      distanceToTarget,
      1
```

```
    );


    Serial.print(
      "m BRG="
    );

    Serial.print(
      targetBearing,
      1
    );
  }


  // Heading
  if (headingValid)
  {
    Serial.print(
      " HDG="
    );

    Serial.print(
      currentDirection,
      1
    );


    Serial.print(
      " SRC="
    );

    Serial.print(
      headingSource
    );


    Serial.print(
      " ERR="
    );

    Serial.print(
      courseError,
      1
    );
  }
  else
  {
    Serial.print(
      " HDG=NONE"
    );
  }


  // Steering
  Serial.print(
    " DEM="
  );

  Serial.print(
    steeringDemand
  );


  Serial.print(
    " S1="
  );

  Serial.print(
    servo1Position
  );


  Serial.print(
    " S2="
  );
```

```
  Serial.print(
    servo2Position
  );


  // Compass
  Serial.print(
    " CMP="
  );

  Serial.print(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    " CHP="
  );

  Serial.print(
    compassHeading,
    1
  );


  Serial.print(
    " NAV="
  );

  Serial.print(
    navCompassHeading,
    1
  );


  // IMU
  Serial.print(
    " IMU="
  );

  Serial.print(
    imuOK
      ? "OK"
      : "FAIL"
  );


  // Servos
  Serial.print(
    " SERVO="
  );

  Serial.println(
    servosReady()
      ? "OK"
      : "FAIL"
  );
}


// ================================================================
// STATE NAME
// ================================================================

const char *stateName(
  SystemState s)
{
  switch (s)
  {
    case STATE_WAIT_READY:
      return "WAIT_READY";

```

```
    case STATE_DEPLOYING:
      return "DEPLOYING";


    case STATE_STABILIZING:
      return "STABILIZING";


    case STATE_AUTONAV:
      return "AUTONAV";


    case STATE_CAPTURED:
      return "CAPTURED";


    case STATE_FAILSAFE:
      return "FAILSAFE";


    case STATE_MANUAL_TEST:
      return "MANUAL_TEST";


    default:
      return "UNKNOWN";
  }
}
```

A.6 Archived source 6: Teensy_Parafoil_Final_SingleServo_MATLAB.ino

Source file: `Teensy_Parafoil_Final_SingleServo_MATLAB.ino`

```
/*
  ================================================================
  TEENSY 4.1 AUTONOMOUS PARAFOIL
  GUIDED LANDING - FINAL LAB TEST
  ================================================================

  TARGET IS FIXED IN CODE.

  CHANGE ONLY:
      TARGET_LAT
      TARGET_LON


  ================================================================
  HARDWARE
  ================================================================

  TEENSY 4.1

  ----------------------------------------------------------------
  FOXEER M10Q 180 GPS
  ----------------------------------------------------------------

      GPS TX  -> Teensy Pin 15 (RX3)
      GPS RX  -> Teensy Pin 14 (TX3)
      GPS GND -> Teensy GND

      GPS UART = Serial3
      GPS BAUD = 38400

  ----------------------------------------------------------------
  I2C BUS
  ----------------------------------------------------------------

      SDA = Teensy Pin 18
      SCL = Teensy Pin 19

  ----------------------------------------------------------------
  QMC5883 COMPASS
```

```
------------------------------------------------------------------

    Address = 0x0D

------------------------------------------------------------------
LSM6DSO32 IMU
------------------------------------------------------------------

    Address = 0x6A

------------------------------------------------------------------
BMP581
------------------------------------------------------------------

    Address = 0x46 or 0x47
    Diagnostic only

------------------------------------------------------------------
ST-3009
------------------------------------------------------------------

    Servo 1 ID = 1
    Servo 2 ID = 2

    Serial1
    Pin 0 = RX1
    Pin 1 = TX1

    1 Mbps
    SERIAL_8N1_HALF_DUPLEX

    Servo neutral = 2048

------------------------------------------------------------------
SG90 DEPLOYMENT SERVO
------------------------------------------------------------------

    Signal = Pin 8

==================================================================
NAVIGATION
==================================================================

Target bearing:
    GPS position -> target

Current heading:

    If speed >= 1.5 m/s:
        GPS course

    If speed < 1.5 m/s:
        QMC5883 compass

Course error:

    target bearing - current heading

    wrapped to -180 ... +180 degrees

Steering:

    error <= +/-5 degrees:
        neutral

    error reaches +/-90 degrees:
        maximum steering

Maximum steering:
    +/-450 counts

Differential:

    Positive demand:
        Servo 1 = 2048 - demand
```

```
          Servo 2 = 2048 + demand

      Negative demand:
          Servo 1 = 2048 - demand
          Servo 2 = 2048 + demand

  Target capture:
      35 m

  ================================================================
  STATE MACHINE
  ================================================================

      WAIT_READY
           |
           | GPS GOOD + Servo 1 + Servo 2
           v
      DEPLOYING
           |
           | 1.5 sec
           v
      STABILIZING
           |
           | 5 sec + GPS + servos
           v
       AUTONAV
           |
           | <=35 m
           v
       CAPTURED

  GPS or servo failure during AUTONAV:
       AUTONAV -> FAILSAFE

  3 sec good GPS + servos:
       FAILSAFE -> AUTONAV

  ================================================================
  SERIAL COMMANDS
  ================================================================

      STATUS
      MOTORTEST
      LEFT
      RIGHT
      CENTER
      AUTONAV
      DEPLOY
      PREP
      RESET
      HELP

  There is NO TARGET serial command.

  ================================================================
*/


// ================================================================
// LIBRARIES
// ================================================================

#include <Arduino.h>
#include <Wire.h>
#include <Servo.h>
#include <TinyGPS++.h>
#include <SCServo.h>
#include <Adafruit_LSM6DSOX.h>

#include <math.h>
#include <string.h>


// ================================================================
// TARGET
```

```
// ================================================================

const double TARGET_LAT = 32.6321769;
const double TARGET_LON = 75.0143889;


// ================================================================
// AUTO DEPLOY
// ================================================================
//
// true  = deploy automatically after GPS + both ST-3009 ready
// false = wait for DEPLOY command
//
// SAFETY: FALSE for bench testing. Use DEPLOY manually.
// Set TRUE only for an intentional autonomous deployment test.
// ================================================================

const bool AUTO_DEPLOY_AFTER_READY = false;


// ================================================================
// GPS
// ================================================================

#define GPS_SERIAL Serial3

const uint32_t GPS_BAUD = 38400;

const uint8_t GPS_RX_PIN = 15;
const uint8_t GPS_TX_PIN = 14;

// GPS data considered stale after this time
const uint32_t GPS_DATA_TIMEOUT_MS = 2500;

// Minimum satellites
const uint8_t GPS_MIN_SATS = 5;

// Maximum acceptable HDOP
const double GPS_MAX_HDOP = 3.5;

// GPS course is used only above this speed
const double GPS_COURSE_MIN_SPEED_MPS = 1.5;


// ================================================================
// I2C
// ================================================================

const uint8_t I2C_SDA_PIN = 18;
const uint8_t I2C_SCL_PIN = 19;


// ================================================================
// QMC5883
// ================================================================

const uint8_t QMC5883_ADDR = 0x0D;


// ----------------------------------------------------------------
// Compass navigation orientation
// ----------------------------------------------------------------
//
// Your tested compass direction was mirrored for navigation.
//

const bool COMPASS_MIRROR_FOR_NAV = true;

const float COMPASS_HEADING_OFFSET_DEG = 0.0f;


// ----------------------------------------------------------------
// Calibration values from your working compass test
// ----------------------------------------------------------------

```

```
const float OFFSET_X = 180.0000f;
const float OFFSET_Y = 336.0000f;
const float OFFSET_Z = 615.0000f;

const float SCALE_X = 0.956484f;
const float SCALE_Y = 0.904406f;
const float SCALE_Z = 1.178127f;


// ================================================================
// LSM6DSO32
// ================================================================

const uint8_t LSM6DSO32_ADDR = 0x6A;

Adafruit_LSM6DSOX lsm6dso;

bool imuOK = false;


// ================================================================
// BMP581
// ================================================================

const uint8_t BMP581_ADDR_1 = 0x46;
const uint8_t BMP581_ADDR_2 = 0x47;

bool bmpOK = false;

uint8_t bmpAddress = 0;


// ================================================================
// GPS VARIABLES
// ================================================================

TinyGPSPlus gps;

bool gpsFix = false;

uint8_t gpsSats = 0;

double gpsHDOP = 99.9;

double currentSpeed = 0.0;

unsigned long lastGPSLocationMs = 0;


// ================================================================
// COMPASS VARIABLES
// ================================================================

bool compassOK = false;

int16_t rawMagX = 0;
int16_t rawMagY = 0;
int16_t rawMagZ = 0;

float magX = 0.0f;
float magY = 0.0f;
float magZ = 0.0f;

float compassHeading = 0.0f;
float navCompassHeading = 0.0f;

uint8_t compassFailCount = 0;


// ================================================================
// IMU VARIABLES
// ================================================================

float accelX = 0.0f;
float accelY = 0.0f;
```

```
float accelZ = 0.0f;

float gyroX = 0.0f;
float gyroY = 0.0f;
float gyroZ = 0.0f;

unsigned long lastSensorUpdateMs = 0;


// ================================================================
// ST-3009
// ================================================================
//
// IMPORTANT:
// SCServo 1.0.2 uses:
//
//     SMS_STS
//

SMS_STS st;


// ----------------------------------------------------------------
// Servo IDs
// ----------------------------------------------------------------

const uint8_t SERVO1_ID = 1;
const uint8_t SERVO2_ID = 2;


// ----------------------------------------------------------------
// Servo positions
// ----------------------------------------------------------------

const int SERVO_NEUTRAL = 2048;

const int SERVO_MIN_POS = 200;
const int SERVO_MAX_POS = 3895;


// ----------------------------------------------------------------
// Steering
// ----------------------------------------------------------------

const int MAX_STEERING_COUNTS = 450;

const int STEERING_SERVO_SPEED = 500;
const int STEERING_SERVO_ACCEL = 50;

const int MAX_SLEW_COUNTS = 60;

// ------------------------------------------------------------
// Steering actuation mode
// ------------------------------------------------------------
// TRUE  = only the servo on the requested side moves.
// FALSE = original differential mode (both servos move opposite).
const bool SINGLE_SERVO_STEERING = true;

// Physical mapping by default:
//   Servo ID 1 = LEFT
//   Servo ID 2 = RIGHT
// Set false if your physical IDs are swapped.
const bool SERVO1_IS_LEFT = true;

// Direction of the active servo relative to neutral.
// -1 means the active servo moves below 2048.
const int LEFT_STEER_DIRECTION = -1;
const int RIGHT_STEER_DIRECTION = -1;


// ----------------------------------------------------------------
// Servo health
// ----------------------------------------------------------------

const unsigned long SERVO_HEALTH_INTERVAL_MS = 2000;
```

```
415
416 bool servo1OK = false;
417 bool servo2OK = false;
418 bool servoBusReady = false;
419
420 unsigned long lastServoHealthMs = 0;
421
422
423 // ---------------------------------------------------------------
424 // Current positions
425 // ---------------------------------------------------------------
426
427 int servo1Position = SERVO_NEUTRAL;
428 int servo2Position = SERVO_NEUTRAL;
429
430 // Real ST-3009 position feedback from ReadPos().
431 int servo1Actual = SERVO_NEUTRAL;
432 int servo2Actual = SERVO_NEUTRAL;
433 bool servo1FeedbackOK = false;
434 bool servo2FeedbackOK = false;
435 unsigned long lastServoFeedbackMs = 0;
436 const unsigned long SERVO_FEEDBACK_INTERVAL_MS = 250;
437
438
439 // ---------------------------------------------------------------
440 // Current navigation demand
441 // ---------------------------------------------------------------
442
443 int steeringDemand = 0;
444
445
446 // ===============================================================
447 // SG90 DEPLOYMENT
448 // ===============================================================
449
450 Servo deployServo;
451
452 const uint8_t DEPLOY_SERVO_PIN = 8;
453
454 const int DEPLOY_ANGLE = 180;
455
456 const int PREP_ANGLE = 140;
457
458 const unsigned long DEPLOY_MOVE_TIME_MS = 1500;
459
460 const unsigned long DEPLOY_STABILIZE_TIME_MS = 5000;
461
462 bool deploymentServoActive = false;
463
464 bool deploymentDone = false;
465
466 unsigned long deploymentStartMs = 0;
467
468 unsigned long stabilizationStartMs = 0;
469
470
471 // ===============================================================
472 // NAVIGATION
473 // ===============================================================
474
475 const double TARGET_CAPTURE_RADIUS_M = 35.0;
476
477 const double STEERING_DEADBAND_DEG = 5.0;
478
479 const double FULL_STEERING_ERROR_DEG = 90.0;
480
481
482 // ===============================================================
483 // STATE MACHINE
484 // ===============================================================
485
486 enum SystemState
487 {
488   STATE_WAIT_READY,
489   STATE_DEPLOYING,
```

```
  STATE_STABILIZING,
  STATE_AUTONAV,
  STATE_CAPTURED,
  STATE_FAILSAFE,
  STATE_MANUAL_TEST
};

SystemState state = STATE_WAIT_READY;

unsigned long stateEnteredMs = 0;


// ================================================================
// NAVIGATION VARIABLES
// ================================================================

double currentLatitude = 0.0;

double currentLongitude = 0.0;

double distanceToTarget = 0.0;

double targetBearing = 0.0;

double currentDirection = 0.0;

double courseError = 0.0;

bool headingValid = false;

const char *headingSource = "NONE";


// ================================================================
// FAILSAFE
// ================================================================

unsigned long failsafeGoodSinceMs = 0;

const unsigned long FAILSAFE_RECOVERY_MS = 3000;


// ================================================================
// SERIAL COMMAND BUFFER
// ================================================================

#define COMMAND_BUFFER_SIZE 120

char commandBuffer[COMMAND_BUFFER_SIZE];

uint16_t commandIndex = 0;


// ================================================================
// TIMERS
// ================================================================

unsigned long lastNavigationUpdateMs = 0;

const unsigned long NAVIGATION_UPDATE_INTERVAL_MS = 100;

unsigned long lastLivePrintMs = 0;

const unsigned long LIVE_PRINT_INTERVAL_MS = 1000;

// Machine-readable telemetry for MATLAB.
unsigned long lastMatlabTelemetryMs = 0;
const unsigned long MATLAB_TELEMETRY_INTERVAL_MS = 200;

unsigned long lastBMPCheckMs = 0;

const unsigned long BMP_CHECK_INTERVAL_MS = 3000;


// ================================================================
```

```
// MOTOR TEST
// ================================================================

bool motorTestActive = false;

uint8_t motorTestStep = 0;

unsigned long motorTestTimer = 0;


// ================================================================
// FORWARD DECLARATIONS
// ================================================================

void processSerialCommands();
void processCommand(char *cmd);

void updateGPS();
void updateCompass();
void updateIMU();
void updateServoHealth();
void updateServoFeedback();

void updateNavigation();
void updateSteering();
void updateStateMachine();

void writeSteeringPositions(int pos1, int pos2);
void centerServos();
void calculateServoTargets(int demand, int &target1, int &target2);
void commandSteering(int demand);

void startDeployment();
void prepareDeploymentServo();
void resetSystem();

void startMotorTest();
void updateMotorTest();

void printStatus();
void printLiveData();
void sendMatlabTelemetry();

bool gpsQualityOK();
bool readQMC5883();
bool initQMC5883();

bool checkI2CDevice(uint8_t address);
bool initIMU();

bool servosReady();

float wrap360(float angle);
double wrap180(double angle);
float navigationCompassHeading();

const char *stateName(SystemState s);

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep
);


// ================================================================
// SETUP
// ================================================================

void setup()
{
  // ------------------------------------------------------------
  // USB SERIAL
  // ------------------------------------------------------------

```

```
  Serial.begin(115200);

  delay(1500);


  Serial.println();
  Serial.println(
    "================================================"
  );

  Serial.println(
    " TEENSY 4.1 AUTONOMOUS PARAFOIL"
  );

  Serial.println(
    " GUIDED LANDING - FINAL LAB TEST"
  );

  Serial.println(
    "================================================"
  );


  // ---------------------------------------------------------------
  // TARGET
  // ---------------------------------------------------------------

  Serial.println();

  Serial.print(
    "TARGET LATITUDE  : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "TARGET LONGITUDE : "
  );

  Serial.println(
    TARGET_LON,
    7
  );


  // ---------------------------------------------------------------
  // I2C
  // ---------------------------------------------------------------

  Wire.begin();

  Wire.setClock(400000);


  Serial.println();

  Serial.println(
    "I2C initialized"
  );

  Serial.println(
    "SDA = Pin 18"
  );

  Serial.println(
    "SCL = Pin 19"
  );


  // ---------------------------------------------------------------
  // QMC5883
```

```
  // ---------------------------------------------------------------

  if (initQMC5883())
  {
    compassOK = true;

    Serial.println(
      "QMC5883 : OK"
    );
  }
  else
  {
    compassOK = false;

    Serial.println(
      "QMC5883 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // LSM6DSO32
  // ---------------------------------------------------------------

  if (initIMU())
  {
    imuOK = true;

    Serial.println(
      "LSM6DSO32 : OK"
    );
  }
  else
  {
    imuOK = false;

    Serial.println(
      "LSM6DSO32 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // BMP581
  // ---------------------------------------------------------------

  if (checkI2CDevice(
        BMP581_ADDR_1))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_1;
  }
  else if (checkI2CDevice(
             BMP581_ADDR_2))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_2;
  }
  else
  {
    bmpOK = false;

    bmpAddress = 0;
  }


  if (bmpOK)
  {
    Serial.print(
      "BMP581 : FOUND at 0x"
    );
```

```
    if (bmpAddress < 16)
      Serial.print("0");

    Serial.println(
      bmpAddress,
      HEX
    );
  }
  else
  {
    Serial.println(
      "BMP581 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // GPS
  // ---------------------------------------------------------------
  //
  // EXACTLY MATCHES YOUR WORKING GPS TEST
  //

  GPS_SERIAL.begin(
    GPS_BAUD
  );


  Serial.println();

  Serial.println(
    "GPS UART  : Serial3"
  );

  Serial.print(
    "GPS RX    : Pin "
  );

  Serial.println(
    GPS_RX_PIN
  );

  Serial.print(
    "GPS TX    : Pin "
  );

  Serial.println(
    GPS_TX_PIN
  );

  Serial.print(
    "GPS BAUD  : "
  );

  Serial.println(
    GPS_BAUD
  );


  // ---------------------------------------------------------------
  // ST-3009
  // ---------------------------------------------------------------

  Serial.println();

  Serial.println(
    "Initializing ST-3009 bus..."
  );


  Serial1.begin(
    1000000,
    SERIAL_8N1_HALF_DUPLEX
  );
```

```
865
866
867   st.pSerial =
868     &Serial1;
869
870
871   delay(500);
872
873
874   // Force immediate first health test
875   lastServoHealthMs =
876     millis() -
877     SERVO_HEALTH_INTERVAL_MS;
878
879
880   updateServoHealth();
881
882
883   if (servosReady())
884   {
885     Serial.println(
886       "ST-3009 ID 1 : OK"
887     );
888
889     Serial.println(
890       "ST-3009 ID 2 : OK"
891     );
892
893
894     centerServos();
895   }
896   else
897   {
898     Serial.println(
899       "ST-3009 : WAITING"
900     );
901   }
902
903
904   // ----------------------------------------------------------------
905   // SG90
906   // ----------------------------------------------------------------
907
908   deployServo.detach();
909
910   deploymentServoActive =
911     false;
912
913
914   // ----------------------------------------------------------------
915   // State
916   // ----------------------------------------------------------------
917
918   state =
919     STATE_WAIT_READY;
920
921   stateEnteredMs =
922     millis();
923
924
925   Serial.println();
926
927   Serial.println(
928     "================================================"
929   );
930
931   Serial.println(
932     "SYSTEM READY"
933   );
934
935   Serial.println(
936     "Target is fixed in code."
937   );
938
939   Serial.println(
```

```
    "================================================"
  );


  Serial.println();

  Serial.println(
    "COMMANDS:"
  );

  Serial.println(
    "STATUS"
  );

  Serial.println(
    "MOTORTEST"
  );

  Serial.println(
    "LEFT"
  );

  Serial.println(
    "RIGHT"
  );

  Serial.println(
    "CENTER"
  );

  Serial.println(
    "AUTONAV"
  );

  Serial.println(
    "DEPLOY"
  );

  Serial.println(
    "PREP"
  );

  Serial.println(
    "RESET"
  );

  Serial.println(
    "HELP"
  );


  Serial.println();

  Serial.println(
    "Waiting for GPS + ST-3009..."
  );
}


// ================================================================
// LOOP
// ================================================================

void loop()
{
  // GPS continuously
  updateGPS();


  // Serial commands
  processSerialCommands();


  // Sensors
  if (millis() -
```

```
    lastSensorUpdateMs >=
    100)
{
  lastSensorUpdateMs =
    millis();

  updateCompass();
  updateIMU();
}


// Servo health
updateServoHealth();

// Real ST-3009 encoder feedback for diagnostics / MATLAB.
updateServoFeedback();


// Motor test
if (motorTestActive)
{
  updateMotorTest();
}


// State
updateStateMachine();


// Navigation
if (millis() -
    lastNavigationUpdateMs >=
    NAVIGATION_UPDATE_INTERVAL_MS)
{
  lastNavigationUpdateMs =
    millis();

  updateNavigation();
  updateSteering();
}


// BMP check
if (millis() -
    lastBMPCheckMs >=
    BMP_CHECK_INTERVAL_MS)
{
  lastBMPCheckMs =
    millis();


  if (!bmpOK)
  {
    if (checkI2CDevice(
          BMP581_ADDR_1))
    {
      bmpOK = true;

      bmpAddress =
        BMP581_ADDR_1;
    }
    else if (checkI2CDevice(
                BMP581_ADDR_2))
    {
      bmpOK = true;

      bmpAddress =
        BMP581_ADDR_2;
    }
  }
}


// Live data
if (millis() -
```

```
      lastLivePrintMs >=
      LIVE_PRINT_INTERVAL_MS)
  {
    lastLivePrintMs =
      millis();

    printLiveData();
  }

  // Machine-readable telemetry for MATLAB.
  sendMatlabTelemetry();
}


// ================================================================
// GPS UPDATE
// ================================================================

void updateGPS()
{
  while (GPS_SERIAL.available() > 0)
  {
    char c =
      (char)GPS_SERIAL.read();

    gps.encode(c);
  }


  // Fresh GPS location
  if (gps.location.isUpdated())
  {
    lastGPSLocationMs =
      millis();
  }


  // Position
  if (gps.location.isValid())
  {
    gpsFix = true;

    currentLatitude =
      gps.location.lat();

    currentLongitude =
      gps.location.lng();
  }
  else
  {
    gpsFix = false;
  }


  // Satellites
  if (gps.satellites.isValid())
  {
    gpsSats =
      gps.satellites.value();
  }
  else
  {
    gpsSats = 0;
  }


  // HDOP
  if (gps.hdop.isValid())
  {
    gpsHDOP =
      gps.hdop.hdop();
  }
  else
  {
    gpsHDOP = 99.9;
```

```
  }


  // Speed
  if (gps.speed.isValid())
  {
    currentSpeed =
      gps.speed.mps();
  }
  else
  {
    currentSpeed = 0.0;
  }
}


// ================================================================
// GPS QUALITY
// ================================================================

bool gpsQualityOK()
{
  if (!gpsFix)
    return false;


  if (lastGPSLocationMs == 0)
    return false;


  if (millis() -
      lastGPSLocationMs >
      GPS_DATA_TIMEOUT_MS)
  {
    return false;
  }


  if (gpsSats <
      GPS_MIN_SATS)
  {
    return false;
  }


  if (gps.hdop.isValid() &&
      gpsHDOP >
      GPS_MAX_HDOP)
  {
    return false;
  }


  return true;
}


// ================================================================
// QMC5883 INIT
// ================================================================

bool initQMC5883()
{
  // Check device
  Wire.beginTransmission(
    QMC5883_ADDR
  );


  if (Wire.endTransmission() != 0)
  {
    return false;
  }

```

```
  // SET/RESET period
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x0B);
  Wire.write(0x01);

  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  delay(10);


  // Control register
  // Same setting used by your
  // working compass test.
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x09);
  Wire.write(0x1D);

  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  delay(100);


  return true;
}


// ================================================================
// QMC5883 READ
// ================================================================

bool readQMC5883()
{
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x00);


  if (Wire.endTransmission(false) != 0)
  {
    return false;
  }


  uint8_t count =
    Wire.requestFrom(
      QMC5883_ADDR,
      (uint8_t)6
    );


  if (count != 6)
  {
    while (Wire.available())
    {
      Wire.read();
    }

    return false;
  }
```

```
1315
1316
1317    uint8_t xl =
1318      Wire.read();
1319
1320    uint8_t xh =
1321      Wire.read();
1322
1323
1324    uint8_t yl =
1325      Wire.read();
1326
1327    uint8_t yh =
1328      Wire.read();
1329
1330
1331    uint8_t zl =
1332      Wire.read();
1333
1334    uint8_t zh =
1335      Wire.read();
1336
1337
1338    rawMagX =
1339      (int16_t)(
1340        (xh << 8) |
1341        xl
1342      );
1343
1344
1345    rawMagY =
1346      (int16_t)(
1347        (yh << 8) |
1348        yl
1349      );
1350
1351
1352    rawMagZ =
1353      (int16_t)(
1354        (zh << 8) |
1355        zl
1356      );
1357
1358
1359    return true;
1360  }
1361
1362
1363  // ================================================================
1364  // COMPASS UPDATE
1365  // ================================================================
1366
1367  void updateCompass()
1368  {
1369    // ------------------------------------------------------------
1370    // Read sensor
1371    // ------------------------------------------------------------
1372
1373    if (!readQMC5883())
1374    {
1375      compassFailCount++;
1376
1377
1378      // Do not immediately declare compass dead.
1379      // Reinitialize after 5 consecutive failures.
1380
1381      if (compassFailCount >= 5)
1382      {
1383        if (initQMC5883())
1384        {
1385          compassOK = true;
1386
1387          compassFailCount = 0;
1388
1389          Serial.println(
```

```
          "[COMPASS] Reinitialized OK"
        );
      }
      else
      {
        compassOK = false;
      }
    }


    return;
  }


  // Successful reading
  compassFailCount = 0;
  compassOK = true;


  // ------------------------------------------------------------
  // Apply your proven calibration
  // ------------------------------------------------------------

  magX =
    ((float)rawMagX -
     OFFSET_X) *
    SCALE_X;


  magY =
    ((float)rawMagY -
     OFFSET_Y) *
    SCALE_Y;


  magZ =
    ((float)rawMagZ -
     OFFSET_Z) *
    SCALE_Z;


  // ------------------------------------------------------------
  // Calculate heading
  // ------------------------------------------------------------

  float angle =
    atan2(
      magY,
      magX
    ) *
    180.0f /
    PI;


  if (angle < 0.0f)
    angle += 360.0f;


  compassHeading =
    wrap360(
      angle +
      COMPASS_HEADING_OFFSET_DEG
    );


  // Navigation heading
  navCompassHeading =
    navigationCompassHeading();
}


// ================================================================
// NAVIGATION COMPASS
// ================================================================
```

```
float navigationCompassHeading()
{
  if (!COMPASS_MIRROR_FOR_NAV)
  {
    return compassHeading;
  }


  return wrap360(
    360.0f -
    compassHeading
  );
}


// ================================================================
// WRAP 0...360
// ================================================================

float wrap360(float angle)
{
  while (angle >= 360.0f)
    angle -= 360.0f;


  while (angle < 0.0f)
    angle += 360.0f;


  return angle;
}


// ================================================================
// WRAP -180...180
// ================================================================

double wrap180(double angle)
{
  while (angle > 180.0)
    angle -= 360.0;


  while (angle < -180.0)
    angle += 360.0;


  return angle;
}


// ================================================================
// IMU INIT
// ================================================================

bool initIMU()
{
  if (!lsm6dso.begin_I2C(
        LSM6DSO32_ADDR,
        &Wire))
  {
    return false;
  }


  lsm6dso.setAccelRange(
    LSM6DS_ACCEL_RANGE_16_G
  );


  lsm6dso.setGyroRange(
    LSM6DS_GYRO_RANGE_2000_DPS
  );

```

```
  lsm6dso.setAccelDataRate(
    LSM6DS_RATE_208_HZ
  );


  lsm6dso.setGyroDataRate(
    LSM6DS_RATE_208_HZ
  );


  return true;
}


// ================================================================
// IMU UPDATE
// ================================================================

void updateIMU()
{
  if (!imuOK)
    return;


  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;


  lsm6dso.getEvent(
    &accel,
    &gyro,
    &temp
  );


  accelX =
    accel.acceleration.x;


  accelY =
    accel.acceleration.y;


  accelZ =
    accel.acceleration.z;


  gyroX =
    gyro.gyro.x;


  gyroY =
    gyro.gyro.y;


  gyroZ =
    gyro.gyro.z;
}


// ================================================================
// I2C CHECK
// ================================================================

bool checkI2CDevice(
  uint8_t address)
{
  Wire.beginTransmission(
    address
  );


  return (
    Wire.endTransmission() == 0
```

```
  );
}


// ================================================================
// SERVO HEALTH
// ================================================================

void updateServoHealth()
{
  if (millis() -
      lastServoHealthMs <
      SERVO_HEALTH_INTERVAL_MS)
  {
    return;
  }


  lastServoHealthMs =
    millis();


  // Ping ID 1
  int result1 =
    st.Ping(
      SERVO1_ID
    );


  delay(10);


  // Ping ID 2
  int result2 =
    st.Ping(
      SERVO2_ID
    );


  servo1OK =
    (result1 >= 0);


  servo2OK =
    (result2 >= 0);


  servoBusReady =
    servo1OK &&
    servo2OK;
}


// ================================================================
// SERVO FEEDBACK
// ================================================================

void updateServoFeedback()
{
  if (!servosReady())
  {
    servo1FeedbackOK = false;
    servo2FeedbackOK = false;
    return;
  }

  if (millis() - lastServoFeedbackMs <
      SERVO_FEEDBACK_INTERVAL_MS)
  {
    return;
  }

  lastServoFeedbackMs = millis();

  int p1 = st.ReadPos(SERVO1_ID);
```

```
  int p2 = st.ReadPos(SERVO2_ID);

  if (p1 >= SERVO_MIN_POS && p1 <= SERVO_MAX_POS)
  {
    servo1Actual = p1;
    servo1FeedbackOK = true;
  }
  else
  {
    servo1FeedbackOK = false;
  }

  if (p2 >= SERVO_MIN_POS && p2 <= SERVO_MAX_POS)
  {
    servo2Actual = p2;
    servo2FeedbackOK = true;
  }
  else
  {
    servo2FeedbackOK = false;
  }
}


// ================================================================
// SERVO READY
// ================================================================

bool servosReady()
{
  return
    servoBusReady &&
    servo1OK &&
    servo2OK;
}


// ================================================================
// CENTER BOTH SERVOS
// ================================================================

void centerServos()
{
  if (!servosReady())
    return;


  st.WritePosEx(
    SERVO1_ID,
    SERVO_NEUTRAL,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  st.WritePosEx(
    SERVO2_ID,
    SERVO_NEUTRAL,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );


  servo1Position =
    SERVO_NEUTRAL;


  servo2Position =
    SERVO_NEUTRAL;


  steeringDemand = 0;
}


```

```
// ================================================================
// DIRECT STEERING COMMAND
// ================================================================
//
// demand = +300:
//
//   Servo 1 = 2048 - 300 = 1748
//   Servo 2 = 2048 + 300 = 2348
//
// demand = -300:
//
//   Servo 1 = 2048 + 300 = 2348
//   Servo 2 = 2048 - 300 = 1748
//

void calculateServoTargets(
  int demand,
  int &target1,
  int &target2)
{
  demand = constrain(
    demand,
    -MAX_STEERING_COUNTS,
    MAX_STEERING_COUNTS
  );

  target1 = SERVO_NEUTRAL;
  target2 = SERVO_NEUTRAL;

  if (!SINGLE_SERVO_STEERING)
  {
    // Original differential steering mode.
    target1 = SERVO_NEUTRAL - demand;
    target2 = SERVO_NEUTRAL + demand;
  }
  else if (demand > 0)
  {
    // Positive demand = LEFT turn.
    int active = SERVO_NEUTRAL +
                 LEFT_STEER_DIRECTION * demand;

    if (SERVO1_IS_LEFT)
      target1 = active;
    else
      target2 = active;
  }
  else if (demand < 0)
  {
    // Negative demand = RIGHT turn.
    int magnitude = abs(demand);
    int active = SERVO_NEUTRAL +
                 RIGHT_STEER_DIRECTION * magnitude;

    if (SERVO1_IS_LEFT)
      target2 = active;
    else
      target1 = active;
  }

  target1 = constrain(
    target1,
    SERVO_MIN_POS,
    SERVO_MAX_POS
  );

  target2 = constrain(
    target2,
    SERVO_MIN_POS,
    SERVO_MAX_POS
  );
}


// ================================================================
// DIRECT STEERING COMMAND
```

```
// ================================================================

void commandSteering(
  int demand)
{
  if (!servosReady())
  {
    Serial.println(
      "ERROR: ST-3009 BUS NOT READY"
    );

    return;
  }

  demand = constrain(
    demand,
    -MAX_STEERING_COUNTS,
    MAX_STEERING_COUNTS
  );

  int pos1;
  int pos2;

  calculateServoTargets(
    demand,
    pos1,
    pos2
  );

  st.WritePosEx(
    SERVO1_ID,
    pos1,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );

  st.WritePosEx(
    SERVO2_ID,
    pos2,
    STEERING_SERVO_SPEED,
    STEERING_SERVO_ACCEL
  );

  servo1Position = pos1;
  servo2Position = pos2;
  steeringDemand = demand;

  Serial.print("DEMAND=");
  Serial.print(demand);
  Serial.print("  S1=");
  Serial.print(pos1);
  Serial.print("  S2=");
  Serial.print(pos2);
  Serial.print("  MODE=");
  Serial.println(
    SINGLE_SERVO_STEERING
      ? "SINGLE_SERVO"
      : "DIFFERENTIAL"
  );
}


// ================================================================
// MOTOR TEST START
// ================================================================

void startMotorTest()
{
  if (!servosReady())
  {
    Serial.println(
      "MOTORTEST BLOCKED: "
      "ST-3009 NOT READY"
    );

```

```
    return;
  }


  motorTestActive =
    true;


  state =
    STATE_MANUAL_TEST;


  motorTestStep = 0;

  motorTestTimer =
    millis();


  centerServos();


  Serial.println();

  Serial.println(
    "======================================="
  );

  Serial.println(
    "       ST-3009 MOTOR TEST"
  );

  Serial.println(
    "======================================="
  );

  Serial.println(
    "CENTER -> LEFT -> CENTER -> RIGHT -> CENTER"
  );

  Serial.println();
}


// ================================================================
// MOTOR TEST UPDATE
// ================================================================

void updateMotorTest()
{
  if (!servosReady())
  {
    motorTestActive =
      false;

    state =
      STATE_WAIT_READY;

    return;
  }


  if (millis() -
      motorTestTimer <
      1800)
  {
    return;
  }


  motorTestTimer =
    millis();


  switch (motorTestStep)
  {
```

```
    case 0:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    case 1:

      Serial.println(
        "TEST: LEFT"
      );

      commandSteering(
        +300
      );

      motorTestStep++;

      break;


    case 2:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    case 3:

      Serial.println(
        "TEST: RIGHT"
      );

      commandSteering(
        -300
      );

      motorTestStep++;

      break;


    case 4:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    default:

      motorTestActive =
        false;

```

```
      state =
        STATE_WAIT_READY;


      centerServos();


      Serial.println();

      Serial.println(
        "MOTOR TEST COMPLETE"
      );

      Serial.println();

      break;
  }
}


// ================================================================
// NAVIGATION CALCULATION
// ================================================================

void updateNavigation()
{
  if (!gpsQualityOK())
  {
    headingValid =
      false;

    headingSource =
      "NONE";

    return;
  }


  // ---------------------------------------------------------------
  // GPS position
  // ---------------------------------------------------------------

  currentLatitude =
    gps.location.lat();


  currentLongitude =
    gps.location.lng();


  // ---------------------------------------------------------------
  // Distance
  // ---------------------------------------------------------------

  distanceToTarget =
    TinyGPSPlus::distanceBetween(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
      TARGET_LON
    );


  // ---------------------------------------------------------------
  // Bearing
  // ---------------------------------------------------------------

  targetBearing =
    TinyGPSPlus::courseTo(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
      TARGET_LON
    );

```

```
2140
2141    targetBearing =
2142      wrap360(
2143        targetBearing
2144      );
2145
2146
2147    // ------------------------------------------------------------
2148    // Determine heading
2149    // ------------------------------------------------------------
2150
2151    headingValid =
2152      false;
2153
2154
2155    headingSource =
2156      "NONE";
2157
2158
2159    // GPS course when moving
2160    if (gps.speed.isValid() &&
2161        gps.course.isValid() &&
2162        currentSpeed >=
2163        GPS_COURSE_MIN_SPEED_MPS)
2164    {
2165      currentDirection =
2166        wrap360(
2167          gps.course.deg()
2168        );
2169
2170
2171      headingValid =
2172        true;
2173
2174
2175      headingSource =
2176        "GPS COURSE";
2177    }
2178
2179
2180    // Compass when slow/stationary
2181    else if (compassOK)
2182    {
2183      currentDirection =
2184        navigationCompassHeading();
2185
2186
2187      headingValid =
2188        true;
2189
2190
2191      headingSource =
2192        "COMPASS";
2193    }
2194
2195
2196    // ------------------------------------------------------------
2197    // Course error
2198    // ------------------------------------------------------------
2199
2200    if (headingValid)
2201    {
2202      courseError =
2203        wrap180(
2204          targetBearing -
2205          currentDirection
2206        );
2207    }
2208  }
2209
2210
2211  // ================================================================
2212  // AUTONOMOUS STEERING
2213  // ================================================================
2214
```

```
void updateSteering()
{
  // Manual motor test owns the servos
  if (motorTestActive)
    return;


  // Servo unavailable
  if (!servosReady())
  {
    steeringDemand = 0;

    return;
  }


  // Only steer during AUTONAV
  if (state !=
      STATE_AUTONAV)
  {
    return;
  }


  // GPS required
  if (!gpsQualityOK())
  {
    centerServos();

    return;
  }


  // Target captured
  if (distanceToTarget <=
      TARGET_CAPTURE_RADIUS_M)
  {
    centerServos();

    return;
  }


  // No heading
  if (!headingValid)
  {
    centerServos();

    return;
  }


  // ------------------------------------------------------------
  // Absolute error
  // ------------------------------------------------------------

  double absoluteError =
    fabs(courseError);


  // ------------------------------------------------------------
  // Deadband
  // ------------------------------------------------------------

  if (absoluteError <=
      STEERING_DEADBAND_DEG)
  {
    steeringDemand = 0;
  }
  else
  {
    double usableError =
      absoluteError -
      STEERING_DEADBAND_DEG;

```

```
    double usableRange =
      FULL_STEERING_ERROR_DEG -
      STEERING_DEADBAND_DEG;


    double normalized =
      usableError /
      usableRange;


    normalized =
      constrain(
        normalized,
        0.0,
        1.0
      );


    int demand =
      (int)round(
        normalized *
        MAX_STEERING_COUNTS
      );


    if (courseError > 0.0)
    {
      steeringDemand =
        demand;
    }
    else
    {
      steeringDemand =
        -demand;
    }
  }


  // ------------------------------------------------------------
  // Target servo positions
  // ------------------------------------------------------------

  int target1;
  int target2;

  calculateServoTargets(
    steeringDemand,
    target1,
    target2
  );


  // ------------------------------------------------------------
  // Slew limit
  // ------------------------------------------------------------

  int new1 =
    slewTo(
      servo1Position,
      target1,
      MAX_SLEW_COUNTS
    );


  int new2 =
    slewTo(
      servo2Position,
      target2,
      MAX_SLEW_COUNTS
    );


  // ------------------------------------------------------------
  // Send only if changed
```

```
  // ------------------------------------------------------------

  if (new1 != servo1Position ||
      new2 != servo2Position)
  {
    st.WritePosEx(
      SERVO1_ID,
      new1,
      STEERING_SERVO_SPEED,
      STEERING_SERVO_ACCEL
    );


    st.WritePosEx(
      SERVO2_ID,
      new2,
      STEERING_SERVO_SPEED,
      STEERING_SERVO_ACCEL
    );


    servo1Position =
      new1;


    servo2Position =
      new2;
  }
}


// ====================================================================
// SLEW LIMIT
// ====================================================================

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep)
{
  if (targetValue >
      currentValue)
  {
    int difference =
      targetValue -
      currentValue;


    if (difference >
        maxStep)
    {
      return currentValue +
             maxStep;
    }


    return targetValue;
  }


  if (targetValue <
      currentValue)
  {
    int difference =
      currentValue -
      targetValue;


    if (difference >
        maxStep)
    {
      return currentValue -
             maxStep;
    }

```

```

    return targetValue;
  }


  return currentValue;
}


// ================================================================
// DEPLOYMENT
// ================================================================

void startDeployment()
{
  if (deploymentDone)
  {
    Serial.println(
      "DEPLOYMENT ALREADY COMPLETED"
    );

    return;
  }


  if (!gpsQualityOK())
  {
    Serial.println(
      "DEPLOY BLOCKED: GPS NOT READY"
    );

    return;
  }


  if (!servosReady())
  {
    Serial.println(
      "DEPLOY BLOCKED: ST-3009 NOT READY"
    );

    return;
  }


  // Center steering first
  centerServos();


  delay(100);


  // Attach SG90
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );


  delay(100);


  // Deployment position
  deployServo.write(
    DEPLOY_ANGLE
  );


  deploymentStartMs =
    millis();


  deploymentServoActive =
    true;


```

```
  deploymentDone =
    true;


  state =
    STATE_DEPLOYING;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "STARTING DEPLOYMENT"
  );

  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    DEPLOY_ANGLE
  );

  Serial.println(
    " degrees"
  );
}


// ================================================================
// SG90 PREP
// ================================================================

void prepareDeploymentServo()
{
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );


  delay(100);


  deployServo.write(
    PREP_ANGLE
  );


  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    PREP_ANGLE
  );

  Serial.println(
    " degrees"
  );


  delay(1000);


  deployServo.detach();


  Serial.println(
```

```
    "SG90 detached"
  );
}


// ================================================================
// STATE MACHINE
// ================================================================

void updateStateMachine()
{
  // ------------------------------------------------------------
  // Manual test
  // ------------------------------------------------------------

  if (state ==
      STATE_MANUAL_TEST)
  {
    return;
  }


  // ------------------------------------------------------------
  // WAIT READY
  // ------------------------------------------------------------

  if (state ==
      STATE_WAIT_READY)
  {
    if (AUTO_DEPLOY_AFTER_READY &&
        !deploymentDone &&
        gpsQualityOK() &&
        servosReady())
    {
      Serial.println();

      Serial.println(
        "GPS + ST-3009 READY"
      );

      Serial.println(
        "AUTO DEPLOY ENABLED"
      );


      startDeployment();
    }


    return;
  }


  // ------------------------------------------------------------
  // DEPLOYING
  // ------------------------------------------------------------

  if (state ==
      STATE_DEPLOYING)
  {
    steeringDemand =
      0;


    if (millis() -
        deploymentStartMs >=
        DEPLOY_MOVE_TIME_MS)
    {
      // Detach SG90
      deployServo.detach();


      deploymentServoActive =
        false;

```

```
      // Center steering
      if (servosReady())
      {
        centerServos();
      }


      // Stabilization starts
      stabilizationStartMs =
        millis();


      state =
        STATE_STABILIZING;


      Serial.println();

      Serial.println(
        "SG90 detached"
      );

      Serial.println(
        "5 second stabilization started"
      );
    }


    return;
  }


  // ------------------------------------------------------------
  // STABILIZING
  // ------------------------------------------------------------

  if (state ==
      STATE_STABILIZING)
  {
    steeringDemand =
      0;


    if (millis() -
        stabilizationStartMs >=
        DEPLOY_STABILIZE_TIME_MS)
    {
      if (gpsQualityOK() &&
          servosReady())
      {
        state =
          STATE_AUTONAV;


        Serial.println();

        Serial.println(
          "========================================"
        );

        Serial.println(
          "AUTONOMOUS NAVIGATION STARTED"
        );

        Serial.println(
          "========================================"
        );
      }
    }


    return;
  }
```

```
  // ------------------------------------------------------------
  // AUTONAV
  // ------------------------------------------------------------

  if (state ==
      STATE_AUTONAV)
  {
    // GPS / servo failure
    if (!gpsQualityOK() ||
        !servosReady())
    {
      steeringDemand =
        0;


      if (servosReady())
      {
        centerServos();
      }


      state =
        STATE_FAILSAFE;


      failsafeGoodSinceMs =
        0;


      Serial.println();

      Serial.println(
        "!!! NAVIGATION FAILSAFE !!!"
      );

      Serial.println(
        "GPS or ST-3009 unavailable"
      );


      return;
    }


    // Target capture
    if (distanceToTarget <=
        TARGET_CAPTURE_RADIUS_M)
    {
      centerServos();


      state =
        STATE_CAPTURED;


      Serial.println();

      Serial.println(
        "========================================"
      );

      Serial.println(
        "TARGET CAPTURED"
      );

      Serial.print(
        "Distance = "
      );

      Serial.print(
        distanceToTarget,
        2
      );

```

```
      Serial.println(
        " m"
      );

      Serial.println(
        "ST-3009 -> CENTER"
      );

      Serial.println(
        "========================================"
      );
    }


    return;
  }


  // ------------------------------------------------------------
  // CAPTURED
  // ------------------------------------------------------------

  if (state ==
      STATE_CAPTURED)
  {
    steeringDemand =
      0;

    return;
  }


  // ------------------------------------------------------------
  // FAILSAFE
  // ------------------------------------------------------------

  if (state ==
      STATE_FAILSAFE)
  {
    steeringDemand =
      0;


    if (servosReady())
    {
      centerServos();
    }


    if (gpsQualityOK() &&
        servosReady())
    {
      if (failsafeGoodSinceMs ==
          0)
      {
        failsafeGoodSinceMs =
          millis();


        Serial.println(
          "GPS + ST-3009 recovered"
        );


        Serial.println(
          "Waiting 3 seconds..."
        );
      }
      else if (millis() -
               failsafeGoodSinceMs >=
               FAILSAFE_RECOVERY_MS)
      {
        state =
          STATE_AUTONAV;

```

```
        failsafeGoodSinceMs =
          0;


        Serial.println(
          "FAILSAFE RECOVERY COMPLETE"
        );


        Serial.println(
          "AUTONAV RESUMED"
        );
      }
    }
    else
    {
      failsafeGoodSinceMs =
        0;
    }


    return;
  }
}


// ================================================================
// SERIAL INPUT
// ================================================================

void processSerialCommands()
{
  while (Serial.available() > 0)
  {
    char c =
      (char)Serial.read();


    // ------------------------------------------------------------
    // Newline
    // ------------------------------------------------------------

    if (c == '\r' ||
        c == '\n')
    {
      if (commandIndex > 0)
      {
        commandBuffer[
          commandIndex] =
          '\0';


        processCommand(
          commandBuffer
        );


        commandIndex =
          0;
      }


      continue;
    }


    // ------------------------------------------------------------
    // Store
    // ------------------------------------------------------------

    if (commandIndex <
        COMMAND_BUFFER_SIZE - 1)
    {
      commandBuffer[
```

```
        commandIndex++] =
        c;
    }
    else
    {
      commandIndex = 0;

      Serial.println(
        "ERROR: COMMAND TOO LONG"
      );
    }
  }
}


// ================================================================
// COMMAND PROCESSOR
// ================================================================

void processCommand(
  char *cmd)
{
  // Remove leading spaces
  while (*cmd == ' ' ||
         *cmd == '\t')
  {
    cmd++;
  }


  // Remove trailing spaces
  int len =
    strlen(cmd);


  while (len > 0 &&
         (cmd[len - 1] == ' ' ||
          cmd[len - 1] == '\t'))
  {
    cmd[len - 1] =
      '\0';

    len--;
  }


  if (len == 0)
    return;


  Serial.print(
    "CMD RECEIVED: ["
  );

  Serial.print(
    cmd
  );

  Serial.println(
    "]"
  );


  // ============================================================
  // STATUS
  // ============================================================

  if (strcmp(
        cmd,
        "STATUS") == 0)
  {
    printStatus();
    return;
  }

```

```
  // ============================================================
  // MOTORTEST
  // ============================================================

  if (strcmp(
        cmd,
        "MOTORTEST") == 0)
  {
    startMotorTest();
    return;
  }


  // ============================================================
  // LEFT
  // ============================================================

  if (strcmp(
        cmd,
        "LEFT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      +300
    );


    return;
  }


  // ============================================================
  // RIGHT
  // ============================================================

  if (strcmp(
        cmd,
        "RIGHT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      -300
    );


    return;
  }


  // ============================================================
  // CENTER
  // ============================================================

  if (strcmp(
        cmd,
        "CENTER") == 0)
  {
    motorTestActive =
      false;

```

```
    if (servosReady())
    {
      centerServos();


      Serial.println(
        "ST-3009 -> CENTER"
      );
    }
    else
    {
      Serial.println(
        "ST-3009 BUS NOT READY"
      );
    }


    state =
      STATE_WAIT_READY;


    return;
  }


  // ============================================================
  // AUTONAV
  // ============================================================

  if (strcmp(
        cmd,
        "AUTONAV") == 0)
  {
    motorTestActive =
      false;


    if (gpsQualityOK() &&
        servosReady())
    {
      state =
        STATE_AUTONAV;


      Serial.println(
        "AUTONAV ENABLED"
      );
    }
    else
    {
      Serial.println(
        "AUTONAV BLOCKED"
      );


      Serial.print(
        "GPS = "
      );

      Serial.println(
        gpsQualityOK()
          ? "GOOD"
          : "BAD"
      );


      Serial.print(
        "SERVO = "
      );

      Serial.println(
        servosReady()
          ? "GOOD"
          : "BAD"
```

```
      );
    }


    return;
  }


  // ================================================================
  // DEPLOY
  // ================================================================

  if (strcmp(
        cmd,
        "DEPLOY") == 0)
  {
    motorTestActive =
      false;


    startDeployment();

    return;
  }


  // ================================================================
  // PREP
  // ================================================================

  if (strcmp(
        cmd,
        "PREP") == 0)
  {
    motorTestActive =
      false;


    prepareDeploymentServo();

    return;
  }


  // ================================================================
  // RESET
  // ================================================================

  if (strcmp(
        cmd,
        "RESET") == 0)
  {
    motorTestActive =
      false;


    resetSystem();

    return;
  }


  // ================================================================
  // HELP
  // ================================================================

  if (strcmp(
        cmd,
        "HELP") == 0)
  {
    Serial.println();

    Serial.println(
      "STATUS"
    );
```

```
    Serial.println(
      "MOTORTEST"
    );

    Serial.println(
      "LEFT"
    );

    Serial.println(
      "RIGHT"
    );

    Serial.println(
      "CENTER"
    );

    Serial.println(
      "AUTONAV"
    );

    Serial.println(
      "DEPLOY"
    );

    Serial.println(
      "PREP"
    );

    Serial.println(
      "RESET"
    );

    Serial.println(
      "HELP"
    );


    Serial.println();

    Serial.println(
      "TARGET IS FIXED IN CODE"
    );


    return;
  }


  // Unknown
  Serial.print(
    "UNKNOWN COMMAND: "
  );

  Serial.println(
    cmd
  );
}


// ================================================================
// RESET
// ================================================================

void resetSystem()
{
  // Stop deployment servo
  if (deploymentServoActive)
  {
    deployServo.detach();

    deploymentServoActive =
      false;
  }

```

```
3340
3341    // Stop motor test
3342    motorTestActive =
3343      false;
3344
3345
3346    // Center if available
3347    if (servosReady())
3348    {
3349      centerServos();
3350    }
3351
3352
3353    // Reset flags
3354    deploymentDone =
3355      false;
3356
3357
3358    steeringDemand =
3359      0;
3360
3361
3362    failsafeGoodSinceMs =
3363      0;
3364
3365
3366    // Return to wait state
3367    state =
3368      STATE_WAIT_READY;
3369
3370
3371    stateEnteredMs =
3372      millis();
3373
3374
3375    Serial.println();
3376
3377    Serial.println(
3378      "========================================"
3379    );
3380
3381    Serial.println(
3382      "SYSTEM RESET"
3383    );
3384
3385    Serial.print(
3386      "TARGET = "
3387    );
3388
3389    Serial.print(
3390      TARGET_LAT,
3391      7
3392    );
3393
3394    Serial.print(
3395      ", "
3396    );
3397
3398    Serial.println(
3399      TARGET_LON,
3400      7
3401    );
3402
3403    Serial.println(
3404      "========================================"
3405    );
3406  }
3407
3408
3409  // ================================================================
3410  // STATUS
3411  // ================================================================
3412
3413  void printStatus()
3414  {
```

```
  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "                    STATUS"
  );

  Serial.println(
    "================================================"
  );


  // System
  Serial.print(
    "State             : "
  );

  Serial.println(
    stateName(state)
  );


  // Target
  Serial.print(
    "Target latitude   : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "Target longitude  : "
  );

  Serial.println(
    TARGET_LON,
    7
  );


  // GPS
  Serial.print(
    "GPS fix           : "
  );

  Serial.println(
    gpsFix
      ? "YES"
      : "NO"
  );


  Serial.print(
    "GPS quality       : "
  );

  Serial.println(
    gpsQualityOK()
      ? "GOOD"
      : "BAD"
  );


  Serial.print(
    "Satellites        : "
  );

  Serial.println(
    gpsSats
```

```
  );


  Serial.print(
    "HDOP               : "
  );

  Serial.println(
    gpsHDOP,
    2
  );


  Serial.print(
    "Latitude           : "
  );

  Serial.println(
    currentLatitude,
    7
  );


  Serial.print(
    "Longitude          : "
  );

  Serial.println(
    currentLongitude,
    7
  );


  Serial.print(
    "Speed              : "
  );

  Serial.print(
    currentSpeed,
    2
  );

  Serial.println(
    " m/s"
  );


  // Navigation
  Serial.print(
    "Distance to target: "
  );

  Serial.print(
    distanceToTarget,
    2
  );

  Serial.println(
    " m"
  );


  Serial.print(
    "Target bearing    : "
  );

  Serial.print(
    targetBearing,
    2
  );

  Serial.println(
    " deg"
  );

```

```
  Serial.print(
    "Current heading   : "
  );

  Serial.print(
    currentDirection,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Heading source    : "
  );

  Serial.println(
    headingSource
  );


  Serial.print(
    "Course error      : "
  );

  Serial.print(
    courseError,
    2
  );

  Serial.println(
    " deg"
  );


  // Compass
  Serial.print(
    "Compass status    : "
  );

  Serial.println(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Raw magnetometer  : "
  );

  Serial.print(
    rawMagX
  );

  Serial.print(
    ", "
  );

  Serial.print(
    rawMagY
  );

  Serial.print(
    ", "
  );

  Serial.println(
    rawMagZ
  );

```

```
  Serial.print(
    "Compass heading   : "
  );

  Serial.print(
    compassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Navigation heading: "
  );

  Serial.print(
    navCompassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  // Servos
  Serial.print(
    "Servo 1           : "
  );

  Serial.println(
    servo1OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 2           : "
  );

  Serial.println(
    servo2OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 1 position  : "
  );

  Serial.println(
    servo1Position
  );


  Serial.print(
    "Servo 2 position  : "
  );

  Serial.println(
    servo2Position
  );

  Serial.print(
    "Servo 1 feedback  : "
  );

  Serial.print(
    servo1Actual
  );
```

```
3715
3716  Serial.println(
3717    servo1FeedbackOK ? " OK" : " FAIL"
3718  );
3719
3720  Serial.print(
3721    "Servo 2 feedback  : "
3722  );
3723
3724  Serial.print(
3725    servo2Actual
3726  );
3727
3728  Serial.println(
3729    servo2FeedbackOK ? " OK" : " FAIL"
3730  );
3731
3732  Serial.print(
3733    "Steering mode     : "
3734  );
3735
3736  Serial.println(
3737    SINGLE_SERVO_STEERING ? "SINGLE SERVO" : "DIFFERENTIAL"
3738  );
3739
3740
3741  Serial.print(
3742    "Steering demand   : "
3743  );
3744
3745  Serial.println(
3746    steeringDemand
3747  );
3748
3749
3750  // IMU
3751  Serial.print(
3752    "IMU               : "
3753  );
3754
3755  Serial.println(
3756    imuOK
3757      ? "OK"
3758      : "FAIL"
3759  );
3760
3761
3762  // BMP
3763  Serial.print(
3764    "BMP581            : "
3765  );
3766
3767
3768  if (bmpOK)
3769  {
3770    Serial.print(
3771      "OK at 0x"
3772    );
3773
3774
3775    if (bmpAddress < 16)
3776      Serial.print("0");
3777
3778
3779    Serial.println(
3780      bmpAddress,
3781      HEX
3782    );
3783  }
3784  else
3785  {
3786    Serial.println(
3787      "NOT FOUND"
3788    );
3789  }
```

```
3790
3791
3792    // Deployment
3793    Serial.print(
3794      "Deployment done   : "
3795    );
3796
3797    Serial.println(
3798      deploymentDone
3799        ? "YES"
3800        : "NO"
3801    );
3802
3803
3804    Serial.println(
3805      "================================================"
3806    );
3807  }
3808
3809
3810  // ================================================================
3811  // LIVE DATA
3812  // ================================================================
3813
3814  void printLiveData()
3815  {
3816    Serial.print(
3817      "[LIVE] State="
3818    );
3819
3820    Serial.print(
3821      stateName(state)
3822    );
3823
3824
3825    // GPS
3826    Serial.print(
3827      " GPS="
3828    );
3829
3830    Serial.print(
3831      gpsQualityOK()
3832        ? "GOOD"
3833        : "BAD"
3834    );
3835
3836
3837    Serial.print(
3838      " SAT="
3839    );
3840
3841    Serial.print(
3842      gpsSats
3843    );
3844
3845
3846    Serial.print(
3847      " HDOP="
3848    );
3849
3850    Serial.print(
3851      gpsHDOP,
3852      1
3853    );
3854
3855
3856    if (gpsFix)
3857    {
3858      Serial.print(
3859        " LAT="
3860      );
3861
3862      Serial.print(
3863        currentLatitude,
3864        6
```

```
    );


    Serial.print(
      " LON="
    );

    Serial.print(
      currentLongitude,
      6
    );
  }


  // Navigation
  if (gpsQualityOK())
  {
    Serial.print(
      " DIST="
    );

    Serial.print(
      distanceToTarget,
      1
    );


    Serial.print(
      "m BRG="
    );

    Serial.print(
      targetBearing,
      1
    );
  }


  // Heading
  if (headingValid)
  {
    Serial.print(
      " HDG="
    );

    Serial.print(
      currentDirection,
      1
    );


    Serial.print(
      " SRC="
    );

    Serial.print(
      headingSource
    );


    Serial.print(
      " ERR="
    );

    Serial.print(
      courseError,
      1
    );
  }
  else
  {
    Serial.print(
      " HDG=NONE"
    );
  }
```

```
3940
3941
3942    // Steering
3943    Serial.print(
3944      " DEM="
3945    );
3946
3947    Serial.print(
3948      steeringDemand
3949    );
3950
3951
3952    Serial.print(
3953      " S1="
3954    );
3955
3956    Serial.print(
3957      servo1Position
3958    );
3959
3960
3961    Serial.print(
3962      " S2="
3963    );
3964
3965    Serial.print(
3966      servo2Position
3967    );
3968
3969    Serial.print(
3970      " FB1="
3971    );
3972
3973    Serial.print(
3974      servo1Actual
3975    );
3976
3977    Serial.print(
3978      " FB2="
3979    );
3980
3981    Serial.print(
3982      servo2Actual
3983    );
3984
3985
3986    // Compass
3987    Serial.print(
3988      " CMP="
3989    );
3990
3991    Serial.print(
3992      compassOK
3993        ? "OK"
3994        : "FAIL"
3995    );
3996
3997
3998    Serial.print(
3999      " CHP="
4000    );
4001
4002    Serial.print(
4003      compassHeading,
4004      1
4005    );
4006
4007
4008    Serial.print(
4009      " NAV="
4010    );
4011
4012    Serial.print(
4013      navCompassHeading,
4014      1
```

```
  );


  // IMU
  Serial.print(
    " IMU="
  );

  Serial.print(
    imuOK
      ? "OK"
      : "FAIL"
  );


  // Servos
  Serial.print(
    " SERVO="
  );

  Serial.println(
    servosReady()
      ? "OK"
      : "FAIL"
  );
}


// ================================================================
// MATLAB TELEMETRY
// ================================================================
// CSV format:
// NAVMAT,time_ms,lat,lon,speed_mps,target_bearing,heading,error,
//        demand,cmd_s1,cmd_s2,fb_s1,fb_s2,distance,state
// ================================================================

void sendMatlabTelemetry()
{
  if (millis() - lastMatlabTelemetryMs <
      MATLAB_TELEMETRY_INTERVAL_MS)
  {
    return;
  }

  lastMatlabTelemetryMs = millis();

  Serial.print("NAVMAT,");
  Serial.print(millis()); Serial.print(",");
  Serial.print(currentLatitude, 7); Serial.print(",");
  Serial.print(currentLongitude, 7); Serial.print(",");
  Serial.print(currentSpeed, 2); Serial.print(",");
  Serial.print(targetBearing, 2); Serial.print(",");
  Serial.print(currentDirection, 2); Serial.print(",");
  Serial.print(courseError, 2); Serial.print(",");
  Serial.print(steeringDemand); Serial.print(",");
  Serial.print(servo1Position); Serial.print(",");
  Serial.print(servo2Position); Serial.print(",");
  Serial.print(servo1Actual); Serial.print(",");
  Serial.print(servo2Actual); Serial.print(",");
  Serial.print(distanceToTarget, 2); Serial.print(",");
  Serial.println(stateName(state));
}


// ================================================================
// STATE NAME
// ================================================================

const char *stateName(
  SystemState s)
{
  switch (s)
  {
    case STATE_WAIT_READY:
      return "WAIT_READY";
```

```
    case STATE_DEPLOYING:
      return "DEPLOYING";


    case STATE_STABILIZING:
      return "STABILIZING";


    case STATE_AUTONAV:
      return "AUTONAV";


    case STATE_CAPTURED:
      return "CAPTURED";


    case STATE_FAILSAFE:
      return "FAILSAFE";


    case STATE_MANUAL_TEST:
      return "MANUAL_TEST";


    default:
      return "UNKNOWN";
  }
}
```

A.7 Archived source 7: Teensy_Parafoil_Final_MATLAB.ino

Source file: `Teensy_Parafoil_Final_MATLAB.ino`

```
/*
=========================================================================
 TEENSY 4.1 AUTONOMOUS PARAFOIL
 FINAL LAB / MATLAB LIVE-TEST TRANSMITTER

 PURPOSE
   - Live GPS navigation toward a target coordinate
   - ST-3009 single-servo steering mode for bench/mechanical testing
   - Actual ST-3009 position feedback
   - MATLAB machine-readable telemetry
   - Manual steering tests
   - GPS/compass heading selection
   - GPS/servo failsafe

 IMPORTANT
   - No PD, wind estimation, glide-speed estimation, or predicted
     landing-point controller is added yet.
   - Those parameters should be measured before tuning a PD controller.
   - TARGET_CAPTURE_ENABLED is FALSE by default. Passing within 35 m
     while still airborne does not guarantee a 35 m landing.

 STEERING MODE
   Servo ID 1 = LEFT
   Servo ID 2 = RIGHT

   SINGLE_SERVO_STEERING = true:
       LEFT  -> only Servo 1 moves
       RIGHT -> only Servo 2 moves
       CENTER -> both servos = 2048

   Bench-verify LEFT_STEER_DIRECTION and RIGHT_STEER_DIRECTION.

 HARDWARE
   GPS: TX -> 15, RX -> 14, Serial3 @ 38400
   I2C: SDA 18, SCL 19
   QMC5883: 0x0D
   LSM6DSO32: 0x6A
   BMP581: 0x46 or 0x47, diagnostic only
```

```
   ST-3009: IDs 1 and 2, Serial1, 1 Mbps, half duplex
   SG90: signal pin 8
   USB Serial: 115200

 COMMANDS
   STATUS
   MOTORTEST
   LEFT
   RIGHT
   CENTER
   AUTONAV
   TARGET <lat> <lon>
   DEPLOY
   PREP
   RESET
   HELP

 MATLAB TELEMETRY
   NAVMAT,<millis>,<lat>,<lon>,<alt>,<speed>,<course>,
   <heading>,<distance>,<bearing>,<error>,<demand>,
   <cmdS1>,<cmdS2>,<fbS1>,<fbS2>,<sats>,<hdop>,
   <state>,<headingSource>,<compass>
======================================================================
*/


// ================================================================
// LIBRARIES
// ================================================================

#include <Arduino.h>
#include <Wire.h>
#include <Servo.h>
#include <TinyGPS++.h>
#include <SCServo.h>
#include <Adafruit_LSM6DSOX.h>

#include <math.h>
#include <string.h>


// ================================================================
// TARGET
// ================================================================

double TARGET_LAT = 32.6321769;
double TARGET_LON = 75.0143889;


// ================================================================
// AUTO DEPLOY
// ================================================================
//
// true  = deploy automatically after GPS + both ST-3009 ready
// false = wait for DEPLOY command
//
// Lab default is FALSE. Use DEPLOY manually after bench checks.
// ================================================================

const bool AUTO_DEPLOY_AFTER_READY = false;


// ================================================================
// GPS
// ================================================================

#define GPS_SERIAL Serial3

const uint32_t GPS_BAUD = 38400;

const uint8_t GPS_RX_PIN = 15;
const uint8_t GPS_TX_PIN = 14;

// GPS data considered stale after this time
const uint32_t GPS_DATA_TIMEOUT_MS = 2500;
```

```
114
115 // Minimum satellites
116 const uint8_t GPS_MIN_SATS = 5;
117
118 // Maximum acceptable HDOP
119 const double GPS_MAX_HDOP = 3.5;
120
121 // GPS course is used only above this speed
122 const double GPS_COURSE_MIN_SPEED_MPS = 1.5;
123
124
125 // ================================================================
126 // I2C
127 // ================================================================
128
129 const uint8_t I2C_SDA_PIN = 18;
130 const uint8_t I2C_SCL_PIN = 19;
131
132
133 // ================================================================
134 // QMC5883
135 // ================================================================
136
137 const uint8_t QMC5883_ADDR = 0x0D;
138
139
140 // ----------------------------------------------------------------
141 // Compass navigation orientation
142 // ----------------------------------------------------------------
143 //
144 // Your tested compass direction was mirrored for navigation.
145 //
146
147 const bool COMPASS_MIRROR_FOR_NAV = true;
148
149 const float COMPASS_HEADING_OFFSET_DEG = 0.0f;
150
151
152 // ----------------------------------------------------------------
153 // Calibration values from your working compass test
154 // ----------------------------------------------------------------
155
156 const float OFFSET_X = 180.0000f;
157 const float OFFSET_Y = 336.0000f;
158 const float OFFSET_Z = 615.0000f;
159
160 const float SCALE_X = 0.956484f;
161 const float SCALE_Y = 0.904406f;
162 const float SCALE_Z = 1.178127f;
163
164
165 // ================================================================
166 // LSM6DSO32
167 // ================================================================
168
169 const uint8_t LSM6DSO32_ADDR = 0x6A;
170
171 Adafruit_LSM6DSOX lsm6dso;
172
173 bool imuOK = false;
174
175
176 // ================================================================
177 // BMP581
178 // ================================================================
179
180 const uint8_t BMP581_ADDR_1 = 0x46;
181 const uint8_t BMP581_ADDR_2 = 0x47;
182
183 bool bmpOK = false;
184
185 uint8_t bmpAddress = 0;
186
187
188 // ================================================================
```

```
// GPS VARIABLES
// ================================================================

TinyGPSPlus gps;

bool gpsFix = false;

uint8_t gpsSats = 0;

double gpsHDOP = 99.9;

double currentSpeed = 0.0;
double currentAltitude = -1.0;

unsigned long lastGPSLocationMs = 0;


// ================================================================
// COMPASS VARIABLES
// ================================================================

bool compassOK = false;

int16_t rawMagX = 0;
int16_t rawMagY = 0;
int16_t rawMagZ = 0;

float magX = 0.0f;
float magY = 0.0f;
float magZ = 0.0f;

float compassHeading = 0.0f;
float navCompassHeading = 0.0f;

uint8_t compassFailCount = 0;


// ================================================================
// IMU VARIABLES
// ================================================================

float accelX = 0.0f;
float accelY = 0.0f;
float accelZ = 0.0f;

float gyroX = 0.0f;
float gyroY = 0.0f;
float gyroZ = 0.0f;

unsigned long lastSensorUpdateMs = 0;


// ================================================================
// ST-3009
// ================================================================
//
// IMPORTANT:
// SCServo 1.0.2 uses:
//
//     SMS_STS
//

SMS_STS st;


// ----------------------------------------------------------------
// Servo IDs
// ----------------------------------------------------------------

const uint8_t SERVO1_ID = 1;
const uint8_t SERVO2_ID = 2;


// ----------------------------------------------------------------
// Servo positions
```

```
// ----------------------------------------------------------------

const int SERVO_NEUTRAL = 2048;

const int SERVO_MIN_POS = 200;
const int SERVO_MAX_POS = 3895;


// ----------------------------------------------------------------
// Steering
// ----------------------------------------------------------------

const int MAX_STEERING_COUNTS = 450;

const int STEERING_SERVO_SPEED = 500;
const int STEERING_SERVO_ACCEL = 50;

const int MAX_SLEW_COUNTS = 60;

// ----------------------------------------------------------------
// Steering mode
// ----------------------------------------------------------------
const bool SINGLE_SERVO_STEERING = true;
const bool SERVO1_IS_LEFT = true;
const int LEFT_STEER_DIRECTION = -1;
const int RIGHT_STEER_DIRECTION = -1;


// ----------------------------------------------------------------
// Servo health
// ----------------------------------------------------------------

const unsigned long SERVO_HEALTH_INTERVAL_MS = 2000;

bool servo1OK = false;
bool servo2OK = false;
bool servoBusReady = false;

unsigned long lastServoHealthMs = 0;


// ----------------------------------------------------------------
// Current positions
// ----------------------------------------------------------------

int servo1Position = SERVO_NEUTRAL;
int servo2Position = SERVO_NEUTRAL;


// ----------------------------------------------------------------
// Current navigation demand
// ----------------------------------------------------------------

int steeringDemand = 0;

int servo1Feedback = -1;
int servo2Feedback = -1;
unsigned long lastServoFeedbackMs = 0;
const unsigned long SERVO_FEEDBACK_INTERVAL_MS = 200;

unsigned long lastMATLABTelemetryMs = 0;
const unsigned long MATLAB_TELEMETRY_INTERVAL_MS = 200;


// ================================================================
// SG90 DEPLOYMENT
// ================================================================

Servo deployServo;

const uint8_t DEPLOY_SERVO_PIN = 8;

const int DEPLOY_ANGLE = 180;

const int PREP_ANGLE = 140;
```

```

const unsigned long DEPLOY_MOVE_TIME_MS = 1500;

const unsigned long DEPLOY_STABILIZE_TIME_MS = 5000;

bool deploymentServoActive = false;

bool deploymentDone = false;

unsigned long deploymentStartMs = 0;

unsigned long stabilizationStartMs = 0;


// ================================================================
// NAVIGATION
// ================================================================

const double TARGET_CAPTURE_RADIUS_M = 35.0;

// Disabled by default: do not stop steering just because the
// vehicle is within 35 m while still airborne.
const bool TARGET_CAPTURE_ENABLED = false;

const double STEERING_DEADBAND_DEG = 5.0;

const double FULL_STEERING_ERROR_DEG = 90.0;


// ================================================================
// STATE MACHINE
// ================================================================

enum SystemState
{
  STATE_WAIT_READY,
  STATE_DEPLOYING,
  STATE_STABILIZING,
  STATE_AUTONAV,
  STATE_CAPTURED,
  STATE_FAILSAFE,
  STATE_MANUAL_TEST
};

SystemState state = STATE_WAIT_READY;

unsigned long stateEnteredMs = 0;


// ================================================================
// NAVIGATION VARIABLES
// ================================================================

double currentLatitude = 0.0;

double currentLongitude = 0.0;

double distanceToTarget = 0.0;

double targetBearing = 0.0;

double currentDirection = 0.0;

double courseError = 0.0;

bool headingValid = false;

const char *headingSource = "NONE";


// ================================================================
// FAILSAFE
// ================================================================

unsigned long failsafeGoodSinceMs = 0;
```

```
414
415 const unsigned long FAILSAFE_RECOVERY_MS = 3000;
416
417
418 // ================================================================
419 // SERIAL COMMAND BUFFER
420 // ================================================================
421
422 #define COMMAND_BUFFER_SIZE 120
423
424 char commandBuffer[COMMAND_BUFFER_SIZE];
425
426 uint16_t commandIndex = 0;
427
428
429 // ================================================================
430 // TIMERS
431 // ================================================================
432
433 unsigned long lastNavigationUpdateMs = 0;
434
435 const unsigned long NAVIGATION_UPDATE_INTERVAL_MS = 100;
436
437 unsigned long lastLivePrintMs = 0;
438
439 const unsigned long LIVE_PRINT_INTERVAL_MS = 1000;
440
441 unsigned long lastBMPCheckMs = 0;
442
443 const unsigned long BMP_CHECK_INTERVAL_MS = 3000;
444
445
446 // ================================================================
447 // MOTOR TEST
448 // ================================================================
449
450 bool motorTestActive = false;
451
452 uint8_t motorTestStep = 0;
453
454 unsigned long motorTestTimer = 0;
455
456
457 // ================================================================
458 // FORWARD DECLARATIONS
459 // ================================================================
460
461 void processSerialCommands();
462 void processCommand(char *cmd);
463
464 void updateGPS();
465 void updateCompass();
466 void updateIMU();
467 void updateServoHealth();
468 void updateServoFeedback();
469 void printMATLABTelemetry();
470
471 void updateNavigation();
472 void updateSteering();
473 void updateStateMachine();
474
475 void writeSteeringPositions(int pos1, int pos2);
476 void centerServos();
477 void commandSteering(int demand);
478
479 void startDeployment();
480 void prepareDeploymentServo();
481 void resetSystem();
482
483 void startMotorTest();
484 void updateMotorTest();
485
486 void printStatus();
487 void printLiveData();
488
```

```
bool gpsQualityOK();
bool readQMC5883();
bool initQMC5883();

bool checkI2CDevice(uint8_t address);
bool initIMU();

bool servosReady();

float wrap360(float angle);
double wrap180(double angle);
float navigationCompassHeading();

const char *stateName(SystemState s);

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep
);


// ================================================================
// SETUP
// ================================================================

void setup()
{
  // ------------------------------------------------------------
  // USB SERIAL
  // ------------------------------------------------------------

  Serial.begin(115200);

  delay(1500);


  Serial.println();
  Serial.println(
    "================================================"
  );

  Serial.println(
    " TEENSY 4.1 AUTONOMOUS PARAFOIL"
  );

  Serial.println(
    " GUIDED LANDING - FINAL LAB TEST"
  );

  Serial.println(
    "================================================"
  );


  // ------------------------------------------------------------
  // TARGET
  // ------------------------------------------------------------

  Serial.println();

  Serial.print(
    "TARGET LATITUDE  : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "TARGET LONGITUDE : "
  );

```

```
  Serial.println(
    TARGET_LON,
    7
  );

  Serial.print("STEERING MODE   : ");
  Serial.println(SINGLE_SERVO_STEERING ? "SINGLE SERVO" : "DIFFERENTIAL");
  Serial.print("SERVO 1 ROLE    : ");
  Serial.println(SERVO1_IS_LEFT ? "LEFT" : "RIGHT");


  // ---------------------------------------------------------------
  // I2C
  // ---------------------------------------------------------------

  Wire.begin();

  Wire.setClock(400000);


  Serial.println();

  Serial.println(
    "I2C initialized"
  );

  Serial.println(
    "SDA = Pin 18"
  );

  Serial.println(
    "SCL = Pin 19"
  );


  // ---------------------------------------------------------------
  // QMC5883
  // ---------------------------------------------------------------

  if (initQMC5883())
  {
    compassOK = true;

    Serial.println(
      "QMC5883 : OK"
    );
  }
  else
  {
    compassOK = false;

    Serial.println(
      "QMC5883 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // LSM6DSO32
  // ---------------------------------------------------------------

  if (initIMU())
  {
    imuOK = true;

    Serial.println(
      "LSM6DSO32 : OK"
    );
  }
  else
  {
    imuOK = false;

    Serial.println(
      "LSM6DSO32 : NOT FOUND"
```

```
  );
}


// ---------------------------------------------------------------
// BMP581
// ---------------------------------------------------------------

if (checkI2CDevice(
      BMP581_ADDR_1))
{
  bmpOK = true;

  bmpAddress =
    BMP581_ADDR_1;
}
else if (checkI2CDevice(
           BMP581_ADDR_2))
{
  bmpOK = true;

  bmpAddress =
    BMP581_ADDR_2;
}
else
{
  bmpOK = false;

  bmpAddress = 0;
}


if (bmpOK)
{
  Serial.print(
    "BMP581 : FOUND at 0x"
  );

  if (bmpAddress < 16)
    Serial.print("0");

  Serial.println(
    bmpAddress,
    HEX
  );
}
else
{
  Serial.println(
    "BMP581 : NOT FOUND"
  );
}


// ---------------------------------------------------------------
// GPS
// ---------------------------------------------------------------
//
// EXACTLY MATCHES YOUR WORKING GPS TEST
//

GPS_SERIAL.begin(
  GPS_BAUD
);


Serial.println();

Serial.println(
  "GPS UART  : Serial3"
);

Serial.print(
  "GPS RX    : Pin "
);
```

```

  Serial.println(
    GPS_RX_PIN
  );

  Serial.print(
    "GPS TX    : Pin "
  );

  Serial.println(
    GPS_TX_PIN
  );

  Serial.print(
    "GPS BAUD  : "
  );

  Serial.println(
    GPS_BAUD
  );


  // ---------------------------------------------------------------
  // ST-3009
  // ---------------------------------------------------------------

  Serial.println();

  Serial.println(
    "Initializing ST-3009 bus..."
  );


  Serial1.begin(
    1000000,
    SERIAL_8N1_HALF_DUPLEX
  );


  st.pSerial =
    &Serial1;


  delay(500);


  // Force immediate first health test
  lastServoHealthMs =
    millis() -
    SERVO_HEALTH_INTERVAL_MS;


  updateServoHealth();


  if (servosReady())
  {
    Serial.println(
      "ST-3009 ID 1 : OK"
    );

    Serial.println(
      "ST-3009 ID 2 : OK"
    );


    centerServos();
  }
  else
  {
    Serial.println(
      "ST-3009 : WAITING"
    );
  }

```

```

  // ---------------------------------------------------------------
  // SG90
  // ---------------------------------------------------------------

  deployServo.detach();

  deploymentServoActive =
    false;


  // ---------------------------------------------------------------
  // State
  // ---------------------------------------------------------------

  state =
    STATE_WAIT_READY;

  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "SYSTEM READY"
  );

  Serial.println(
    "Target can be updated with TARGET lat lon."
  );

  Serial.println(
    "================================================"
  );


  Serial.println();

  Serial.println(
    "COMMANDS:"
  );

  Serial.println(
    "STATUS"
  );

  Serial.println(
    "MOTORTEST"
  );

  Serial.println(
    "LEFT"
  );

  Serial.println(
    "RIGHT"
  );

  Serial.println(
    "CENTER"
  );

  Serial.println(
    "AUTONAV"
  );

  Serial.println(
    "DEPLOY"
  );

```

```
  Serial.println(
    "PREP"
  );

  Serial.println(
    "RESET"
  );

  Serial.println(
    "HELP"
  );


  Serial.println();

  Serial.println(
    "Waiting for GPS + ST-3009..."
  );
}


// ================================================================
// LOOP
// ================================================================

void loop()
{
  // GPS continuously
  updateGPS();


  // Serial commands
  processSerialCommands();


  // Sensors
  if (millis() -
      lastSensorUpdateMs >=
      100)
  {
    lastSensorUpdateMs =
      millis();

    updateCompass();
    updateIMU();
  }


  // Servo health
  updateServoHealth();

  // Actual servo feedback for MATLAB
  updateServoFeedback();


  // Motor test
  if (motorTestActive)
  {
    updateMotorTest();
  }


  // State
  updateStateMachine();


  // Navigation
  if (millis() -
      lastNavigationUpdateMs >=
      NAVIGATION_UPDATE_INTERVAL_MS)
  {
    lastNavigationUpdateMs =
      millis();

    updateNavigation();
```

```
    updateSteering();
  }


  // BMP check
  if (millis() -
      lastBMPCheckMs >=
      BMP_CHECK_INTERVAL_MS)
  {
    lastBMPCheckMs =
      millis();


    if (!bmpOK)
    {
      if (checkI2CDevice(
            BMP581_ADDR_1))
      {
        bmpOK = true;

        bmpAddress =
          BMP581_ADDR_1;
      }
      else if (checkI2CDevice(
                  BMP581_ADDR_2))
      {
        bmpOK = true;

        bmpAddress =
          BMP581_ADDR_2;
      }
    }
  }


  // Live data
  if (millis() -
      lastLivePrintMs >=
      LIVE_PRINT_INTERVAL_MS)
  {
    lastLivePrintMs =
      millis();

    printLiveData();
  }

  if (millis() -
      lastMATLABTelemetryMs >=
      MATLAB_TELEMETRY_INTERVAL_MS)
  {
    lastMATLABTelemetryMs = millis();
    printMATLABTelemetry();
  }
}


// ================================================================
// GPS UPDATE
// ================================================================

void updateGPS()
{
  while (GPS_SERIAL.available() > 0)
  {
    char c =
      (char)GPS_SERIAL.read();

    gps.encode(c);
  }


  // Fresh GPS location
  if (gps.location.isUpdated())
  {
    lastGPSLocationMs =
```

```
      millis();
  }


  // Position
  if (gps.location.isValid())
  {
    gpsFix = true;

    currentLatitude =
      gps.location.lat();

    currentLongitude =
      gps.location.lng();
  }
  else
  {
    gpsFix = false;
  }


  // Satellites
  if (gps.satellites.isValid())
  {
    gpsSats =
      gps.satellites.value();
  }
  else
  {
    gpsSats = 0;
  }


  // HDOP
  if (gps.hdop.isValid())
  {
    gpsHDOP =
      gps.hdop.hdop();
  }
  else
  {
    gpsHDOP = 99.9;
  }


  // Speed
  if (gps.speed.isValid())
  {
    currentSpeed =
      gps.speed.mps();
  }
  else
  {
    currentSpeed = 0.0;
  }

  if (gps.altitude.isValid())
    currentAltitude = gps.altitude.meters();
  else
    currentAltitude = -1.0;
}


// ================================================================
// GPS QUALITY
// ================================================================

bool gpsQualityOK()
{
  if (!gpsFix)
    return false;


  if (lastGPSLocationMs == 0)
    return false;
```

```

  if (millis() -
      lastGPSLocationMs >
      GPS_DATA_TIMEOUT_MS)
  {
    return false;
  }


  if (gpsSats <
      GPS_MIN_SATS)
  {
    return false;
  }


  if (gps.hdop.isValid() &&
      gpsHDOP >
      GPS_MAX_HDOP)
  {
    return false;
  }


  return true;
}


// ================================================================
// QMC5883 INIT
// ================================================================

bool initQMC5883()
{
  // Check device
  Wire.beginTransmission(
    QMC5883_ADDR
  );


  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  // SET/RESET period
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x0B);
  Wire.write(0x01);

  if (Wire.endTransmission() != 0)
  {
    return false;
  }


  delay(10);


  // Control register
  // Same setting used by your
  // working compass test.
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x09);
  Wire.write(0x1D);

  if (Wire.endTransmission() != 0)
```

```
  {
    return false;
  }


  delay(100);


  return true;
}


// ================================================================
// QMC5883 READ
// ================================================================

bool readQMC5883()
{
  Wire.beginTransmission(
    QMC5883_ADDR
  );

  Wire.write(0x00);


  if (Wire.endTransmission(false) != 0)
  {
    return false;
  }


  uint8_t count =
    Wire.requestFrom(
      QMC5883_ADDR,
      (uint8_t)6
    );


  if (count != 6)
  {
    while (Wire.available())
    {
      Wire.read();
    }

    return false;
  }


  uint8_t xl =
    Wire.read();

  uint8_t xh =
    Wire.read();


  uint8_t yl =
    Wire.read();

  uint8_t yh =
    Wire.read();


  uint8_t zl =
    Wire.read();

  uint8_t zh =
    Wire.read();


  rawMagX =
    (int16_t)(
      (xh << 8) |
      xl
    );
```

```

  rawMagY =
    (int16_t)(
      (yh << 8) |
      yl
    );


  rawMagZ =
    (int16_t)(
      (zh << 8) |
      zl
    );


  return true;
}


// ================================================================
// COMPASS UPDATE
// ================================================================

void updateCompass()
{
  // ------------------------------------------------------------
  // Read sensor
  // ------------------------------------------------------------

  if (!readQMC5883())
  {
    compassFailCount++;


    // Do not immediately declare compass dead.
    // Reinitialize after 5 consecutive failures.

    if (compassFailCount >= 5)
    {
      if (initQMC5883())
      {
        compassOK = true;

        compassFailCount = 0;

        Serial.println(
          "[COMPASS] Reinitialized OK"
        );
      }
      else
      {
        compassOK = false;
      }
    }


    return;
  }


  // Successful reading
  compassFailCount = 0;
  compassOK = true;


  // ------------------------------------------------------------
  // Apply your proven calibration
  // ------------------------------------------------------------

  magX =
    ((float)rawMagX -
     OFFSET_X) *
    SCALE_X;

```

```
  magY =
    ((float)rawMagY -
     OFFSET_Y) *
    SCALE_Y;


  magZ =
    ((float)rawMagZ -
     OFFSET_Z) *
    SCALE_Z;


  // ------------------------------------------------------------
  // Calculate heading
  // ------------------------------------------------------------

  float angle =
    atan2(
      magY,
      magX
    ) *
    180.0f /
    PI;


  if (angle < 0.0f)
    angle += 360.0f;


  compassHeading =
    wrap360(
      angle +
      COMPASS_HEADING_OFFSET_DEG
    );


  // Navigation heading
  navCompassHeading =
    navigationCompassHeading();
}


// ================================================================
// NAVIGATION COMPASS
// ================================================================

float navigationCompassHeading()
{
  if (!COMPASS_MIRROR_FOR_NAV)
  {
    return compassHeading;
  }


  return wrap360(
    360.0f -
    compassHeading
  );
}


// ================================================================
// WRAP 0...360
// ================================================================

float wrap360(float angle)
{
  while (angle >= 360.0f)
    angle -= 360.0f;


  while (angle < 0.0f)
    angle += 360.0f;

```

```

  return angle;
}


// ================================================================
// WRAP -180...180
// ================================================================

double wrap180(double angle)
{
  while (angle > 180.0)
    angle -= 360.0;


  while (angle < -180.0)
    angle += 360.0;


  return angle;
}


// ================================================================
// IMU INIT
// ================================================================

bool initIMU()
{
  if (!lsm6dso.begin_I2C(
        LSM6DSO32_ADDR,
        &Wire))
  {
    return false;
  }


  lsm6dso.setAccelRange(
    LSM6DS_ACCEL_RANGE_16_G
  );


  lsm6dso.setGyroRange(
    LSM6DS_GYRO_RANGE_2000_DPS
  );


  lsm6dso.setAccelDataRate(
    LSM6DS_RATE_208_HZ
  );


  lsm6dso.setGyroDataRate(
    LSM6DS_RATE_208_HZ
  );


  return true;
}


// ================================================================
// IMU UPDATE
// ================================================================

void updateIMU()
{
  if (!imuOK)
    return;


  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;

```

```

  lsm6dso.getEvent(
    &accel,
    &gyro,
    &temp
  );


  accelX =
    accel.acceleration.x;


  accelY =
    accel.acceleration.y;


  accelZ =
    accel.acceleration.z;


  gyroX =
    gyro.gyro.x;


  gyroY =
    gyro.gyro.y;


  gyroZ =
    gyro.gyro.z;
}


// ================================================================
// I2C CHECK
// ================================================================

bool checkI2CDevice(
  uint8_t address)
{
  Wire.beginTransmission(
    address
  );


  return (
    Wire.endTransmission() == 0
  );
}


// ================================================================
// SERVO HEALTH
// ================================================================

void updateServoHealth()
{
  if (millis() -
      lastServoHealthMs <
      SERVO_HEALTH_INTERVAL_MS)
  {
    return;
  }


  lastServoHealthMs =
    millis();


  // Ping ID 1
  int result1 =
    st.Ping(
      SERVO1_ID
    );

```

```
  delay(10);


  // Ping ID 2
  int result2 =
    st.Ping(
      SERVO2_ID
    );


  servo1OK =
    (result1 >= 0);


  servo2OK =
    (result2 >= 0);


  servoBusReady =
    servo1OK &&
    servo2OK;
}


// ================================================================
// SERVO READY
// ================================================================

bool servosReady()
{
  return
    servoBusReady &&
    servo1OK &&
    servo2OK;
}


// ================================================================
// CENTER BOTH SERVOS
// ================================================================

void centerServos()
{
  if (!servosReady()) return;

  st.WritePosEx(SERVO1_ID, SERVO_NEUTRAL, STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
  st.WritePosEx(SERVO2_ID, SERVO_NEUTRAL, STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);

  servo1Position = SERVO_NEUTRAL;
  servo2Position = SERVO_NEUTRAL;
  steeringDemand = 0;
}

// ================================================================
// STEERING DEMAND -> SERVO TARGETS
// ================================================================
// demand > 0 = LEFT
// demand < 0 = RIGHT
// SINGLE_SERVO_STEERING=true makes only the requested servo move.

void steeringTargetsFromDemand(int demand, int &target1, int &target2)
{
  target1 = SERVO_NEUTRAL;
  target2 = SERVO_NEUTRAL;

  demand = constrain(demand, -MAX_STEERING_COUNTS, MAX_STEERING_COUNTS);

  if (!SINGLE_SERVO_STEERING)
  {
    target1 = SERVO_NEUTRAL - demand;
    target2 = SERVO_NEUTRAL + demand;
    return;
  }

```

```
  if (demand > 0)
  {
    if (SERVO1_IS_LEFT)
      target1 = SERVO_NEUTRAL + LEFT_STEER_DIRECTION * demand;
    else
      target2 = SERVO_NEUTRAL + RIGHT_STEER_DIRECTION * demand;
  }
  else if (demand < 0)
  {
    int magnitude = -demand;

    if (SERVO1_IS_LEFT)
      target2 = SERVO_NEUTRAL + RIGHT_STEER_DIRECTION * magnitude;
    else
      target1 = SERVO_NEUTRAL + LEFT_STEER_DIRECTION * magnitude;
  }

  target1 = constrain(target1, SERVO_MIN_POS, SERVO_MAX_POS);
  target2 = constrain(target2, SERVO_MIN_POS, SERVO_MAX_POS);
}

void commandSteering(int demand)
{
  if (!servosReady())
  {
    Serial.println("ERROR: ST-3009 BUS NOT READY");
    return;
  }

  demand = constrain(demand, -MAX_STEERING_COUNTS, MAX_STEERING_COUNTS);

  int pos1, pos2;
  steeringTargetsFromDemand(demand, pos1, pos2);

  st.WritePosEx(SERVO1_ID, pos1, STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
  st.WritePosEx(SERVO2_ID, pos2, STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);

  servo1Position = pos1;
  servo2Position = pos2;
  steeringDemand = demand;

  Serial.print("DEMAND=");
  Serial.print(demand);
  Serial.print("  CMD_S1=");
  Serial.print(pos1);
  Serial.print("  CMD_S2=");
  Serial.println(pos2);
}

// ================================================================
// MOTOR TEST START
// ================================================================

void startMotorTest()
{
  if (!servosReady())
  {
    Serial.println(
      "MOTORTEST BLOCKED: "
      "ST-3009 NOT READY"
    );

    return;
  }


  motorTestActive =
    true;


  state =
    STATE_MANUAL_TEST;


  motorTestStep = 0;
```

```

  motorTestTimer =
    millis();


  centerServos();


  Serial.println();

  Serial.println(
    "======================================="
  );

  Serial.println(
    "       ST-3009 MOTOR TEST"
  );

  Serial.println(
    "======================================="
  );

  Serial.println(
    "CENTER -> LEFT -> CENTER -> RIGHT -> CENTER"
  );

  Serial.println();
}


// ================================================================
// MOTOR TEST UPDATE
// ================================================================

void updateMotorTest()
{
  if (!servosReady())
  {
    motorTestActive =
      false;

    state =
      STATE_WAIT_READY;

    return;
  }


  if (millis() -
      motorTestTimer <
      1800)
  {
    return;
  }


  motorTestTimer =
    millis();


  switch (motorTestStep)
  {
    case 0:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


```

```
    case 1:

      Serial.println(
        "TEST: LEFT"
      );

      commandSteering(
        +300
      );

      motorTestStep++;

      break;


    case 2:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    case 3:

      Serial.println(
        "TEST: RIGHT"
      );

      commandSteering(
        -300
      );

      motorTestStep++;

      break;


    case 4:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    default:

      motorTestActive =
        false;


      state =
        STATE_WAIT_READY;


      centerServos();


      Serial.println();

      Serial.println(
        "MOTOR TEST COMPLETE"
      );

```

```
      Serial.println();

      break;
  }
}


// ================================================================
// NAVIGATION CALCULATION
// ================================================================

void updateNavigation()
{
  if (!gpsQualityOK())
  {
    headingValid =
      false;

    headingSource =
      "NONE";

    return;
  }


  // ---------------------------------------------------------------
  // GPS position
  // ---------------------------------------------------------------

  currentLatitude =
    gps.location.lat();


  currentLongitude =
    gps.location.lng();


  // ---------------------------------------------------------------
  // Distance
  // ---------------------------------------------------------------

  distanceToTarget =
    TinyGPSPlus::distanceBetween(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
      TARGET_LON
    );


  // ---------------------------------------------------------------
  // Bearing
  // ---------------------------------------------------------------

  targetBearing =
    TinyGPSPlus::courseTo(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
      TARGET_LON
    );


  targetBearing =
    wrap360(
      targetBearing
    );


  // ---------------------------------------------------------------
  // Determine heading
  // ---------------------------------------------------------------

  headingValid =
    false;
```

```

  headingSource =
    "NONE";


  // GPS course when moving
  if (gps.speed.isValid() &&
      gps.course.isValid() &&
      currentSpeed >=
      GPS_COURSE_MIN_SPEED_MPS)
  {
    currentDirection =
      wrap360(
        gps.course.deg()
      );


    headingValid =
      true;


    headingSource =
      "GPS COURSE";
  }


  // Compass when slow/stationary
  else if (compassOK)
  {
    currentDirection =
      navigationCompassHeading();


    headingValid =
      true;


    headingSource =
      "COMPASS";
  }


  // ------------------------------------------------------------
  // Course error
  // ------------------------------------------------------------

  if (headingValid)
  {
    courseError =
      wrap180(
        targetBearing -
        currentDirection
      );
  }
}


// ================================================================
// AUTONOMOUS STEERING
// ================================================================

void updateSteering()
{
  if (motorTestActive) return;

  if (!servosReady())
  {
    steeringDemand = 0;
    return;
  }

  if (state != STATE_AUTONAV) return;

  if (!gpsQualityOK())
```

```
  {
    centerServos();
    return;
  }

  if (TARGET_CAPTURE_ENABLED && distanceToTarget <= TARGET_CAPTURE_RADIUS_M)
  {
    centerServos();
    return;
  }

  if (!headingValid)
  {
    centerServos();
    return;
  }

  double absoluteError = fabs(courseError);

  if (absoluteError <= STEERING_DEADBAND_DEG)
  {
    steeringDemand = 0;
  }
  else
  {
    double usableError = absoluteError - STEERING_DEADBAND_DEG;
    double usableRange = FULL_STEERING_ERROR_DEG - STEERING_DEADBAND_DEG;
    double normalized = constrain(usableError / usableRange, 0.0, 1.0);
    int demand = (int)round(normalized * MAX_STEERING_COUNTS);

    if (courseError > 0.0) steeringDemand = demand;
    else steeringDemand = -demand;
  }

  int target1, target2;
  steeringTargetsFromDemand(steeringDemand, target1, target2);

  int new1 = slewTo(servo1Position, target1, MAX_SLEW_COUNTS);
  int new2 = slewTo(servo2Position, target2, MAX_SLEW_COUNTS);

  if (new1 != servo1Position || new2 != servo2Position)
  {
    st.WritePosEx(SERVO1_ID, new1, STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
    st.WritePosEx(SERVO2_ID, new2, STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);

    servo1Position = new1;
    servo2Position = new2;
  }
}

// ================================================================
// SLEW LIMIT
// ================================================================

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep)
{
  if (targetValue >
      currentValue)
  {
    int difference =
      targetValue -
      currentValue;


    if (difference >
        maxStep)
    {
      return currentValue +
             maxStep;
    }


```

```
    return targetValue;
  }


  if (targetValue <
      currentValue)
  {
    int difference =
      currentValue -
      targetValue;


    if (difference >
        maxStep)
    {
      return currentValue -
             maxStep;
    }


    return targetValue;
  }


  return currentValue;
}


// ================================================================
// DEPLOYMENT
// ================================================================

void startDeployment()
{
  if (deploymentDone)
  {
    Serial.println(
      "DEPLOYMENT ALREADY COMPLETED"
    );

    return;
  }


  if (!gpsQualityOK())
  {
    Serial.println(
      "DEPLOY BLOCKED: GPS NOT READY"
    );

    return;
  }


  if (!servosReady())
  {
    Serial.println(
      "DEPLOY BLOCKED: ST-3009 NOT READY"
    );

    return;
  }


  // Center steering first
  centerServos();


  delay(100);


  // Attach SG90
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );
```

```

  delay(100);


  // Deployment position
  deployServo.write(
    DEPLOY_ANGLE
  );


  deploymentStartMs =
    millis();


  deploymentServoActive =
    true;


  deploymentDone =
    true;


  state =
    STATE_DEPLOYING;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "======================================="
  );

  Serial.println(
    "STARTING DEPLOYMENT"
  );

  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    DEPLOY_ANGLE
  );

  Serial.println(
    " degrees"
  );
}


// ================================================================
// SG90 PREP
// ================================================================

void prepareDeploymentServo()
{
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );


  delay(100);


  deployServo.write(
    PREP_ANGLE
  );


  Serial.print(
```

```
    "SG90 -> "
  );

  Serial.print(
    PREP_ANGLE
  );

  Serial.println(
    " degrees"
  );


  delay(1000);


  deployServo.detach();


  Serial.println(
    "SG90 detached"
  );
}


// ================================================================
// STATE MACHINE
// ================================================================

void updateStateMachine()
{
  // ----------------------------------------------------------------
  // Manual test
  // ----------------------------------------------------------------

  if (state ==
      STATE_MANUAL_TEST)
  {
    return;
  }


  // ----------------------------------------------------------------
  // WAIT READY
  // ----------------------------------------------------------------

  if (state ==
      STATE_WAIT_READY)
  {
    if (AUTO_DEPLOY_AFTER_READY &&
        !deploymentDone &&
        gpsQualityOK() &&
        servosReady())
    {
      Serial.println();

      Serial.println(
        "GPS + ST-3009 READY"
      );

      Serial.println(
        "AUTO DEPLOY ENABLED"
      );


      startDeployment();
    }


    return;
  }


  // ----------------------------------------------------------------
  // DEPLOYING
  // ----------------------------------------------------------------
```

```
2289
2290  if (state ==
2291      STATE_DEPLOYING)
2292  {
2293    steeringDemand =
2294      0;
2295
2296
2297    if (millis() -
2298        deploymentStartMs >=
2299        DEPLOY_MOVE_TIME_MS)
2300    {
2301      // Detach SG90
2302      deployServo.detach();
2303
2304
2305      deploymentServoActive =
2306        false;
2307
2308
2309      // Center steering
2310      if (servosReady())
2311      {
2312        centerServos();
2313      }
2314
2315
2316      // Stabilization starts
2317      stabilizationStartMs =
2318        millis();
2319
2320
2321      state =
2322        STATE_STABILIZING;
2323
2324
2325      Serial.println();
2326
2327      Serial.println(
2328        "SG90 detached"
2329      );
2330
2331      Serial.println(
2332        "5 second stabilization started"
2333      );
2334    }
2335
2336
2337    return;
2338  }
2339
2340
2341  // ------------------------------------------------------------
2342  // STABILIZING
2343  // ------------------------------------------------------------
2344
2345  if (state ==
2346      STATE_STABILIZING)
2347  {
2348    steeringDemand =
2349      0;
2350
2351
2352    if (millis() -
2353        stabilizationStartMs >=
2354        DEPLOY_STABILIZE_TIME_MS)
2355    {
2356      if (gpsQualityOK() &&
2357          servosReady())
2358      {
2359        state =
2360          STATE_AUTONAV;
2361
2362
2363        Serial.println();
```

```
      Serial.println(
        "========================================"
      );

      Serial.println(
        "AUTONOMOUS NAVIGATION STARTED"
      );

      Serial.println(
        "========================================"
      );
    }
  }


  return;
}


// ------------------------------------------------------------
// AUTONAV
// ------------------------------------------------------------

if (state ==
    STATE_AUTONAV)
{
  // GPS / servo failure
  if (!gpsQualityOK() ||
      !servosReady())
  {
    steeringDemand =
      0;


    if (servosReady())
    {
      centerServos();
    }


    state =
      STATE_FAILSAFE;


    failsafeGoodSinceMs =
      0;


    Serial.println();

    Serial.println(
      "!!! NAVIGATION FAILSAFE !!!"
    );

    Serial.println(
      "GPS or ST-3009 unavailable"
    );


    return;
  }


  // Target capture
  if (TARGET_CAPTURE_ENABLED &&
      distanceToTarget <=
      TARGET_CAPTURE_RADIUS_M)
  {
    centerServos();


    state =
      STATE_CAPTURED;

```

```
      Serial.println();

      Serial.println(
        "========================================"
      );

      Serial.println(
        "TARGET CAPTURED"
      );

      Serial.print(
        "Distance = "
      );

      Serial.print(
        distanceToTarget,
        2
      );

      Serial.println(
        " m"
      );

      Serial.println(
        "ST-3009 -> CENTER"
      );

      Serial.println(
        "========================================"
      );
    }


    return;
  }


  // ------------------------------------------------------------
  // CAPTURED
  // ------------------------------------------------------------

  if (state ==
      STATE_CAPTURED)
  {
    steeringDemand =
      0;

    return;
  }


  // ------------------------------------------------------------
  // FAILSAFE
  // ------------------------------------------------------------

  if (state ==
      STATE_FAILSAFE)
  {
    steeringDemand =
      0;


    if (servosReady())
    {
      centerServos();
    }


    if (gpsQualityOK() &&
        servosReady())
    {
      if (failsafeGoodSinceMs ==
          0)
      {
```

```
        failsafeGoodSinceMs =
          millis();


        Serial.println(
          "GPS + ST-3009 recovered"
        );


        Serial.println(
          "Waiting 3 seconds..."
        );
      }
      else if (millis() -
               failsafeGoodSinceMs >=
               FAILSAFE_RECOVERY_MS)
      {
        state =
          STATE_AUTONAV;


        failsafeGoodSinceMs =
          0;


        Serial.println(
          "FAILSAFE RECOVERY COMPLETE"
        );


        Serial.println(
          "AUTONAV RESUMED"
        );
      }
    }
    else
    {
      failsafeGoodSinceMs =
        0;
    }


    return;
  }
}


// ================================================================
// SERIAL INPUT
// ================================================================

void processSerialCommands()
{
  while (Serial.available() > 0)
  {
    char c =
      (char)Serial.read();


    // ------------------------------------------------------------
    // Newline
    // ------------------------------------------------------------

    if (c == '\r' ||
        c == '\n')
    {
      if (commandIndex > 0)
      {
        commandBuffer[
          commandIndex] =
          '\0';


        processCommand(
          commandBuffer
```

```
        );


        commandIndex =
          0;
      }


      continue;
    }


    // ------------------------------------------------------------
    // Store
    // ------------------------------------------------------------

    if (commandIndex <
        COMMAND_BUFFER_SIZE - 1)
    {
      commandBuffer[
        commandIndex++] =
        c;
    }
    else
    {
      commandIndex = 0;

      Serial.println(
        "ERROR: COMMAND TOO LONG"
      );
    }
  }
}


// ================================================================
// COMMAND PROCESSOR
// ================================================================

void processCommand(
  char *cmd)
{
  // Remove leading spaces
  while (*cmd == ' ' ||
         *cmd == '\t')
  {
    cmd++;
  }


  // Remove trailing spaces
  int len =
    strlen(cmd);


  while (len > 0 &&
         (cmd[len - 1] == ' ' ||
          cmd[len - 1] == '\t'))
  {
    cmd[len - 1] =
      '\0';

    len--;
  }


  if (len == 0)
    return;


  Serial.print(
    "CMD RECEIVED: ["
  );

  Serial.print(
```

```
    cmd
  );

  Serial.println(
    "]"
  );


  // ================================================================
  // STATUS
  // ================================================================

  if (strcmp(
        cmd,
        "STATUS") == 0)
  {
    printStatus();
    return;
  }


  // ================================================================
  // MOTORTEST
  // ================================================================

  if (strcmp(
        cmd,
        "MOTORTEST") == 0)
  {
    startMotorTest();
    return;
  }


  // ================================================================
  // LEFT
  // ================================================================

  if (strcmp(
        cmd,
        "LEFT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      +300
    );


    return;
  }


  // ================================================================
  // RIGHT
  // ================================================================

  if (strcmp(
        cmd,
        "RIGHT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;

```

```
    commandSteering(
      -300
    );


    return;
  }


  // ================================================================
  // CENTER
  // ================================================================

  if (strcmp(
        cmd,
        "CENTER") == 0)
  {
    motorTestActive =
      false;


    if (servosReady())
    {
      centerServos();


      Serial.println(
        "ST-3009 -> CENTER"
      );
    }
    else
    {
      Serial.println(
        "ST-3009 BUS NOT READY"
      );
    }


    state =
      STATE_WAIT_READY;


    return;
  }


  // ================================================================
  // TARGET
  // ================================================================

  if (strncmp(cmd, "TARGET ", 7) == 0)
  {
    double newLat = 0.0;
    double newLon = 0.0;

    int parsed = sscanf(cmd + 7, "%lf %lf", &newLat, &newLon);

    if (parsed == 2 &&
        newLat >= -90.0 && newLat <= 90.0 &&
        newLon >= -180.0 && newLon <= 180.0)
    {
      TARGET_LAT = newLat;
      TARGET_LON = newLon;

      Serial.println("TARGET UPDATED");
      Serial.print("LAT = ");
      Serial.println(TARGET_LAT, 7);
      Serial.print("LON = ");
      Serial.println(TARGET_LON, 7);
    }
    else
    {
      Serial.println("Usage: TARGET LAT LON");
    }

```

```
    return;
  }


  // ============================================================
  // AUTONAV
  // ============================================================

  if (strcmp(
        cmd,
        "AUTONAV") == 0)
  {
    motorTestActive =
      false;


    if (gpsQualityOK() &&
        servosReady())
    {
      state =
        STATE_AUTONAV;


      Serial.println(
        "AUTONAV ENABLED"
      );
    }
    else
    {
      Serial.println(
        "AUTONAV BLOCKED"
      );


      Serial.print(
        "GPS = "
      );

      Serial.println(
        gpsQualityOK()
          ? "GOOD"
          : "BAD"
      );


      Serial.print(
        "SERVO = "
      );

      Serial.println(
        servosReady()
          ? "GOOD"
          : "BAD"
      );
    }


    return;
  }


  // ============================================================
  // DEPLOY
  // ============================================================

  if (strcmp(
        cmd,
        "DEPLOY") == 0)
  {
    motorTestActive =
      false;


    startDeployment();

```

```
    return;
  }


  // ============================================================
  // PREP
  // ============================================================

  if (strcmp(
        cmd,
        "PREP") == 0)
  {
    motorTestActive =
      false;


    prepareDeploymentServo();

    return;
  }


  // ============================================================
  // RESET
  // ============================================================

  if (strcmp(
        cmd,
        "RESET") == 0)
  {
    motorTestActive =
      false;


    resetSystem();

    return;
  }


  // ============================================================
  // HELP
  // ============================================================

  if (strcmp(
        cmd,
        "HELP") == 0)
  {
    Serial.println();

    Serial.println(
      "STATUS"
    );

    Serial.println(
      "MOTORTEST"
    );

    Serial.println(
      "LEFT"
    );

    Serial.println(
      "RIGHT"
    );

    Serial.println(
      "CENTER"
    );

    Serial.println(
      "AUTONAV"
    );

    Serial.println(
```

```
      "DEPLOY"
    );

    Serial.println(
      "PREP"
    );

    Serial.println(
      "RESET"
    );

    Serial.println(
      "HELP"
    );

    Serial.println(
      "TARGET <lat> <lon>"
    );


    Serial.println();

    Serial.println(
      "TARGET CAN BE UPDATED FROM MATLAB"
    );


    return;
  }


  // Unknown
  Serial.print(
    "UNKNOWN COMMAND: "
  );

  Serial.println(
    cmd
  );
}


// ================================================================
// RESET
// ================================================================

void resetSystem()
{
  // Stop deployment servo
  if (deploymentServoActive)
  {
    deployServo.detach();

    deploymentServoActive =
      false;
  }


  // Stop motor test
  motorTestActive =
    false;


  // Center if available
  if (servosReady())
  {
    centerServos();
  }


  // Reset flags
  deploymentDone =
    false;

```

```
  steeringDemand =
    0;


  failsafeGoodSinceMs =
    0;


  // Return to wait state
  state =
    STATE_WAIT_READY;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "SYSTEM RESET"
  );

  Serial.print(
    "TARGET = "
  );

  Serial.print(
    TARGET_LAT,
    7
  );

  Serial.print(
    ", "
  );

  Serial.println(
    TARGET_LON,
    7
  );

  Serial.println(
    "========================================"
  );
}


// ================================================================
// STATUS
// ================================================================

void printStatus()
{
  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "                    STATUS"
  );

  Serial.println(
    "================================================"
  );


  // System
  Serial.print(
    "State             : "
```

```
  );

  Serial.println(
    stateName(state)
  );


  // Target
  Serial.print(
    "Target latitude   : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "Target longitude  : "
  );

  Serial.println(
    TARGET_LON,
    7
  );


  // GPS
  Serial.print(
    "GPS fix           : "
  );

  Serial.println(
    gpsFix
      ? "YES"
      : "NO"
  );


  Serial.print(
    "GPS quality       : "
  );

  Serial.println(
    gpsQualityOK()
      ? "GOOD"
      : "BAD"
  );


  Serial.print(
    "Satellites        : "
  );

  Serial.println(
    gpsSats
  );


  Serial.print(
    "HDOP              : "
  );

  Serial.println(
    gpsHDOP,
    2
  );


  Serial.print(
    "Latitude          : "
  );

  Serial.println(
```

```
    currentLatitude,
    7
  );


  Serial.print(
    "Longitude         : "
  );

  Serial.println(
    currentLongitude,
    7
  );


  Serial.print(
    "Speed             : "
  );

  Serial.print(
    currentSpeed,
    2
  );

  Serial.println(
    " m/s"
  );


  // Navigation
  Serial.print(
    "Distance to target: "
  );

  Serial.print(
    distanceToTarget,
    2
  );

  Serial.println(
    " m"
  );


  Serial.print(
    "Target bearing    : "
  );

  Serial.print(
    targetBearing,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Current heading   : "
  );

  Serial.print(
    currentDirection,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Heading source    : "
  );
```

```
  Serial.println(
    headingSource
  );


  Serial.print(
    "Course error      : "
  );

  Serial.print(
    courseError,
    2
  );

  Serial.println(
    " deg"
  );


  // Compass
  Serial.print(
    "Compass status    : "
  );

  Serial.println(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Raw magnetometer  : "
  );

  Serial.print(
    rawMagX
  );

  Serial.print(
    ", "
  );

  Serial.print(
    rawMagY
  );

  Serial.print(
    ", "
  );

  Serial.println(
    rawMagZ
  );


  Serial.print(
    "Compass heading   : "
  );

  Serial.print(
    compassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Navigation heading: "
  );

```

```
  Serial.print(
    navCompassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  // Servos
  Serial.print(
    "Servo 1           : "
  );

  Serial.println(
    servo1OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 2           : "
  );

  Serial.println(
    servo2OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 1 position  : "
  );

  Serial.println(
    servo1Position
  );


  Serial.print(
    "Servo 2 position  : "
  );

  Serial.println(
    servo2Position
  );


  Serial.print(
    "Steering demand   : "
  );

  Serial.println(
    steeringDemand
  );


  // IMU
  Serial.print(
    "IMU               : "
  );

  Serial.println(
    imuOK
      ? "OK"
      : "FAIL"
  );


  // BMP
  Serial.print(
    "BMP581            : "
```

```
  );


  if (bmpOK)
  {
    Serial.print(
      "OK at 0x"
    );


    if (bmpAddress < 16)
      Serial.print("0");


    Serial.println(
      bmpAddress,
      HEX
    );
  }
  else
  {
    Serial.println(
      "NOT FOUND"
    );
  }


  // Deployment
  Serial.print(
    "Deployment done   : "
  );

  Serial.println(
    deploymentDone
      ? "YES"
      : "NO"
  );


  Serial.println(
    "================================================"
  );
}


// ================================================================
// SERVO FEEDBACK
// ================================================================

void updateServoFeedback()
{
  if (millis() - lastServoFeedbackMs < SERVO_FEEDBACK_INTERVAL_MS) return;

  lastServoFeedbackMs = millis();

  if (!servosReady())
  {
    servo1Feedback = -1;
    servo2Feedback = -1;
    return;
  }

  int p1 = st.ReadPos(SERVO1_ID);
  int p2 = st.ReadPos(SERVO2_ID);

  if (p1 >= 0) servo1Feedback = p1;
  if (p2 >= 0) servo2Feedback = p2;
}

// ================================================================
// MATLAB TELEMETRY
// ================================================================

void printMATLABTelemetry()
{
```

```
  double gpsCourse = -1.0;
  if (gps.course.isValid()) gpsCourse = gps.course.deg();

  Serial.print("NAVMAT,");
  Serial.print(millis()); Serial.print(",");
  Serial.print(currentLatitude, 7); Serial.print(",");
  Serial.print(currentLongitude, 7); Serial.print(",");
  Serial.print(currentAltitude, 2); Serial.print(",");
  Serial.print(currentSpeed, 2); Serial.print(",");
  Serial.print(gpsCourse, 2); Serial.print(",");
  Serial.print(currentDirection, 2); Serial.print(",");
  Serial.print(distanceToTarget, 2); Serial.print(",");
  Serial.print(targetBearing, 2); Serial.print(",");
  Serial.print(courseError, 2); Serial.print(",");
  Serial.print(steeringDemand); Serial.print(",");
  Serial.print(servo1Position); Serial.print(",");
  Serial.print(servo2Position); Serial.print(",");
  Serial.print(servo1Feedback); Serial.print(",");
  Serial.print(servo2Feedback); Serial.print(",");
  Serial.print(gpsSats); Serial.print(",");
  Serial.print(gpsHDOP, 2); Serial.print(",");
  Serial.print(stateName(state)); Serial.print(",");
  Serial.print(headingSource); Serial.print(",");
  Serial.println(navCompassHeading, 2);
}


// ================================================================
// LIVE DATA
// ================================================================

void printLiveData()
{
  Serial.print(
    "[LIVE] State="
  );

  Serial.print(
    stateName(state)
  );


  // GPS
  Serial.print(
    " GPS="
  );

  Serial.print(
    gpsQualityOK()
      ? "GOOD"
      : "BAD"
  );


  Serial.print(
    " SAT="
  );

  Serial.print(
    gpsSats
  );


  Serial.print(
    " HDOP="
  );

  Serial.print(
    gpsHDOP,
    1
  );


  if (gpsFix)
  {
```

```
    Serial.print(
      " LAT="
    );

    Serial.print(
      currentLatitude,
      6
    );


    Serial.print(
      " LON="
    );

    Serial.print(
      currentLongitude,
      6
    );
  }


  // Navigation
  if (gpsQualityOK())
  {
    Serial.print(
      " DIST="
    );

    Serial.print(
      distanceToTarget,
      1
    );


    Serial.print(
      "m BRG="
    );

    Serial.print(
      targetBearing,
      1
    );
  }


  // Heading
  if (headingValid)
  {
    Serial.print(
      " HDG="
    );

    Serial.print(
      currentDirection,
      1
    );


    Serial.print(
      " SRC="
    );

    Serial.print(
      headingSource
    );


    Serial.print(
      " ERR="
    );

    Serial.print(
      courseError,
      1
    );
```

```
  }
  else
  {
    Serial.print(
      " HDG=NONE"
    );
  }


  // Steering
  Serial.print(
    " DEM="
  );

  Serial.print(
    steeringDemand
  );


  Serial.print(
    " S1="
  );

  Serial.print(
    servo1Position
  );


  Serial.print(
    " S2="
  );

  Serial.print(
    servo2Position
  );

  Serial.print(
    " FB1="
  );

  Serial.print(
    servo1Feedback
  );

  Serial.print(
    " FB2="
  );

  Serial.print(
    servo2Feedback
  );


  // Compass
  Serial.print(
    " CMP="
  );

  Serial.print(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    " CHP="
  );

  Serial.print(
    compassHeading,
    1
  );

```

```
  Serial.print(
    " NAV="
  );

  Serial.print(
    navCompassHeading,
    1
  );


  // IMU
  Serial.print(
    " IMU="
  );

  Serial.print(
    imuOK
      ? "OK"
      : "FAIL"
  );


  // Servos
  Serial.print(
    " SERVO="
  );

  Serial.println(
    servosReady()
      ? "OK"
      : "FAIL"
  );
}


// ================================================================
// STATE NAME
// ================================================================

const char *stateName(
  SystemState s)
{
  switch (s)
  {
    case STATE_WAIT_READY:
      return "WAIT_READY";


    case STATE_DEPLOYING:
      return "DEPLOYING";


    case STATE_STABILIZING:
      return "STABILIZING";


    case STATE_AUTONAV:
      return "AUTONAV";


    case STATE_CAPTURED:
      return "CAPTURED";


    case STATE_FAILSAFE:
      return "FAILSAFE";


    case STATE_MANUAL_TEST:
      return "MANUAL_TEST";


    default:
      return "UNKNOWN";
  }
```

```
}
```

A.8 Archived source 8: Teensy_Parafoil_FINAL_SERVO_FIXED.ino

Source file: `Teensy_Parafoil_FINAL_SERVO_FIXED.ino`

```
/*
=====================================================================
 TEENSY 4.1 AUTONOMOUS PARAFOIL
 FINAL LAB / MATLAB LIVE-TEST TRANSMITTER

 PURPOSE
   - Live GPS navigation toward a target coordinate
   - ST-3009 single-servo steering mode for bench/mechanical testing
   - Actual ST-3009 position feedback
   - MATLAB machine-readable telemetry
   - Manual steering tests
   - GPS/compass heading selection
   - GPS/servo failsafe

 IMPORTANT
   - No PD, wind estimation, glide-speed estimation, or predicted
     landing-point controller is added yet.
   - Those parameters should be measured before tuning a PD controller.
   - TARGET_CAPTURE_ENABLED is FALSE by default. Passing within 35 m
     while still airborne does not guarantee a 35 m landing.

 STEERING MODE
   Servo ID 1 = LEFT
   Servo ID 2 = RIGHT

   SINGLE_SERVO_STEERING = true:
       LEFT  -> only Servo 1 moves
       RIGHT -> only Servo 2 moves
       CENTER -> both servos = 2048

   Bench-verify LEFT_STEER_DIRECTION and RIGHT_STEER_DIRECTION.

 HARDWARE
   GPS: TX -> 15, RX -> 14, Serial3 @ 38400
   I2C: SDA 18, SCL 19
   QMC5883: 0x0D
   LSM6DSO32: 0x6A
   BMP581: 0x46 or 0x47, diagnostic only
   ST-3009: IDs 1 and 2, Serial1, 1 Mbps, half duplex
   SG90: signal pin 8
   USB Serial: 115200

 COMMANDS
   STATUS
   MOTORTEST
   LEFT
   RIGHT
   CENTER
   AUTONAV
   TARGET <lat> <lon>
   DEPLOY
   PREP
   RESET
   HELP

 MATLAB TELEMETRY
   NAVMAT,<millis>,<lat>,<lon>,<alt>,<speed>,<course>,
   <heading>,<distance>,<bearing>,<error>,<demand>,
   <cmdS1>,<cmdS2>,<fbS1>,<fbS2>,<sats>,<hdop>,
   <state>,<headingSource>,<compass>
=====================================================================
*/


// ===================================================================
// LIBRARIES
// ===================================================================
```

```

#include <Arduino.h>
#include <Wire.h>
#include <Servo.h>
#include <TinyGPS++.h>
#include <SCServo.h>
#include <Adafruit_LSM6DSOX.h>

#include <math.h>
#include <string.h>


// ================================================================
// TARGET
// ================================================================

double TARGET_LAT = 32.6321769;
double TARGET_LON = 75.0143889;


// ================================================================
// AUTO DEPLOY
// ================================================================
//
// true  = deploy automatically after GPS + both ST-3009 ready
// false = wait for DEPLOY command
//
// Lab default is FALSE. Use DEPLOY manually after bench checks.
// ================================================================

const bool AUTO_DEPLOY_AFTER_READY = false;


// ================================================================
// GPS
// ================================================================

#define GPS_SERIAL Serial3

const uint32_t GPS_BAUD = 38400;

const uint8_t GPS_RX_PIN = 15;
const uint8_t GPS_TX_PIN = 14;

// GPS data considered stale after this time
const uint32_t GPS_DATA_TIMEOUT_MS = 2500;

// Minimum satellites
const uint8_t GPS_MIN_SATS = 5;

// Maximum acceptable HDOP
const double GPS_MAX_HDOP = 3.5;

// GPS course is used only above this speed
const double GPS_COURSE_MIN_SPEED_MPS = 1.5;


// ================================================================
// I2C
// ================================================================

const uint8_t I2C_SDA_PIN = 18;
const uint8_t I2C_SCL_PIN = 19;


// ================================================================
// QMC5883
// ================================================================

const uint8_t QMC5883_ADDR = 0x0D;


// ----------------------------------------------------------------
// Compass navigation orientation
// ----------------------------------------------------------------
```

```
//
// Your tested compass direction was mirrored for navigation.
//

const bool COMPASS_MIRROR_FOR_NAV = true;

const float COMPASS_HEADING_OFFSET_DEG = 0.0f;


// ----------------------------------------------------------------
// Calibration values from your working compass test
// ----------------------------------------------------------------

const float OFFSET_X = 180.0000f;
const float OFFSET_Y = 336.0000f;
const float OFFSET_Z = 615.0000f;

const float SCALE_X = 0.956484f;
const float SCALE_Y = 0.904406f;
const float SCALE_Z = 1.178127f;


// ================================================================
// LSM6DSO32
// ================================================================

const uint8_t LSM6DSO32_ADDR = 0x6A;

Adafruit_LSM6DSOX lsm6dso;

bool imuOK = false;


// ================================================================
// BMP581
// ================================================================

const uint8_t BMP581_ADDR_1 = 0x46;
const uint8_t BMP581_ADDR_2 = 0x47;

bool bmpOK = false;

uint8_t bmpAddress = 0;


// ================================================================
// GPS VARIABLES
// ================================================================

TinyGPSPlus gps;

bool gpsFix = false;

uint8_t gpsSats = 0;

double gpsHDOP = 99.9;

double currentSpeed = 0.0;
double currentAltitude = -1.0;

unsigned long lastGPSLocationMs = 0;


// ================================================================
// COMPASS VARIABLES
// ================================================================

bool compassOK = false;

int16_t rawMagX = 0;
int16_t rawMagY = 0;
int16_t rawMagZ = 0;

float magX = 0.0f;
float magY = 0.0f;
```

```
float magZ = 0.0f;

float compassHeading = 0.0f;
float navCompassHeading = 0.0f;

uint8_t compassFailCount = 0;


// ================================================================
// IMU VARIABLES
// ================================================================

float accelX = 0.0f;
float accelY = 0.0f;
float accelZ = 0.0f;

float gyroX = 0.0f;
float gyroY = 0.0f;
float gyroZ = 0.0f;

unsigned long lastSensorUpdateMs = 0;


// ================================================================
// ST-3009
// ================================================================
//
// IMPORTANT:
// SCServo 1.0.2 uses:
//
//     SMS_STS
//

SMS_STS st;


// ----------------------------------------------------------------
// Servo IDs
// ----------------------------------------------------------------

const uint8_t SERVO1_ID = 1;
const uint8_t SERVO2_ID = 2;


// ----------------------------------------------------------------
// Servo positions
// ----------------------------------------------------------------

const int SERVO_NEUTRAL = 2048;

const int SERVO_MIN_POS = 200;
const int SERVO_MAX_POS = 3895;


// ----------------------------------------------------------------
// Steering
// ----------------------------------------------------------------

const int MAX_STEERING_COUNTS = 450;

const int STEERING_SERVO_SPEED = 2400;
const int STEERING_SERVO_ACCEL = 50;

const int MAX_SLEW_COUNTS = 60;

// ------------------------------------------------------------
// Steering mode
// ------------------------------------------------------------
const bool SINGLE_SERVO_STEERING = true;
const bool SERVO1_IS_LEFT = true;
const int LEFT_STEER_DIRECTION = -1;
const int RIGHT_STEER_DIRECTION = -1;


// ----------------------------------------------------------------
```

```
// Servo health
// ----------------------------------------------------------------

const unsigned long SERVO_HEALTH_INTERVAL_MS = 2000;

bool servo1OK = false;
bool servo2OK = false;
bool servoBusReady = false;

unsigned long lastServoHealthMs = 0;


// ----------------------------------------------------------------
// Current positions
// ----------------------------------------------------------------

int servo1Position = SERVO_NEUTRAL;
int servo2Position = SERVO_NEUTRAL;


// ----------------------------------------------------------------
// Current navigation demand
// ----------------------------------------------------------------

int steeringDemand = 0;

int servo1Feedback = -1;
int servo2Feedback = -1;
unsigned long lastServoFeedbackMs = 0;
const unsigned long SERVO_FEEDBACK_INTERVAL_MS = 500;

unsigned long lastMATLABTelemetryMs = 0;
const unsigned long MATLAB_TELEMETRY_INTERVAL_MS = 250;


// ================================================================
// SG90 DEPLOYMENT
// ================================================================

Servo deployServo;

const uint8_t DEPLOY_SERVO_PIN = 8;

const int DEPLOY_ANGLE = 180;

const int PREP_ANGLE = 140;

const unsigned long DEPLOY_MOVE_TIME_MS = 1500;

const unsigned long DEPLOY_STABILIZE_TIME_MS = 5000;

bool deploymentServoActive = false;

bool deploymentDone = false;

unsigned long deploymentStartMs = 0;

unsigned long stabilizationStartMs = 0;


// ================================================================
// NAVIGATION
// ================================================================

const double TARGET_CAPTURE_RADIUS_M = 35.0;

// Disabled by default: do not stop steering just because the
// vehicle is within 35 m while still airborne.
const bool TARGET_CAPTURE_ENABLED = false;

const double STEERING_DEADBAND_DEG = 5.0;

const double FULL_STEERING_ERROR_DEG = 90.0;


```

```
// ================================================================
// STATE MACHINE
// ================================================================

enum SystemState
{
  STATE_WAIT_READY,
  STATE_DEPLOYING,
  STATE_STABILIZING,
  STATE_AUTONAV,
  STATE_CAPTURED,
  STATE_FAILSAFE,
  STATE_MANUAL_TEST
};

SystemState state = STATE_WAIT_READY;

unsigned long stateEnteredMs = 0;


// ================================================================
// NAVIGATION VARIABLES
// ================================================================

double currentLatitude = 0.0;

double currentLongitude = 0.0;

double distanceToTarget = 0.0;

double targetBearing = 0.0;

double currentDirection = 0.0;

double courseError = 0.0;

bool headingValid = false;

const char *headingSource = "NONE";


// ================================================================
// FAILSAFE
// ================================================================

unsigned long failsafeGoodSinceMs = 0;

const unsigned long FAILSAFE_RECOVERY_MS = 3000;


// ================================================================
// SERIAL COMMAND BUFFER
// ================================================================

#define COMMAND_BUFFER_SIZE 120

char commandBuffer[COMMAND_BUFFER_SIZE];

uint16_t commandIndex = 0;


// ================================================================
// TIMERS
// ================================================================

unsigned long lastNavigationUpdateMs = 0;

const unsigned long NAVIGATION_UPDATE_INTERVAL_MS = 100;

unsigned long lastLivePrintMs = 0;

const unsigned long LIVE_PRINT_INTERVAL_MS = 1000;

unsigned long lastBMPCheckMs = 0;

```

```
const unsigned long BMP_CHECK_INTERVAL_MS = 3000;


// ================================================================
// MOTOR TEST
// ================================================================

bool motorTestActive = false;

uint8_t motorTestStep = 0;

unsigned long motorTestTimer = 0;


// ================================================================
// FORWARD DECLARATIONS
// ================================================================

void processSerialCommands();
void processCommand(char *cmd);

void updateGPS();
void updateCompass();
void updateIMU();
void updateServoHealth();
void updateServoFeedback();
void printMATLABTelemetry();
void printServoDiagnostics();
bool pingServo(uint8_t id);
int readServoPosition(uint8_t id);

void updateNavigation();
void updateSteering();
void updateStateMachine();

void writeSteeringPositions(int pos1, int pos2);
void centerServos();
void commandSteering(int demand);

void startDeployment();
void prepareDeploymentServo();
void resetSystem();

void startMotorTest();
void updateMotorTest();

void printStatus();
void printLiveData();

bool gpsQualityOK();
bool readQMC5883();
bool initQMC5883();

bool checkI2CDevice(uint8_t address);
bool initIMU();

bool servosReady();

float wrap360(float angle);
double wrap180(double angle);
float navigationCompassHeading();

const char *stateName(SystemState s);

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep
);


// ================================================================
// SETUP
// ================================================================

```

```
void setup()
{
  // ----------------------------------------------------------------
  // USB SERIAL
  // ----------------------------------------------------------------

  Serial.begin(115200);

  delay(1500);


  Serial.println();
  Serial.println(
    "================================================"
  );

  Serial.println(
    " TEENSY 4.1 AUTONOMOUS PARAFOIL"
  );

  Serial.println(
    " GUIDED LANDING - FINAL LAB TEST"
  );

  Serial.println(
    "================================================"
  );


  // ----------------------------------------------------------------
  // TARGET
  // ----------------------------------------------------------------

  Serial.println();

  Serial.print(
    "TARGET LATITUDE  : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "TARGET LONGITUDE : "
  );

  Serial.println(
    TARGET_LON,
    7
  );

  Serial.print("STEERING MODE   : ");
  Serial.println(SINGLE_SERVO_STEERING ? "SINGLE SERVO" : "DIFFERENTIAL");
  Serial.print("SERVO 1 ROLE    : ");
  Serial.println(SERVO1_IS_LEFT ? "LEFT" : "RIGHT");


  // ----------------------------------------------------------------
  // I2C
  // ----------------------------------------------------------------

  Wire.begin();

  Wire.setClock(400000);


  Serial.println();

  Serial.println(
    "I2C initialized"
  );

```

```
  Serial.println(
    "SDA = Pin 18"
  );

  Serial.println(
    "SCL = Pin 19"
  );


  // ---------------------------------------------------------------
  // QMC5883
  // ---------------------------------------------------------------

  if (initQMC5883())
  {
    compassOK = true;

    Serial.println(
      "QMC5883 : OK"
    );
  }
  else
  {
    compassOK = false;

    Serial.println(
      "QMC5883 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // LSM6DSO32
  // ---------------------------------------------------------------

  if (initIMU())
  {
    imuOK = true;

    Serial.println(
      "LSM6DSO32 : OK"
    );
  }
  else
  {
    imuOK = false;

    Serial.println(
      "LSM6DSO32 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // BMP581
  // ---------------------------------------------------------------

  if (checkI2CDevice(
        BMP581_ADDR_1))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_1;
  }
  else if (checkI2CDevice(
             BMP581_ADDR_2))
  {
    bmpOK = true;

    bmpAddress =
      BMP581_ADDR_2;
  }
  else
  {
```

```
    bmpOK = false;

    bmpAddress = 0;
  }


  if (bmpOK)
  {
    Serial.print(
      "BMP581 : FOUND at 0x"
    );

    if (bmpAddress < 16)
      Serial.print("0");

    Serial.println(
      bmpAddress,
      HEX
    );
  }
  else
  {
    Serial.println(
      "BMP581 : NOT FOUND"
    );
  }


  // ---------------------------------------------------------------
  // GPS
  // ---------------------------------------------------------------
  //
  // EXACTLY MATCHES YOUR WORKING GPS TEST
  //

  GPS_SERIAL.begin(
    GPS_BAUD
  );


  Serial.println();

  Serial.println(
    "GPS UART  : Serial3"
  );

  Serial.print(
    "GPS RX    : Pin "
  );

  Serial.println(
    GPS_RX_PIN
  );

  Serial.print(
    "GPS TX    : Pin "
  );

  Serial.println(
    GPS_TX_PIN
  );

  Serial.print(
    "GPS BAUD  : "
  );

  Serial.println(
    GPS_BAUD
  );


  // ---------------------------------------------------------------
  // ST-3009
  // ---------------------------------------------------------------

```

```
  Serial.println();

  Serial.println(
    "Initializing ST-3009 bus..."
  );


  Serial1.begin(
    1000000,
    SERIAL_8N1_HALF_DUPLEX
  );


  st.pSerial =
    &Serial1;


  delay(500);


  // Force immediate first health test
  lastServoHealthMs =
    millis() -
    SERVO_HEALTH_INTERVAL_MS;


  updateServoHealth();


  Serial.print("SERVO PING 1 : ");
  Serial.println(servo1OK ? "OK" : "FAIL");

  Serial.print("SERVO PING 2 : ");
  Serial.println(servo2OK ? "OK" : "FAIL");

  if (servosReady())
  {
    centerServos();
    Serial.println("ST-3009 BUS : READY");
  }
  else
  {
    Serial.println("ST-3009 BUS : NOT READY");
    Serial.println("Check power, common GND, DATA bus, and IDs 1/2.");
  }


  // ------------------------------------------------------------------
  // SG90
  // ------------------------------------------------------------------

  deployServo.detach();

  deploymentServoActive =
    false;


  // ------------------------------------------------------------------
  // State
  // ------------------------------------------------------------------

  state =
    STATE_WAIT_READY;

  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
```

```
    "SYSTEM READY"
  );

  Serial.println(
    "Target can be updated with TARGET lat lon."
  );

  Serial.println(
    "================================================"
  );


  Serial.println();

  Serial.println(
    "COMMANDS:"
  );

  Serial.println(
    "STATUS"
  );

  Serial.println(
    "MOTORTEST"
  );

  Serial.println(
    "LEFT"
  );

  Serial.println(
    "RIGHT"
  );

  Serial.println(
    "CENTER"
  );

  Serial.println(
    "AUTONAV"
  );

  Serial.println(
    "DEPLOY"
  );

  Serial.println(
    "PREP"
  );

  Serial.println(
    "RESET"
  );

  Serial.println(
    "HELP"
  );


  Serial.println();

  Serial.println(
    "Waiting for GPS + ST-3009..."
  );

  Serial.println();
  Serial.println("SERVO BENCH TEST:");
  Serial.println("  PING1  PING2  POS1  POS2");
  Serial.println("  LEFT   RIGHT  CENTER");
  Serial.println("  SERVOTEST");
}


// ================================================================
// LOOP
```

```
// ================================================================

void loop()
{
  // GPS continuously
  updateGPS();


  // Serial commands
  processSerialCommands();


  // Sensors
  if (millis() -
      lastSensorUpdateMs >=
      100)
  {
    lastSensorUpdateMs =
      millis();

    updateCompass();
    updateIMU();
  }


  // Servo health
  updateServoHealth();

  // Actual servo feedback for MATLAB
  updateServoFeedback();


  // Motor test
  if (motorTestActive)
  {
    updateMotorTest();
  }


  // State
  updateStateMachine();


  // Navigation
  if (millis() -
      lastNavigationUpdateMs >=
      NAVIGATION_UPDATE_INTERVAL_MS)
  {
    lastNavigationUpdateMs =
      millis();

    updateNavigation();
    updateSteering();
  }


  // BMP check
  if (millis() -
      lastBMPCheckMs >=
      BMP_CHECK_INTERVAL_MS)
  {
    lastBMPCheckMs =
      millis();


    if (!bmpOK)
    {
      if (checkI2CDevice(
            BMP581_ADDR_1))
      {
        bmpOK = true;

        bmpAddress =
          BMP581_ADDR_1;
      }
```

```
      else if (checkI2CDevice(
                 BMP581_ADDR_2))
      {
        bmpOK = true;

        bmpAddress =
          BMP581_ADDR_2;
      }
    }
  }


  // Live data
  if (millis() -
      lastLivePrintMs >=
      LIVE_PRINT_INTERVAL_MS)
  {
    lastLivePrintMs =
      millis();

    printLiveData();
  }

  if (millis() -
      lastMATLABTelemetryMs >=
      MATLAB_TELEMETRY_INTERVAL_MS)
  {
    lastMATLABTelemetryMs = millis();
    printMATLABTelemetry();
  }
}


// ================================================================
// GPS UPDATE
// ================================================================

void updateGPS()
{
  while (GPS_SERIAL.available() > 0)
  {
    char c =
      (char)GPS_SERIAL.read();

    gps.encode(c);
  }


  // Fresh GPS location
  if (gps.location.isUpdated())
  {
    lastGPSLocationMs =
      millis();
  }


  // Position
  if (gps.location.isValid())
  {
    gpsFix = true;

    currentLatitude =
      gps.location.lat();

    currentLongitude =
      gps.location.lng();
  }
  else
  {
    gpsFix = false;
  }


  // Satellites
  if (gps.satellites.isValid())
```

```
  {
    gpsSats =
      gps.satellites.value();
  }
  else
  {
    gpsSats = 0;
  }


  // HDOP
  if (gps.hdop.isValid())
  {
    gpsHDOP =
      gps.hdop.hdop();
  }
  else
  {
    gpsHDOP = 99.9;
  }


  // Speed
  if (gps.speed.isValid())
  {
    currentSpeed =
      gps.speed.mps();
  }
  else
  {
    currentSpeed = 0.0;
  }

  if (gps.altitude.isValid())
    currentAltitude = gps.altitude.meters();
  else
    currentAltitude = -1.0;
}


// ================================================================
// GPS QUALITY
// ================================================================

bool gpsQualityOK()
{
  if (!gpsFix)
    return false;


  if (lastGPSLocationMs == 0)
    return false;


  if (millis() -
      lastGPSLocationMs >
      GPS_DATA_TIMEOUT_MS)
  {
    return false;
  }


  if (gpsSats <
      GPS_MIN_SATS)
  {
    return false;
  }


  if (gps.hdop.isValid() &&
      gpsHDOP >
      GPS_MAX_HDOP)
  {
    return false;
  }
```

```
1118
1119
1120    return true;
1121  }
1122
1123
1124  // ================================================================
1125  // QMC5883 INIT
1126  // ================================================================
1127
1128  bool initQMC5883()
1129  {
1130    // Check device
1131    Wire.beginTransmission(
1132      QMC5883_ADDR
1133    );
1134
1135
1136    if (Wire.endTransmission() != 0)
1137    {
1138      return false;
1139    }
1140
1141
1142    // SET/RESET period
1143    Wire.beginTransmission(
1144      QMC5883_ADDR
1145    );
1146
1147    Wire.write(0x0B);
1148    Wire.write(0x01);
1149
1150    if (Wire.endTransmission() != 0)
1151    {
1152      return false;
1153    }
1154
1155
1156    delay(10);
1157
1158
1159    // Control register
1160    // Same setting used by your
1161    // working compass test.
1162    Wire.beginTransmission(
1163      QMC5883_ADDR
1164    );
1165
1166    Wire.write(0x09);
1167    Wire.write(0x1D);
1168
1169    if (Wire.endTransmission() != 0)
1170    {
1171      return false;
1172    }
1173
1174
1175    delay(100);
1176
1177
1178    return true;
1179  }
1180
1181
1182  // ================================================================
1183  // QMC5883 READ
1184  // ================================================================
1185
1186  bool readQMC5883()
1187  {
1188    Wire.beginTransmission(
1189      QMC5883_ADDR
1190    );
1191
1192    Wire.write(0x00);
```

```

  if (Wire.endTransmission(false) != 0)
  {
    return false;
  }


  uint8_t count =
    Wire.requestFrom(
      QMC5883_ADDR,
      (uint8_t)6
    );


  if (count != 6)
  {
    while (Wire.available())
    {
      Wire.read();
    }

    return false;
  }


  uint8_t xl =
    Wire.read();

  uint8_t xh =
    Wire.read();


  uint8_t yl =
    Wire.read();

  uint8_t yh =
    Wire.read();


  uint8_t zl =
    Wire.read();

  uint8_t zh =
    Wire.read();


  rawMagX =
    (int16_t)(
      (xh << 8) |
      xl
    );


  rawMagY =
    (int16_t)(
      (yh << 8) |
      yl
    );


  rawMagZ =
    (int16_t)(
      (zh << 8) |
      zl
    );


  return true;
}


// ================================================================
// COMPASS UPDATE
// ================================================================
```

```

void updateCompass()
{
  // ------------------------------------------------------------
  // Read sensor
  // ------------------------------------------------------------

  if (!readQMC5883())
  {
    compassFailCount++;


    // Do not immediately declare compass dead.
    // Reinitialize after 5 consecutive failures.

    if (compassFailCount >= 5)
    {
      if (initQMC5883())
      {
        compassOK = true;

        compassFailCount = 0;

        Serial.println(
          "[COMPASS] Reinitialized OK"
        );
      }
      else
      {
        compassOK = false;
      }
    }


    return;
  }


  // Successful reading
  compassFailCount = 0;
  compassOK = true;


  // ------------------------------------------------------------
  // Apply your proven calibration
  // ------------------------------------------------------------

  magX =
    ((float)rawMagX -
     OFFSET_X) *
    SCALE_X;


  magY =
    ((float)rawMagY -
     OFFSET_Y) *
    SCALE_Y;


  magZ =
    ((float)rawMagZ -
     OFFSET_Z) *
    SCALE_Z;


  // ------------------------------------------------------------
  // Calculate heading
  // ------------------------------------------------------------

  float angle =
    atan2(
      magY,
      magX
    ) *
    180.0f /
```

```
    PI;


  if (angle < 0.0f)
    angle += 360.0f;


  compassHeading =
    wrap360(
      angle +
      COMPASS_HEADING_OFFSET_DEG
    );


  // Navigation heading
  navCompassHeading =
    navigationCompassHeading();
}


// ================================================================
// NAVIGATION COMPASS
// ================================================================

float navigationCompassHeading()
{
  if (!COMPASS_MIRROR_FOR_NAV)
  {
    return compassHeading;
  }


  return wrap360(
    360.0f -
    compassHeading
  );
}


// ================================================================
// WRAP 0...360
// ================================================================

float wrap360(float angle)
{
  while (angle >= 360.0f)
    angle -= 360.0f;


  while (angle < 0.0f)
    angle += 360.0f;


  return angle;
}


// ================================================================
// WRAP -180...180
// ================================================================

double wrap180(double angle)
{
  while (angle > 180.0)
    angle -= 360.0;


  while (angle < -180.0)
    angle += 360.0;


  return angle;
}
```

```
// ================================================================
// IMU INIT
// ================================================================

bool initIMU()
{
  if (!lsm6dso.begin_I2C(
        LSM6DSO32_ADDR,
        &Wire))
  {
    return false;
  }


  lsm6dso.setAccelRange(
    LSM6DS_ACCEL_RANGE_16_G
  );


  lsm6dso.setGyroRange(
    LSM6DS_GYRO_RANGE_2000_DPS
  );


  lsm6dso.setAccelDataRate(
    LSM6DS_RATE_208_HZ
  );


  lsm6dso.setGyroDataRate(
    LSM6DS_RATE_208_HZ
  );


  return true;
}


// ================================================================
// IMU UPDATE
// ================================================================

void updateIMU()
{
  if (!imuOK)
    return;


  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;


  lsm6dso.getEvent(
    &accel,
    &gyro,
    &temp
  );


  accelX =
    accel.acceleration.x;


  accelY =
    accel.acceleration.y;


  accelZ =
    accel.acceleration.z;


  gyroX =
    gyro.gyro.x;

```

```
  gyroY =
    gyro.gyro.y;


  gyroZ =
    gyro.gyro.z;
}


// ================================================================
// I2C CHECK
// ================================================================

bool checkI2CDevice(
  uint8_t address)
{
  Wire.beginTransmission(
    address
  );


  return (
    Wire.endTransmission() == 0
  );
}


// ================================================================
// SERVO HEALTH
// ================================================================

bool pingServo(uint8_t id)
{
  int result = st.Ping(id);
  delay(15);
  return (result >= 0);
}

int readServoPosition(uint8_t id)
{
  int p = st.ReadPos(id);
  delay(15);
  return p;
}

void updateServoHealth()
{
  if (millis() -
      lastServoHealthMs <
      SERVO_HEALTH_INTERVAL_MS)
  {
    return;
  }

  lastServoHealthMs = millis();

  servo1OK = pingServo(SERVO1_ID);
  servo2OK = pingServo(SERVO2_ID);

  servoBusReady = servo1OK && servo2OK;
}


// ================================================================
// SERVO READY
// ================================================================

bool servosReady()
{
  return
    servoBusReady &&
    servo1OK &&
    servo2OK;
}
```

```
1568
1569
1570  // ================================================================
1571  // CENTER BOTH SERVOS
1572  // ================================================================
1573
1574  void centerServos()
1575  {
1576    if (!servosReady())
1577    {
1578      Serial.println("CENTER BLOCKED: ST-3009 BUS NOT READY");
1579      return;
1580    }
1581
1582    // Avoid repeatedly sending identical commands.
1583    if (servo1Position != SERVO_NEUTRAL)
1584    {
1585      st.WritePosEx(SERVO1_ID, SERVO_NEUTRAL,
1586                    STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
1587      delay(15);
1588    }
1589
1590    if (servo2Position != SERVO_NEUTRAL)
1591    {
1592      st.WritePosEx(SERVO2_ID, SERVO_NEUTRAL,
1593                    STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
1594      delay(15);
1595    }
1596
1597    servo1Position = SERVO_NEUTRAL;
1598    servo2Position = SERVO_NEUTRAL;
1599    steeringDemand = 0;
1600
1601    Serial.println("SERVO CENTERED: S1=2048 S2=2048");
1602  }
1603
1604  // ================================================================
1605  // STEERING DEMAND -> SERVO TARGETS
1606  // ================================================================
1607  // demand > 0 = LEFT
1608  // demand < 0 = RIGHT
1609  // SINGLE_SERVO_STEERING=true makes only the requested servo move.
1610
1611  void steeringTargetsFromDemand(int demand, int &target1, int &target2)
1612  {
1613    target1 = SERVO_NEUTRAL;
1614    target2 = SERVO_NEUTRAL;
1615
1616    demand = constrain(demand, -MAX_STEERING_COUNTS, MAX_STEERING_COUNTS);
1617
1618    if (!SINGLE_SERVO_STEERING)
1619    {
1620      target1 = SERVO_NEUTRAL - demand;
1621      target2 = SERVO_NEUTRAL + demand;
1622      return;
1623    }
1624
1625    if (demand > 0)
1626    {
1627      if (SERVO1_IS_LEFT)
1628        target1 = SERVO_NEUTRAL + LEFT_STEER_DIRECTION * demand;
1629      else
1630        target2 = SERVO_NEUTRAL + RIGHT_STEER_DIRECTION * demand;
1631    }
1632    else if (demand < 0)
1633    {
1634      int magnitude = -demand;
1635
1636      if (SERVO1_IS_LEFT)
1637        target2 = SERVO_NEUTRAL + RIGHT_STEER_DIRECTION * magnitude;
1638      else
1639        target1 = SERVO_NEUTRAL + LEFT_STEER_DIRECTION * magnitude;
1640    }
1641
1642    target1 = constrain(target1, SERVO_MIN_POS, SERVO_MAX_POS);
```

```
  target2 = constrain(target2, SERVO_MIN_POS, SERVO_MAX_POS);
}

void commandSteering(int demand)
{
  if (!servosReady())
  {
    Serial.println("ERROR: ST-3009 BUS NOT READY");
    Serial.println("Run PING1 and PING2. If either fails, fix the bus first.");
    return;
  }

  demand = constrain(demand, -MAX_STEERING_COUNTS, MAX_STEERING_COUNTS);

  int pos1, pos2;
  steeringTargetsFromDemand(demand, pos1, pos2);

  Serial.print("SERVO COMMAND: demand=");
  Serial.print(demand);
  Serial.print(" S1=");
  Serial.print(pos1);
  Serial.print(" S2=");
  Serial.println(pos2);

  // Send only when the target differs. A short gap makes the half-duplex
  // bus easier to observe and avoids back-to-back packet ambiguity.
  if (pos1 != servo1Position)
  {
    st.WritePosEx(SERVO1_ID, pos1,
                  STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
    delay(15);
    servo1Position = pos1;
  }

  if (pos2 != servo2Position)
  {
    st.WritePosEx(SERVO2_ID, pos2,
                  STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
    delay(15);
    servo2Position = pos2;
  }

  steeringDemand = demand;

  Serial.print("COMMAND ACCEPTED: S1=");
  Serial.print(servo1Position);
  Serial.print(" S2=");
  Serial.println(servo2Position);
}

// ================================================================
// MOTOR TEST START
// ================================================================

void startMotorTest()
{
  if (!servosReady())
  {
    Serial.println(
      "MOTORTEST BLOCKED: "
      "ST-3009 NOT READY"
    );

    return;
  }


  motorTestActive =
    true;


  state =
    STATE_MANUAL_TEST;


```

```
  motorTestStep = 0;

  motorTestTimer =
    millis();


  centerServos();


  Serial.println();

  Serial.println(
    "======================================="
  );

  Serial.println(
    "       ST-3009 MOTOR TEST"
  );

  Serial.println(
    "======================================="
  );

  Serial.println(
    "CENTER -> LEFT -> CENTER -> RIGHT -> CENTER"
  );

  Serial.println();
}


// ================================================================
// MOTOR TEST UPDATE
// ================================================================

void updateMotorTest()
{
  if (!servosReady())
  {
    motorTestActive =
      false;

    state =
      STATE_WAIT_READY;

    return;
  }


  if (millis() -
      motorTestTimer <
      1800)
  {
    return;
  }


  motorTestTimer =
    millis();


  switch (motorTestStep)
  {
    case 0:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;

```

```
    case 1:

      Serial.println(
        "TEST: LEFT"
      );

      commandSteering(
        +300
      );

      motorTestStep++;

      break;


    case 2:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    case 3:

      Serial.println(
        "TEST: RIGHT"
      );

      commandSteering(
        -300
      );

      motorTestStep++;

      break;


    case 4:

      Serial.println(
        "TEST: CENTER"
      );

      commandSteering(0);

      motorTestStep++;

      break;


    default:

      motorTestActive =
        false;


      state =
        STATE_WAIT_READY;


      centerServos();


      Serial.println();

      Serial.println(
        "MOTOR TEST COMPLETE"
      );
```

```
      Serial.println();

      break;
  }
}


// ================================================================
// NAVIGATION CALCULATION
// ================================================================

void updateNavigation()
{
  if (!gpsQualityOK())
  {
    headingValid =
      false;

    headingSource =
      "NONE";

    return;
  }


  // ---------------------------------------------------------------
  // GPS position
  // ---------------------------------------------------------------

  currentLatitude =
    gps.location.lat();


  currentLongitude =
    gps.location.lng();


  // ---------------------------------------------------------------
  // Distance
  // ---------------------------------------------------------------

  distanceToTarget =
    TinyGPSPlus::distanceBetween(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
      TARGET_LON
    );


  // ---------------------------------------------------------------
  // Bearing
  // ---------------------------------------------------------------

  targetBearing =
    TinyGPSPlus::courseTo(
      currentLatitude,
      currentLongitude,
      TARGET_LAT,
      TARGET_LON
    );


  targetBearing =
    wrap360(
      targetBearing
    );


  // ---------------------------------------------------------------
  // Determine heading
  // ---------------------------------------------------------------

  headingValid =
```

```
    false;


  headingSource =
    "NONE";


  // GPS course when moving
  if (gps.speed.isValid() &&
      gps.course.isValid() &&
      currentSpeed >=
      GPS_COURSE_MIN_SPEED_MPS)
  {
    currentDirection =
      wrap360(
        gps.course.deg()
      );


    headingValid =
      true;


    headingSource =
      "GPS COURSE";
  }


  // Compass when slow/stationary
  else if (compassOK)
  {
    currentDirection =
      navigationCompassHeading();


    headingValid =
      true;


    headingSource =
      "COMPASS";
  }


  // ------------------------------------------------------------
  // Course error
  // ------------------------------------------------------------

  if (headingValid)
  {
    courseError =
      wrap180(
        targetBearing -
        currentDirection
      );
  }
}


// ================================================================
// AUTONOMOUS STEERING
// ================================================================

void updateSteering()
{
  if (motorTestActive) return;

  if (!servosReady())
  {
    steeringDemand = 0;
    return;
  }

  if (state != STATE_AUTONAV) return;

```

```
  if (!gpsQualityOK())
  {
    centerServos();
    return;
  }

  if (TARGET_CAPTURE_ENABLED && distanceToTarget <= TARGET_CAPTURE_RADIUS_M)
  {
    centerServos();
    return;
  }

  if (!headingValid)
  {
    Serial.println("AUTONAV: NO VALID HEADING -> CENTER");
    centerServos();
    return;
  }

  double absoluteError = fabs(courseError);

  if (absoluteError <= STEERING_DEADBAND_DEG)
  {
    steeringDemand = 0;
  }
  else
  {
    double usableError = absoluteError - STEERING_DEADBAND_DEG;
    double usableRange = FULL_STEERING_ERROR_DEG - STEERING_DEADBAND_DEG;
    double normalized = constrain(usableError / usableRange, 0.0, 1.0);
    int demand = (int)round(normalized * MAX_STEERING_COUNTS);

    if (courseError > 0.0) steeringDemand = demand;
    else steeringDemand = -demand;
  }

  int target1, target2;
  steeringTargetsFromDemand(steeringDemand, target1, target2);

  int new1 = slewTo(servo1Position, target1, MAX_SLEW_COUNTS);
  int new2 = slewTo(servo2Position, target2, MAX_SLEW_COUNTS);

  if (new1 != servo1Position || new2 != servo2Position)
  {
    if (new1 != servo1Position)
    {
      st.WritePosEx(SERVO1_ID, new1,
                    STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
      delay(8);
      servo1Position = new1;
    }

    if (new2 != servo2Position)
    {
      st.WritePosEx(SERVO2_ID, new2,
                    STEERING_SERVO_SPEED, STEERING_SERVO_ACCEL);
      delay(8);
      servo2Position = new2;
    }
  }
}

// ================================================================
// SLEW LIMIT
// ================================================================

int slewTo(
  int currentValue,
  int targetValue,
  int maxStep)
{
  if (targetValue >
      currentValue)
  {
    int difference =
```

```
      targetValue -
      currentValue;


    if (difference >
        maxStep)
    {
      return currentValue +
             maxStep;
    }


    return targetValue;
  }


  if (targetValue <
      currentValue)
  {
    int difference =
      currentValue -
      targetValue;


    if (difference >
        maxStep)
    {
      return currentValue -
             maxStep;
    }


    return targetValue;
  }


  return currentValue;
}


// ================================================================
// DEPLOYMENT
// ================================================================

void startDeployment()
{
  if (deploymentDone)
  {
    Serial.println(
      "DEPLOYMENT ALREADY COMPLETED"
    );

    return;
  }


  if (!gpsQualityOK())
  {
    Serial.println(
      "DEPLOY BLOCKED: GPS NOT READY"
    );

    return;
  }


  if (!servosReady())
  {
    Serial.println(
      "DEPLOY BLOCKED: ST-3009 NOT READY"
    );

    return;
  }

```

```
  // Center steering first
  centerServos();


  delay(100);


  // Attach SG90
  deployServo.attach(
    DEPLOY_SERVO_PIN
  );


  delay(100);


  // Deployment position
  deployServo.write(
    DEPLOY_ANGLE
  );


  deploymentStartMs =
    millis();


  deploymentServoActive =
    true;


  deploymentDone =
    true;


  state =
    STATE_DEPLOYING;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "STARTING DEPLOYMENT"
  );

  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    DEPLOY_ANGLE
  );

  Serial.println(
    " degrees"
  );
}


// ================================================================
// SG90 PREP
// ================================================================

void prepareDeploymentServo()
{
  deployServo.attach(
    DEPLOY_SERVO_PIN
```

```
  );


  delay(100);


  deployServo.write(
    PREP_ANGLE
  );


  Serial.print(
    "SG90 -> "
  );

  Serial.print(
    PREP_ANGLE
  );

  Serial.println(
    " degrees"
  );


  delay(1000);


  deployServo.detach();


  Serial.println(
    "SG90 detached"
  );
}


// ====================================================================
// STATE MACHINE
// ====================================================================

void updateStateMachine()
{
  // ----------------------------------------------------------------
  // Manual test
  // ----------------------------------------------------------------

  if (state ==
      STATE_MANUAL_TEST)
  {
    return;
  }


  // ----------------------------------------------------------------
  // WAIT READY
  // ----------------------------------------------------------------

  if (state ==
      STATE_WAIT_READY)
  {
    if (AUTO_DEPLOY_AFTER_READY &&
        !deploymentDone &&
        gpsQualityOK() &&
        servosReady())
    {
      Serial.println();

      Serial.println(
        "GPS + ST-3009 READY"
      );

      Serial.println(
        "AUTO DEPLOY ENABLED"
      );

```

```
      startDeployment();
    }


    return;
  }


  // ------------------------------------------------------------
  // DEPLOYING
  // ------------------------------------------------------------

  if (state ==
      STATE_DEPLOYING)
  {
    steeringDemand =
      0;


    if (millis() -
        deploymentStartMs >=
        DEPLOY_MOVE_TIME_MS)
    {
      // Detach SG90
      deployServo.detach();


      deploymentServoActive =
        false;


      // Center steering
      if (servosReady())
      {
        centerServos();
      }


      // Stabilization starts
      stabilizationStartMs =
        millis();


      state =
        STATE_STABILIZING;


      Serial.println();

      Serial.println(
        "SG90 detached"
      );

      Serial.println(
        "5 second stabilization started"
      );
    }


    return;
  }


  // ------------------------------------------------------------
  // STABILIZING
  // ------------------------------------------------------------

  if (state ==
      STATE_STABILIZING)
  {
    steeringDemand =
      0;

```

```
    if (millis() -
        stabilizationStartMs >=
        DEPLOY_STABILIZE_TIME_MS)
    {
      if (gpsQualityOK() &&
          servosReady())
      {
        state =
          STATE_AUTONAV;


        Serial.println();

        Serial.println(
          "======================================="
        );

        Serial.println(
          "AUTONOMOUS NAVIGATION STARTED"
        );

        Serial.println(
          "======================================="
        );
      }
    }


    return;
  }


  // ------------------------------------------------------------
  // AUTONAV
  // ------------------------------------------------------------

  if (state ==
      STATE_AUTONAV)
  {
    // GPS / servo failure
    if (!gpsQualityOK() ||
        !servosReady())
    {
      steeringDemand =
        0;


      if (servosReady())
      {
        centerServos();
      }


      state =
        STATE_FAILSAFE;


      failsafeGoodSinceMs =
        0;


      Serial.println();

      Serial.println(
        "!!! NAVIGATION FAILSAFE !!!"
      );

      Serial.println(
        "GPS or ST-3009 unavailable"
      );


      return;
    }
```

```

  // Target capture
  if (TARGET_CAPTURE_ENABLED &&
      distanceToTarget <=
      TARGET_CAPTURE_RADIUS_M)
  {
    centerServos();


    state =
      STATE_CAPTURED;


    Serial.println();

    Serial.println(
      "========================================"
    );

    Serial.println(
      "TARGET CAPTURED"
    );

    Serial.print(
      "Distance = "
    );

    Serial.print(
      distanceToTarget,
      2
    );

    Serial.println(
      " m"
    );

    Serial.println(
      "ST-3009 -> CENTER"
    );

    Serial.println(
      "========================================"
    );
  }


  return;
}


// ------------------------------------------------------------
// CAPTURED
// ------------------------------------------------------------

if (state ==
    STATE_CAPTURED)
{
  steeringDemand =
    0;

  return;
}


// ------------------------------------------------------------
// FAILSAFE
// ------------------------------------------------------------

if (state ==
    STATE_FAILSAFE)
{
  steeringDemand =
    0;


```

```
    if (servosReady())
    {
      centerServos();
    }


    if (gpsQualityOK() &&
        servosReady())
    {
      if (failsafeGoodSinceMs ==
          0)
      {
        failsafeGoodSinceMs =
          millis();


        Serial.println(
          "GPS + ST-3009 recovered"
        );


        Serial.println(
          "Waiting 3 seconds..."
        );
      }
      else if (millis() -
               failsafeGoodSinceMs >=
               FAILSAFE_RECOVERY_MS)
      {
        state =
          STATE_AUTONAV;


        failsafeGoodSinceMs =
          0;


        Serial.println(
          "FAILSAFE RECOVERY COMPLETE"
        );


        Serial.println(
          "AUTONAV RESUMED"
        );
      }
    }
    else
    {
      failsafeGoodSinceMs =
        0;
    }


    return;
  }
}


// ================================================================
// SERIAL INPUT
// ================================================================

void processSerialCommands()
{
  while (Serial.available() > 0)
  {
    char c =
      (char)Serial.read();


    // ------------------------------------------------------------
    // Newline
    // ------------------------------------------------------------
```

```
    if (c == '\r' ||
        c == '\n')
    {
      if (commandIndex > 0)
      {
        commandBuffer[
          commandIndex] =
          '\0';


        processCommand(
          commandBuffer
        );


        commandIndex =
          0;
      }


      continue;
    }


    // ---------------------------------------------------------
    // Store
    // ---------------------------------------------------------

    if (commandIndex <
        COMMAND_BUFFER_SIZE - 1)
    {
      commandBuffer[
        commandIndex++] =
        c;
    }
    else
    {
      commandIndex = 0;

      Serial.println(
        "ERROR: COMMAND TOO LONG"
      );
    }
  }
}


// ================================================================
// COMMAND PROCESSOR
// ================================================================

void processCommand(
  char *cmd)
{
  // Remove leading spaces
  while (*cmd == ' ' ||
         *cmd == '\t')
  {
    cmd++;
  }


  // Remove trailing spaces
  int len =
    strlen(cmd);


  while (len > 0 &&
         (cmd[len - 1] == ' ' ||
          cmd[len - 1] == '\t'))
  {
    cmd[len - 1] =
      '\0';

    len--;
```

```
  }


  if (len == 0)
    return;


  Serial.print(
    "CMD RECEIVED: ["
  );

  Serial.print(
    cmd
  );

  Serial.println(
    "]"
  );


  // ================================================================
  // STATUS
  // ================================================================

  if (strcmp(
        cmd,
        "STATUS") == 0)
  {
    printStatus();
    return;
  }


  // ================================================================
  // MOTORTEST
  // ================================================================

  if (strcmp(
        cmd,
        "MOTORTEST") == 0)
  {
    startMotorTest();
    return;
  }


  // ================================================================
  // SERVO DIAGNOSTICS
  // ================================================================

  if (strcmp(cmd, "PING1") == 0)
  {
    Serial.print("PING ID 1 : ");
    Serial.println(pingServo(SERVO1_ID) ? "OK" : "FAIL");
    return;
  }

  if (strcmp(cmd, "PING2") == 0)
  {
    Serial.print("PING ID 2 : ");
    Serial.println(pingServo(SERVO2_ID) ? "OK" : "FAIL");
    return;
  }

  if (strcmp(cmd, "POS1") == 0)
  {
    int p = readServoPosition(SERVO1_ID);
    Serial.print("POS ID 1 : ");
    Serial.println(p);
    return;
  }

  if (strcmp(cmd, "POS2") == 0)
  {
    int p = readServoPosition(SERVO2_ID);
```

```
    Serial.print("POS ID 2 : ");
    Serial.println(p);
    return;
  }

  if (strcmp(cmd, "SERVOTEST") == 0)
  {
    printServoDiagnostics();
    centerServos();
    delay(500);
    commandSteering(+300);
    delay(1800);
    centerServos();
    delay(500);
    commandSteering(-300);
    delay(1800);
    centerServos();
    return;
  }


  // ================================================================
  // LEFT
  // ================================================================

  if (strcmp(
        cmd,
        "LEFT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      +300
    );


    return;
  }


  // ================================================================
  // RIGHT
  // ================================================================

  if (strcmp(
        cmd,
        "RIGHT") == 0)
  {
    motorTestActive =
      false;


    state =
      STATE_MANUAL_TEST;


    commandSteering(
      -300
    );


    return;
  }


  // ================================================================
  // CENTER
  // ================================================================

```

```
  if (strcmp(
        cmd,
        "CENTER") == 0)
  {
    motorTestActive =
      false;


    if (servosReady())
    {
      centerServos();


      Serial.println(
        "ST-3009 -> CENTER"
      );
    }
    else
    {
      Serial.println(
        "ST-3009 BUS NOT READY"
      );
    }


    state =
      STATE_WAIT_READY;


    return;
  }


  // ============================================================
  // TARGET
  // ============================================================

  if (strncmp(cmd, "TARGET ", 7) == 0)
  {
    double newLat = 0.0;
    double newLon = 0.0;

    int parsed = sscanf(cmd + 7, "%lf %lf", &newLat, &newLon);

    if (parsed == 2 &&
        newLat >= -90.0 && newLat <= 90.0 &&
        newLon >= -180.0 && newLon <= 180.0)
    {
      TARGET_LAT = newLat;
      TARGET_LON = newLon;

      Serial.println("TARGET UPDATED");
      Serial.print("LAT = ");
      Serial.println(TARGET_LAT, 7);
      Serial.print("LON = ");
      Serial.println(TARGET_LON, 7);
    }
    else
    {
      Serial.println("Usage: TARGET LAT LON");
    }

    return;
  }


  // ============================================================
  // AUTONAV
  // ============================================================

  if (strcmp(
        cmd,
        "AUTONAV") == 0)
  {
    motorTestActive =
```

```
      false;


    if (gpsQualityOK() &&
        servosReady())
    {
      state =
        STATE_AUTONAV;


      Serial.println(
        "AUTONAV ENABLED"
      );
    }
    else
    {
      Serial.println(
        "AUTONAV BLOCKED"
      );


      Serial.print(
        "GPS = "
      );

      Serial.println(
        gpsQualityOK()
          ? "GOOD"
          : "BAD"
      );


      Serial.print(
        "SERVO = "
      );

      Serial.println(
        servosReady()
          ? "GOOD"
          : "BAD"
      );
    }


    return;
  }


  // ============================================================
  // DEPLOY
  // ============================================================

  if (strcmp(
        cmd,
        "DEPLOY") == 0)
  {
    motorTestActive =
      false;


    startDeployment();

    return;
  }


  // ============================================================
  // PREP
  // ============================================================

  if (strcmp(
        cmd,
        "PREP") == 0)
  {
    motorTestActive =
```

```
      false;


    prepareDeploymentServo();

    return;
  }


  // ============================================================
  // RESET
  // ============================================================

  if (strcmp(
        cmd,
        "RESET") == 0)
  {
    motorTestActive =
      false;


    resetSystem();

    return;
  }


  // ============================================================
  // HELP
  // ============================================================

  if (strcmp(
        cmd,
        "HELP") == 0)
  {
    Serial.println();

    Serial.println(
      "STATUS"
    );

    Serial.println(
      "MOTORTEST"
    );

    Serial.println("PING1");
    Serial.println("PING2");
    Serial.println("POS1");
    Serial.println("POS2");
    Serial.println("SERVOTEST");

    Serial.println(
      "LEFT"
    );

    Serial.println(
      "RIGHT"
    );

    Serial.println(
      "CENTER"
    );

    Serial.println(
      "AUTONAV"
    );

    Serial.println(
      "DEPLOY"
    );

    Serial.println(
      "PREP"
    );

```

```
    Serial.println(
      "RESET"
    );

    Serial.println(
      "HELP"
    );

    Serial.println(
      "TARGET <lat> <lon>"
    );


    Serial.println();

    Serial.println(
      "TARGET CAN BE UPDATED FROM MATLAB"
    );


    return;
  }


  // Unknown
  Serial.print(
    "UNKNOWN COMMAND: "
  );

  Serial.println(
    cmd
  );
}


// ================================================================
// RESET
// ================================================================

void resetSystem()
{
  // Stop deployment servo
  if (deploymentServoActive)
  {
    deployServo.detach();

    deploymentServoActive =
      false;
  }


  // Stop motor test
  motorTestActive =
    false;


  // Center if available
  if (servosReady())
  {
    centerServos();
  }


  // Reset flags
  deploymentDone =
    false;


  steeringDemand =
    0;


  failsafeGoodSinceMs =
    0;

```

```
  // Return to wait state
  state =
    STATE_WAIT_READY;


  stateEnteredMs =
    millis();


  Serial.println();

  Serial.println(
    "========================================"
  );

  Serial.println(
    "SYSTEM RESET"
  );

  Serial.print(
    "TARGET = "
  );

  Serial.print(
    TARGET_LAT,
    7
  );

  Serial.print(
    ", "
  );

  Serial.println(
    TARGET_LON,
    7
  );

  Serial.println(
    "========================================"
  );
}


// ================================================================
// STATUS
// ================================================================

void printStatus()
{
  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "                    STATUS"
  );

  Serial.println(
    "================================================"
  );


  // System
  Serial.print(
    "State             : "
  );

  Serial.println(
    stateName(state)
  );

```

```
  // Target
  Serial.print(
    "Target latitude   : "
  );

  Serial.println(
    TARGET_LAT,
    7
  );


  Serial.print(
    "Target longitude  : "
  );

  Serial.println(
    TARGET_LON,
    7
  );


  // GPS
  Serial.print(
    "GPS fix           : "
  );

  Serial.println(
    gpsFix
      ? "YES"
      : "NO"
  );


  Serial.print(
    "GPS quality       : "
  );

  Serial.println(
    gpsQualityOK()
      ? "GOOD"
      : "BAD"
  );


  Serial.print(
    "Satellites        : "
  );

  Serial.println(
    gpsSats
  );


  Serial.print(
    "HDOP              : "
  );

  Serial.println(
    gpsHDOP,
    2
  );


  Serial.print(
    "Latitude          : "
  );

  Serial.println(
    currentLatitude,
    7
  );


  Serial.print(
    "Longitude         : "
```

```
  );

  Serial.println(
    currentLongitude,
    7
  );


  Serial.print(
    "Speed             : "
  );

  Serial.print(
    currentSpeed,
    2
  );

  Serial.println(
    " m/s"
  );


  // Navigation
  Serial.print(
    "Distance to target: "
  );

  Serial.print(
    distanceToTarget,
    2
  );

  Serial.println(
    " m"
  );


  Serial.print(
    "Target bearing    : "
  );

  Serial.print(
    targetBearing,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Current heading   : "
  );

  Serial.print(
    currentDirection,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Heading source    : "
  );

  Serial.println(
    headingSource
  );


  Serial.print(
```

```
    "Course error      : "
  );

  Serial.print(
    courseError,
    2
  );

  Serial.println(
    " deg"
  );


  // Compass
  Serial.print(
    "Compass status    : "
  );

  Serial.println(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Raw magnetometer  : "
  );

  Serial.print(
    rawMagX
  );

  Serial.print(
    ", "
  );

  Serial.print(
    rawMagY
  );

  Serial.print(
    ", "
  );

  Serial.println(
    rawMagZ
  );


  Serial.print(
    "Compass heading   : "
  );

  Serial.print(
    compassHeading,
    2
  );

  Serial.println(
    " deg"
  );


  Serial.print(
    "Navigation heading: "
  );

  Serial.print(
    navCompassHeading,
    2
  );

  Serial.println(
    " deg"
```

```
  );


  // Servos
  Serial.print(
    "Servo 1           : "
  );

  Serial.println(
    servo1OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 2           : "
  );

  Serial.println(
    servo2OK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    "Servo 1 position  : "
  );

  Serial.println(
    servo1Position
  );


  Serial.print(
    "Servo 2 position  : "
  );

  Serial.println(
    servo2Position
  );


  Serial.print(
    "Steering demand   : "
  );

  Serial.println(
    steeringDemand
  );


  // IMU
  Serial.print(
    "IMU               : "
  );

  Serial.println(
    imuOK
      ? "OK"
      : "FAIL"
  );


  // BMP
  Serial.print(
    "BMP581            : "
  );


  if (bmpOK)
  {
    Serial.print(
      "OK at 0x"
```

```
    );


    if (bmpAddress < 16)
      Serial.print("0");


    Serial.println(
      bmpAddress,
      HEX
    );
  }
  else
  {
    Serial.println(
      "NOT FOUND"
    );
  }


  // Deployment
  Serial.print(
    "Deployment done   : "
  );

  Serial.println(
    deploymentDone
      ? "YES"
      : "NO"
  );


  Serial.println(
    "================================================"
  );
}


// ===================================================================
// SERVO FEEDBACK
// ===================================================================

void updateServoFeedback()
{
  if (millis() - lastServoFeedbackMs < SERVO_FEEDBACK_INTERVAL_MS)
    return;

  lastServoFeedbackMs = millis();

  if (!servosReady())
  {
    servo1Feedback = -1;
    servo2Feedback = -1;
    return;
  }

  int p1 = readServoPosition(SERVO1_ID);
  int p2 = readServoPosition(SERVO2_ID);

  if (p1 >= 0) servo1Feedback = p1;
  if (p2 >= 0) servo2Feedback = p2;
}

// ===================================================================
// SERVO DIAGNOSTICS
// ===================================================================

void printServoDiagnostics()
{
  Serial.println();
  Serial.println("================ SERVO DIAGNOSTICS ================");

  bool p1 = pingServo(SERVO1_ID);
  bool p2 = pingServo(SERVO2_ID);

```

```
  Serial.print("PING ID 1 : ");
  Serial.println(p1 ? "OK" : "FAIL");

  Serial.print("PING ID 2 : ");
  Serial.println(p2 ? "OK" : "FAIL");

  if (p1)
  {
    int a = readServoPosition(SERVO1_ID);
    Serial.print("POS ID 1  : ");
    Serial.println(a);
  }
  else
  {
    Serial.println("POS ID 1  : unavailable");
  }

  if (p2)
  {
    int b = readServoPosition(SERVO2_ID);
    Serial.print("POS ID 2  : ");
    Serial.println(b);
  }
  else
  {
    Serial.println("POS ID 2  : unavailable");
  }

  Serial.print("BUS READY : ");
  Serial.println((p1 && p2) ? "YES" : "NO");
  Serial.println("==================================================");
}

// ================================================================
// MATLAB TELEMETRY
// ================================================================

void printMATLABTelemetry()
{
  double gpsCourse = -1.0;
  if (gps.course.isValid()) gpsCourse = gps.course.deg();

  Serial.print("NAVMAT,");
  Serial.print(millis()); Serial.print(",");
  Serial.print(currentLatitude, 7); Serial.print(",");
  Serial.print(currentLongitude, 7); Serial.print(",");
  Serial.print(currentAltitude, 2); Serial.print(",");
  Serial.print(currentSpeed, 2); Serial.print(",");
  Serial.print(gpsCourse, 2); Serial.print(",");
  Serial.print(currentDirection, 2); Serial.print(",");
  Serial.print(distanceToTarget, 2); Serial.print(",");
  Serial.print(targetBearing, 2); Serial.print(",");
  Serial.print(courseError, 2); Serial.print(",");
  Serial.print(steeringDemand); Serial.print(",");
  Serial.print(servo1Position); Serial.print(",");
  Serial.print(servo2Position); Serial.print(",");
  Serial.print(servo1Feedback); Serial.print(",");
  Serial.print(servo2Feedback); Serial.print(",");
  Serial.print(gpsSats); Serial.print(",");
  Serial.print(gpsHDOP, 2); Serial.print(",");
  Serial.print(stateName(state)); Serial.print(",");
  Serial.print(headingSource); Serial.print(",");
  Serial.println(navCompassHeading, 2);
}


// ================================================================
// LIVE DATA
// ================================================================

void printLiveData()
{
  Serial.print(
    "[LIVE] State="
  );
```

```
  Serial.print(
    stateName(state)
  );


  // GPS
  Serial.print(
    " GPS="
  );

  Serial.print(
    gpsQualityOK()
      ? "GOOD"
      : "BAD"
  );


  Serial.print(
    " SAT="
  );

  Serial.print(
    gpsSats
  );


  Serial.print(
    " HDOP="
  );

  Serial.print(
    gpsHDOP,
    1
  );


  if (gpsFix)
  {
    Serial.print(
      " LAT="
    );

    Serial.print(
      currentLatitude,
      6
    );


    Serial.print(
      " LON="
    );

    Serial.print(
      currentLongitude,
      6
    );
  }


  // Navigation
  if (gpsQualityOK())
  {
    Serial.print(
      " DIST="
    );

    Serial.print(
      distanceToTarget,
      1
    );


    Serial.print(
      "m BRG="
```

```
    );

    Serial.print(
      targetBearing,
      1
    );
  }


  // Heading
  if (headingValid)
  {
    Serial.print(
      " HDG="
    );

    Serial.print(
      currentDirection,
      1
    );


    Serial.print(
      " SRC="
    );

    Serial.print(
      headingSource
    );


    Serial.print(
      " ERR="
    );

    Serial.print(
      courseError,
      1
    );
  }
  else
  {
    Serial.print(
      " HDG=NONE"
    );
  }


  // Steering
  Serial.print(
    " DEM="
  );

  Serial.print(
    steeringDemand
  );


  Serial.print(
    " S1="
  );

  Serial.print(
    servo1Position
  );


  Serial.print(
    " S2="
  );

  Serial.print(
    servo2Position
  );

```

```
  Serial.print(
    " FB1="
  );

  Serial.print(
    servo1Feedback
  );

  Serial.print(
    " FB2="
  );

  Serial.print(
    servo2Feedback
  );


  // Compass
  Serial.print(
    " CMP="
  );

  Serial.print(
    compassOK
      ? "OK"
      : "FAIL"
  );


  Serial.print(
    " CHP="
  );

  Serial.print(
    compassHeading,
    1
  );


  Serial.print(
    " NAV="
  );

  Serial.print(
    navCompassHeading,
    1
  );


  // IMU
  Serial.print(
    " IMU="
  );

  Serial.print(
    imuOK
      ? "OK"
      : "FAIL"
  );


  // Servos
  Serial.print(
    " SERVO="
  );

  Serial.println(
    servosReady()
      ? "OK"
      : "FAIL"
  );
}


// ================================================================
```

```
// STATE NAME
// ================================================================

const char *stateName(
  SystemState s)
{
  switch (s)
  {
    case STATE_WAIT_READY:
      return "WAIT_READY";


    case STATE_DEPLOYING:
      return "DEPLOYING";


    case STATE_STABILIZING:
      return "STABILIZING";


    case STATE_AUTONAV:
      return "AUTONAV";


    case STATE_CAPTURED:
      return "CAPTURED";


    case STATE_FAILSAFE:
      return "FAILSAFE";


    case STATE_MANUAL_TEST:
      return "MANUAL_TEST";


    default:
      return "UNKNOWN";
  }
}
```

A.9 Archived source 9: ST3009_SERVO_BUS_TEST_ONLY.ino

Source file: `ST3009_SERVO_BUS_TEST_ONLY.ino`

```
/*
  ST-3009 SERVO BUS TEST ONLY
  Teensy 4.1

  Serial1: 1 Mbps, half duplex
  Servo ID 1 = left
  Servo ID 2 = right
  Neutral = 2048

  Commands from USB Serial @ 115200:
    PING1
    PING2
    POS1
    POS2
    CENTER
    LEFT
    RIGHT
    TEST

  LEFT  -> ID1 = 1748, ID2 = 2048
  RIGHT -> ID1 = 2048, ID2 = 1748
*/

#include <Arduino.h>
#include <SCServo.h>

SMS_STS st;

```

```
const uint8_t LEFT_ID = 1;
const uint8_t RIGHT_ID = 2;

const int CENTER_POS = 2048;
const int TEST_POS = 300;
const int SPEED = 2400;
const int ACCEL = 50;

void pauseBus() {
  delay(20);
}

bool pingServo(uint8_t id) {
  int r = st.Ping(id);
  pauseBus();
  return r >= 0;
}

int readPosition(uint8_t id) {
  int p = st.ReadPos(id);
  pauseBus();
  return p;
}

void moveServo(uint8_t id, int position) {
  st.WritePosEx(id, position, SPEED, ACCEL);
  pauseBus();
}

void printPosition(uint8_t id, const char *name) {
  int p = readPosition(id);
  Serial.print(name);
  Serial.print(" feedback = ");
  Serial.println(p);
}

void centerAll() {
  Serial.println("CENTER: S1=2048 S2=2048");
  moveServo(LEFT_ID, CENTER_POS);
  moveServo(RIGHT_ID, CENTER_POS);
}

void setup() {
  Serial.begin(115200);
  delay(1500);

  Serial.println();
  Serial.println("============================================");
  Serial.println("ST-3009 SERVO BUS TEST ONLY");
  Serial.println("============================================");

  Serial1.begin(1000000, SERIAL_8N1_HALF_DUPLEX);
  st.pSerial = &Serial1;
  delay(300);

  Serial.println("Serial1 = 1 Mbps HALF DUPLEX");
  Serial.println("Testing servo IDs 1 and 2...");

  bool p1 = pingServo(LEFT_ID);
  bool p2 = pingServo(RIGHT_ID);

  Serial.print("PING ID 1 = ");
  Serial.println(p1 ? "OK" : "FAIL");

  Serial.print("PING ID 2 = ");
  Serial.println(p2 ? "OK" : "FAIL");

  if (p1) printPosition(LEFT_ID, "Servo 1");
  if (p2) printPosition(RIGHT_ID, "Servo 2");

  Serial.println();
  Serial.println("Commands:");
  Serial.println("CENTER");
  Serial.println("LEFT");
  Serial.println("RIGHT");
```

```
  Serial.println("TEST");
  Serial.println("PING1 / PING2");
  Serial.println("POS1 / POS2");

  centerAll();
}

void handleCommand(String cmd) {
  cmd.trim();
  cmd.toUpperCase();

  if (cmd == "PING1") {
    Serial.println(pingServo(LEFT_ID) ? "PING1 OK" : "PING1 FAIL");
    return;
  }

  if (cmd == "PING2") {
    Serial.println(pingServo(RIGHT_ID) ? "PING2 OK" : "PING2 FAIL");
    return;
  }

  if (cmd == "POS1") {
    printPosition(LEFT_ID, "Servo 1");
    return;
  }

  if (cmd == "POS2") {
    printPosition(RIGHT_ID, "Servo 2");
    return;
  }

  if (cmd == "CENTER") {
    centerAll();
    return;
  }

  if (cmd == "LEFT") {
    Serial.println("LEFT: ID1 -> 1748, ID2 -> 2048");
    moveServo(LEFT_ID, CENTER_POS - TEST_POS);
    moveServo(RIGHT_ID, CENTER_POS);
    return;
  }

  if (cmd == "RIGHT") {
    Serial.println("RIGHT: ID1 -> 2048, ID2 -> 1748");
    moveServo(LEFT_ID, CENTER_POS);
    moveServo(RIGHT_ID, CENTER_POS - TEST_POS);
    return;
  }

  if (cmd == "TEST") {
    centerAll();
    delay(700);

    Serial.println("TEST LEFT");
    moveServo(LEFT_ID, CENTER_POS - TEST_POS);
    delay(1500);

    Serial.println("TEST CENTER");
    centerAll();
    delay(700);

    Serial.println("TEST RIGHT");
    moveServo(RIGHT_ID, CENTER_POS - TEST_POS);
    delay(1500);

    Serial.println("TEST CENTER");
    centerAll();

    return;
  }

  if (cmd.length() > 0) {
    Serial.println("Unknown command");
  }
```

```
}

void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    handleCommand(cmd);
  }
}
```

A.10 Archived source 10: parafoil_nav_test.py

Source file: `parafoil_nav_test.py`

```

"""
parafoil_nav_test.py

Desktop test bench for the Teensy parafoil navigation logic.

What it tests:
- GPS position -> distance to fixed target
- GPS position -> target bearing
- heading -> course error
- +/-180 degree wrap
- +/-5 degree steering deadband
- linear steering demand up to +/-450 counts at +/-90 deg error
- differential ST-3009 positions around neutral 2048

This is a logic simulator. It does NOT need a GPS.

Optional:
- With pyserial installed and the Teensy code modified to accept
  "NAVTEST LAT LON HEADING", this script can also send the simulated
  test points to the Teensy so you can watch the REAL ST-3009 servos.
"""

from __future__ import annotations

import math
import time
import argparse
from dataclasses import dataclass

# ---------------------------------------------------------------------
# YOUR TARGET
# ---------------------------------------------------------------------

TARGET_LAT = 32.639340
TARGET_LON = 75.014533

# ---------------------------------------------------------------------
# MATCH YOUR ARDUINO VALUES
# ---------------------------------------------------------------------

SERVO_NEUTRAL = 2048
MAX_STEERING_COUNTS = 450

SERVO_MIN_POS = 200
SERVO_MAX_POS = 3895

STEERING_DEADBAND_DEG = 5.0
FULL_STEERING_ERROR_DEG = 90.0


@dataclass
class NavResult:
    distance_m: float
    target_bearing_deg: float
    heading_deg: float
    error_deg: float
    demand: int
    servo1: int
    servo2: int
```

```

def wrap360(angle: float) -> float:
    return angle % 360.0


def wrap180(angle: float) -> float:
    angle = (angle + 180.0) % 360.0 - 180.0
    # Keep +180 if the input was positive rather than always -180.
    if angle == -180.0:
        return 180.0
    return angle


def distance_bearing(
    lat1: float,
    lon1: float,
    lat2: float,
    lon2: float,
) -> tuple[float, float]:
    """Equivalent navigation quantities to GPS distance/bearing."""
    earth_radius_m = 6371000.0

    phi1 = math.radians(lat1)
    phi2 = math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlambda = math.radians(lon2 - lon1)

    a = (
        math.sin(dphi / 2.0) ** 2
        + math.cos(phi1)
        * math.cos(phi2)
        * math.sin(dlambda / 2.0) ** 2
    )
    a = max(0.0, min(1.0, a))

    c = 2.0 * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
    distance_m = earth_radius_m * c

    # Initial bearing, clockwise from north.
    y = math.sin(dlambda) * math.cos(phi2)
    x = (
        math.cos(phi1) * math.sin(phi2)
        - math.sin(phi1) * math.cos(phi2) * math.cos(dlambda)
    )

    bearing = math.degrees(math.atan2(y, x))
    bearing = wrap360(bearing)

    return distance_m, bearing


def steering_from_error(error_deg: float) -> tuple[int, int, int]:
    """
    Match the current Arduino steering logic:
      <= 5 deg -> zero
      90 deg   -> +/-450
      between  -> linear
    """

    absolute_error = abs(error_deg)

    if absolute_error <= STEERING_DEADBAND_DEG:
        demand = 0
    else:
        usable_error = absolute_error - STEERING_DEADBAND_DEG
        usable_range = FULL_STEERING_ERROR_DEG - STEERING_DEADBAND_DEG

        normalized = usable_error / usable_range
        normalized = max(0.0, min(1.0, normalized))

        magnitude = round(normalized * MAX_STEERING_COUNTS)

        demand = magnitude if error_deg > 0.0 else -magnitude

```

```
    servo1 = SERVO_NEUTRAL - demand
    servo2 = SERVO_NEUTRAL + demand

    servo1 = max(SERVO_MIN_POS, min(SERVO_MAX_POS, servo1))
    servo2 = max(SERVO_MIN_POS, min(SERVO_MAX_POS, servo2))

    return demand, servo1, servo2


def run_case(
    lat: float,
    lon: float,
    heading_deg: float,
) -> NavResult:

    distance_m, target_bearing = distance_bearing(
        lat, lon, TARGET_LAT, TARGET_LON
    )

    heading = wrap360(heading_deg)

    error = wrap180(target_bearing - heading)

    demand, servo1, servo2 = steering_from_error(error)

    return NavResult(
        distance_m=distance_m,
        target_bearing_deg=target_bearing,
        heading_deg=heading,
        error_deg=error,
        demand=demand,
        servo1=servo1,
        servo2=servo2,
    )


def print_result(name: str, result: NavResult) -> None:
    print(f"\n{name}")
    print("-" * 70)
    print(f"Distance to target : {result.distance_m:8.2f} m")
    print(f"Target bearing     : {result.target_bearing_deg:8.2f} deg")
    print(f"Current heading    : {result.heading_deg:8.2f} deg")
    print(f"Course error       : {result.error_deg:8.2f} deg")
    print(f"Steering demand    : {result.demand:8d}")
    print(f"ST-3009 Servo 1    : {result.servo1:8d}")
    print(f"ST-3009 Servo 2    : {result.servo2:8d}")


def run_basic_tests() -> None:
    """
    Test position is approximately 500 m north of the target.
    Since it is on the same longitude, target bearing is about 180 deg.
    """
    test_lat = 32.6438366
    test_lon = TARGET_LON

    print("\n================================================")
    print(" PARAFOIL NAVIGATION LOGIC TEST")
    print("================================================")
    print(f"Target: {TARGET_LAT:.7f}, {TARGET_LON:.7f}")

    cases = [
        ("TEST 1: aligned", 180.0),
        ("TEST 2: +5 deg deadband edge", 175.0),
        ("TEST 3: +10 deg error", 170.0),
        ("TEST 4: +45 deg error", 135.0),
        ("TEST 5: +90 deg error", 90.0),
        ("TEST 6: -10 deg error", 190.0),
        ("TEST 7: -45 deg error", 225.0),
        ("TEST 8: -90 deg error", 270.0),
    ]

    for name, heading in cases:
        result = run_case(test_lat, test_lon, heading)
        print_result(name, result)
```

```

def run_wrap_tests() -> None:
    """
    Important because navigation can cross 0/360 degrees.
    """
    print("\n================================================")
    print(" ANGLE WRAP TESTS")
    print("================================================")

    tests = [
        (1.0, 359.0),
        (359.0, 1.0),
        (0.0, 180.0),
        (180.0, 0.0),
        (350.0, 10.0),
        (10.0, 350.0),
    ]

    for target, heading in tests:
        error = wrap180(target - heading)
        print(
            f"Target={target:6.1f}  "
            f"Heading={heading:6.1f}  "
            f"Error={error:7.1f}"
        )


def run_automatic_heading_sweep() -> None:
    """
    Hold the simulated position fixed and sweep heading from 0 to 350 deg.
    This shows how the steering command changes over all headings.
    """
    print("\n================================================")
    print(" HEADING SWEEP")
    print("================================================")

    test_lat = 32.6438366
    test_lon = TARGET_LON

    print(
        "Heading(deg)  Bearing(deg)  Error(deg)  "
        "Demand  Servo1  Servo2"
    )
    print("-" * 70)

    for heading in range(0, 360, 10):
        result = run_case(test_lat, test_lon, float(heading))
        print(
            f"{result.heading_deg:11.1f}  "
            f"{result.target_bearing_deg:12.1f}  "
            f"{result.error_deg:9.1f}  "
            f"{result.demand:6d}  "
            f"{result.servo1:6d}  "
            f"{result.servo2:6d}"
        )


def run_path_simulation() -> None:
    """
    Simulate several positions around the target.
    This is useful for seeing whether the navigation direction changes
    sensibly as the simulated vehicle moves.
    """
    print("\n================================================")
    print(" POSITION / HEADING SCENARIO")
    print("================================================")

    scenarios = [
        ("North of target", 32.6438366, 75.014533, 90.0),
        ("South of target", 32.6348434, 75.014533, 270.0),
        ("East of target", 32.639340, 75.019928, 180.0),
        ("West of target", 32.639340, 75.009138, 0.0),
        ("NE of target", 32.6438366, 75.019928, 180.0),
        ("SW of target", 32.6348434, 75.009138, 0.0),
```

```
    ]

    for name, lat, lon, heading in scenarios:
        result = run_case(lat, lon, heading)
        print_result(name, result)


def send_navtest_to_teensy(
    port: str,
    baud: int,
    lat: float,
    lon: float,
    heading: float,
) -> None:
    """
    Optional hardware-in-the-loop test.

    Requires:
        pip install pyserial

    Requires the Teensy sketch to support:
        NAVTEST LAT LON HEADING
    """
    try:
        import serial
    except ImportError as exc:
        raise SystemExit(
            "pyserial is not installed.\n"
            "Install it with: pip install pyserial"
        ) from exc

    command = f"NAVTEST {lat:.7f} {lon:.7f} {heading:.2f}\n"

    print(f"\nOpening {port} at {baud} baud...")
    with serial.Serial(port, baudrate=baud, timeout=1) as ser:
        time.sleep(2.0)  # allow Teensy USB serial reset
        ser.write(command.encode("ascii"))

        print(f"Sent: {command.strip()}")
        print("Teensy response:")
        start = time.time()

        while time.time() - start < 3.0:
            line = ser.readline().decode(
                "utf-8", errors="replace"
            ).strip()

            if line:
                print(line)


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Parafoil navigation / ST-3009 logic tester"
    )

    parser.add_argument(
        "--port",
        help="Teensy serial port, e.g. COM5",
    )

    parser.add_argument(
        "--baud",
        type=int,
        default=115200,
        help="Teensy USB serial baud rate",
    )

    parser.add_argument("--lat", type=float)
    parser.add_argument("--lon", type=float)
    parser.add_argument("--heading", type=float)

    parser.add_argument(
        "--sweep",
        action="store_true",
```

```
        help="Run 0-350 degree heading sweep",
    )

    parser.add_argument(
        "--wrap",
        action="store_true",
        help="Run angle wrap tests",
    )

    parser.add_argument(
        "--scenarios",
        action="store_true",
        help="Run position/heading scenarios",
    )

    args = parser.parse_args()

    if (
        args.port
        and args.lat is not None
        and args.lon is not None
        and args.heading is not None
    ):
        send_navtest_to_teensy(
            args.port,
            args.baud,
            args.lat,
            args.lon,
            args.heading,
        )
        return

    run_basic_tests()

    if args.wrap:
        run_wrap_tests()

    if args.sweep:
        run_automatic_heading_sweep()

    if args.scenarios:
        run_path_simulation()


if __name__ == "__main__":
    main()
```

A.11 Archived source 11: parafoil_nav_test_corrected.py

Source file: `parafoil_nav_test_corrected.py`

```
"""
PARAFOIL NAVIGATION TEST BENCH

Run as a normal Python program:
    python parafoil_nav_test_corrected.py

Full tests:
    python parafoil_nav_test_corrected.py --wrap --sweep --assert

Interactive test:
    python parafoil_nav_test_corrected.py --interactive

Optional Teensy hardware-in-the-loop test:
    python -m pip install pyserial
    python parafoil_nav_test_corrected.py --port COM5 --lat 32.6438366 --lon 75.014533 --heading 135

This program does NOT need a GPS. It simulates GPS position and heading,
then calculates the target bearing, course error, steering demand and
ST-3009 positions.
"""

import argparse
```

```
import math
import time
from dataclasses import dataclass

# ================================================================
# FIXED TARGET
# ================================================================
TARGET_LAT = 32.639340
TARGET_LON = 75.014533

# ================================================================
# SAME STEERING VALUES AS YOUR TEENSY CODE
# ================================================================
SERVO_NEUTRAL = 2048
SERVO_MIN_POS = 200
SERVO_MAX_POS = 3895
MAX_STEERING_COUNTS = 450
STEERING_DEADBAND_DEG = 5.0
FULL_STEERING_ERROR_DEG = 90.0


@dataclass
class NavResult:
    latitude: float
    longitude: float
    distance_m: float
    target_bearing: float
    heading: float
    course_error: float
    demand: int
    servo1: int
    servo2: int


def wrap360(angle: float) -> float:
    result = angle % 360.0
    return 0.0 if abs(result) < 1e-12 else result


def wrap180(angle: float) -> float:
    result = (angle + 180.0) % 360.0 - 180.0
    if result == -180.0 and angle > 0.0:
        result = 180.0
    return 0.0 if abs(result) < 1e-12 else result


def distance_and_bearing(lat1: float, lon1: float,
                         lat2: float, lon2: float) -> tuple[float, float]:
    """Return distance in metres and initial bearing in degrees."""
    earth_radius = 6371000.0

    phi1 = math.radians(lat1)
    phi2 = math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlambda = math.radians(lon2 - lon1)

    a = (
        math.sin(dphi / 2.0) ** 2
        + math.cos(phi1) * math.cos(phi2)
        * math.sin(dlambda / 2.0) ** 2
    )
    a = max(0.0, min(1.0, a))
    c = 2.0 * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
    distance = earth_radius * c

    y = math.sin(dlambda) * math.cos(phi2)
    x = (
        math.cos(phi1) * math.sin(phi2)
        - math.sin(phi1) * math.cos(phi2) * math.cos(dlambda)
    )
    bearing = wrap360(math.degrees(math.atan2(y, x)))
    return distance, bearing


def calculate_steering(error_deg: float) -> tuple[int, int, int]:
```

```
    """Match the Teensy steering logic."""
    absolute_error = abs(error_deg)

    if absolute_error <= STEERING_DEADBAND_DEG:
        demand = 0
    else:
        usable_error = absolute_error - STEERING_DEADBAND_DEG
        usable_range = FULL_STEERING_ERROR_DEG - STEERING_DEADBAND_DEG
        normalized = usable_error / usable_range
        normalized = max(0.0, min(1.0, normalized))
        magnitude = int(round(normalized * MAX_STEERING_COUNTS))
        demand = magnitude if error_deg > 0.0 else -magnitude

    servo1 = SERVO_NEUTRAL - demand
    servo2 = SERVO_NEUTRAL + demand

    servo1 = max(SERVO_MIN_POS, min(SERVO_MAX_POS, servo1))
    servo2 = max(SERVO_MIN_POS, min(SERVO_MAX_POS, servo2))
    return demand, servo1, servo2


def calculate_navigation(latitude: float, longitude: float,
                         heading_deg: float) -> NavResult:
    distance, target_bearing = distance_and_bearing(
        latitude, longitude, TARGET_LAT, TARGET_LON
    )
    heading = wrap360(heading_deg)
    error = wrap180(target_bearing - heading)
    demand, servo1, servo2 = calculate_steering(error)

    return NavResult(
        latitude, longitude, distance, target_bearing,
        heading, error, demand, servo1, servo2
    )


def print_result(title: str, r: NavResult) -> None:
    print("\n" + "=" * 68)
    print(title)
    print("=" * 68)
    print(f"Current GPS       : {r.latitude:.7f}, {r.longitude:.7f}")
    print(f"Target GPS        : {TARGET_LAT:.7f}, {TARGET_LON:.7f}")
    print(f"Distance          : {r.distance_m:.2f} m")
    print(f"Target bearing    : {r.target_bearing:.2f} deg")
    print(f"Current heading   : {r.heading:.2f} deg")
    print(f"Course error      : {r.course_error:.2f} deg")
    print(f"Steering demand   : {r.demand}")
    print(f"ST-3009 Servo 1   : {r.servo1}")
    print(f"ST-3009 Servo 2   : {r.servo2}")


def basic_tests() -> None:
    """Vehicle is about 500 m north of target; target bearing is ~180 deg."""
    lat = 32.6438366
    lon = TARGET_LON

    cases = [
        ("1 - ALIGNED", 180.0),
        ("2 - +5 DEG DEAD BAND", 175.0),
        ("3 - +10 DEG ERROR", 170.0),
        ("4 - +45 DEG ERROR", 135.0),
        ("5 - +90 DEG ERROR", 90.0),
        ("6 - -10 DEG ERROR", 190.0),
        ("7 - -45 DEG ERROR", 225.0),
        ("8 - -90 DEG ERROR", 270.0),
    ]

    print("\n" + "#" * 68)
    print("PARAFOIL NAVIGATION LOGIC TEST")
    print("#" * 68)
    for title, heading in cases:
        print_result(title, calculate_navigation(lat, lon, heading))


def assertion_tests() -> None:
```

```
    """Automatic pass/fail checks."""
    tests = []

    tests.append((abs(wrap180(181.0) + 179.0) < 1e-9,
                  "+181 wraps to -179"))
    tests.append((abs(wrap180(-181.0) - 179.0) < 1e-9,
                  "-181 wraps to +179"))

    d, s1, s2 = calculate_steering(0.0)
    tests.append((d == 0 and s1 == 2048 and s2 == 2048,
                  "0 deg -> neutral"))

    d, s1, s2 = calculate_steering(5.0)
    tests.append((d == 0 and s1 == 2048 and s2 == 2048,
                  "+5 deg -> deadband"))

    d, s1, s2 = calculate_steering(-5.0)
    tests.append((d == 0 and s1 == 2048 and s2 == 2048,
                  "-5 deg -> deadband"))

    d, s1, s2 = calculate_steering(90.0)
    tests.append((d == 450 and s1 == 1598 and s2 == 2498,
                  "+90 deg -> +450"))

    d, s1, s2 = calculate_steering(-90.0)
    tests.append((d == -450 and s1 == 2498 and s2 == 1598,
                  "-90 deg -> -450"))

    print("\n" + "#" * 68)
    print("AUTOMATIC PASS/FAIL TESTS")
    print("#" * 68)

    failed = 0
    for ok, name in tests:
        print(f"[{'PASS' if ok else 'FAIL'}] {name}")
        failed += int(not ok)

    print(f"\n{len(tests) - failed} passed, {failed} failed")
    if failed:
        raise SystemExit(1)


def wrap_tests() -> None:
    print("\n" + "#" * 68)
    print("ANGLE WRAP TESTS")
    print("#" * 68)
    print(f"{'Target':>10} {'Heading':>10} {'Error':>10}")
    print("-" * 34)

    for target, heading in [
        (1.0, 359.0),
        (359.0, 1.0),
        (350.0, 10.0),
        (10.0, 350.0),
        (0.0, 180.0),
        (180.0, 0.0),
    ]:
        error = wrap180(target - heading)
        print(f"{target:10.1f} {heading:10.1f} {error:10.1f}")


def heading_sweep() -> None:
    print("\n" + "#" * 68)
    print("HEADING SWEEP")
    print("#" * 68)
    lat = 32.6438366
    lon = TARGET_LON
    print(f"{'Head':>8} {'Bearing':>9} {'Error':>9} {'Demand':>9} {'S1':>7} {'S2':>7}")
    print("-" * 55)

    for heading in range(0, 360, 10):
        r = calculate_navigation(lat, lon, float(heading))
        print(
            f"{r.heading:8.1f} {r.target_bearing:9.1f} "
            f"{r.course_error:9.1f} {r.demand:9d} "
```

```
            f"{r.servo1:7d} {r.servo2:7d}"
        )


def interactive_test() -> None:
    print("\n" + "#" * 68)
    print("INTERACTIVE TEST")
    print("#" * 68)
    print("Enter: latitude longitude heading")
    print("Example: 32.6438366 75.014533 135")
    print("Type q to exit.")

    while True:
        try:
            line = input("\nLAT LON HEADING > ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            return

        if line.lower() == "q":
            return

        parts = line.split()
        if len(parts) != 3:
            print("Please enter exactly 3 values.")
            continue

        try:
            lat, lon, heading = map(float, parts)
        except ValueError:
            print("Invalid number.")
            continue

        if not -90 <= lat <= 90:
            print("Latitude must be between -90 and +90.")
            continue
        if not -180 <= lon <= 180:
            print("Longitude must be between -180 and +180.")
            continue

        print_result(
            "INTERACTIVE RESULT",
            calculate_navigation(lat, lon, heading),
        )


def send_to_teensy(port: str, baud: int,
                   lat: float, lon: float, heading: float) -> None:
    """Optional hardware-in-the-loop test using pyserial."""
    try:
        import serial
    except ImportError:
        print("pyserial is not installed.")
        print("Run: python -m pip install pyserial")
        raise SystemExit(1)

    command = f"NAVTEST {lat:.7f} {lon:.7f} {heading:.2f}\n"

    print(f"Opening {port} at {baud} baud...")
    try:
        with serial.Serial(port, baudrate=baud, timeout=1) as ser:
            time.sleep(2.0)
            ser.reset_input_buffer()
            ser.write(command.encode("ascii"))
            ser.flush()

            print(f"Sent: {command.strip()}")
            print("Teensy response:")

            start = time.time()
            while time.time() - start < 4.0:
                line = ser.readline().decode("utf-8", errors="replace").strip()
                if line:
                    print(line)
    except Exception as exc:
```

```
        print(f"Could not use {port}: {exc}")
        raise SystemExit(1)


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Parafoil GPS navigation and ST-3009 logic test bench."
    )
    parser.add_argument("--wrap", action="store_true")
    parser.add_argument("--sweep", action="store_true")
    parser.add_argument("--assert", dest="do_assert", action="store_true")
    parser.add_argument("--interactive", action="store_true")

    parser.add_argument("--port")
    parser.add_argument("--baud", type=int, default=115200)
    parser.add_argument("--lat", type=float)
    parser.add_argument("--lon", type=float)
    parser.add_argument("--heading", type=float)

    args = parser.parse_args()

    # Hardware-in-the-loop mode
    if args.port or any(v is not None for v in (args.lat, args.lon, args.heading)):
        if not args.port or args.lat is None or args.lon is None or args.heading is None:
            parser.error("Hardware mode requires --port --lat --lon --heading")
        send_to_teensy(args.port, args.baud, args.lat, args.lon, args.heading)
        return

    # Default test
    basic_tests()

    if args.do_assert:
        assertion_tests()
    if args.wrap:
        wrap_tests()
    if args.sweep:
        heading_sweep()
    if args.interactive:
        interactive_test()


if __name__ == "__main__":
    main()
```

A.12 Archived source 12: targeted_landing_simulation.py

Source file: `targeted_landing_simulation.py`

```
import math
import random
import argparse
from dataclasses import dataclass
import matplotlib.pyplot as plt

# ======================== USER PARAMETERS ========================
TARGET_LAT = 32.639340
TARGET_LON = 75.014533

# Default launch point: 500 m north, 300 m west of target.
START_NORTH_M = 500.0
START_EAST_M = -300.0
INITIAL_ALTITUDE_M = 600.0

# Replace these with measured parafoil values when available.
GLIDE_SPEED_MPS = 8.0
DESCENT_RATE_MPS = 4.0
MAX_TURN_RATE_DEG_S = 25.0   # simplified controller model

# Same steering settings used by your Teensy logic.
SERVO_NEUTRAL = 2048
MAX_STEERING_COUNTS = 450
STEERING_DEADBAND_DEG = 5.0
FULL_STEERING_ERROR_DEG = 90.0
```

```
MAX_SLEW_COUNTS = 60
TARGET_CAPTURE_RADIUS_M = 35.0
DT_S = 0.10
MAX_FLIGHT_TIME_S = 300.0

@dataclass
class State:
    t: float
    alt: float
    north: float
    east: float
    heading: float
    ground_course: float
    ground_speed: float
    distance: float
    bearing: float
    error: float
    demand: float
    servo1: int
    servo2: int
    captured: bool

def wrap360(a):
    return a % 360.0

def wrap180(a):
    return (a + 180.0) % 360.0 - 180.0

def distance_bearing(dn, de):
    distance = math.hypot(dn, de)
    bearing = wrap360(math.degrees(math.atan2(de, dn)))
    return distance, bearing

def velocity_from_heading(heading, speed):
    r = math.radians(heading)
    return math.cos(r) * speed, math.sin(r) * speed

def heading_from_velocity(vn, ve):
    return wrap360(math.degrees(math.atan2(ve, vn)))

def steering_demand(error):
    ae = abs(error)
    if ae <= STEERING_DEADBAND_DEG:
        return 0
    usable = ae - STEERING_DEADBAND_DEG
    span = FULL_STEERING_ERROR_DEG - STEERING_DEADBAND_DEG
    magnitude = round(max(0.0, min(1.0, usable / span)) * MAX_STEERING_COUNTS)
    return magnitude if error > 0 else -magnitude

def slew(previous, target):
    d = max(-MAX_SLEW_COUNTS, min(MAX_SLEW_COUNTS, target - previous))
    return previous + d

def servo_positions(demand):
    s1 = max(200, min(3895, round(SERVO_NEUTRAL - demand)))
    s2 = max(200, min(3895, round(SERVO_NEUTRAL + demand)))
    return s1, s2

def run(wind_east=0.0, wind_north=0.0, seed=1):
    random.seed(seed)
    t = 0.0
    alt = INITIAL_ALTITUDE_M
    north = START_NORTH_M
    east = START_EAST_M
    heading = distance_bearing(-north, -east)[1]
    demand = 0.0
    captured = False
    hist = []

    while alt > 0 and t < MAX_FLIGHT_TIME_S:
        dn, de = -north, -east
        dist, bearing = distance_bearing(dn, de)

        if not captured and dist <= TARGET_CAPTURE_RADIUS_M:
            captured = True
```

```
        if captured:
            target = 0
            error = 0.0
        else:
            error = wrap180(bearing - heading)
            target = steering_demand(error)

        demand = slew(demand, target)
        s1, s2 = servo_positions(demand)

        turn_rate = (demand / MAX_STEERING_COUNTS) * MAX_TURN_RATE_DEG_S
        heading = wrap360(heading + turn_rate * DT_S)

        air_n, air_e = velocity_from_heading(heading, GLIDE_SPEED_MPS)
        ground_n = air_n + wind_north
        ground_e = air_e + wind_east
        ground_speed = math.hypot(ground_n, ground_e)
        ground_course = heading_from_velocity(ground_n, ground_e)

        north += ground_n * DT_S
        east += ground_e * DT_S
        alt = max(0.0, alt - DESCENT_RATE_MPS * DT_S)
        t += DT_S

        hist.append(State(t, alt, north, east, heading, ground_course,
                          ground_speed, dist, bearing, error, demand,
                          s1, s2, captured))

    landing_error = math.hypot(north, east)
    return hist, landing_error, t, captured, north, east

def print_result(hist, err, t, captured, north, east, wind_e, wind_n):
    print('\n' + '=' * 72)
    print('PARAFOIL TARGETED LANDING SIMULATION')
    print('=' * 72)
    print(f'Target GPS        : {TARGET_LAT:.7f}, {TARGET_LON:.7f}')
    print(f'Start offset       : N {START_NORTH_M:+.1f} m, E {START_EAST_M:+.1f} m')
    print(f'Initial altitude   : {INITIAL_ALTITUDE_M:.1f} m')
    print(f'Glide speed        : {GLIDE_SPEED_MPS:.2f} m/s')
    print(f'Descent rate       : {DESCENT_RATE_MPS:.2f} m/s')
    print(f'Wind               : N {wind_n:+.2f} m/s, E {wind_e:+.2f} m/s')
    print(f'Flight time        : {t:.2f} s')
    print(f'Landing offset     : N {north:+.2f} m, E {east:+.2f} m')
    print(f'Landing error      : {err:.2f} m')
    print(f'Target captured    : {"YES" if captured else "NO"}')
    print(f'35 m requirement   : {"PASS" if err <= TARGET_CAPTURE_RADIUS_M else "FAIL"}')
    print('=' * 72)

def plot_trajectory(hist):
    plt.figure(figsize=(8, 7))
    plt.plot([s.east for s in hist], [s.north for s in hist], label='Parafoil path')
    plt.scatter([START_EAST_M], [START_NORTH_M], label='Start', s=70)
    plt.scatter([0], [0], marker='x', label='Target', s=100)
    c = plt.Circle((0, 0), TARGET_CAPTURE_RADIUS_M, fill=False,
                   linestyle='--', label='35 m capture radius')
    plt.gca().add_patch(c)
    plt.xlabel('East (m)')
    plt.ylabel('North (m)')
    plt.title('Targeted Landing Trajectory')
    plt.axis('equal')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()

def plot_altitude(hist):
    plt.figure(figsize=(8, 5))
    plt.plot([s.t for s in hist], [s.alt for s in hist])
    plt.xlabel('Time (s)')
    plt.ylabel('Altitude (m)')
    plt.title('Descent Profile')
    plt.grid(True)
    plt.tight_layout()

def plot_steering(hist):
```

```
    plt.figure(figsize=(8, 5))
    plt.plot([s.t for s in hist], [s.demand for s in hist])
    plt.axhline(0, linestyle='--')
    plt.axhline(MAX_STEERING_COUNTS, linestyle=':')
    plt.axhline(-MAX_STEERING_COUNTS, linestyle=':')
    plt.xlabel('Time (s)')
    plt.ylabel('Steering demand (counts)')
    plt.title('ST-3009 Steering Demand')
    plt.grid(True)
    plt.tight_layout()

def wind_sweep():
    winds = list(range(0, 7))
    errors = []
    for w in winds:
        _, e, _, _, _, _ = run(wind_east=w, seed=1)
        errors.append(e)
        print(f'Wind {w:>2} m/s -> landing error {e:>8.2f} m')
    plt.figure(figsize=(8, 5))
    plt.plot(winds, errors, marker='o')
    plt.axhline(TARGET_CAPTURE_RADIUS_M, linestyle='--', label='35 m requirement')
    plt.xlabel('Constant east wind (m/s)')
    plt.ylabel('Landing error (m)')
    plt.title('Landing Error vs Wind')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()

def main():
    p = argparse.ArgumentParser()
    p.add_argument('--wind', type=float, default=0.0)
    p.add_argument('--wind-north', type=float, default=0.0)
    p.add_argument('--wind-sweep', action='store_true')
    p.add_argument('--no-plots', action='store_true')
    args = p.parse_args()

    hist, err, t, captured, n, e = run(
        wind_east=args.wind,
        wind_north=args.wind_north,
        seed=1,
    )
    print_result(hist, err, t, captured, n, e, args.wind, args.wind_north)

    if args.wind_sweep:
        print('\nWIND SWEEP')
        wind_sweep()
    elif not args.no_plots:
        plot_trajectory(hist)
        plot_altitude(hist)
        plot_steering(hist)

    if not args.no_plots:
        plt.show()

if __name__ == '__main__':
    main()
```

A.13 Archived source 13: teensy_matlab_telemetry_addon.ino

Source file: `teensy_matlab_telemetry_addon.ino`

```
// ============================================================================
// TEENSY -> MATLAB LIVE TELEMETRY ADD-ON
// ============================================================================
// Add these pieces to your existing guided landing sketch.
// The MATLAB program expects one machine-readable line:
// NAVMAT,millis,lat,lon,speed,gpsCourse,servo1FB,servo2FB,demand,state
//
// Example:
// NAVMAT,123456,32.6438366,75.014533,2.31,182.40,2028,2068,20,AUTONAV
//
// IMPORTANT:
// servo1FB / servo2FB below are REAL ReadPos() feedback values.
```

```
// Your existing sketch currently stores commanded positions in
// servo1Position / servo2Position; those are not the same as actual feedback.
// ============================================================================

int servo1Feedback = -1;
int servo2Feedback = -1;
unsigned long lastMatlabTelemetryMs = 0;
unsigned long lastServoFeedbackMatlabMs = 0;
const unsigned long MATLAB_TELEMETRY_INTERVAL_MS = 200;
const unsigned long MATLAB_SERVO_FEEDBACK_INTERVAL_MS = 200;

void updateServoFeedbackForMatlab()
{
  if (millis() - lastServoFeedbackMatlabMs < MATLAB_SERVO_FEEDBACK_INTERVAL_MS)
    return;

  lastServoFeedbackMatlabMs = millis();

  if (!servosReady())
  {
    servo1Feedback = -1;
    servo2Feedback = -1;
    return;
  }

  int p1 = st.ReadPos(SERVO1_ID);
  delay(3);
  int p2 = st.ReadPos(SERVO2_ID);

  if (p1 >= 0)
    servo1Feedback = p1;

  if (p2 >= 0)
    servo2Feedback = p2;
}

const char* matlabStateName()
{
  switch (state)
  {
    case STATE_WAIT_READY:  return "WAIT_READY";
    case STATE_DEPLOYING:   return "DEPLOYING";
    case STATE_STABILIZING: return "STABILIZING";
    case STATE_AUTONAV:     return "AUTONAV";
    case STATE_CAPTURED:    return "CAPTURED";
    case STATE_FAILSAFE:    return "FAILSAFE";
    case STATE_MANUAL_TEST: return "MANUAL_TEST";
    default:                return "UNKNOWN";
  }
}

void sendMatlabTelemetry()
{
  if (millis() - lastMatlabTelemetryMs < MATLAB_TELEMETRY_INTERVAL_MS)
    return;

  lastMatlabTelemetryMs = millis();

  if (!gpsFix)
    return;

  Serial.print("NAVMAT,");
  Serial.print(millis());
  Serial.print(",");
  Serial.print(currentLatitude, 7);
  Serial.print(",");
  Serial.print(currentLongitude, 7);
  Serial.print(",");
  Serial.print(currentSpeed, 2);
  Serial.print(",");

  if (gps.course.isValid())
    Serial.print(gps.course.deg(), 2);
  else
    Serial.print(-1.0, 2);
```

```
88
89    Serial.print(",");
90    Serial.print(servo1Feedback);
91    Serial.print(",");
92    Serial.print(servo2Feedback);
93    Serial.print(",");
94    Serial.print(steeringDemand);
95    Serial.print(",");
96    Serial.println(matlabStateName());
97  }
98
99  // ============================================================================
100 // In loop(), add:
101 //
102 // updateServoFeedbackForMatlab();
103 // sendMatlabTelemetry();
104 // ============================================================================
```

A.14 Archived source 14: live_parafoil_target_monitor.m

Source file: `live_parafoil_target_monitor.m`

```
1  %% LIVE PARAFOIL TARGETED-LANDING MONITOR
2  % Reads telemetry from the Teensy USB COM port and shows:
3  %   1) live GPS position and path
4  %   2) target bearing and distance
5  %   3) actual movement course from GPS
6  %   4) whether the vehicle is approaching or moving away
7  %   5) actual ST-3009 servo feedback positions
8  %   6) servo differential / steering direction
9  %   7) target alignment
10 %
11 % IMPORTANT:
12 % With the current hardware, GPS is connected to Teensy Serial3 and the
13 % servos are connected to Teensy Serial1. Therefore MATLAB should normally
14 % read the TEENSY USB COM port. The Teensy forwards GPS + servo feedback
15 % in one CSV line.
16 %
17 % Expected Teensy line:
18 % NAVMAT,millis,lat,lon,speed,gpsCourse,s1FB,s2FB,demand,state
19 %
20 % Example:
21 % NAVMAT,123456,32.6438366,75.014533,2.31,182.4,2028,2068,20,AUTONAV
22
23 clear;
24 clc;
25 close all;
26
27 %% ===================== USER SETTINGS ================================
28
29 COM_PORT = "COM5";       % CHANGE THIS
30 BAUD_RATE = 115200;      % Teensy USB Serial Monitor baud
31
32 TARGET_LAT = 32.639340;
33 TARGET_LON = 75.014533;
34
35 TARGET_CAPTURE_RADIUS_M = 35.0;
36
37 % Minimum movement speed before GPS course is considered reliable.
38 MIN_SPEED_MPS = 1.5;
39
40 % Maximum age allowed for an update before status becomes stale.
41 STALE_TIMEOUT_S = 3.0;
42
43 % Servo neutral and maximum differential.
44 SERVO_NEUTRAL = 2048;
45 MAX_STEERING_COUNTS = 450;
46
47 % If your physical mechanism turns opposite to the assumed sign, change this
48 % from +1 to -1. This affects only the displayed steering interpretation.
49 POSITIVE_STEERING_SIGN = +1;
50
51 % Keep last N GPS samples in the plot.
```

```
MAX_HISTORY = 2000;

%% ==================== SERIAL PORT ================================

fprintf('Opening Teensy port %s at %d baud...\n', COM_PORT, BAUD_RATE);

s = serialport(COM_PORT, BAUD_RATE);
configureTerminator(s, "LF");
configureCallback(s, "off");
flush(s);

fprintf('Connected. Waiting for NAVMAT telemetry...\n');

%% ==================== FIGURE =====================================

fig = figure( ...
    'Name', 'Parafoil Live Targeted Landing Monitor', ...
    'NumberTitle', 'off', ...
    'Color', 'w');

% -------------------- Plot 1: trajectory --------------------------
ax1 = subplot(2,2,1);
hPath = animatedline(ax1, ...
    'LineWidth', 1.5);
hCurrent = plot(ax1, NaN, NaN, 'o', 'MarkerSize', 8, 'LineWidth', 2);
hTarget = plot(ax1, 0, 0, 'x', 'MarkerSize', 12, 'LineWidth', 2);
theta = linspace(0, 2*pi, 200);
hCapture = plot(ax1, TARGET_CAPTURE_RADIUS_M*cos(theta), TARGET_CAPTURE_RADIUS_M*sin(theta), '--');

grid(ax1, 'on');
axis(ax1, 'equal');
xlabel(ax1, 'East from target (m)');
ylabel(ax1, 'North from target (m)');
title(ax1, 'Live GPS Trajectory');

% -------------------- Plot 2: distance ----------------------------
ax2 = subplot(2,2,2);
hDistance = animatedline(ax2, 'LineWidth', 1.5);
yline(ax2, TARGET_CAPTURE_RADIUS_M, '--');
grid(ax2, 'on');
xlabel(ax2, 'Time (s)');
ylabel(ax2, 'Distance to target (m)');
title(ax2, 'Distance to Target');

% -------------------- Plot 3: servo feedback ----------------------
ax3 = subplot(2,2,3);
hS1 = animatedline(ax3, 'LineWidth', 1.5);
hS2 = animatedline(ax3, 'LineWidth', 1.5);
yline(ax3, SERVO_NEUTRAL, '--');
grid(ax3, 'on');
xlabel(ax3, 'Time (s)');
ylabel(ax3, 'Servo position');
title(ax3, 'Actual ST-3009 Feedback');
legend(ax3, 'Servo 1', 'Servo 2', 'Neutral', 'Location', 'best');

% -------------------- Plot 4: alignment ---------------------------
ax4 = subplot(2,2,4);
hAlignment = animatedline(ax4, 'LineWidth', 1.5);
yline(ax4, 0, '--');
grid(ax4, 'on');
xlabel(ax4, 'Time (s)');
ylabel(ax4, 'Bearing error (deg)');
title(ax4, 'Movement Course vs Target Bearing');

%% ==================== LIVE STATE ==================================

historyTime = [];
historyLat = [];
historyLon = [];
historyNorth = [];
historyEast = [];
historyDistance = [];
historyTargetBearing = [];
historyCourse = [];
historyCourseError = [];
```

```
historyS1 = [];
historyS2 = [];
historyDemand = [];

lastLat = NaN;
lastLon = NaN;
lastTime = NaN;
lastDistance = NaN;
lastUpdateTime = tic;

% Counters for approach/away determination.
approachCount = 0;
awayCount = 0;

fprintf('\n');
fprintf('============================================================\n');
fprintf(' LIVE PARAFOIL TARGETED-LANDING MONITOR\n');
fprintf(' Target: %.7f, %.7f\n', TARGET_LAT, TARGET_LON);
fprintf('============================================================\n');

%% ==================== MAIN LOOP ================================

try
    while isvalid(fig)

        if s.NumBytesAvailable == 0
            drawnow limitrate;
            pause(0.01);
            continue;
        end

        line = strtrim(readline(s));

        % Ignore normal human-readable Teensy lines.
        if ~startsWith(line, "NAVMAT,")
            continue;
        end

        parts = split(line, ',');

        % Required fields:
        % 1 NAVMAT
        % 2 millis
        % 3 latitude
        % 4 longitude
        % 5 speed
        % 6 GPS course
        % 7 servo1 feedback
        % 8 servo2 feedback
        % 9 steering demand
        % 10 state
        if numel(parts) < 10
            continue;
        end

        tArduino = str2double(parts(2)) / 1000.0;
        lat = str2double(parts(3));
        lon = str2double(parts(4));
        gpsSpeed = str2double(parts(5));
        gpsCourse = str2double(parts(6));
        s1 = str2double(parts(7));
        s2 = str2double(parts(8));
        demand = str2double(parts(9));
        state = char(parts(10));

        if any(isnan([tArduino lat lon gpsSpeed s1 s2 demand]))
            continue;
        end

        % ---------------------------------------------------------
        % GPS target geometry
        % ---------------------------------------------------------

        [distanceM, targetBearingDeg] = distanceAndBearing( ...
            lat, lon, TARGET_LAT, TARGET_LON);
```

```
202
203         % ------------------------------------------------------------
204         % Movement course from consecutive GPS points
205         % ------------------------------------------------------------
206         movementCourse = NaN;
207
208         if ~isnan(lastLat) && ~isnan(lastLon) && ~isnan(lastTime)
209             dt = tArduino - lastTime;
210
211             if dt > 0
212                 [stepDistanceM, stepBearing] = distanceAndBearing( ...
213                     lastLat, lastLon, lat, lon);
214
215                 if stepDistanceM > 0.5
216                     movementCourse = stepBearing;
217                 end
218             end
219         end
220
221         % If GPS course is valid and speed is high enough, use it as the
222         % primary course; otherwise use course calculated from positions.
223         if ~isnan(gpsCourse) && gpsCourse >= 0 && gpsSpeed >= MIN_SPEED_MPS
224             actualCourse = wrap360(gpsCourse);
225             courseSource = 'GPS COURSE';
226         elseif ~isnan(movementCourse)
227             actualCourse = wrap360(movementCourse);
228             courseSource = 'POSITION DELTA';
229         else
230             actualCourse = NaN;
231             courseSource = 'NONE';
232         end
233
234         % ------------------------------------------------------------
235         % Bearing error: where motion is going vs where target is
236         % ------------------------------------------------------------
237         if ~isnan(actualCourse)
238             movementBearingError = wrap180( ...
239                 targetBearingDeg - actualCourse);
240         else
241             movementBearingError = NaN;
242         end
243
244         % ------------------------------------------------------------
245         % Servo differential
246         % ------------------------------------------------------------
247         if s1 < 0 || s2 < 0
248             measuredDemand = NaN;
249         else
250             measuredDemand = (s2 - s1) / 2.0;
251         end
252
253         % Clamp to expected range for display only.
254         if ~isnan(measuredDemand)
255             measuredDemand = max( ...
256                 -MAX_STEERING_COUNTS, ...
257                 min(MAX_STEERING_COUNTS, measuredDemand));
258         end
259
260         steeringDirection = "NEUTRAL";
261
262         if measuredDemand > 5
263             if POSITIVE_STEERING_SIGN > 0
264                 steeringDirection = "POSITIVE STEERING";
265             else
266                 steeringDirection = "NEGATIVE STEERING";
267             end
268         elseif measuredDemand < -5
269             if POSITIVE_STEERING_SIGN > 0
270                 steeringDirection = "NEGATIVE STEERING";
271             else
272                 steeringDirection = "POSITIVE STEERING";
273             end
274         end
275
276         % ------------------------------------------------------------
```

```
        % Is the vehicle actually approaching the target?
        % ---------------------------------------------------------------
        distanceTrend = "STARTING";

        if ~isnan(lastDistance)
            deltaDistance = distanceM - lastDistance;

            if deltaDistance < -0.5
                approachCount = approachCount + 1;
                awayCount = 0;
            elseif deltaDistance > 0.5
                awayCount = awayCount + 1;
                approachCount = 0;
            end

            if approachCount >= 2
                distanceTrend = "APPROACHING TARGET";
            elseif awayCount >= 2
                distanceTrend = "MOVING AWAY";
            else
                distanceTrend = "NEARLY STABLE";
            end
        end

        % ---------------------------------------------------------------
        % Direction classification
        % ---------------------------------------------------------------
        if isnan(movementBearingError)
            directionStatus = "WAITING FOR MOTION";
        elseif distanceM <= TARGET_CAPTURE_RADIUS_M
            directionStatus = "INSIDE 35 m";
        elseif abs(movementBearingError) <= 15
            directionStatus = "GOING TOWARD TARGET";
        elseif abs(movementBearingError) <= 45
            directionStatus = "MOSTLY TOWARD TARGET";
        elseif abs(movementBearingError) <= 90
            directionStatus = "SIDEWAYS / NEEDS TURN";
        else
            directionStatus = "GOING AWAY FROM TARGET";
        end

        % ---------------------------------------------------------------
        % Overall status
        % ---------------------------------------------------------------
        if distanceM <= TARGET_CAPTURE_RADIUS_M
            overallStatus = "TARGET ZONE";
        elseif strcmp(distanceTrend, "MOVING AWAY")
            overallStatus = "WARNING: MOVING AWAY";
        elseif strcmp(directionStatus, "GOING AWAY FROM TARGET")
            overallStatus = "WARNING: WRONG DIRECTION";
        elseif strcmp(directionStatus, "GOING TOWARD TARGET")
            overallStatus = "ON TRACK";
        else
            overallStatus = directionStatus;
        end

        % ---------------------------------------------------------------
        % History
        % ---------------------------------------------------------------

        [northM, eastM] = localOffsetFromTarget( ...
            lat, lon, TARGET_LAT, TARGET_LON);

        historyTime(end+1) = tArduino;
        historyLat(end+1) = lat;
        historyLon(end+1) = lon;
        historyNorth(end+1) = northM;
        historyEast(end+1) = eastM;
        historyDistance(end+1) = distanceM;
        historyTargetBearing(end+1) = targetBearingDeg;
        historyCourse(end+1) = actualCourse;
        historyCourseError(end+1) = movementBearingError;
        historyS1(end+1) = s1;
        historyS2(end+1) = s2;
        historyDemand(end+1) = measuredDemand;
```

```
352
353         if numel(historyTime) > MAX_HISTORY
354             historyTime(1) = [];
355             historyLat(1) = [];
356             historyLon(1) = [];
357             historyNorth(1) = [];
358             historyEast(1) = [];
359             historyDistance(1) = [];
360             historyTargetBearing(1) = [];
361             historyCourse(1) = [];
362             historyCourseError(1) = [];
363             historyS1(1) = [];
364             historyS2(1) = [];
365             historyDemand(1) = [];
366         end
367
368         % ---------------------------------------------------------------
369         % Console status
370         % ---------------------------------------------------------------
371
372         fprintf(['\rDIST=%7.1f m | TB=%6.1f | COURSE=%6.1f | ', ...
373                  'ERR=%+6.1f | S1=%4d | S2=%4d | ', ...
374                  '%.28s'], ...
375             distanceM, ...
376             targetBearingDeg, ...
377             actualCourse, ...
378             movementBearingError, ...
379             round(s1), ...
380             round(s2), ...
381             overallStatus);
382
383         % ---------------------------------------------------------------
384         % Update plots
385         % ---------------------------------------------------------------
386
387         addpoints(hPath, eastM, northM);
388         set(hCurrent, 'XData', eastM, 'YData', northM);
389
390         addpoints(hDistance, tArduino, distanceM);
391         addpoints(hS1, tArduino, s1);
392         addpoints(hS2, tArduino, s2);
393
394         if ~isnan(movementBearingError)
395             addpoints(hAlignment, tArduino, movementBearingError);
396         end
397
398         % Keep trajectory centered around target.
399         margin = 100;
400         maxRange = max(500, ...
401             max(abs([historyEast historyNorth])) + margin);
402
403         xlim(ax1, [-maxRange maxRange]);
404         ylim(ax1, [-maxRange maxRange]);
405
406         drawnow limitrate;
407
408         % ---------------------------------------------------------------
409         % Save previous sample
410         % ---------------------------------------------------------------
411
412         lastLat = lat;
413         lastLon = lon;
414         lastTime = tArduino;
415         lastDistance = distanceM;
416         lastUpdateTime = tic;
417     end
418
419 catch ME
420     fprintf('\nMonitor stopped: %s\n', ME.message);
421 end
422
423 clear s;
424
425 %% ====================================================================
426 % LOCAL FUNCTIONS
```

```
% ======================================================================

function [distanceM, bearingDeg] = distanceAndBearing(lat1, lon1, lat2, lon2)
    % Haversine distance + initial bearing.
    R = 6371000.0;

    phi1 = deg2rad(lat1);
    phi2 = deg2rad(lat2);
    dPhi = deg2rad(lat2 - lat1);
    dLambda = deg2rad(lon2 - lon1);

    a = sin(dPhi/2).^2 + ...
        cos(phi1).*cos(phi2).*sin(dLambda/2).^2;

    a = max(0, min(1, a));

    c = 2 * atan2(sqrt(a), sqrt(1-a));
    distanceM = R * c;

    y = sin(dLambda) .* cos(phi2);
    x = cos(phi1).*sin(phi2) - ...
        sin(phi1).*cos(phi2).*cos(dLambda);

    bearingDeg = wrap360(rad2deg(atan2(y, x)));
end

function angle = wrap360(angle)
    angle = mod(angle, 360);
    if angle < 0
        angle = angle + 360;
    end
end

function angle = wrap180(angle)
    angle = mod(angle + 180, 360) - 180;
end

function [northM, eastM] = localOffsetFromTarget(lat, lon, targetLat, targetLon)
    % Local tangent-plane approximation, accurate enough for this plot scale.
    R = 6371000.0;

    northM = deg2rad(lat - targetLat) * R;

    meanLat = deg2rad((lat + targetLat) / 2);
    eastM = deg2rad(lon - targetLon) * R * cos(meanLat);
end
```

A.15 Archived source 15: live_parafoil_target_monitor_FINAL.m

Source file: `live_parafoil_target_monitor_FINAL.m`

```
%% ======================================================================
%  LIVE PARAFOIL TARGETED-LANDING MONITOR
%  MATLAB <-> Teensy 4.1
%  ======================================================================
%
%  PURPOSE
%    - Read live telemetry from Teensy over USB COM port
%    - Enter target latitude/longitude here
%    - Send target to Teensy
%    - Plot live GPS trajectory
%    - Show distance and target bearing
%    - Show GPS course and controller heading
%    - Show course error and steering demand
%    - Show commanded AND actual ST-3009 positions
%    - Tell whether the vehicle is approaching the target
%    - Save the complete run to CSV when the monitor closes
%
%  REQUIRED TELEMETRY FORMAT FROM TEENSY
%    NAVMAT,millis,lat,lon,alt,speed,course,heading,distance,bearing,
%    error,demand,cmdS1,cmdS2,fbS1,fbS2,sats,hdop,state,
%    headingSource,compass
%
```

```
%  IMPORTANT
%    This MATLAB program is a live MONITOR and data logger.
%    It does not yet implement wind compensation or a PD controller.
%    Do not use the current 35 m circle as proof of final landing
%    accuracy while the vehicle is still airborne.
%% ================================================================

clear;
clc;
close all;

%% ======================== USER SETTINGS ========================

COM_PORT = "COM5";          % CHANGE THIS
BAUD_RATE = 115200;

% Target entered here and sent to Teensy.
TARGET_LAT = 32.6321769;
TARGET_LON = 75.0143889;

% Target capture/reference circle for visualization only.
TARGET_RADIUS_M = 35;

% Do NOT automatically start AUTONAV or DEPLOY.
SEND_TARGET_ON_START = true;
SEND_AUTONAV_ON_START = false;
SEND_DEPLOY_ON_START = false;

% GPS course is normally useful only when moving.
MIN_GPS_COURSE_SPEED = 1.5;

% Update graphics approximately every N telemetry packets.
PLOT_UPDATE_EVERY = 1;

%% ======================== CONNECT ================================

fprintf('\n================================================\n');
fprintf(' LIVE PARAFOIL MATLAB MONITOR\n');
fprintf('================================================\n');
fprintf('COM port : %s\n', COM_PORT);
fprintf('Baud     : %d\n', BAUD_RATE);
fprintf('Target   : %.7f, %.7f\n', TARGET_LAT, TARGET_LON);

try
    sp = serialport(COM_PORT, BAUD_RATE, "Timeout", 1);
catch ME
    error(['Could not open %s. Check the Teensy COM port and make sure ', ...
           'Arduino Serial Monitor/another program is not using it.\n\n%s'], ...
           COM_PORT, ME.message);
end

configureTerminator(sp, "LF");
flush(sp);
pause(1.5);
flush(sp);

fprintf('Serial connection: OK\n');

%% ======================== SEND TARGET ============================

if SEND_TARGET_ON_START
    sendCommand(sprintf('TARGET %.7f %.7f', TARGET_LAT, TARGET_LON));
    pause(0.2);
end

% Request a human-readable status.
sendCommand('STATUS');
pause(0.2);

if SEND_AUTONAV_ON_START
    sendCommand('AUTONAV');
end

if SEND_DEPLOY_ON_START
    sendCommand('DEPLOY');
```

```
end

%% ======================== DATA ARRAYS ============================

time_ms = [];
lat = [];
lon = [];
alt = [];
speed = [];
gpsCourse = [];
heading = [];
distance = [];
targetBearing = [];
controllerError = [];
demand = [];
cmdS1 = [];
cmdS2 = [];
fbS1 = [];
fbS2 = [];
sats = [];
hdop = [];
stateText = strings(0,1);
headingSource = strings(0,1);
compass = [];

%% ======================== FIGURE ================================

fig = figure( ...
    'Name', 'Parafoil Targeted Landing - Live MATLAB Monitor', ...
    'NumberTitle', 'off', ...
    'Color', 'w', ...
    'WindowKeyPressFcn', @keyHandler);

fig.Position = [100 80 1250 760];

layout = tiledlayout(fig, 2, 2, ...
    'TileSpacing', 'compact', ...
    'Padding', 'compact');

% ---- Trajectory ----
ax1 = nexttile(layout, 1);
hold(ax1, 'on');
grid(ax1, 'on');
axis(ax1, 'equal');
xlabel(ax1, 'East (m)');
ylabel(ax1, 'North (m)');
title(ax1, 'Live GPS Trajectory');

pathPlot = plot(ax1, NaN, NaN, '-', 'LineWidth', 1.5);
currentPlot = plot(ax1, NaN, NaN, 'o', 'MarkerSize', 8, 'LineWidth', 2);
targetPlot = plot(ax1, NaN, NaN, 'x', 'MarkerSize', 12, 'LineWidth', 2);

% Target radius is drawn once target coordinates have a local reference.
resolution = 72;
circleTheta = linspace(0, 2*pi, resolution);
circlePlot = plot(ax1, NaN, NaN, '--', 'LineWidth', 1.0);
legend(ax1, {'GPS path', 'Current', 'Target', '35 m radius'}, ...
    'Location', 'best');

% ---- Steering ----
ax2 = nexttile(layout, 2);
hold(ax2, 'on');
grid(ax2, 'on');
xlabel(ax2, 'Time (s)');
ylabel(ax2, 'Steering demand');
title(ax2, 'Steering Command');

steerPlot = plot(ax2, NaN, NaN, 'LineWidth', 1.5);
yline(ax2, 450, '--');
yline(ax2, -450, '--');
yline(ax2, 0, ':');
ylim(ax2, [-500 500]);

% ---- Distance ----
ax3 = nexttile(layout, 3);
```

```
hold(ax3, 'on');
grid(ax3, 'on');
xlabel(ax3, 'Time (s)');
ylabel(ax3, 'Distance to target (m)');
title(ax3, 'Distance to Target');
distPlot = plot(ax3, NaN, NaN, 'LineWidth', 1.5);
yline(ax3, TARGET_RADIUS_M, '--', '35 m reference');

% ---- Text dashboard ----
ax4 = nexttile(layout, 4);
axis(ax4, 'off');

infoText = text( ...
    ax4, 0.02, 0.98, '', ...
    'Units', 'normalized', ...
    'VerticalAlignment', 'top', ...
    'FontName', 'Consolas', ...
    'FontSize', 11);

%% ========================= MAIN LOOP ==============================

packetCount = 0;
lastStatus = "WAITING FOR NAVMAT";
lastUpdateTime = tic;

% Local plot reference.
haveReference = false;
refLat = TARGET_LAT;
refLon = TARGET_LON;

targetEast = 0;
targetNorth = 0;

while isgraphics(fig)

    while sp.NumBytesAvailable > 0

        raw = readline(sp);
        line = strtrim(string(raw));

        % Ignore normal human-readable Teensy messages.
        if ~startsWith(line, "NAVMAT,")
            continue;
        end

        parts = split(line, ",");

        % Expected number of fields:
        % 1 NAVMAT
        % 2 millis
        % 3 lat
        % 4 lon
        % 5 alt
        % 6 speed
        % 7 course
        % 8 heading
        % 9 distance
        %10 bearing
        %11 error
        %12 demand
        %13 cmdS1
        %14 cmdS2
        %15 fbS1
        %16 fbS2
        %17 sats
        %18 hdop
        %19 state
        %20 headingSource
        %21 compass

        if numel(parts) < 21
            continue;
        end

        try
```

```
            k = numel(time_ms) + 1;

            time_ms(k,1) = str2double(parts(2));
            lat(k,1) = str2double(parts(3));
            lon(k,1) = str2double(parts(4));
            alt(k,1) = str2double(parts(5));
            speed(k,1) = str2double(parts(6));
            gpsCourse(k,1) = str2double(parts(7));
            heading(k,1) = str2double(parts(8));
            distance(k,1) = str2double(parts(9));
            targetBearing(k,1) = str2double(parts(10));
            controllerError(k,1) = str2double(parts(11));
            demand(k,1) = str2double(parts(12));
            cmdS1(k,1) = str2double(parts(13));
            cmdS2(k,1) = str2double(parts(14));
            fbS1(k,1) = str2double(parts(15));
            fbS2(k,1) = str2double(parts(16));
            sats(k,1) = str2double(parts(17));
            hdop(k,1) = str2double(parts(18));
            stateText(k,1) = parts(19);
            headingSource(k,1) = parts(20);
            compass(k,1) = str2double(parts(21));

        catch
            continue;
        end

        packetCount = packetCount + 1;

        % Establish local ENU-style plotting reference at first valid GPS.
        if ~haveReference && isfinite(lat(k)) && isfinite(lon(k))
            refLat = lat(k);
            refLon = lon(k);
            haveReference = true;

            [targetNorth, targetEast] = localNE( ...
                TARGET_LAT, TARGET_LON, refLat, refLon);
        end

        if haveReference
            [northPath, eastPath] = localNE( ...
                lat, lon, refLat, refLon);

            [targetNorth, targetEast] = localNE( ...
                TARGET_LAT, TARGET_LON, refLat, refLon);

            set(pathPlot, ...
                'XData', eastPath, ...
                'YData', northPath);

            set(currentPlot, ...
                'XData', eastPath(end), ...
                'YData', northPath(end));

            set(targetPlot, ...
                'XData', targetEast, ...
                'YData', targetNorth);

            set(circlePlot, ...
                'XData', targetEast + TARGET_RADIUS_M*cos(circleTheta), ...
                'YData', targetNorth + TARGET_RADIUS_M*sin(circleTheta));
        end

        elapsed_s = (time_ms - time_ms(1)) / 1000;

        set(steerPlot, ...
            'XData', elapsed_s, ...
            'YData', demand);

        set(distPlot, ...
            'XData', elapsed_s, ...
            'YData', distance);

        % Make the plot axes follow the data without excessive zooming.
        if packetCount == 1 || mod(packetCount, 20) == 0
```

```
            if haveReference
                allEast = [eastPath(:); targetEast];
                allNorth = [northPath(:); targetNorth];
                pad = max(50, 0.10 * max( ...
                    1, max([max(abs(allEast)), max(abs(allNorth))])));

                xlim(ax1, [min(allEast)-pad, max(allEast)+pad]);
                ylim(ax1, [min(allNorth)-pad, max(allNorth)+pad]);
            end

            xlim(ax2, [max(0, elapsed_s(end)-120), max(120, elapsed_s(end))]);
            xlim(ax3, [max(0, elapsed_s(end)-120), max(120, elapsed_s(end))]);
        end

        % ---------------------------------------------------------
        % Navigation interpretation
        % ---------------------------------------------------------

        trackError = NaN;
        closingSpeed = NaN;
        movementStatus = "NO MOVEMENT DATA";

        if isfinite(gpsCourse(k)) && speed(k) >= MIN_GPS_COURSE_SPEED
            trackError = wrap180(targetBearing(k) - gpsCourse(k));
            closingSpeed = speed(k) * cosd(trackError);

            if closingSpeed > 0.5
                movementStatus = "GOING TOWARD TARGET";
            elseif closingSpeed < -0.5
                movementStatus = "GOING AWAY FROM TARGET";
            else
                movementStatus = "CROSSING / WEAK CLOSING";
            end
        elseif numel(distance) >= 5
            recent = distance(max(1,end-4):end);
            slope = mean(diff(recent));

            if slope < -0.10
                movementStatus = "APPROACHING TARGET";
            elseif slope > 0.10
                movementStatus = "MOVING AWAY FROM TARGET";
            else
                movementStatus = "DISTANCE STABLE";
            end
        end

        if distance(k) <= TARGET_RADIUS_M
            radiusStatus = "INSIDE 35 m CIRCLE";
        else
            radiusStatus = "OUTSIDE 35 m CIRCLE";
        end

        if k > 1
            distanceTrend = mean(diff(distance(max(1,end-4):end)));
        else
            distanceTrend = NaN;
        end

        fbError1 = fbS1(k) - cmdS1(k);
        fbError2 = fbS2(k) - cmdS2(k);

        lastStatus = movementStatus;

        % ---------------------------------------------------------
        % Dashboard text
        % ---------------------------------------------------------

        txt = sprintf([ ...
            'TARGET\n' ...
            '  LAT : %.7f\n' ...
            '  LON : %.7f\n\n' ...
            'CURRENT GPS\n' ...
            '  LAT : %.7f\n' ...
            '  LON : %.7f\n' ...
            '  ALT : %.1f m\n' ...
```

```
            '  SAT : %.0f\n' ...
            '  HDOP: %.2f\n\n' ...
            'NAVIGATION\n' ...
            '  DISTANCE       : %.2f m\n' ...
            '  TARGET BEARING : %.2f deg\n' ...
            '  GPS COURSE     : %.2f deg\n' ...
            '  CONTROLLER HDG : %.2f deg\n' ...
            '  HEADING SOURCE : %s\n' ...
            '  COURSE ERROR   : %.2f deg\n' ...
            '  CLOSING SPEED  : %.2f m/s\n\n' ...
            'STEERING\n' ...
            '  DEMAND         : %d\n' ...
            '  CMD S1         : %d\n' ...
            '  CMD S2         : %d\n' ...
            '  FB  S1         : %d\n' ...
            '  FB  S2         : %d\n' ...
            '  FB ERROR S1    : %d\n' ...
            '  FB ERROR S2    : %d\n\n' ...
            'STATUS\n' ...
            '  %s\n' ...
            '  %s\n' ...
            '  State          : %s\n' ...
            '  Distance trend : %.3f m/sample\n'], ...
            TARGET_LAT, TARGET_LON, ...
            lat(k), lon(k), alt(k), sats(k), hdop(k), ...
            distance(k), targetBearing(k), gpsCourse(k), ...
            heading(k), char(headingSource(k)), controllerError(k), ...
            closingSpeed, demand(k), cmdS1(k), cmdS2(k), ...
            fbS1(k), fbS2(k), fbError1, fbError2, ...
            movementStatus, radiusStatus, char(stateText(k)), ...
            distanceTrend);

        set(infoText, 'String', txt);

        if strcmp(movementStatus, "GOING TOWARD TARGET") ...
                || strcmp(movementStatus, "APPROACHING TARGET")
            set(infoText, 'Color', [0 0.45 0]);
        elseif strcmp(movementStatus, "GOING AWAY FROM TARGET") ...
                || strcmp(movementStatus, "MOVING AWAY FROM TARGET")
            set(infoText, 'Color', [0.75 0 0]);
        else
            set(infoText, 'Color', [0 0 0]);
        end

        title(ax1, sprintf( ...
            'Live GPS Trajectory | %s | D = %.1f m', ...
            movementStatus, distance(k)));

        drawnow limitrate;
    end

    drawnow limitrate;
    pause(0.01);
end

%% ======================== SAVE LOG ==============================

try
    if ~isempty(time_ms)
        Log = table( ...
            time_ms, lat, lon, alt, speed, gpsCourse, heading, ...
            distance, targetBearing, controllerError, demand, ...
            cmdS1, cmdS2, fbS1, fbS2, sats, hdop, ...
            stateText, headingSource, compass);

        filename = sprintf( ...
            'parafoil_live_log_%s.csv', ...
            datestr(now, 'yyyymmdd_HHMMSS'));

        writetable(Log, filename);

        fprintf('\nSaved log: %s\n', filename);
    end
catch ME
    fprintf('\nCould not save CSV log: %s\n', ME.message);
```

```
end

% Return servos to center before closing if the connection is valid.
try
    if isvalid(sp)
        writeline(sp, 'CENTER');
        pause(0.2);
        clear sp;
    end
catch
end

fprintf('MATLAB monitor stopped.\n');

%% ======================== LOCAL FUNCTIONS ======================

function sendCommand(command)
    % This function is nested/uses the base workspace serial object.
    spLocal = evalin('base', 'sp');

    if ~isvalid(spLocal)
        return;
    end

    fprintf('[MATLAB -> TEENSY] %s\n', command);
    writeline(spLocal, command);
end

function keyHandler(~, event)
    % Keyboard control for the Teensy.
    %
    % A = AUTONAV
    % D = DEPLOY
    % P = PREP
    % C = CENTER
    % L = LEFT
    % R = RIGHT
    % M = MOTORTEST
    % X = RESET
    % T = resend TARGET
    % Q = quit

    switch lower(event.Key)
        case 'a'
            sendCommand('AUTONAV');

        case 'd'
            sendCommand('DEPLOY');

        case 'p'
            sendCommand('PREP');

        case 'c'
            sendCommand('CENTER');

        case 'l'
            sendCommand('LEFT');

        case 'r'
            sendCommand('RIGHT');

        case 'm'
            sendCommand('MOTORTEST');

        case 'x'
            sendCommand('RESET');

        case 't'
            targetLatLocal = evalin('base', 'TARGET_LAT');
            targetLonLocal = evalin('base', 'TARGET_LON');
            sendCommand(sprintf( ...
                'TARGET %.7f %.7f', ...
                targetLatLocal, targetLonLocal));

        case 'q'
```

```
            try
                if isgraphics(gcf)
                    close(gcf);
                end
            catch
            end
    end
end

function [north, east] = localNE(lat, lon, refLat, refLon)
    % Approximate local north/east coordinates for plotting.
    % Accurate enough for the local test area used here.
    R = 6371000;

    north = deg2rad(lat - refLat) .* R;

    east = deg2rad(lon - refLon) .* R .* cosd(refLat);
end

function value = wrap180(value)
    value = mod(value + 180, 360) - 180;
end
```

B Artifact inventory and provenance notes

The archive was reconstructed from the available conversation-uploaded files, generated working files, prior project memory, and the user-visible test results in the conversation. Some early artifacts have nearly identical names/content (for example `Pasted code(10).cpp`, `Pasted code(20261005-071236).cpp`, and the 20261003 pasted text). They are retained separately because they represent separate uploaded/edited artifacts.

The current context package is intentionally conservative: when the source did not establish a value, it is labeled as unknown, assumed, placeholder, or unresolved. In particular, simulation parameters must not be mistaken for measured parafoil performance.

C Final hand-off summary for another AI

A useful first question for the next model is not "how do we add more control sophistication?" but "why does the known-good ST-3009 bus test work while the integrated navigation code does not?" The strongest evidence currently available is the latest screenshot: HDOP was 5.13 while the integrated code accepts only HDOP ≤ 3.5. The integrated navigation function exits early on failed GPS quality, clears heading validity, and the steering routine therefore centers/does nothing. This is a deterministic software gate and should be tested before changing servo wiring or tuning a PD controller.

A robust continuation path is to create an explicit LAB mode with a relaxed HDOP threshold and a lower GPS-course threshold for walking tests, while keeping a separate strict FLIGHT mode. Then verify, in order: (1) AUTONAV state, (2) heading source = GPS COURSE, (3) nonzero course error, (4) nonzero steering demand, (5) changing commanded servo target, (6) changing actual servo feedback, and (7) GPS trajectory/distance moving toward the target. Only after those are stable should advanced landing prediction and PD control be introduced.